

**TIMEX SINCLAIR 2068
PERSONAL COLOR COMPUTER
TECHNICAL REFERENCE MANUAL**

**Prepared by
V. C. Corcoran
and
M. H. Branigin**

**TIMEX COMPUTER CORPORATION
Waterbury CT 06720
© May 1984**

Second Edition Printing
Published Exclusively by :
TIME DESIGNS MAGAZINE CO.
COLTON, OREGON 97017
© JANUARY 1986

PREFACE

This manual is dedicated to the many individuals associated with the Timex Computer Corporation in the development and production of the TS2068. Our special thanks to Nan Parsons who prepared the TS2068 Schematic and other drawings used in this manual.

While every effort has been made to make this document complete and accurate, use of the technical information contained herein is at user's sole risk. The Timex Corp. or its affiliates, and Time Designs Magazine Company assume no responsibility or liability for the safety or performance of any product manufactured relying on the technical data contained herein, or any liability, loss, damage, or expense sustained by reason of any claim that such products infringe any patent or other industrial property right. The Second Edition of this Technical Manual has been reedited by Tim Woods. Special thanks to Bob Orrfelt and Dave Clifford for technical assistance.

If you would like to receive information on a magazine and other publications for the Timex Sinclair 2068, direct your inquiry to: Time Designs Magazine Company, 29722 Hult Rd., Colton, OR 97017.

January 12, 2016

This version of the manual was imported from the online PDF version by cutting a pasting the text and appropriate drawings from the PDF file into a Microsoft Word format document. Additional figures were drawn where the PDF versions were either difficult to interpret or known to have errors. The PDF output from Word was generated by CutePDF Writer.

Since this is a cut and paste from the PDF, there are likely errors in some of the text. It was found during the cut and paste operation that while the PDF document text seemed correct, the cut operation would sometimes mix text up in a way that word groups from a particular sentence would be transposed to different locations in the cut text. If any users of this manual find errors in the text, I would appreciate a message on the Yahoo TS2068 group detailing the problem and I will repair the Word Document.

The detailed description of the sound chip and the Z80 processor have been deleted from this version, but files containing the data sheets are included in this document's ZIP files.

Much of the source code in the original manual has not been reproduced in this document, but a full listing of the Home and EXROM banks is appended. The source should be usable with any standard Z80 assembler with a few changes for the assembler pseudo-ops.

Timex Sinclair 2068 Technical Manual (2nd Edition), Copyright 1986 by the Time Designs Magazine Company. Reproduction of this document in whole or in part by any means without expressed written permission from Time Designs, is prohibited by law. This manual was printed by Toad'l Litho Printing and Composition, Oregon City, OR 97045.

Introduction.....	6
1.1 TS2068 Overview	6
1.1.1 Hardware Overview	6
.....	7
.....	7
1.1.2 System Software Overview.....	8
1.1.3 Cartridge Software Overview	11
2 Hardware Guide.....	12
2.1 Description of Major Hardware Functions	12
2.1.1 AC Adapter	13
2.1.2 Voltage Regulation	13
2.1.3 Z-80A CPU	13
2.1.4 ROM	14
2.1.5 32k RAM (Address 8000-FFFFH).....	14
2.1.6 Sound Generator	14
2.1.7 Joystick Port Operation.....	15
2.1.8 Control Logic (SCLD)	18
2.1.9 Keyboard.....	25
2.1.10 16K Video Display RAM	27
2.1.11 Video Generation	32
2.1.12 Cassette Tape I/O	34
2.1.13 Port Map.....	34
2.2 Schematic Diagram	37
2.3 Unit Absolute Ratings.....	38
2.4 Interfaces and Connectors.....	38
2.4.1 System Bus Connector - P1	38
2.4.2 Cartridge Connector.....	42
2.4.3 Cassette I/O	45
2.4.4 Joystick I/O	45
2.4.5 AC Adapter Power Plug.....	46
2.4.6 Composite Monitor Output	47
2.4.7 RF Output.....	47
2.4.8 Keyboard Interface.....	47
3 System Software Guide.....	49
3.1 Identifier.....	49
3.2 ROM Organization and Services	49
3.2.1 Home ROM Organization and Services	49
3.2.2 Extension ROM	50
3.3 RAM Organization and Services	54
3.3.1 System Variables	54
3.3.2 System Configuration Table	54
3.3.3 Machine Stack.....	54
3.3.4 OS Ram Routines.....	54
4 System I/O Guide.....	64

4.1	I/O Channels	64
4.1.1	Keyboard.....	66
4.1.2	Video Screen.....	68
4.1.3	2040 Dot Matrix Printer.....	71
4.2	Cassette Tape	71
4.3	Joysticks.....	74
4.4	Software Generated Sound (BEEP)	74
4.5	Sound Chip.....	75
5	Advanced Concepts	75
5.1	Cartridge Software/Hardware	75
5.1.1	LROS	75
5.1.2	AROS.....	77
5.1.3	EPROM Cartridge Board Application	80
5.2	Advanced Video Modes.....	82
5.2.1	Dual Screen Mode.....	84
5.2.2	High Resolution Graphics Mode	85
5.2.3	64 Column Mode	88
5.2.4	Other	89
5.3	Other Advanced Concepts	89
5.3.1	Interruption Fielding	89
5.3.2	BASIC AROS Variables.....	89
6	Known "Bugs" and corrections.....	90
6.1	LROS and Autostart Machine Code AROS.....	90
6.1.1	Keyboard.....	90
6.1.2	Calculator.....	90
6.1.3	Chunk 3.....	90
6.1.4	AROS under LROS.....	90
6.2	Machine Code AROS	90
6.3	BASIC AROS	91
6.3.1	USR Function.....	91
6.3.2	FOR/NEXT	91
6.3.3	Advanced Video Modes.....	91
6.4	Video Mode Change Service	91
6.4.1	Available Memory Test	91
6.4.2	RAMTOP.....	91
6.4.3	New Command	92
6.4.4	VIDMOD	92
6.4.5	Interrupt Inhibit.....	93
6.5	OS RAM ROUTINES.....	93
6.5.1	Function Dispatcher	93
6.5.2	Get Status	93
6.5.3	Put Word	93
6.5.4	Bank Enable and Restore Status	93
6.5.5	Save Status and Restore Status	94
6.5.6	Call Bank	94
6.5.7	GET_NUMBER.....	97

6.5.8	XFER_BYTES.....	97
6.6	General.....	97
6.6.1	Enter Key Anomalies.....	97
6.6.2	ON_ERR_GOTO.....	97
6.6.3	Sound Command Parameters.....	98
6.6.4	Scroll? User Responses.....	98
6.6.5	Delete Key Anomalies.....	98
7	Home ROM Source Code.....	105
8	EXROM Source Code.....	314
9	Assembler Include File.....	387

Introduction

This manual provides detailed technical information on the Timex Sinclair 2068 Personal Color Computer. In conjunction with the TS2068 User Manual, it is intended to assist the reader in understanding the architecture, hardware and software features, programming techniques and I/O techniques pertaining to the TS2068.

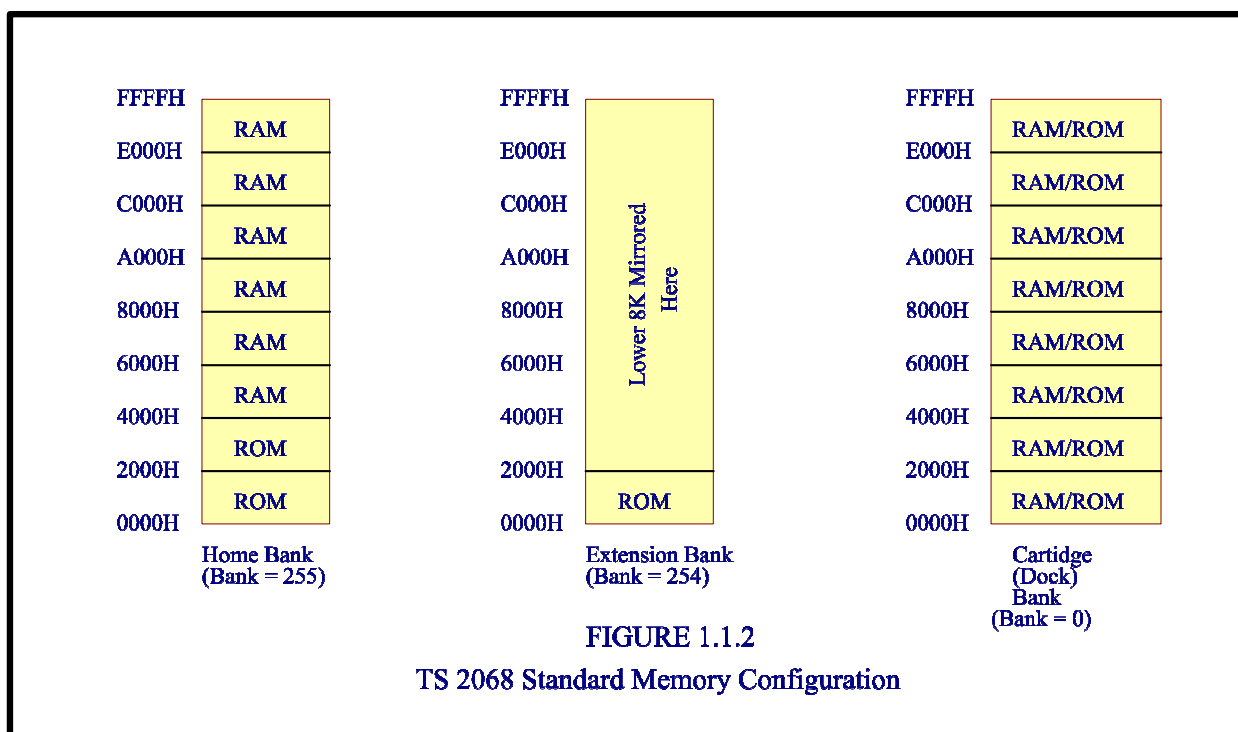
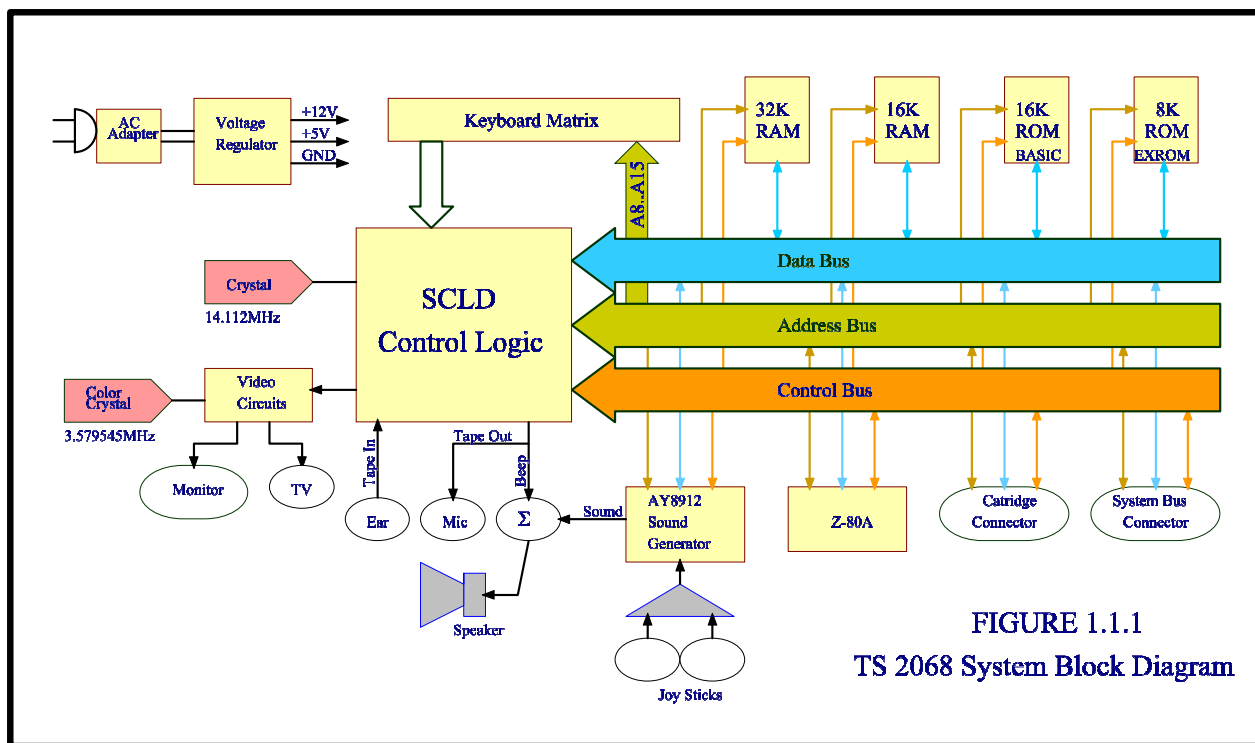
1.1 TS2068 Overview

1.1.1 Hardware Overview

Figure 1.1-1 is a block diagram of major functional components connections. These components are:

- Control Logic - SCLD (Standard Cell
- CPU - Z80A Microprocessor
- the TS2068 showing the
- and their logical
- Logic Device)
- RAM- 48K Random Access Memory
- ROM- 24K System Read-Only Memory (16K plus 8K Extension)
- System Bus Connector
- Cartridge Connector
- Sound Generator/Speaker
- Video Circuits
- Cassette READ/WRITE
- Joystick Connectors

The TS2068 Cartridge Connector provides for the plug-in of cartridges containing programmed ROM's with up to 64K of addressable memory. The full 64K is not normally utilized (e.g., due to need for access to RAM for the machine stack). See Section 5.1 for details. Figure 1.1-2 shows the standard TS2068 memory configuration comprised of the Home Bank, the ROM Extension Bank and the Dock (Cartridge) Bank. This memory is selectable as eight 8K 'chunks' with the Home Bank being enabled by default, i.e., any chunk not selected in the Extension or Dock Bank is automatically enabled in the Home Bank. Memory selection and I/O are controlled via the I/O ports. These topics are covered in detail in later sections.



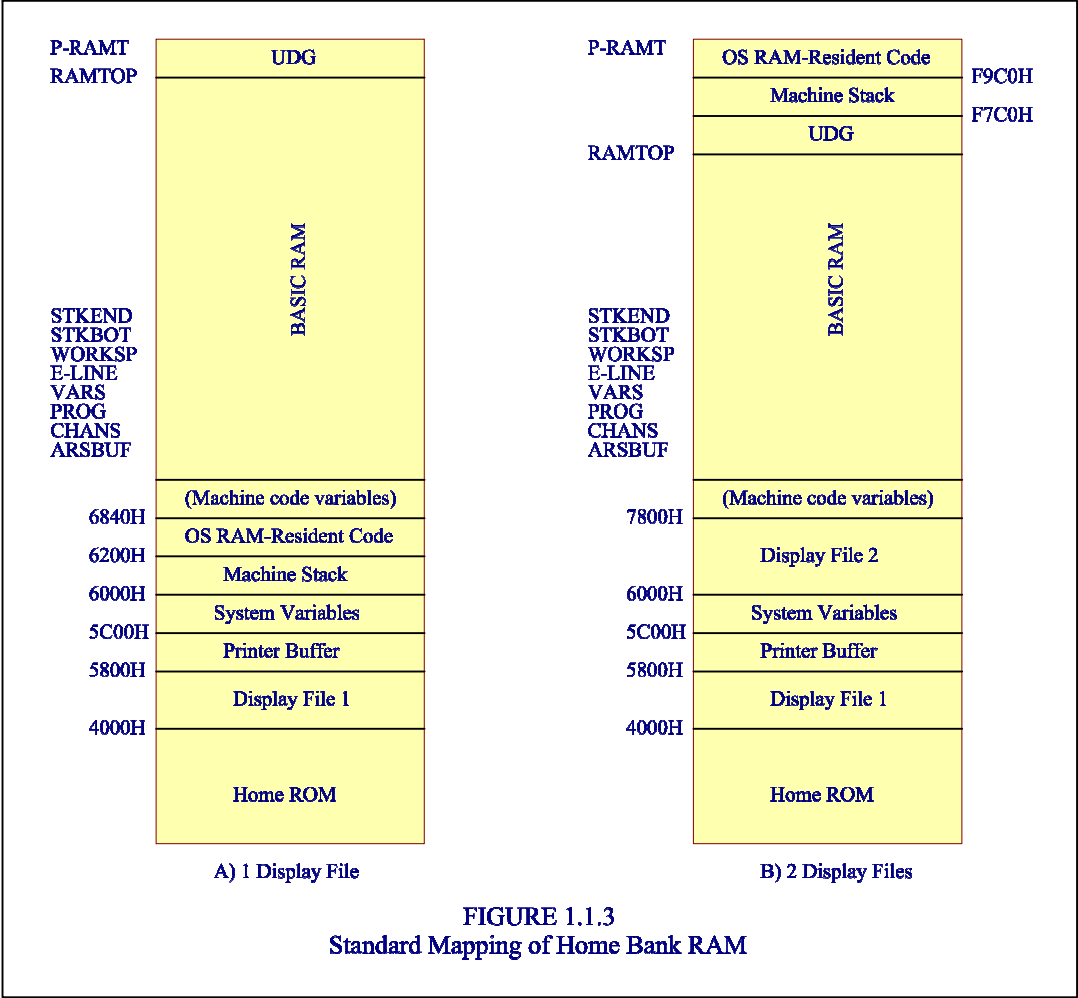
1.1.2 System Software Overview

The TS2068 System Software resides in the Home ROM, the Extension ROM, and dedicated RAM. It supports the following functions:

- System Initialization
- BASIC Interpreter (including BASIC cartridge support)
- BASIC I/O for Standard Peripherals
 - keyboard
 - video screen
 - 2040 32-col. dot matrix printer
 - cassette tape
 - joysticks
 - software generated sound (BEEP)
 - programmable sound chip (SOUND)
- Video Mode Change Service
- Interruption Servicing (Z80 Int. Mode 1)
- Bank Switching/Data Transfer Services
- Function Dispatcher (provides access to selected system routines via a Service Code input)

In addition, portions of the Home Bank RAM are used for the machine stack, the BASIC system variables, the Printer Buffer and the Display Files. Figure 1.1-3 shows the standard mapping of the Home Bank RAM and the mapping necessary when the second display file is to be used with the BASIC interpreter still functional. The Video Mode Change Service routine makes these memory modifications.

Note: There is no direct support of the second display file via BASIC or in the system ROM I/O routines. Figure 1.1-4 is a Flowchart of the System Initialization process.



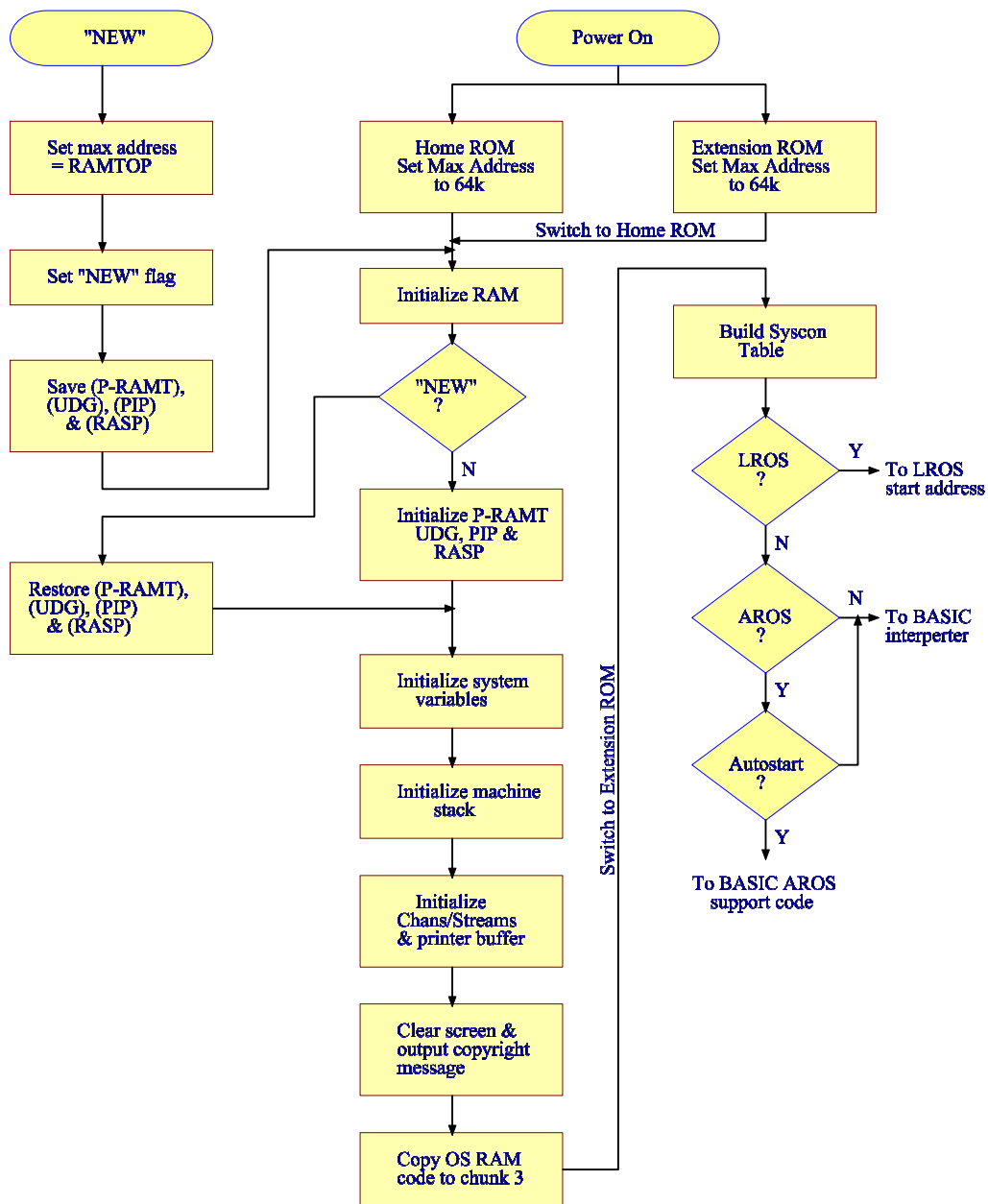


FIGURE 1.4.4
System Initialization

1.1.3 Cartridge Software Overview

The TS2068 supports two basic types of Cartridge or ROM-Oriented Software designated as LROS (Language ROM-Oriented Software) and AROS (Application ROM-Oriented Software) which plug into the cartridge connector. They are identified via overhead bytes at Location 0 for an LROS or 32768 (8000H) for an AROS. The fundamental difference is that an LROS contains 280 machine code in memory chunk 0 and is in total control of the TS2068 hardware including the RESTART implementation and Interruption Mode setting and handling, while an AROS is dependent on the System ROM or an LROS for these functions if needed. An AROS written in BASIC, which may also include machine code accessed via the USR function, is supported from the System ROM BASIC Interpreter and is mapped beginning in memory chunk 4. An AROS may also be written entirely in Z80 machine code. An AROS written in any other high-level language would require an LROS supporting that language and would have to be integrated with the LROS in a single cartridge. See Sections 3.2.1.2, BASIC AROS Support and 5.1, Cartridge Software/Hardware, for additional details.

2 Hardware Guide

2.1 Description of Major Hardware Functions

Figure 1.1-1 shows a simplified block diagram of the TS2068. The following functional units are described in the following sections:

Section	Functional Unit
2.1.1	AC Adapter
2.1.2	Voltage Regulation
2.1.3	Z-80A CPU
2.1.3.1	Address Bus
2.1.3.2	Data Bus
2.1.3.3	Control Signals
2.1.3.4	Opcode Fetch
2.1.3.5	Memory Read/Write
2.1.3.6	I/O Read/Write
2.1.3.7	Maskable interrupt
2.1.3.8	Non-maskable interrupt (NMI)
2.1.4	ROM
2.1.5	32K RAM
2.1.6	Sound Generator
2.1.7	Joystick ports
2.1.8	Control Logic
2.1.8.1	Bank Selection Logic
2.1.8.2	Z80 Clock Generator
2.1.8.3	Display file Access
2.1.8.4	Interrupt Generation
2.1.9	Keyboard
2.1.10	16K Video Display RAM
2.1.11	Video generation
2.1.11.1	Composite Video
2.1.11.2	RF Modulator
2.1.12	Cassette I/O
2.1.13	Port Map

2.1.1 AC Adapter

The AC Adapter transforms 117V AC (Nominal) to filtered DC via a step down transformer, full-wave bridge rectifier, and filter capacitor to supply from 14 to 25 volts at 1 amp over the AC voltage variation range of 105 to 130 V AC. Transformer isolation exceeds 1500 volts.

2.1.2 Voltage Regulation

Unregulated DC from the AC Adapter is supplied for regulation through a bi-filar torroidal inductor which reduces conducted line emanation for FCC compliance and through the power-ON/OFF switch located on the left side of the TS2068. This switch voltage is supplied to the System Bus Connector (see Section 2.4) and for regulation to the +12 V regulator and the +5 V regulator.

The 12V regulator is a 78L12 series 5V regulator is a switching supply circuit.

Characteristics are as follows:

SUPPLY	VOLTAGE RANGE	CURRENT RANGE
5V	4.75 - 5.25V	200ma - 1.0A
12V	11.5 - 12.5V	20ma - 100ma

2.1.3 Z-80A CPU

See the attached Z80 data sheet PDF files for detailed information about the CPU.

2.1.3.1 Z-80A Clock

The Z80 clock frequency is 3.528MHz derived in the SCLD by dividing the 14.112MHz master clock by four.

2.1.3.2 Z-80A Interrupts

2.1.3.2.1 Non-maskable interrupt

The non-maskable interrupt is not used in the TS2068. The input to the Z80 is available on the system expansion connector.

2.1.3.2.2 Maskable interrupt

The TS2068 sets the Z80 for MODE1 interrupts. This mode causes a subroutine call to location 0038H in the Z80 address space. If chunk 0 is switched to the dock bank, the code must either disable interrupts or provide an interrupt handler at this address.

2.1.4 ROM

The system includes both a 16K byte ROM and an 8K byte ROM mapped into the address space as follows:

U16 is a 16kX8 23128 mask programmed ROM and is mapped into the HOME bank at locations 0000H..3FFFH.

U20 is a 8kX8 2364 mask programmed ROM and is mapped into the EXTENSION bank at 0000H..1FFFH.

Section 2.1.8.1 describes the selection of the Home Bank and Expansion Bank via the control logic. The devices involved are a 23128 and a 2364 for the 16K byte (128K-bit) and the 8K byte (64K-bit) ROM's respectively. Direct replacement of these devices with 27128 and 2764 EPROM's is not possible since pins 1 and 27 must be maintained in the high state for those devices.

2.1.5 32k RAM (Address 8000-FFFFH)

The upper 32K of RAM is composed of four 200ns 4416's (16K x 4 dynamic RAMs).

2.1.6 Sound Generator

The Programmable Sound Generator (GI 8912) is accessed via Ports 0F5H (Address) and 0F6H (Data). The basic registers in the PSG which produce the programmed sounds include:

- Tone Generators: Produce the basic square wave tone frequencies for each channel (A, B, C).
- Noise Generator: Produces a frequency modulated pseudo-random pulse width square wave output.
- Mixers: Combine the outputs of the Tone Generators and the Noise Generator. One for each channel (A, B, C).
- Amplitude Control: Provides the D/A Converters with either a fixed or variable amplitude pattern. The fixed amplitude is under direct CPU control; the variable amplitude is accomplished by using the output of the Envelope Generator.
- Envelope Generator: Produces an envelope pattern which can be used to amplitude modulate the output of each Mixer.
- D/A Converters: The three D/A Converters each produce up to a 16-level output signal as determined by the Amplitude Control.
- An additional register is shown in the PSG Block Diagram (Figure 2.1.6-1) which has nothing directly to do with the production of sound -- this is the I/O Port (A). Data to/from the CPU may be read/written to/from the 8-bit I/O Port without affecting any other function of the PSG. **The TS 2068 uses the I/O Port to access the joysticks.**

The PSG clock is 1.76475 which is generated in the SCLD by dividing the 14.112MHz master frequency by 8.

For a detailed description of the PSG theory of operation, see the included data sheet PDF. The PDF was downloaded from <<http://www.s100computers.com>>.

2.1.6.1 I/O Port Data Store (Register R14)

Register R14 functions as an intermediate data storage register between the PSG/CPU data bus (DA7-DA0) and the I/O Port (IOA7-IOAO). This port is available for reading the joysticks. Using register R14 for the transfer of I/O data has no effect at all on sound generation.

To output data from the CPU bus to a peripheral device connected to I/O Port A would require the following steps:

1. Latch address R7 (select Enable register)
2. Write data to PSG (setting R7, B6=1)
3. Latch address R14 (select IOA register)
4. Write data to PSG (data to be output on I/O Port A)

To input data from I/O Port A to the CPU bus would require the following:

1. Latch address R7 (select Enable register)
2. Write data to PSG (setting R7 B6=0)
3. Latch address R14 (select IOA register)
4. Read data from PSG (data from I/O Port A)

Note: once loaded with data in the output mode, the data will remain on the I/O port until changed either by loading different data, by applying a reset (grounding the Reset pin), or by switching to the input mode.

Note: When in the input mode, the contents of register R14 will follow the signals applied to the I/O port, However, transfer of this data to the CPU bus requires a "read" operation as described above.

2.1.7 Joystick Port Operation

The joystick port (Register 14 of the Sound Chip - Section 2.1.6.1) is read via an IN-instruction directed at port F6H with selection of activating data from the left (player 1) or right (player 2) determined by Address bits 8 and 9 as shown in Figure 2.1.7-1. In order to address Register 14, a 0EH must be written to port F5H (Sound Generator Address) prior to reading joystick data. Section 4.4 describes the software sequence necessary to control this hardware.

In the example of Figure 2.1.7-1, the joystick, shown schematically in the lower left of the drawing, is composed of a movable center stick which is pushed up to touch the up-contact and, therefore, electronically connects pin-8 to pin-1. In this state, a read of port F6H with address bit A8 high, causes actions as follows:

- (1) Address A8 high turns on transistor Q8
- (2) Q8 drives cable pin-8 low
- (3) The movable center stick of the joystick in contact with the up-contact results in a conductive path from cable pin-8 to cable pin-1.
- (4) Pin-1 low results in a 0 in bit position 0 of the I/O register via the isolation diode.

The various positions of the stick similarly result in various bits being read from the I/O register.

Note: +5 volts and ground are available on the connector so +5V logic could be attached to the joystick port.

The above note is incorrect: the errata from the original technical manual states that the ground pins are not connected on the TS2068 circuit board. Therefore logic chips cannot be used on the joystick ports.

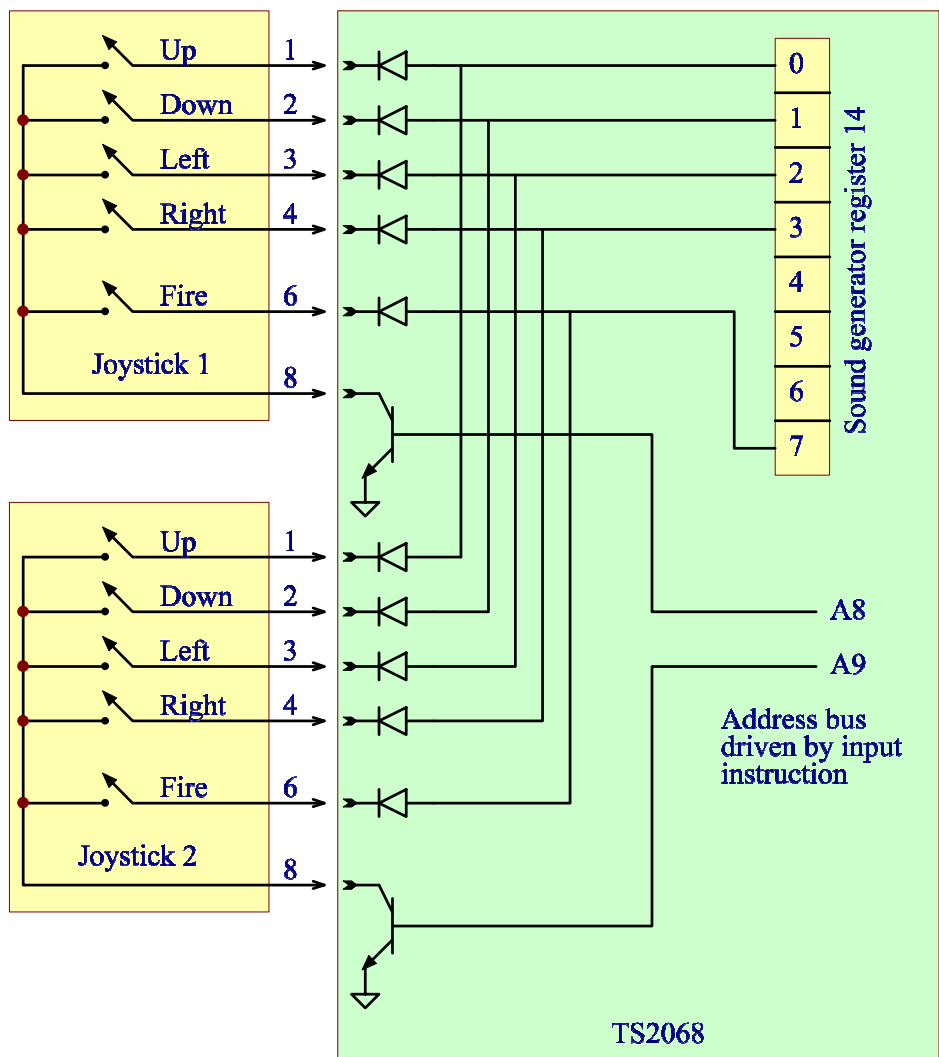


FIGURE 2.1.1-1
Joystick Port Operation

2.1.8 Control Logic (SCLD)

The control logic of the TS2068 is primarily a Standard Cell Logic Device in a 68-pin JEDEC leaded carrier package and includes the following major functions:

SECTION	FUNCTION
2.1.8.1	Bank Selection Logic
2.1.8.2	Z-80 Clock Generation
2.1.8.3	Display Timing, DMA Display File Access, Attribute Control, and Pixel Data Serial Shift
2.1.8.4	Interruption Generation

BEEP Output (See Section 2.1.13.2)

CASSETTE I/O (See Section 2.1.12).

Additionally, Table 2.1.8-1 provides a description of the function of each SCLD I/O pin. See the System Schematic in Appendix D for pin numbering.

2.1.8.1 Bank Selection Logic

The TS2068 is a Z-80 based computer, therefore it can directly address only 64K bytes of memory via its 16-bit address. Additionally, since the Z-80 has no relocation or indirection capability, the conventional technique of extending the memory space available to the Z-80 is bank switching. The TS2068 provides extended bank switching by allowing selection of memory in 8K "chunks" which are identified by bank number and chunk number as illustrated in Figure 2.1.8-1 for the internal bank selection logic. The externally sourced *BE (Bank Enable) signal can be used by external logic to disable the internally controlled memories.

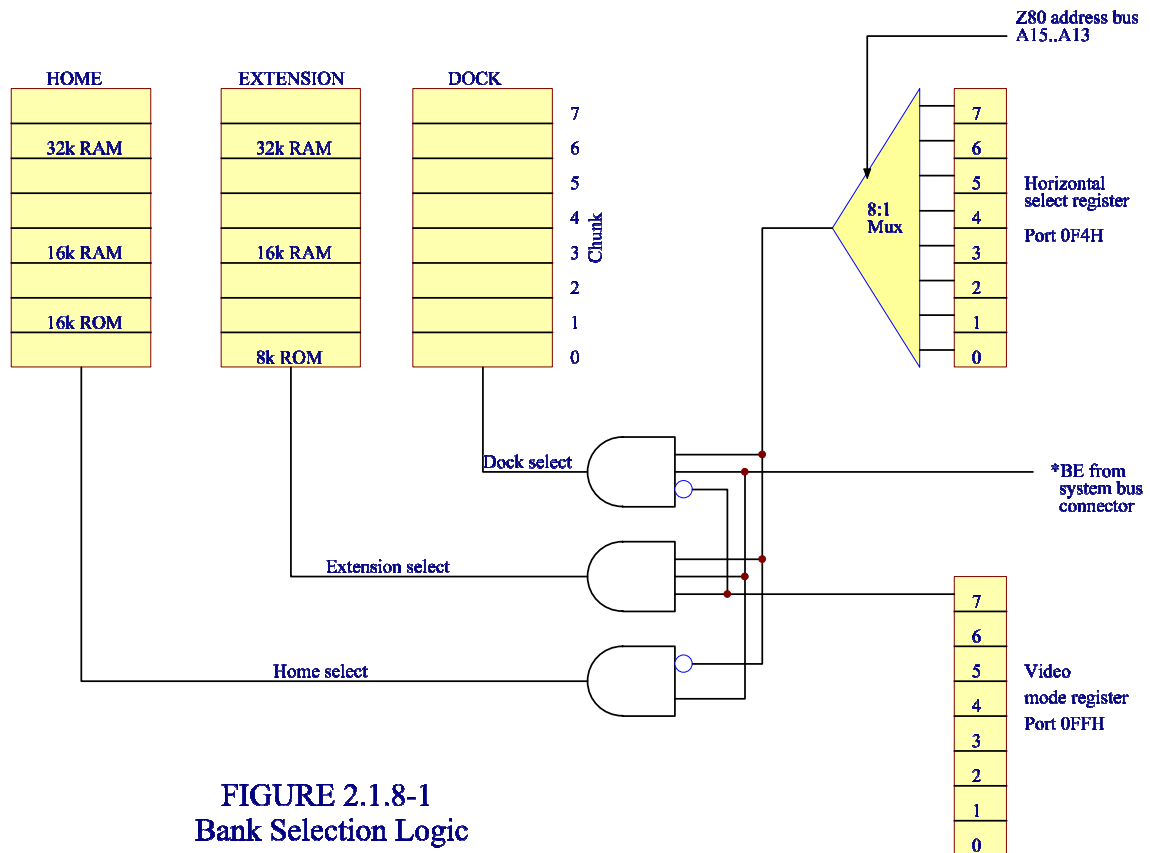


FIGURE 2.1.8-1
Bank Selection Logic

Table 2.1.8-1 SCLD I/O Pin Function Definitions Note a leading asterisk (*) indicates active low			
Symbol	Name	Pin direction	Function
A0-A7 A13-A15	Address bus	in	Address lines from Z80
D0-D7	Data bus	in/out	Data Bus inputs/outputs from/to Z80A through U9-74LS245 or inputs from display RAM (16K) - U6 and U7
KB0-KB4	Keyboard Outputs	in	Inputs from 5 lines of keyboard matrix - goes low at one of 8 address line (active low) sequences on I/O Request
A7R	A7+Refresh	out	To refresh and address 8th bit address line input of RAM memory (not display) of 32K of 4416 RAM's (Home Bank 8000H to FFFFH)
MA0-MA7	Muxed Adrs.Bus	out	Display memory muxed address bus and refresh
*TS	Tri-State Display Memory Ctl.	out	Tri-State control for address and data buffers when CPU is addressing display memory at same time display controller is addressing the display memory
φCPU	Clock to CPU	out	CLK - Clock to Z80A CPU which is interrupted to stop CPU when CPU wants to address display RAM at same time as display controller
*RD	Read Direction Control to SCLD	out	To control read/write direction of 74LS245 Data Bus Buffer between CPU and SCLD
*ROMCS	Home ROM Chip Select	out	To activate the 16K Home ROM (first 16K) when memory selection (MS) is set to Home Bank
*RAS1	Row Address Strobe #1	out	To activate row address strobe for display memory only during emory read/write, refresh and display read
*CAS1	Column Address Strobe #1	out	To activate column address strobe for display memory only (2nd 16K) during memory read/write and display read
*CAS2	Column Address Strobe #2	out	To activate column address strobe for Home Bank RAM (3rd 16K).

Table 2.1.8-1 SCLD I/O Pin Function Definitions Note a leading asterisk (*) indicates active low			
Symbol	Name	Pin direction	Function
*CAS3	Column Address Strobe #2	out	To activate column address strobe for display memory only (2nd 16K) during memory read/write and display read
*DRAMWE	Dynamic RAM Write Enable	out	When active low, enables a write into the display RAM only
MUX	MUX Control of RAM Address	out	MUX control to 74LS157 (UIO & UII) to multiplex the row and column addresses to all dynamic RAM's
V	Chroma Vector V	out	Color vector level for quadrature (R-Y) input to video modulator
Y	Luminance Y	out	Luminance (brightness) control level
*RD	Read from CPU	in	CPU is reading from a memory or I/O location
*WR	Write from CPU	in	CPU is writing to a memory or I/O location
*MREQ	Memory Request	in	CPU is requesting access to a memory location to read or write
*IORQ	I/O Request	in	CPU is requesting access to an I/O location to read or write
*RFSH	Refresh	in	CPU is generating a refresh address to refresh dynamic RAM's
Tape In	Tape Input	in	Magnetic tape signal input
*BE	Bank Enable	in	When active low, indicates that internal memory ' disabled (Home, Extension and SDock Banks) and an external memory is in use
*EXROM	Extension ROM Select	out	Active low chip select signal for Extension ROM
VCC	+5 Volt Power	in	Power (+5V) input to SCLD
*INT	Interrupt to CPU	out	Interrupts CPU to handle keyboard strobing and timer for PAUSE command. Open drain N channel with internal pull-up
*ROSCS	ROS Chip Select	out	ROM-Oriented Software (Cartridge Bank) Chip Select
Spkr/Tape out	Speaker and Tape Output	out	Digital output to magnetic tape and to sound amplifier for speaker output
φC	Clock "C"	out	Clock for sound chip 1.764 MHz.

Table 2.1.8-1 SCLD I/O Pin Function Definitions Note a leading asterisk (*) indicates active low			
Symbol	Name	Pin direction	Function
BDIR	Bus Direction to Sound Chip	out	A bus direction control signal to the PSG. When high the sound chip either receives a write to PSG or latches addresses from the data bus
BC1	Bus Control to Sound Chip	out	A bus control signal to the PSG. When high the sound chip either is read to data bus or latches addresses from the data bus
Osc Out	Oscillator Out	out	Xtal Oscillator amplifier output to drive crystal
Osc In	Oscillator In	in	Xtal Oscillator amplifier input to sense crystal signal
U	Chroma Vector U	out	Color vector level for quadrature (B-Y) input to video modulator
GND	Ground	in	Ground return of SCLD
*φ	Buffered Clock	out	Buffered CPU clock to outside (J1 - connector)
R	Red Color Output	out	Produce color signals to RGB monitor (TTL level)
G	Green Color output	out	Produce color signals to RGB monitor (TTL level)
B	Blue Color output	out	Produce color signals to RGB monitor (TTL level)

2.1.8.2 Z80 Clock Generation

The oscillator circuit utilizes an AT-cut quartz crystal at 14.112 MHz. This oscillator feeds a divide by 4 chain to generate the 3.528 MHz clock for the CPU (0 CPU). This clock runs continuously except when the CPU addresses the 16K bytes of RAM containing the video display file at the same time the video display processor logic requires access to that same RAM. For this contention case the CPU clock is stopped in the high state until the video display processor access has been completed, then the CPU clock continues in its normal manner.

2.1.8.3 Display File Hardware Control and Timing

The 14.112 MHz oscillator is also used to drive the counter chain deriving video timing. By dividing the 14.112 MHz. signal by 896 a 15.75 KHz horizontal sweep frequency is generated. The 15.75 KHz signal feeds a g-stage counter which counts from 0 to 106H (262 decimal) developing the 60.1145 Hz vertical sync. See Figure 2.1.8-2.

During each horizontal scan the video display processor accesses, in the standard video mode, 32 bytes of pixel data plus 32 bytes of attributes by 32 memory accesses reading 2 bytes per access in RAM page mode, i.e. the low order address bits are provided to the RAM once via RAS activation, then the data byte is read during the first activation of CAS and the attribute byte is read during the second activation of CAS. The page mode operation is completed by deactivating RAS. (See Fig. 2.1.8-2.)

The accessed pixel data is serially shifted out to the video generation circuitry at a rate of 1 bit each 142 nanoseconds (7.056 MHz) resulting in the need to fetch a new data/attribute pair each 1.134 microseconds during the horizontal scan time. The shifted out pixel information is used to control the selection of the 3 paper color (pixel=0) or 3 ink color (pixel=1) bits to be gated out as the R, G, and B signals. When FLASH is enabled by the attribute byte, the INK and PAPER field information is swapped at the 1.879 Hz. flash rate. The R, G, and B signals control the D-to-A converter which generates the proper U, V, and T outputs for use by the 1889 to create composite video.

The address information provided to the RAM's during RAS and CAS times is as shown in Figure 2.1.8-2. This address generation logic explains the non-sequential nature of the video display as described in Section 2.1.10.

Display Pixel Data Address Range 4000H - 57FFh															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	R	Q	M	L	K	P	O	N	I	H	G	F	D
CAS1A								RAS							

Display Attribute Data Address Range 5800H - 5AFFh															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	0	R	Q	P	O	N	I	H	G	F	D
CAS1A								RAS							

Video Timing Counter Chain															
60Hz Sync	MSB Line		LSB Line			Y Pixel (8 bit group)				Column D is clocked by 1.764MHz					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S	R	Q	P	O	N	M	L	K	J	I	H	G	F	E	D
Divide by 262									Divide by 7			Divide by 16			

Notes for the video timing chain:

- The counter is fed by the 1.764MHz clock at D
- Counter stage J generates the 15,750Hz horizontal sync signal
- Counter state S generates the 60Hz vertical sync signal.

2.1.8.4 Interruption Generation (17ms)

During the vertical blanking interval (once each 15.635ms) the SCLD, if enabled by the INTEN bit (Bit 6) of I/O Port FFH, activates the *INT signal which directly connects to the *INT input to the Z80. A CPU maskable interruption can then occur, as described in Section 2.1.3.7, if enabled.

2.1.9 Keyboard

The keyboard for the TS 2068 has forty-two (42) hard keys (typewriter style) with tactile feel utilizing an over-dead-center type of rubber spring pad and a carbon pill that hits the P.C. board, just under the keyboard, to short-out a pair of closely placed precious metal contacts. The read-out matrix is an eight by five cross point switching as shown in Figure 2.1.9-1.

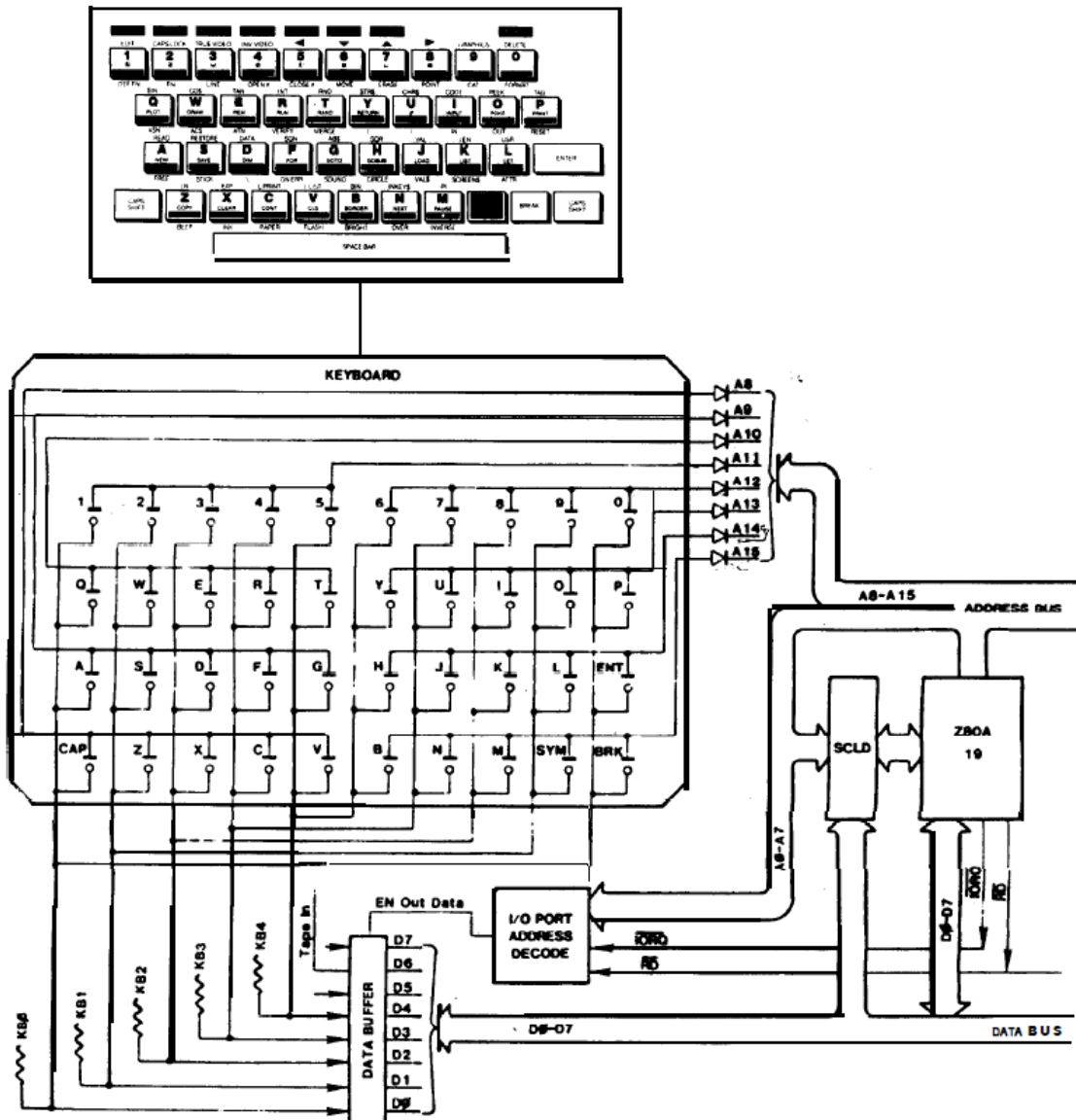
Each switch closure connects one of the eight high order address lines (by going low through a diode) to one of the five input lines to the SCLD (KBO through KB4).

Scanning is by software algorithm as described in Section 4.1.1. During the IN instruction, address bits A0-A7=FEH select the Keyboard I/O port while bits A8-A15 select the particular 5 keys to be sampled during the particular IN instruction execution. For example, an IN instruction directed at the keyboard I/O port with address bit A8 low and A9-A15 high will supply 0's on KB0, KB1, KB2, KB3, and/or KB4 if the CAP SHIFT, Z, X, C, and/or V keys are respectively depressed.

Note: when reading the I/O port FEH, data bits D5-D7 are not part of the keyboard information.

Section 2.4.7 details the connection of the keyboard to the main P.C. board.

Figure 2.1 .9. KEYBOARD SCHEMATIC



2.1.10 16K Video Display RAM

The 16K-byte video display RAM, composed of two 4416's, is isolated from the Z80A CPU by the SCLD control logic and buffers to allow the video display processor to access pixel and attribute data from the display files independent of the CPU (see Section 2.1.8.3).

The Video Display RAM is located in Chunks 2 and 3 of the Home Bank, beginning at 400DH and 600DH respectively. Figure 2.1.10-1 illustrates the organization of the Primary Display File located at 4000H. The second display file utilizes the same organization. Based on the video mode set via Port FFH, the video hardware accesses the RAM for pixel data and attribute control information.

FIGURE 1.2.10-1
Display File Organization (Normal Mode)

Block 0 Character bit map								
Scan Line	32 byte Line 0		32 byte Line 1		32 byte Line 2		32 byte Line 3	
0	4000	401F	4020	403F	4040	405F	4060	407F
1	4100	411F	4120	413F	4140	415F	4160	417F
2	4200	421F	4220	423F	4240	425F	4260	427F
3	4300	431F	4320	433F	4340	435F	4360	437F
4	4400	441F	4420	443F	4440	445F	4460	447F
5	4500	451F	4520	453F	4540	455F	4560	457F
6	4600	461F	4620	463F	4640	465F	4660	467F
7	4700	471F	4720	473F	4740	475F	4760	477F
	Char Pos. 0/0	Char Pos. 0/31	Char Pos. 1/0	Char Pos. 1/31	Char Pos. 2/0	Char Pos. 2/31	Char Pos. 3/0	Char Pos. 3/31

Block 0 Character bit map								
Scan Line	32 byte Line 4		32 byte Line 5		32 byte Line 6		32 byte Line 7	
0	4080	409F	40A0	40BF	40C0	40DF	40E0	40FF
1	4180	419F	41A0	41BF	41C0	41DF	41E0	41FF
2	4280	429F	42A0	42BF	42C0	42DF	42E0	42FF
3	4380	439F	43A0	43BF	43C0	43DF	43E0	43FF
4	4480	449F	44A0	44BF	44C0	44DF	44E0	44FF
5	4580	459F	45A0	45BF	45C0	45DF	45E0	45FF
6	4680	469F	46A0	46BF	46C0	46DF	46E0	46FF
7	4780	479F	47A0	47BF	47C0	47DF	47E0	47FF
	Char Pos. 4/0	Char Pos. 4/31	Char Pos. 5/0	Char Pos. 5/31	Char Pos. 6/0	Char Pos. 6/31	Char Pos. 7/0	Char Pos. 7/31

FIGURE 1.2.10-1
Display File Organization (Normal Mode)
(Continued)

Block 1 Character bit map								
Scan Line	32 byte Line 8		32 byte Line 9		32 byte Line 10		32 byte Line 11	
0	4800	481F	4820	483F	4840	485F	4860	487F
1	4900	491F	4920	493F	4940	495F	4960	497F
2	4A00	4A1F	4A20	4A3F	4A40	4A5F	4A60	4A7F
3	4B00	4B1F	4B20	4B3F	4B40	4B5F	4B60	4B7F
4	4C00	4C1F	4C20	4C3F	4C40	4C5F	4C60	4C7F
5	4D00	4D1F	4D20	4D3F	4D40	4D5F	4D60	4D7F
6	4E00	4E1F	4E20	4E3F	4E40	4E5F	4E60	4E7F
7	4F00	4F1F	4F20	4F3F	4F40	4F5F	4F60	4F7F
	Char Pos. 8/0	Char Pos. 8/31	Char Pos. 9/0	Char Pos. 9/31	Char Pos. 10/0	Char Pos. 10/31	Char Pos. 11/0	Char Pos. 11/31

Block 1 Character bit map								
Scan Line	32 byte Line 12		32 byte Line 13		32 byte Line 14		32 byte Line 15	
0	4880	489F	48A0	48BF	48C0	48DF	48E0	48FF
1	4980	499F	49A0	49BF	49C0	49DF	49E0	49FF
2	4A80	4A9F	4AA0	4ABF	4AC0	4ADF	4AE0	4AFF
3	4B80	4B9F	4BA0	4BBF	4BC0	4BDF	4BE0	4BFF
4	4C80	4C9F	4CA0	4CBF	4CC0	4CDF	4CE0	4CFF
5	4D80	4D9F	4DA0	4DBF	4DC0	4DDF	4DE0	4DFF
6	4E80	4E9F	4EA0	4EBF	4EC0	4EDF	4EE0	4EFF
7	4F80	4F9F	4FA0	4FBF	4FC0	4FDF	4FE0	4FFF
	Char Pos. 12/0	Char Pos. 12/31	Char Pos. 13/0	Char Pos. 13/31	Char Pos. 14/0	Char Pos. 14/31	Char Pos. 15/0	Char Pos. 15/31

FIGURE 1.2.10-1
Display File Organization (Normal Mode)
(Continued)

Block 2 Character bit map								
Scan Line	32 byte Line 16		32 byte Line 17		32 byte Line 18		32 byte Line 19	
0	5000	501F	5020	503F	5040	505F	5060	507F
1	5100	511F	5120	513F	5140	515F	5160	517F
2	5200	521F	5220	523F	5240	525F	5260	527F
3	5300	531F	5320	533F	5340	535F	5360	537F
4	5400	541F	5420	543F	5440	545F	5460	547F
5	5500	551F	5520	553F	5540	555F	5560	557F
6	5600	561F	5620	563F	5640	565F	5660	567F
7	5700	571F	5720	573F	5740	575F	5760	577F
	Char Pos. 16/0	Char Pos. 16/31	Char Pos. 17/0	Char Pos. 17/31	Char Pos. 18/0	Char Pos. 18/31	Char Pos. 19/0	Char Pos. 19/31

Block 2 Character bit map								
Scan Line	32 byte Line 20		32 byte Line 21		32 byte Line 22		32 byte Line 23	
0	5080	509F	50A0	50BF	50C0	50DF	50E0	50FF
1	5180	519F	51A0	51BF	51C0	51DF	51E0	51FF
2	5280	529F	52A0	52BF	52C0	52DF	52E0	52FF
3	5380	539F	53A0	53BF	53C0	53DF	53E0	53FF
4	5480	549F	54A0	54BF	54C0	54DF	54E0	54FF
5	5580	559F	55A0	55BF	55C0	55DF	55E0	55FF
6	5680	569F	56A0	56BF	56C0	56DF	56E0	56FF
7	5780	579F	57A0	57BF	57C0	57DF	57E0	57FF
	Char Pos. 20/0	Char Pos. 20/31	Char Pos. 21/0	Char Pos. 21/31	Char Pos. 22/0	Char Pos. 22/31	Char Pos. 23/0	Char Pos. 23/31

Attribute Byte Addresses							
Line 0		Line 1		Line 2		Line 3	
Block 0	5800	581F	5820	583F	5840	585F	5860 587F
	Line 4		Line 5		Line 6		Line 7
	5880	589F	58A0	58BF	58C0	58DF	58E0 58FF
	Line 8		Line 9		Line 10		Line 11
Block 1	5900	591F	5920	593F	5940	595F	5960 597F
	Line 12		Line 13		Line 14		Line 15
	5980	599F	59A0	59BF	59C0	59DF	59E0 59FF
	Line 16		Line 17		Line 18		Line 19
Block 2	5A00	5A1F	5A20	5A3F	5A40	5A5F	5A60 5A7F
	Line 20		Line 21		Line 22		Line 23
	5A80	5A9F	5AA0	5ABF	5AC0	5ADF	5AE0 5AFF

2.1.11 Video Generation

2.1.11.1 Composite Video

The U, V, and Y signals from the SCLD are supplied to the LM1889 and associated circuitry to produce composite video and modulated RF. This circuitry produces color vectors at approximately the following angles:

PHASE	TS 2068 (Degrees)	NTSC STANDARD (Degrees)
Blue	350	350
Magenta	64	62
Red	116	112
Green	242	240
Cyan	284	284
Yellow	170	170
Reference	224	180

The Front Porch, Sync Pulse, Back Porch, and Color Burst portions of the composite video signal are illustrated in Figure 2.1.11-1. In proper adjustment the following should be observed:

Sync Pulse	= 40 +/- 2 IRE units
Color Burst	= 35 to 45 IRE units
Color Burst Freq.	= 3.579545 MHz. +/- 70 Hz

The following three facts may aid in understanding problems with certain monitors.

1. The color burst is not synchronous with the waveform since it is generated from the 3.579545 MHz crystal and the waveform is derived from the 14.112 MHz crystal. The result is observed ripples at color boundaries, e.g. green to magenta.
2. The color burst duration is 8 cycles while standard TV broadcast stations provide 9 cycles. This "short" burst is a problem for some monitors.
3. The color burst starts 6.4 microseconds from the leading edge of sync. Many monitors are designed to expect this start as early as 5.3 microseconds, thus these monitors may not produce color when attached to the TS 2068.

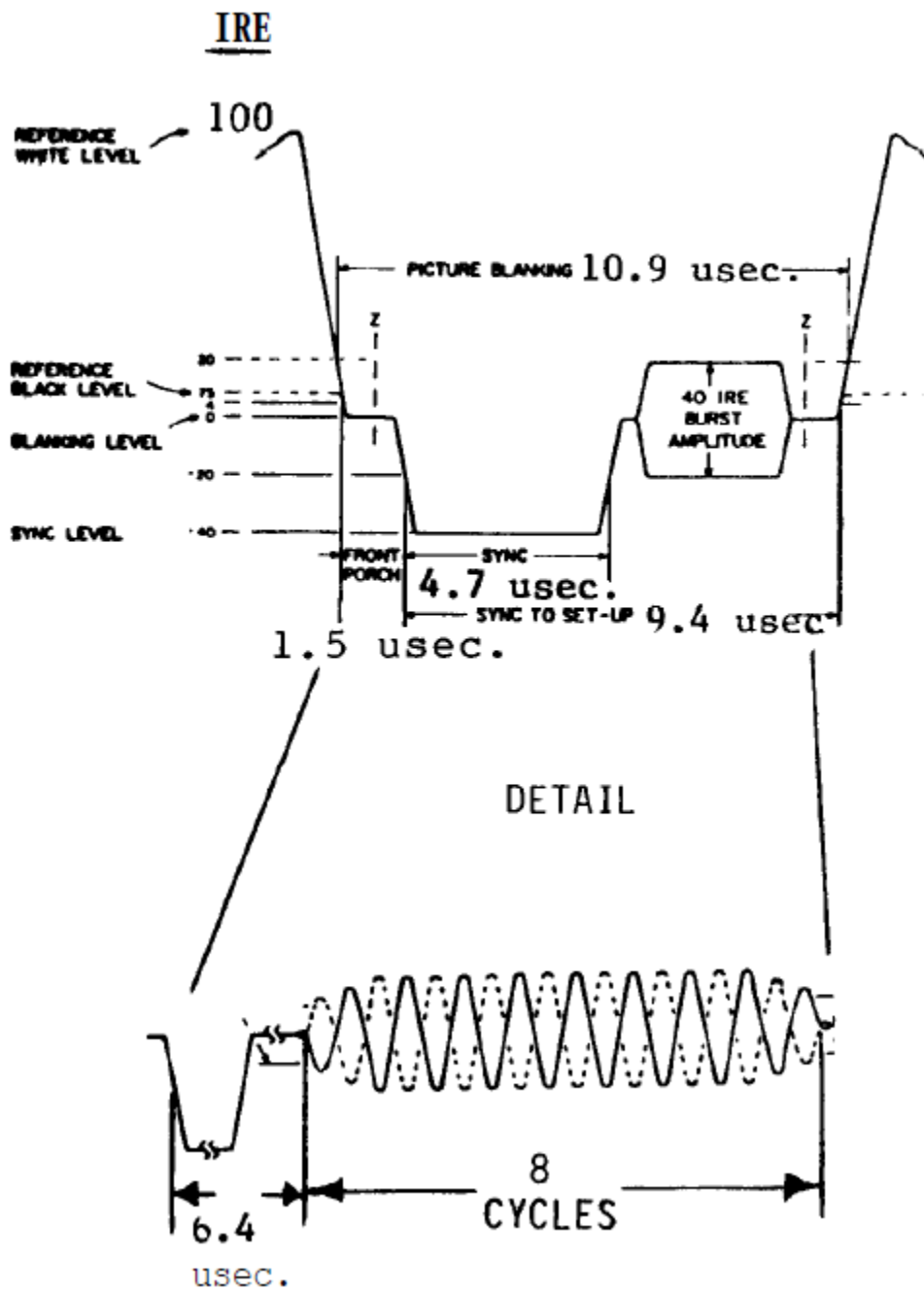


FIGURE 2.1.11-1
Composite Video Signals

2.1.11.2 Video Modulator

The composite video information is used to AM modulate the selected channel frequency via the LM1889 and associated Channel 2/3 tank circuitry. The modulated output is filtered through the output filter network to reduce harmonic generation to comply with FCC requirements. The RF circuitry is physically contained inside the RF-can at the rear left corner of the PCB (at the RF output jack). 75 ohms is the output impedance.

2.1.12 Cassette Tape I/O

See Sections 2.1.13.2, 2.4.3 and 4.2.

2.1.13 Port Map

Table 2.1.13-1 summarizes the I/O addressing of ports utilized by the TS 2068. Details of the data bits of each of these ports is provided by the following sections.

Function	Port Address		Operation	Reference
	Hex	Decimal		
Display Enhancement Control	FF	255	R/W	2.1.10, 2.1.13.1, 3.2.2.3, 5.2
Keyboard/Tape I/O	FE	254	R/W	2.1.9, 2.1.13.2, 2.4.3, 4.1.1, 4.2
Reserved	FD	253	R/W	Bank switching - not implemented
Reserved	FC	252	R/W	Bank switching - not implemented
TS 2040,Printer	FB	251	R/W	2.1.13.3, 4.1.3
Sound Chip & Joystick Data	F6	246	R/W	2.1.6, 2.1.7, 2.1.13.4 2.4.4, 4.3, 4.5
Sound Chip Address	F5	245	W	2.1.6, 2.1.7, 2.1.13.4 2.4.4, 4.3, 4.5
Horizontal Select Register	F4	244	R/W	2.1.8.1

2.1.13.1 Display Enhancement Control (Port FFH)

The display enhancement control register within the SCLD controls:

- a) Selection of Enhanced Video Modes
- b) Ink selection for 64-Column Mode
- c) Enable/Inhibit the 17 ms interruption to the Z80
- d) Selection of Extension ROM or Cartridge (see Section 2.1.8.1)

D7	D6	D5	D4	D3	D2	D1	D0
Exrom/Cartridge Select (See 2.1.8.1)	Inhibit 17ms interrupt (0 enables)	64 column Mode Ink/Paper Selection 000 - Black/White 001 - Blue/Yellow 010 - Red/Cyan 011 - Magenta/Green 100 - Green/Magenta 101 - Cyan/Red 110 - Yellow/Blue 111 - White/Black			Video Mode Selection 000 - Normal (primary display file) 001 - Second display file 010 - Hi res. graphics 110 - 64 column mode Other combinations may produce unpredictable results.		

2.1.13.2 Keyboard/Tape I/O (Port FEH)

I/O Read							
D7	D6	D5	D4	D3	D2	D1	D0
Not used	Tape Input	Not used	Keyboard Input (See 2.1.9)				

I/O Write							
D7	D6	D5	D4	D3	D2	D1	D0
Not used			Sound Out (Beep)	Tape output (See 4.2)	Border Color 000 - Black 001 - Blue 010 - Red 011 - Magenta 100 - Green 101 - Cyan 110 - Yellow 111 - White		

2.1.13.3 Printer I/O (Port 1XXXX0XX)

I/O Read							
D7	D6	D5	D4	D3	D2	D1	D0
Start of Paper	Printer not configured						Ready for next Pixel

I/O Write							
D7	D6	D5	D4	D3	D2	D1	D0
Pixel to print 0 = none 1 = black					Motor control 0 = on 1 = off	Motor select 0 - fast 1 - slow	Not used

2.1.13.4 Sound Chip and Joystick I/O (Ports F5 and F6)

Ports F5H and F6H are used to control and access the Sound Generator and the Joysticks. Details of the registers available via these ports is contained in Sections 2.1.6 and 2.1.7 and the AY8910 data sheet.

I/O Read							
D7	D6	D5	D4	D3	D2	D1	D0
Start of Paper	Printer not configured						Ready for next Pixel

2.1.13.5 Horizontal Select Register I/O (Port F4)

The HSR addressed via Port F4H is used in the control of the Bank Switching logic as detailed in Section 2.1.8. Each bit, when set, enables the corresponding 8K memory "chunk" in either the Dock Bank (Port FF, Bit 7=0) or the Extension ROM Bank (Port FF, Bit 7=1). The HSR must be set to all zeroes in order to enable the entire Home Bank.

2.2 Schematic Diagram

Under construction

2.3 Unit Absolute Ratings

FUNCTION	DESCRIPTION	MIN	MAX
T _s	Storage Temperature	-40c	+65C
VAC	AC Line Voltage	105v	130v
T _a	Operating Ambient Temperature	0C	40C
V _{fn}	Voltage on any Logic pin	-0.3v	+5.3v
V _{fn} (EAR)	EAR input Peak AC	-2.0v	+5.0v
V _{dc} (IN)	Input DC Voltage	14.75V	26V

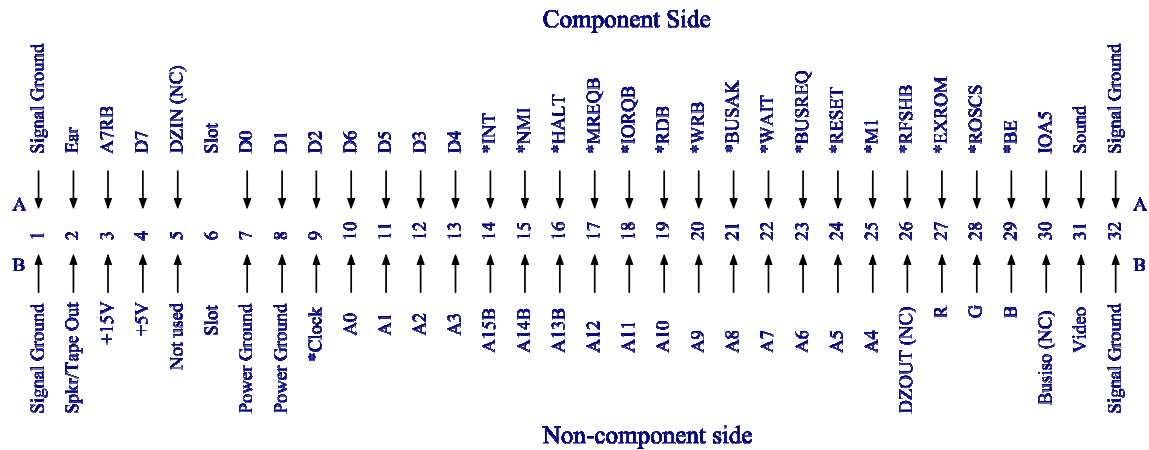
2.4 Interfaces and Connectors

The TS2068 has a number of specialized interfaces that are accessible via the following connectors:

CONNECTOR	TYPE	LOCATION
System Bus	2X32 Card Edge	Right Rear
Cartridge	2X18 Card Edge	Under TCC door
MIC	1/8" Mini Phone	Rear
EAR	1/8" Mini Phone	Rear
Player 1	Joystick 9-pin "D"	Left Side
Player 2	Joystick 9-pin "D"	Right Side
Monitor	RCA Phono	Rear
TV	RCA Phono	Rear
Keyboard	14-pin SIP	Inside-Left Rear
AC Adapter		Rear

2.4.1 System Bus Connector - P1

The TS2068 provides a 2 X 32 pin connector, which is designated as P1, at the right rear corner of the console. The mechanical, functional, and electrical requirements of the system buss connector are detailed in the attached data sheet. The data sheet connector must be modified by opening the slot at each end to allow the system PCB to pass through.



Connector is as seen from the front of the computer

FIGURE 2.4.1-2
P1 Connector Signal Layout

Table 2.4.1-1		
P1 Signal Definitions		
Pin #	Signal Name	Description
1A	GND	Signal Ground
1B	GND	Signal Ground
2A	EAR	EAR Input
2B	SPKR/TAPE OUT	Speaker/Tape Output
3A	A7RB	Refresh Address Bit 7 Buffered
3B	+15v	+15 Volts DC
4A	D7	Data Bus Bit 7
4B	+5v	+5 Volts
5A	DZIN	Daisy In (Not Connected)
5B	Not Used	
6A	Slot	
6B	Slot	
7A	D0	Data Bus Bit 0
7B	GND	Power Ground
8A	D1	Data Bus Bit 1
8B	GND	Power Ground
9A	D2	Data Bus Bit 2
9B	*CLK	CPU Clock (Inverted)
10A	D6	Data Bus Bit 6
10B	A0	Address Bus Bit 0
11A	D5	Data Bus Bit 5
11B	A1	Address Bus Bit 1
12A	D3	Data Bus Bit 3
12B	A2	Address Bus Bit 2

Table 2.4.1-1		
P1 Signal Definitions		
13A	D4	Data Bus Bit 4
13B	A3	Address Bus Bit 3
14A	*INT	Interrupt Bequest (Active Low)
14B	A15B	Address Bus Bit 15, Buffered
15A	*NMI	Non-Maskable Int.(Active Low)
15B	A14B	Address Bus Bit 14, Buffered
16A	*HALT	CPU HALT Indicator (Active Low)
16B	A13B	Address Bus Bit 13, Buffered
17A	*MEMRQB	Memory Request (Active Low),Bfrd.
17B	A12	Address Bus Bit 12
18A	*IRQB	I/O Request (Active Low), Bfrd.
18B	A11	Address Bus Bit 11
19A	*RDB	Read (Active Low), Buffered
19B	A10	Addre3ss Bus Bit 10
20A	*WRB	Write (Active Low), Buffered
20B	A9	Address Bus Bit 9
21A	*BUSAK	Bus Acknowledge (Active Low)
21B	A8	Address Bus Bit 8
22A	*WAIT	CPU WAIT (Active Low)
22B	A7	Address Bus Bit 7
23A	*BUSRQ	Bus Request (Active Low)
23B	A6	Address Bus Bit 6
24A	*RESET	CPU Reset (Active Low)
24B	A5	Address Bus Bit 5
25A	*M1	CPU MI State (Active Low)
25B	A4	Address Bus Bit 4
26A	*RFSHB	Refresh (Active Low),Buffered
26B	DZOUT	Daisy Out (Not Connected)
27A	*EXROM	Extension ROM Enable (Active Low)
27B	R	Color Signal - Red
28A	*ROSCS	ROS Chip Select (Active Low) (Dock Bank Enable)
28B	G	Color Signal - Green
29A	*BE	Bank Enable (Active Low)
29B	B	Color Signal - Blue
30A	IOA5	
30B	BUSISO	
31A	SOUND	Analog Sound Signal Output(O-5V)
31B	VIDEO	Composite Video Signal Output
32A	GND	Signal Ground
32B	GND	Signal Ground

2.4.1.1 Attachment of an RGB monitor

The TS 2068 provides via the PI rear-edge connector the ability to attach an RGB monitor for excellent picture clarity and resolution. The TTL-level logic signals appear directly on the rear-edge connector of the TS 2068 -- the necessary synch signals can be derived from the simple synch stripper/separator circuit described here.

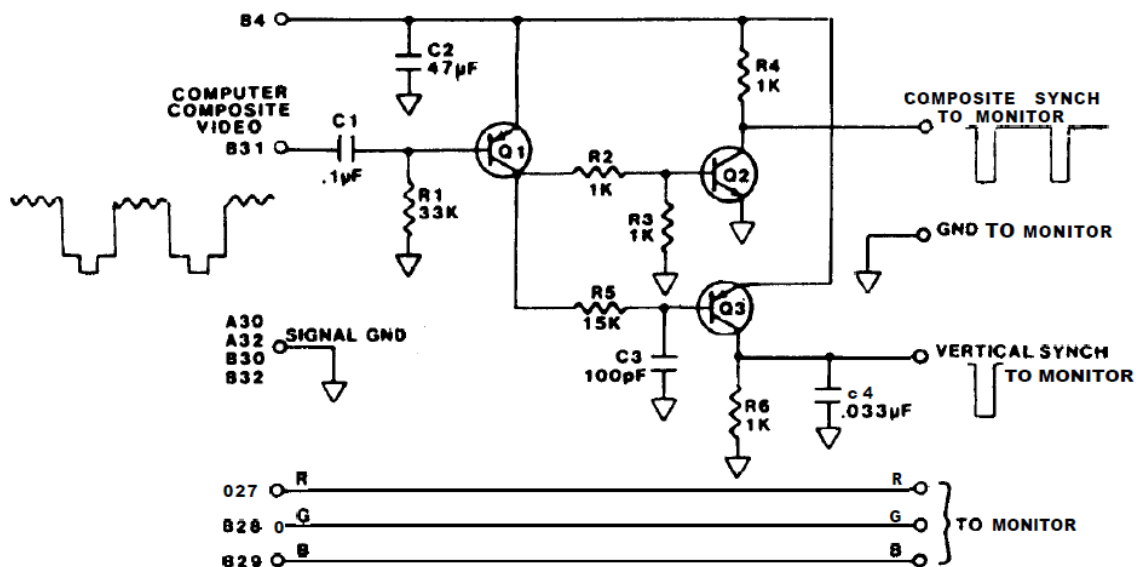
The Schematic of Figure 2.4.1-3 shows the required connections and electronics. Attachment is via the 64-pin keyed PI connector. Shielding should not normally be required, but ferrite beads are recommended on each wire to minimize EMI, TVI, etc.

Circuit Operation - R1 and the base-emitter junction of Q1 operate as a DC restoration circuit with current flowing only when the composite video input signal from connector pin B31 is at the synch level. With the charge maintained on C1, Q1 conducts only during the synch pulse interval (not during the color burst time). During this conduction interval, the composite synch signal appears in inverted form on the collector of Q1. The Q2 stage simply re-inverts the signal, providing at its collector a composite synch signal for the connected monitor.

To provide a separated Vertical synch pulse, R5 and C3 filter the output of Q1 to partially eliminate the Horizontal synch pulses which are shorter than the Vertical synch pulses. The partially filtered inverted signal is re-inverted by Q3, then R6 and C4 complete the elimination of the Horizontal synch pulses so that a separate Vertical synch pulse is supplied for the attached monitor.

Signals R, G, and B from connector pins B27, B28, and B29 can be supplied directly to the attached monitor.

Q1, Q3: 2N2907 or equivalent Q2: 2N2222 or equivalent



2.4.2 Cartridge Connector

The TS2068 provides a 2 X 18 pin connector (designated J4 on the schematic) under the door at the front right of the console. The table and figures listed below detail the mechanical, functional, and electrical requirements and limits of the J4 Cartridge Connector.

FIGURE 2.4.2-1
J4 Mating PCB Mechanical Requirements

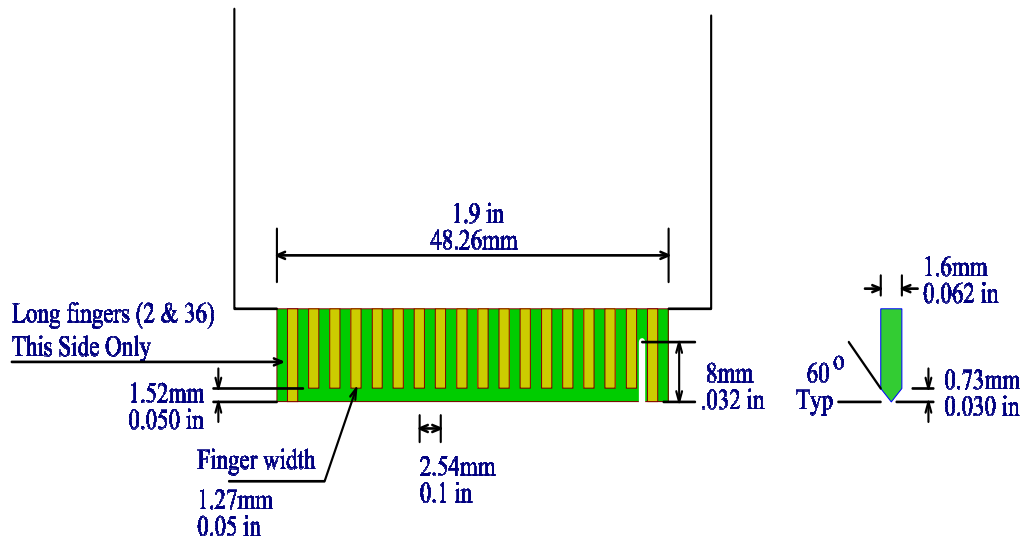


FIGURE 2.2.2-2
J4 Signal Layout
(View from front of computer)

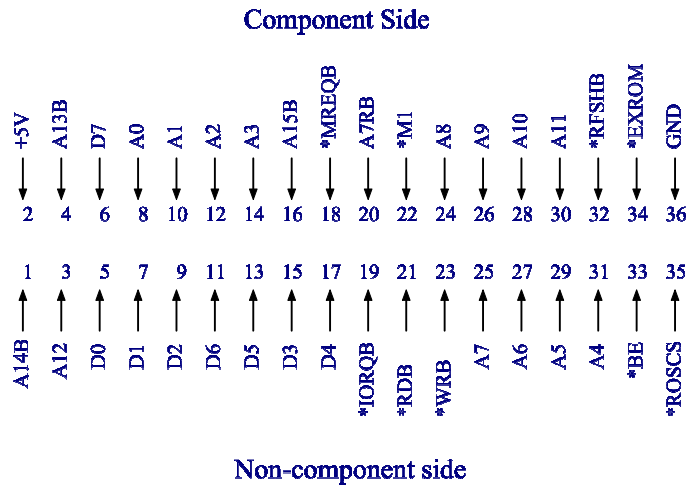


Table 2.4.2-1		
J4 Signal Definitions		
Pin #	Signal Name	Description
1	A14B	Address Bus Bit 14, Buffered
2	+5V	+5 volts DC
3	A12	Address Bus Bit 12
4	A13B	Address Bus Bit 13, Buffered
5	D0	Data Bus Bit 0
6	D7	Data Bus Bit 7
7	D1	Data Bus Bit 1
8	A0	Address Bus Bit 0
9	D2	Data Bus Bit 2
10	A1	Address Bus Bit 1
11	D6	Data Bus Bit 6
12	A2	Address Bus Bit 2
13	D5	Data Bus Bit 5
14	A3	Address Bus Bit 3
15	D3	Data Bus Bit 3
16	A15B	Address Bus Bit 15, Buffered
17	D4	Data Bus Bit 4
18	*MREQB	Memory Request (Active Low), Bfrd.
19	*IORQB	I/O Request (Active Low), Buffered
20	A7RB	Refresh Address Bit 7, Buffered
21	*RDB	Read (Active Low), Buffered
22	*M1	CPU M1 State (Active Low)
23	*WRB	Write (Active Low), Buffered
24	A8	Address Bus Bit 8
25	A7	Address Bus Bit 7
26	A9	Address Bus Bit 9
27	A6	Address Bus Bit 6
28	A10	Address Bus Bit 10
30	A5	Address Bus Bit 5
30	A11	Address Bus Bit 11
31	A4	Address Bus Bit 4
32	*RFSHB	Refresh (Active Low), Buffered
33	*BE	Bank Enable (Active Low)
34	*EXROM	Extension ROM Enable (Active Low)
35	*ROSCS	ROS Chip Select (Active Low)
36	GND	(Dock Bank Enable) Ground

2.4.3 Cassette I/O

The EAR and MIC connectors provided on the rear of the TS2068 are 1/8" mini-phone jacks requiring 1/8" mini-phone plugs as mating connectors.

The MIC output is filtered by a low-pass filter with a breakpoint of 2.5KHz and provides a signal output of 0.15 to 0.67 V p-p.

The EAR input is filtered by a low-pass filter with a breakpoint of 23 KHz. Input voltages should be between 4.0 and 10.0 V p-p.

2.4.4 Joystick I/O

The joystick input connectors, one TS2068 case, are standard D-pin "D" use with 5-switch type joysticks.

Connector layout and the function of on each side of the type connectors for each pin is given in Figure 2.4.4-1 and Table 2.4.4-1, respectively.

FIGURE 2.4.4-1
JOYSTICK CONNECTOR

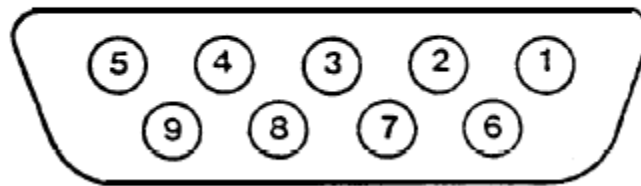
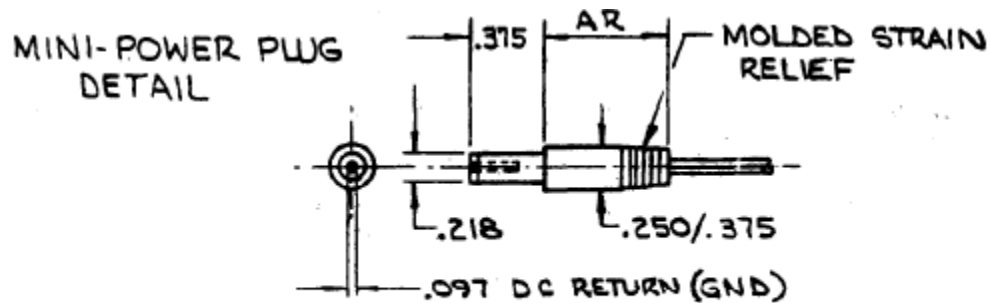


Table 2.4.4-1			
Joystick Connectory Signal Assignment			
Pin #	Signal Name	I/O Port Bit	Function
1	*DIR1	0	STICK UP
2	*DIR1	1	STICK DOWN
3	*DIR1	2	STICK LEFT
4	*DIR1	3	STICK RIGHT
5			not used
6	*BUTTON	7	PUSH BUTTON
7	+5V		5 VOLT POWER
8	*READ_STB		ADDRESS BIT 8 OR 9*
9	GND		POWER GROUND

*When Address Bit 8 is high, the READ strobe to the left joystick is driven low.
When address Bit 9 is high, the READ strobe to the right joystick is driven low.

2.4.5 AC Adapter Power Plug

The AC Adapter provided with the TS 2068 provides unregulated DC to the unit as described in Section 2.1.1 Mechanical details of the plug which mates to the TS 2068 are shown below:



2.4.6 Composite Monitor Output

The MONITOR output on the rear of the TS2068 provides a 1V p-p (+/- 20%) composite color video signal output to an RCA phono jack which is mated by a standard phono plug into a 75 ohm coax cable. See Section 2.1.11.1.

2.4.7 RF Output

The TV output on the rear of the TS2068 provides a modulated color video signal on VHF Channel 2 or Channel 3 as selected by the channel select switch on the bottom of the unit. Connection to the RCA phono jack output should be via a standard phono plug and 75 ohm coax cable. See Section 2.1.11.2.

Channel frequencies provided are

Channel 2 55,250 +/- 100 KHz

Channel 3 61,250 +/- 100 KHz

Output levels are less than 3 milliwatts as limited by the Federal Communications Commission.

2.4.8 Keyboard Interface

Located on the PCB inside the TS 2068 is a 14-pin single-in-line flex cable connector (AMP TRIO-MATE P/N 1-520315-4 or equivalent). Signals are as listed below:

PIN	SIGNAL
0	GND
1	KB0
2	KB1
3	KB2
4	KB3
5	KB4
6	CR6/A11
7	CR7/A10
8	CR8/A9
9	CR9/A12
10	CR10/A13B
11	CR11/A8
12	CR12/A14B
13	CR13/A15B

Any modification to or replacement of the keyboard supplied must consider the following:

- (1) Contact resistance less than 200 ohms.
- (2) Bounce less than 10 ms.
- (3) Capacitance per line less than 20 pF (0 or 1 key depressed); less than 40 pF (more than 1 key depressed).

3 System Software Guide

3.1 Identifier

Location 13 (13H) of the Home Bank ROM is used to identify the revision level of the System Software. The initial version is identified by this location having a value of 255 (FFH). Any subsequent versions will decrement this value by 1, e.g., the first revision would be identified by a value of 254 (FEH). This identifier should be used to conditionally apply patches or execute "workarounds" identified as necessary with a particular version of the System Software.

3.2 ROM Organization and Services

3.2.1 Home ROM Organization and Services

3.2.1.1 Fixed Entry Points

Home ROM Location 0 is the entry to the system initialization code upon power-up (Ref. Figure 1.1-4). Locations 8 through 48 (8H through 30H) are the Z80 RESTART entry points for the following functions:

RESTART	FUNCTION
8	ERROR - Error exit from BASIC (Address on Stack points to Error Number)
15 (0FH)	WRCH - Write Character (Code in A) to Current Output Channel as established by SELECT (Address of output routine pointed to by System Variable CURCHL). (See Section 4.0).
24 (18H)	IGN SP - Return in A the current significant character in the Program Line (Address in System Variable CH ADD) skipping over spaces and- control characters except End-of-Line (0DH=ENTER)
32 (20H)	NXT_IS - Like IGN SP but returns in A the Next Significant Character.
40 (28H)	CALCTR - Entry to Calculator Routines.
48 (30H)	COPYUP - Make room for BC Bytes of temporary workspace just before address in System Variable STKBOT by copying up memory between there and the address in STKEND, adjusting affected pointers. Returns DE=1st Byte of Space; HL=Last.

Location 56 (38H) is the entry to service the hardware generated interruption which occurs approximately every 1/60 of a second (16.67ms). Z80 Int. Mode 1 is used. This interruption is used to scan the keyboard (call to routine UPD K - see Section 4.1.1). It is also used to update the Frame Counter (3 bytes pointed to by the System Variable FRAMES) used by the RANDOMIZE instruction.

Location 102 (66H) is the entry point for the NMI interruption, but this interruption is not used in the TS2068 design. (See Section 2.1.3.8 NMI Interruption.)

3.2.1.2 BASIC AROS Support

BASIC Application Cartridges are supported by special code in the Home ROM. A program line is copied from the cartridge to a buffer in the Home RAM (ARSBUF) and is then executed from there by the BASIC Interpreter. When a READ command is executed, the line containing the appropriate DATA statement is also copied from the cartridge to the RAM. The cartridge memory is enabled only for search and copy operations for both program lines and DATA statements, and when executing a USR function, otherwise the entire Home Bank is enabled while executing in the BASIC Interpreter. There is no support for User-Defined Functions which insert the expanded definition parameters directly into the program and then require search of the program area to find these parameters whenever a function is invoked. See Section 5.1, Cartridge Software/Hardware, for additional details on BASIC AROS.

3.2.2 Extension ROM

3.2.2.1 Fixed Entry Points

Extension ROM Location 0 contains code to pass control to the initialization code in the Home ROM. (Figure 1.1-4).

Extension ROM Location 56 (38H) is the interruption fielder. Control is passed to the System RAM code (See Section 3.3.3) to bank switch to the Home Bank and call the interruption service routines after which the state of the machine is restored and control returns to the interrupted process. Figure 3.2.2-1 shows the Extension ROM Interruption Fielder code.

3.2.2.2 General

The balance of the Extension ROM contains the following major components:

- Final Phase of System Initialization (See Figure 1.1-4)
- Cassette tape I/O (see Section 4.2)
- Change Video Mode Service
- OS RAM routines including the Function Dispatcher (copied to RAM at System Initialization) (see Section 3.3.3)
- Function Dispatcher Jump Table

FIGURE 3.2.2-1
Extension ROM Interruption Fielder

LOCATION	OBJECT CODE	SOURCE CODE	COMMENTS
0038	F5	PUSH AF	;Save AF
0039	F3	DI	
003A	3AC25C	LD A,(VIDMOD)	;Disable Ints.
003D	A7	AND A	;Test Vidmod
003E	00	NOP	
003F	2804	JR Z,CHK3	;Vidmod=0
0041	F1	POP AF	;Restore AF
0042	C36EFA	JP INT7	;Chunk7 if Vidmod not 0
0045	F1	CHK3 POP AF	;Restore AF
0046	C3AE62	JP INT3	;Chunk 3 if Vidmod = 0

3.2.2.3 Video Mode Change Service

The routine CHNG VID takes as input a single byte in Register3 which designates the desired video mode as shown in Table 3.2.2-1. All non-zero values involve access to the second display file located at 6000H-7AFFH. When the mode change requires remapping of the RAM (see Figure 1.1-3), the necessary relocation (BASIC program, machine stack, OS RAM code, UDG area, etc.) and modifications (system variables, RAM code internal addresses, stack pointer, etc.) are done by this service. The desired video mode is written to Port OFFH, Bits 0-5, and the System Variable VIDMOD (5CC2H) is updated. The second display file is cleared to zeros on initial access (for Dual Screen Mode and High Resolution Graphics Mode, this results in a black screen since 0 yields attributes of black ink on black paper). If there is not enough free memory to do the necessary remapping, Error 4, Out of Memory is given.

Access to this service via the Function Dispatcher cannot be made consistently for various reasons. An Interface Routine is given in Section 3.2.2.4, to be executed from the Home RAM, which provides access to the Video Mode Change Service as well as other Extension ROM routines.

See Sections 4.1.2 and 5.2 for discussion of video screen support software. See Section 6.4 for details on known problems and corrections related to the Video Mode Change Service.

TABLE 3.2.2-1
INPUT TO VIDEO MODE CHANGE SERVICE

Value In A	Video Mode	DESCRIPTION
0	Normal	Primary Display File Only(Close 2nd Display File if Open)
128 (80H)	Dual Screen	Two Display Files Available. Primary Display File Active at Screen.
1	Dual Screen	Two Display Files Available. Second Display File Active at Screen
2	High Resolution Graphics	Primary Display File contains data for 256X192 pixels. Second Display File contains 6144 Attribute Bytes, each one controlling 8X1 pixels. NOTE 1.

64-Column Modes		
	Ink	Paper
6	Black	White
14 (0EH)	Blue	Yellow
22 (16H)	Red	Cyan
30 (1EH)	Magenta	Green
38 (26H)	Green	Magenta
46 (2EH)	Cyan	Red
54 (36H)	Yellow	Blue
62 (3EH)	White	Black

The two display files are combined to provide a 64 column X 24 line screen. Even columns are derived from data in the Primary Display File and odd columns from the 2nd Display File. Bits 3-5 of the mode select the ink color which determines the complementary paper color. The Flash and Bright Attributes are fixed at 0; the Border is fixed at the paper color. NOTE 1.

NOTE 1: The areas of memory normally used for Attribute Bytes are not accessed by the video hardware in this mode.

3.2.2.4 Extension ROM Interface Routine

The Extension ROM routines W TAPE (Write from RAM to Tape), R-TAPE (Read from Tape to RAM) (see Section 4.2) and CHNG VID (see Section 3.2.2.2) may be of interest to the machine code programmer. Because of a conflict with the use of the IX Register, the tape routines cannot be successfully accessed via the Function Dispatcher. Because the Change Video Mode Service may involve relocating the OS RAM routines (including the Function Dispatcher), and for other reasons, it also cannot be consistently accessed using the Function Dispatcher. Figure 3.2.2-2 gives a sample routine, to be executed from the Home RAM, which can be used to bank switch to the Extension ROM and call directly to the desired service. Appendix A contains an Extension ROM Map giving the addresses of these and other routines.

FIGURE 3.2.2-2 EXTENSION ROM INTERFACE ROUTINE

```

; EXTENSION ROM INTERFACE ROUTINE
R_TAPE      EQU    00FCH      ; READ TAPE ROUTINE
W_TAPE      EQU    0068H      ; WRITE TAPE ROUTINE
CHNG_VID    EQU    0E8EH      ; CHANGE VIDEO MODE ROUTINE
VIDMOD      EQU    5CC2H      ; VIDEO MODE SYSTEM VARIABLE
;
;                                     ; CALL READTP WITH REGISTERS SET
;                                     ; UP FOR R_TAPE ROUTINE
READTP LD    HL, R_TAPE      ; ADDRESS TO HL
          CALL IFRTN         ; ENABLE EXT./EXECUTE QTN
          JR     EXIT        ; RESTORE HOME BANK AND RETURN
;
WRITETP     LD    HL, W_TAPE  ; ADDRESS TO HL
          CALL IFRTN         ;
          JR     EXIT
;
CHGVID LD    HL, CHG_VID
          PUSH AF            ; SAVE VIDEO MODE
          CALL IFRTN
;                                     ; COMPENSATE FOR BUG IN
;                                     ; CHNG_VID RTN WHICH SETS
;                                     ; VIDMOD = 0 INSTEAD OF 80H
;                                     ; WHEN BOTH DISPLAY FILES
;                                     ; ARE OPEN
          POP AF            ; TEST VIDEO MODE
          CP    80H         ; AGAINST 80H
          JR    NZ, EXIT    ;
          LD    (VIDMOD), A  ; SET VIDMOD = 80H
EXIT        LD    A, (HSSAVE) ; GET PREV HSR
          OUT   (0F4H), A    ; RESTORE
          IN    A, (0FFH)    ; READ PORT 0FFH
          RES   7, A         ; TURN OFF ROM SEL
          OUT   (0FFH), A    ;
          EI              ;
          RET
;
HSSAVE      DEFB 0          ; SAVE HOR. SEL. (PORT 0F4H)
;
IFRTN       DI              ; MASK INTERRUPTS
          PUSH AF           ; PRESERVE CALLER'S ACCUM
          IN    A, (0FFH)   ; EXT. ROM SELECT BIT
          SET   7, A        ; SEL EXT. ROM
          OUT   (0FFH), A   ;
          IN    A, (0F4H)   ; HSR FOR DOCK/EXT.
          LD    (HSSAV), A  ; SAVE
          LD    A, 1        ; SELECT CHUNK 0 IN EXT. ROM
          OUT   (0F4H), A   ;
          POP AF           ; RESTORE CALLER'S ACCUM
          JP    (HL)        ; EXECUTE TARGET SUBROUTINE
;                                     ; RETURN TO CALLER OF IFRTN

```

3.3 RAM Organization and Services

3.3.1 System Variables

RAM beginning at 23552 (5COOH) is dedicated to the BASIC System Variables as defined in Appendix D of the TS 2068 User Manual and in Appendix B of this document. The area from the end of the defined variables (STRMNM - 23755 (5CCB)) to 24297 (5EE9H) is reserved for expansion of the System Variables, but is not used by the Operating System in the current TS 2068.

3.3.2 System Configuration Table

The area from 24298 (5EEAH) to 24575 (5FFFH) is reserved for the System Configuration Table (SYSCON). This table is built at system initialization time and is comprised of an 8 byte entry for AROS, a 4 byte entry for LROS, followed by eleven 24-byte entries for proposed expansion banks and an End-of-Table marker. In the original TS 2068 the actual usage of this table is limited to the 12 bytes for software cartridge identification (see Section 5.1 for details of the LROS and AROS Overhead Bytes).

3.3.3 Machine Stack

The TS 2068 reserves 512 (200H) bytes of RAM for the Machine Stack. The Machine Stack pointer is initialized to a value of 6200H (value also in System Variable (MSTBOT); the pointer is decremented as items are pushed onto the stack (the pointer may also be modified directly by software). While the area reserved for the stack extends to 6000H, there is no actual check made to enforce this limit.

Note: The Machine Stack is located in the same memory area as the second display file. The CHNG VID routine relocates the stack to the memory area from 0F7C0H to 0F8BFH, and modifies the Stack Pointer and MSTBOT (0F8C0H), as well as other affected system variables, when initializing the second display file. (See Section 3.2.2.3.)

3.3.4 OS Ram Routines

The code for the following Operating System functions is copied from the Extension ROM to Chunk 3 of the RAM at System initialization time. Since this is in the same memory area as the second display file, this code must be relocated, along with the machine stack, if the second display file is to be used. The CHNG VID routine does the necessary relocation and modifications. (Section 3.2.2.3.)

Because this code is not in a fixed location, access to the OS RAM routines is conditional on the current video mode. The standard technique employed is to test the value in the System Variable VIDMOD at location 23746 (5CC2H). A zero indicates that the second display file is not in use and that the OS RAM routines are therefore in Chunk 3; any non-zero value indicates that the routines are in Chunk 7.

NOTE: This design implies that Chunks 2, 3 and 7 are always enabled in the Home Bank RAM whenever the System ROM and/or RAM routines are being used.

The OS RAM routines are contained in Module "Dispatch" which is included in Appendix A.

3.3.4.1 RAM Interruption Handler

Chunk 3 Entry: 62AEH

Chunk 7 Entry: FA6EH

The user must enter with bank status and Z80 registers intact, with address from point of interruption on the stack. The RAM interruption handler saves state, including memory selection, enables the Home Bank, updates the Frame Counter, calls the keyboard scan routine in the Home ROM, restores state, and returns to the interrupted process. The RAM Interruption handler is used whenever the interruption occurs while the Extension ROM is enabled, See Figure 3.2.2-1, Extension ROM Interruption Fielder. This same technique can be used for interruption processing in another bank, e.g. if an LROS wanted to use the standard system ROM keyboard scanning routines.

3.3.4.2 RAM Service Routines

Table 3.3.4-1 lists the RAM service routines which are designed to facilitate communication between memory banks. Those with Service Codes are accessible via the Function Dispatcher.

Label	Service Code (Decimal)	Location (Hex)		Description
		1 Disp File	2 Disp Files	
GET_WORD		6316	FAD6	Returns in HL the word from the address in HL in the bank specified in B.
PUT_WORD		6336	FAFB	Writes the word in DE to the address in HL in the bank specified in B.
GET_STATUS	14	6405	FBC5	Returns current memory selection (Horizontal Select byte - low active) in C for the bank specified in B. Preserves Bank # in B for Home, Ext. or Dock.
GET_CHUNK		644D	FC0D	Returns a single byte mask in A with all bits 0 except the one corresponding to the chunk for the address in HL.
GET_NUMBER	15	645E	FCIE	Returns in Reg. A the bank number currently controlling the address in HL.
BANK_ENABLE		6499	FC59	Enables the memory selected (Horizontal Select byte - low active) in the specified bank. (Bank # in B; Mem.Sel.in C)

Label	Service Code (Decimal)	Location (Hex)		Description
		1 Disp File	2 Disp Files	
GOTO_BANK		6572	F032	Transfers control to the specified address after enabling the memory selected in the specified bank. Parameters passed on stack by pushing target address, then Bank #/Mem.Select prior to calling GOTO BANK. (Return address is discarded).
CALL_BANK		65D0	FD90	Like GOTO BANK except saves current bankstatus, calls target address, and restores status prior to returning to user. Two additional parameters are passed on stack prior to doing call to CALL BANK. These are PRM_OUT (16-bits) following by PRM_IN (16 bits) as described for the function dispatcher
XFER_BYTES		6722	FEE2	Copies n byte(s) from specified source to specified destination in either ascending or descending order. Source and destination can be in the same or different banks and can be in shadowing chunks, but neither source nor destination can pass a "chunk" (8K) boundary since only the chunks containing the starting source and destination addresses are explicitly enabled. Parameters passed on stack by pushing: Source Bank/Dest.Bank Source Address Dest. Address Length Direction: 0 = Ascending 1 = Descending)
NOTE: See Appendix A for listing of these routines. See Section 6.0 for known corrections to the routines.				

3.3.4.3 Function Dispatcher

Chunk 3 Entry: 6200H

Chunk 7 Entry: F9C0H

The Function Dispatcher provides, a common interface to a number of system routines via a Service Code and Jump Flag parameter passed on the machine stack. Table 3.3.4-2 lists the routines in Service Code order. Codes for routines that are known to not be successfully accessible via the Function Dispatcher have been deleted (marked Reserved). However, there is no guarantee that those on the list can be accessed without problems. Some ROM routines require data in a particular format, e.g. BASIC floating point number(s), both standard and special integer format, on the Calculator Stack which is located between (STKBOT) and (STKEND) (see Appendix C of the TS 2068 User Manual). An effort has been made to include information on register usage and functionality, but some of the ROM routines are so tightly tied to the BASIC Interpreter that they would require analysis which is beyond the scope of this document. These have been flagged with an Asterisk, but included in the list for documentation purposes only. Most of the routines which are directly implementing a BASIC command or function have two different action sequences based on the INTPT Flag (Bit 7 of FLAGS) which distinguishes syntax checking (Flag=0) from actual execution (Flag=1).

In order to use the Function Dispatcher, first set up any memory and stack (both machine and/or calculator) locations as if invoking the desired service directly. Then push the parameter(s) for the Dispatcher on the machine stack in the order outlined below. Finally, set up the registers as if invoking the desired service directly and call the Dispatcher based on its current location (Chunk 3 if VIDMOD=0 or Chunk 7 if VIDMOD has a non-zero value).

1. PRM OUT 16 bits - Number of bytes of parameter data being passed on the stack to the specified Service (number of stack "pushes" * 2). Zero if no parameters being passed. E.g., to pass 4 bytes:
 LD HL,4
 PUSH HL
 This parameter is passed to the Dispatcher only if the Jump Flag (SVC CODE) Bit 15) is not set. NOTE: This parameter refers to machine stack entries only, not to the Calculator Stack.
2. PRM IN 16 bits - Number of bytes of parameter data to be passed back from the specified Service (number of stack "pushes" * 2). Zero if no parameters to be passed back. This parameter is passed to the Dispatcher only if the Jump Flag (SVC CODE Bit 15) is not set.

NOTE: This parameter-refers to machine stack entries only, not to the Calculator Stack.

3. SVC_CODE 16 bits - Bits 0-14 identify the Service to be invoked. Bit 15 (Jump Flag) is set if no return is desired (jump to Service rather than call). Bit 15 is zero if return is desired. E.g, to call K SCAN using Service Code 136:
LD HL,136 or LD HL,88H
PUSH HL PUSH HL

Addendum To TS 2068 Function Dispatcher Services:

On page 84, COLOR and HIFLSH (service codes 85 and 86) cannot always be accessed through the Function Dispatcher, due to resetting of the carry flag by the FD. COLOR may be accessed by setting the registers as described in the manual, and then coding CALL #23DE. HIFLSH can be accessed similarly by coding CALL #2410.

TABLE 3.3.4-2
TS 2068 Function Dispatcher Services

Service	Service Code	Description
	1-13 01H-0DH	Reserved
GET STATUS	14 (0EH)	Returns Memory Selection (Low Active) in C for Bank # in B
GET NUMBER	15 (0FH)	Returns Bank # in A for Address in HL
	16-24 (10-18H)	Reserved
UPD K	25 (19H)	Process Keyboard Input (See Section 4.1 . 1)
PARP	26 (1AH)	Generates DE+1 Cycles of a Tone having the Period 8N+236 to 8N+246 T-States. HL=N. (See 4.4)
BEEP	27 (1BH)	BEEP Command - processes parameters on Calculator Stack. Exits via PARP. (See 4.4)
K_DUMP	28 (1CH)	COPY Command. Dumps Primary Display File to Printer. (See 4.1.3)
SENDTV	29 (1DH)	Char. Output to Screen/Printer. Character Code in A. (See 4.1.2)
SETAT	30 (1EH)	Set Print Position to value in BC. C = Line # (0-23); C=Column # (0-31)
ATTBYT	31 (1FH)	Set Attribute Byte for Display File Address in HL using ATTR_T, MASK_T and P-FLAG.
R_ATTTS	32 (20H)	Permanent Attribute Info. to Temporary Attribute Variables
CLLHS	33 (21H)	Clear Lower Screen (Primary Display File)
CLS	34 (22H)	Clear Entire Screen(Primary Display File)
DUMPPR	35 (23H)	Print/Clear Print Buffer. (See 4.1.3)
PRSCAN	36 (24H)	Send scan line (32 bytes to printer) Pixel address in HL Number of scans remaining in B (1-8) (See 4.1.3)
DESLUG	37 (25H)	Remove Number Slugs from Edit Line Buffer (Address in HL)
K_NEW	38 (26H)	NEW command. See Fig. 1.1-4
INIT	39 (27H)	Initialize: DE=Maximum RAM Address. A=0 for Power-On; = -1 (FFH) for NEW. (See Fig.1.1-4)

TABLE 3.3.4-2 TS 2068 Function Dispatcher Services		
Service	Service Code	Description
INCH	40 (28H)	Input Character to A from currently Selected Channel. Returns NC if no input.
SELECT	41 (29H)	Select Channel (Stream) - # in A. (See 4.1)
INSERT	42 (2AH)	Insert BC Bytes before byte whose address is in HL. Copies up all from HL to (STKEND) and updates affected system variables. Returns BC = 0; DE = address of last byte of inserted space; HL = address of byte before first.
RESET	43 (2BH)	Reset Calculator Stack. Sets (STKEND) =(STKBOT) (MEM)=MEMBOT (5C92H).
CLOSE	44 (2CH)	CLOSE # Command. Channel # on Calculator Stack. (See 4.1 for more info. on OPEN and CLOSE)
CLCHAN	45 (2DH)	Close Channel. BC=Value from STRMS (Index into CHANS).
OPEN	46 (2EH)	OPEN # Command. Channel # and Device Spec. on Calculator Stack
OPCHAN	47 (2FH)	Open Channel. Device Spec. on Calculator Stack. DE=pointer into STRMS based on Ch.#. (See 4.1 for more info. on OPEN and CLOSE)
CAT	48 (30H)	CAT Command (Not Applicable)
ERASE	49 (31H)	ERASE Command (Not Applicable)
FORMAT	50 (32H)	FORMAT Command (Not Applicable)
MOVE	51 (33H)	MOVE Command (Not Applicable)
FLASHA	52 (34H)	Flash Char.in A to Screen. (Calls SENDTV; assumes Lower Screen selected. Used to Flash Cursor.)
FIND_L	53 (35H)	Find BASIC Program Line with the number in HL. If Line found, returns Z and Address of Line in HL, else returns NZ and HL contains either address of line with next larger line number or points to the Variables area if there is no larger line number. Requested Line No. returned in BC and Address of Preceding Line in DE (DE=HL if no preceding line).
SUBL_IN	54 (36H)	Finds either the D'th statement (D=Statement #; E=0) or 1st statement whose keyword token matches E (D=0), in a line pointed to by HL. If the D'th statement is found, returns Z and HL and (CH ADD) both point to 1 byte before-statement. (If line contains exactly D-1 statements, then the next line counts as the D'th.). If match on E is found, then returns NZ,NC and both HL and (CH ADD) point to keyword. D is decremented by the number of statements looked at (e.g. D= -2 if two statements). If no match on E then returns NZ,C with both HL and (CH ADD) pointing to End-of-Line byte (0DH).
RECLEN	55 (37H)	Returns in BC the length of the record pointed to by HL. Sets DE to HL+BC. The record can be a program line, or a string or numeric variable or array.
DELREC	56 (38H)	Delete record pointed to by HL having length BC from Program or Variables memory. Updates affected system variables.
PUT_BC	57 (39H)	Converts number in BC from binary to ASCII and outputs to currently selected channel, If BC less than 0, outputs a 0.
SYNTAX	58 (3AH)	Check syntax of command or program line in Edit Line Buffer (E_LINE). ERR NR= -1 if no errors, otherwise contains Error Number-1.
EXCUTE	59 (3BH)	Execute command(s) from Edit Line buffer.
FOR	60 (3CH)	FOR command. *

TABLE 3.3.4-2 TS 2068 Function Dispatcher Services		
Service	Service Code	Description
STOP	61 (3DH)	STOP command. Does RESTART 8 with Error No. 9.
NEXT	62 (3EH)	NEXT command. *
READ	63 (3FH)	READ command. *
DATA	64 (40H)	DATA statement. *
RESTBC	65 (41H)	RESTORE command - Line No. in BC
RAND	66 (42H)	RANDomize command. Sets seed for Random Number Generator based on Parameter on Calculator Stack. If parameter is non-zero, value is loaded to SEED; if zero, value in FRAMES is loaded to SEED.
CONT	67 (43H)	CONT command. Loads values from OLDPPC and OSPPC to EWPPC and NSPPC and returns. Inside the BASIC Interpreter, this results in executing from Line No. in NEWPPC, Statement No. in NSPPC.
JUMP	68 (44H)	Jump to Line - Loads Line Number from Calculator Stack to NEWPPC and sets NSPPC to 0 and returns.
FIX_U1	69 (45H)	Converts Floating Point number on Calculator Stack to a single byte unsigned binary value in A (uses FP2A). Does RESTART 8 for Error B if number out of range.
FIX_U	70 (46H)	Converts Floating Point number on Calculator Stack to a 2-byte unsigned binary value in BC (uses FP2BC). Error B if number out of range.
CLEAR	71 (47H)	CLEAR command. Processes parameter on Calculator Stack to value in BC for CLR BC.
CLR_BC	72 (48H)	Value in BC is new RAMTOP. Deletes Variables, clears screen, and Calculator Stack, etc.
GO_SUB	73 (49H)	GO SUB command. Inserts a 3-byte GO-SUB Block into the machine stack above the 2 most recent entries. The Block consists of current Line No. (2 bytes) and Statement No. (1 byte) to be used when RETURN is executed. Then calls JUMP to process GO SUB parameter and returns. At return to caller, machine stack consists of top of stack at point GO SUB was called, followed by 3-byte entry (Line No. MSB/Line No. LSB/Statement No.).
CHK_SZ	74 (4AH)	Checks if room for BC 80 (50H) bytes between (STK;ND) and (RAMTOP). Addition of 80 bytes is "left-over" from Spectrum to guarantee minimum machine stack where the stack was at the top of RAM. Error 4 if not enough room.
RETURN	75 (4BH)	RETURN command. Retrieves most recent GO SUB Block from Machine Stack (SP+4), loads data to NEWPPC and NSPPC and returns. Error 7 if MSB Line No.=3EH (End of Stack Marker).
PAUSE	76 (4CH)	PAUSE command. Processes parameter on Calculator Stack to BC then waits BC frames or until key is depressed. (Uses HALT instruction, so interruptions must be enabled)
BREAK?	77 (4DH)	Reads BREAK key. Returns NC if it is pressed and ON ERROR is not active.
DEF	78 (4EH)	Define Function.*
K_LPR	79 (4FH)	LPRINT - Selects Channel 3 and processes items in LPRINT statement for output via WRCH.
K_PRIN	80 (50H)	PRINT - Selects Channel 2 and processes items in PRINT statement for output via WRCH (same code used for K_LPR).
P_SEQ	81 (51H)	Code used by K_LPR and K_PRIN to process output-data and controls in BASIC statement (address in CH ADD).

TABLE 3.3.4-2 TS 2068 Function Dispatcher Services		
Service	Service Code	Description
INPUT	82 (52H)	INPUT command. Selects Channel 1 and processes I/O for Keyboard/Lower Screen using a buffer at (WORKSP) for input. *
I_SEQ	83 (53H)	Code used by INPUT to process input items and controls in BASIC statement (address in CH ADD).
NOTKB?	84 (54H)	Returns Z if current channel is Keyboard/Lower Screen (device specification="K").
COLOR	85 (55H)	Adjusts system variables ATTR T, MASK T and P FLAG for color code in D (0-9). Enter with C set to set Ink or NC set to set Paper. Error K if D is invalid.
HIFLSH	86 (56H)	Adjusts system variables (ATTR T and MASK T) for Flash/Bright code in D (0, 1 or 8) else Error K. Enter with C for Flash or NC for Bright.
SCRMBL	87 (57H)	Returns in HL the primary display file address for the pixel with coordinates in BC (B=Y;C=X). Returns in A the bit no (0-7) where 0=lefthand or most significant bit. Error B if Y is greater than 175.
PLOT	88 (58H)	PLOT command. Processes X/Y parameters on the Calculator Stack to BC for plotting of pixel via PLOTBC.
PLOTBC	89 (59H)	Deals with pixel for coordinates in BC (B=Y; C=X). Processes using P FLAG for Inverse and Over attributes. Updates Attribute File and sets COORDS=BC.
GET_XY	90 (5AH)	Converts a pair of numbers from the Calculator Stack to 2 single byte numbers. Top number goes to B and second to C. D=sign of B and E=sign of C (+1 or -1). Used by PLOT and other routines.
CIRCLE	91 (5BH)	CIRCLE command. Calculates successive plot positions from the parameters in the BASIC statement. *
DRAW	92 (5CH)	DRAW command. Calculates successive plot positions from the parameters in the BASIC statement. *
DRAW_L	93 (5DH)	Plots a straight line from current position (COORDS) based on parameters from Calculator Stack (X,Y). *
EXPRN	94 (5EH)	Evaluates expression in BASIC program line (CH ADD), putting value on Calculator Stack. *
F_SCRN	95 (5FH)	SCREEN\$ function. Matches screen line/col. position (parameters on Calculator Stack) against standard ASCII character set. Returns BC=0 if no find. BC=1 and DE points to Char. Code byte if match found.
F_ATTR	96 (60H)	ATTR function. Returns attribute byte value controlling screen pixel position based on parameters on Calculator Stack (X,Y).
RND	97 (61H)	RND function. Uses value in SEED to generate a pseudo-random number which is placed on the Calculator Stack (Floating Point number).
F_PI	98 (62H)	PI function. Places value of PI on Calculator Stack.
F_INKY	99 (63H)	INKEY\$ function. Scans keyboard and puts character code byte in (WORKSP) if key detected. In any case, pushes Regs. AEDCB onto Calculator Stack - BC=0 if no input; =1 if char. code stored; DE=address of char. code byte.
FIND_N	100 (64H)	Find Variable. Searches Variables area for match against identifier pointed to by CH ADD. Adjusts bit NO of FLAGS (Bit 6) for type (1=numeric; 0=string). Also used to find formal parameters for User Defined Functions. *

TABLE 3.3.4-2
TS 2068 Function Dispatcher Services

Service	Service Code	Description
PSHSTR	101 (65H)	Push String - Clears bit NO of FLAGS and pushes Regs. AEDCB onto Calculator Stack adjusting (STKNXT) upwards. DE contains address of string; BC contains length.
PAEDCB	102 (66H)	Same code as for PSHSTR but preserves state of bit NO of FLAGS (Bit 6).
LET	103 (67H)	LET command. Processes existing or creates new variables. *
POPSTR	104 (68H)	Pop String - Pops end of Calculator Stack ((STKNXT)-1 through (STKNXT)-5) to Regs. BCDEA, adjusting (STKNXT) downwards.
DIM	105 (69H)	DIM statement. Creates or initializes numeric or string arrays. *
STKUSN	106 (6AH)	Stack Unsigned Number - inputs a floating point number onto the Calculator Stack from a series of ASCII characters addressed by (CH_ADD). The first character is already in Reg. A (either decimal point, binary token or digit).
STK_A	107 (6BH)	1-byte unsigned integer in A to top of Calculator Stack (binary to floating point). Loads 0 to B and A to C, then executes STK BC.
STK_BC	108 (6CH)	2-byte unsigned integer in BC to top of Calculator Stack (binary to floating point).
ININT	109 (6DH)	Converts a series of ASCII digits pointed to by (CH ADD) into an unsigned floating point integer on the Calculator Stack. First character is in A on entry. Terminates when non-digit found.
FP2BC	110 (6EH)	Pops top of Calculator Stack (floating point number) and puts in BC, rounded to nearest integer. Returns NZ if value is negative. Returns C if number exceeded maximum 2-byte value (65535). Range: -65535 to +65535.
FP2A	111 (6FH)	Pops top of Calculator Stack (floating point number) and puts in A, rounded to nearest integer. Returns NZ if value is negative. Returns C if number exceeded maximum 1-byte value (255). Range: -255 to +255.
OUTPUT	112 (70H)	outputs number on top of Calculator Stack to currently selected channel via WRCH. (Converts from floating point to ASCII.)
Full explanation of the following Calculator Routines is beyond the scope of this document.		
SUB	113 (71H)	Subtract floating point format numbers (HL) minus (DE). (DE) assumed to be (HL) + 5
ADD	114 (72H)	Add (HL) + (DE). See SUB.
MULT	115 (73H)	Integer multiply HL * DE. Returns C if overflow.
TIMES	116 (74H)	Floating Point Multiply (HL) * (DE).
DIVIDE	117 (75H)	Floating Point Divide (HL)/(DE).
TRUNC	118 (76H)	Truncates a floating point number (HL) towards zero to an integer. Assumes (DE) = (HL) + 5.
FLOAT	119 (77H)	Converts number (HL) to floating point format. Assumes HL points to an integer in 5-byte format.
INTDIV	120 (78H)	Replaces top two numbers on Calculator Stack (X and Y) by X Mod Y and the integer quotient INT (X/Y). Returns with DE and HL = Calc.Stack Pointers.
INT	121 (79H)	Replaces the top of the Calculator Stack by its integer part. Returns with HL = top of Calc. Stack and DE = next free space.
EXP	122 (7AH)	Replaces the top of the Calculator Stack, X, by EXP(X). Returns with DE and HL = Calc. Stack Pointers.
LN	123 (7BH)	Replaces the top of the Calculator Stack by its natural logarithm. Returns DE and HL = Calc. Stack Pointers.

TABLE 3.3.4-2
TS 2068 Function Dispatcher Services

Service	Service Code	Description
ANGLE	124 (7CH)	Replaces the top of the Calculator Stack (X) by Y where Y is greater than or equal to -1 and less than or equal to +1 and the $SIN X = SIN (PI/2 * Y)$.
COS	125 (7DH)	Replaces the top of the calculator stack by its COSINE
SIN	126 (7EH)	Replaces the top of the Calculator Stack by its SINE.
TAN	127 (7FH)	Replaces the top of the Calculator Stack by its TANGENT.
ATN	128 (80H)	Replaces the top of the Calculator Stack by its inverse TANGENT.
ASN	129 (81H)	Replaces the top of the Calculator Stack by its inverse SINE.
ACS	130 (82H)	Replaces the top of the Calculator Stack by its inverse COSINE.
ROOT	131 (83H)	Replaces the top of the Calculator Stack by its Square Root.
TO_THE	132 (84H)	Replaces the top two numbers on the Calculator Stack (X, Y) by $x^{**}y$.
End of floating point routines		
RDCH	133 (85H)	Wait for character from currently selected channel (calls INCH). Returns character code in A. See 4.1.1.
SENDCH	134 (86H)	Write character whose code is in A to currently selected output channel. See 4.1.2.
WRCH	135 (87H)	See 3.2.1.1, RESTART 16.
K_SCAN	136 (88H)	Keyboard Scan. See 4.1.1
P_LFT	137 (89H)	Backspace. Sets current column position back 1 for selected device. (System Variable updated is S POSN, SPOSNL, or P POSN for Screen, Lower Screen or Printer respectively.)
P_RT	138 (8AH)	Outputs a space to currently selected device.
P_NL	139 (8BH)	End-of-Line. Sets current position to start of next line if screen, or outputs printer buffer if printer.
PUTMES	140 (8CH)	output message to currently selected device. DE points to base of message table which contains variable length ASCII coded messages. The first byte of the table and the last byte of each message must have the most significant bit set. Register A contains the message number, numbered from 0 upwards.
K_CLS	141 (8DH)	CLS command. Executes both CLS and CLLHS.
SCRL	142 (8EH)	Scrolls entire screen (primary display file) up 1 line.
F_PNT	143 (8FH)	POINT function. Processes X,Y parameters from Calculator Stack to BC. Returns unsigned integer value = 0 or 1 on Calculator Stack reflecting state of pixel at coordinates X/Y.
DRAWLN	144 (90H)	Same as DRAW L but enter with BC register containing coordinates, B=Y and C=X.
PUT_LN	145 (91H)	Output Line Number as 4 digits, right aligned and space filled to currently selected output channel. HL points to MSB of Z-byte Line Number.

4 System I/O Guide

4.1 I/O Channels

The TS 2068 software architecture supports up to 19 I/O Channels or "Streams", numbered from -3 through 15. Those numbered less than 0 are "hidden" or reserved for system use; Channels 0 through 15 are available for assignment via the OPEN # command which has the following format:

OPEN # n,s

where n is the Channel number (0-15) and s is the Device Specification, e.g. "K" (keyboard), "S" (screen) or "P" (printer).

Channels 0 through 3 are initialized at power-on or execution of a NEW command to support the standard system devices and character I/O functions as shown in Figure 4.1-1. Channels 4-15 are considered "Closed". You can re-assign the standard I/O, e.g. OPEN # 2,"P" will direct all PRINT and LIST commands to the 2040 Printer instead of the screen. You can also assign Channels 4-15 and then direct I/O by including the Channel number (or a variable equated to the channel number) in the I/O statement, e.g. PRINT # Above Support for other than the standard system devices described is not implemented in the original version of the TS 2068 and attempts to OPEN Channels or "Streams" using other than the standard device specifications ("K", "S" or "P") will result in an error message. One possibility for adding BASIC support for new devices is to intercept the I/O error on OPEN and other commands such as CAT and FORMAT via ON ERR and interpret the BASIC program line using your own machine code routines.

Figure 4.1-1			
Channel/ Stream #	Status	Device Specification	Command/Function
-3	Reserved for OS	"K"	Keyboard/Lower Screen
-2	Reserved for OS	"S"	Main Screen
-1	Reserved for OS	"R"	RAM Write (not used)
0	User available	"K"	Output to Lower Screen
1	User available	"K"	INPUT command
2	User available	"S"	PRINT/LIST commands
3	User available	"P"	LPRINT/LLIST commands

The Channel architecture is implemented by a number of tables located in both ROM and RAM.

- A. STRMS STRMS is a 38 byte table (2 bytes for each of the 19 channels) located in the System Variables area beginning at 23568 (5C10H). It is initialized at power-on or NEW to the following values:

Location	Value	Channel	Notes
5C10H	0100	Channel -3	Copied from SMINIT in module EDIT of the Home ROM
5C12H	0600	Channel -2	
5C14H	0600	Channel -1	
5C16H	0100	Channel 0	
5C18H	0100	Channel 1	
5C1AH	0600	Channel 2	
5C1CH	1000	Channel 3	
5C1EH	0000	Channel 4	
***	***	***	Remaining streams
5C34H	0000	Channel 15	

This table is accessed using $((\text{Ch.\#} * 2) + 16\text{H})$ as an index added to 5C00H. The 2-byte value in the table is an index into the CHANS area of memory which contains the addresses of the I/O routines for the selected channel. If the 2-byte value is zero, the Channel is closed. The STRMS table is modified via the OPEN # and CLOSE # commands. When a Channel is OPENed, the device specification is used to obtain the 2-byte value to be inserted. This value is taken from the table STRMINIT in module EDIT of the Home ROM. When Channels 0 through 3 are CLOSEed, the values are restored to those used at power-on time. All others are cleared to zero.

- B. CHANS The CHANS System Variable at 23631 (5C4FH) contains the address of a 21-byte table initialized at power-on or execution of a NEW command to support "stream" I/O to the four standard system devices ("K", "S", "R" and "P"). Each table entry is 5 bytes long and is indexed by the value obtained from the STRMS table added to (CHANS)-1. Each entry has the following format:

Output Routine Address	2 Bytes
Input Routine Address	2 Bytes
Device Specification	1 Byte

This table is copied from CHINIT in module EDIT of the Home ROM. The last byte of the table contains an 80H which will immediately precede the first line of the BASIC Program (PROG). Whenever an I/O operation is performed, the appropriate Channel is "selected", i.e. its number is used as an index into STRMS to obtain the offset into the CHANS table. This offset is added to (CHANS)-1 and the resultant pointer is loaded into the System Variable CURCHL for use by the next character I/O operation (WRCH/RDCH). The device specification from CHANS is used to find and execute the initialization routine in SELTAB.

- C. SELTAB The Select Table is located in the EDIT module of the Home ROM and contains offsets to device dependent initialization routines for the standard devices "K", "S" and "P".
- D. SPEC_T The Specification Table is located in the CHANS module of the Home ROM and contains offsets to device dependent OPEN routines for the standard devices "K", "S" and "P". It is accessed whenever an OPEN # is executed.
- E. CL_TAB The Close Table is located in the CHANS module of the Home ROM and contains offsets to device dependent CLOSE routines for the standard system devices "K", "S" and "P". It is accessed whenever a CLOSE # is executed.

The following sections describe the standard system I/O devices supported via Channel I/O.

4.1.1 Keyboard

The low-level routines supporting keyboard input are executed every 1/60 of a second out of the Interruption Handler (Location 56 (38H)). The controlling routine is labelled UPD K. This routine calls K SCAN to determine if any key(s) are currently being depressed, controls the debouncing and repeat algorithms, calls K BASE to determine the Base Code, calls CHCODE to translate the Base Code based on Mode (e.g. "K", "G" or "E" Mode), and finally, stores the resultant keystroke code in LAST K and sets the flag KEYHIT. Figure 4.1.1-1 illustrates the mode control variable and associated flags and Figure 4.1.1-2 contains flowcharts of the keyboard support routines.

The character input routine associated with Device Spec. "K" is labeled IN K. The entry address is obtained using the pointer in CURCHL when Channel 1 has been Selected and the Character I/O Input routines RDCH/INCH are executed. The IN K routine tests the KEYHIT flag to detect the presence of input from the keyboard. When the KEYHIT flag=1, the contents of LAST K are returned to the requestor.

Keyboard flowcharts here

4.1.2 Video Screen

The TS 2068 system software supports I/O in the primary display file only. See Section 2.1.10 for the display file organization. The screen, which is 32 columns X 24 lines, is partitioned into two parts, the main or upper screen (22 lines) and the lower screen (2 lines). The lower portion of the screen is used for output of system messages and to echo input from the keyboard of BASIC commands, BASIC program lines, or data. The lower screen expands as needed for multi-line input, scrolling the entire screen upwards. The variable DF SZ reflects the number of lines in the lower screen (default=2).

Character output to the screen is done using the Channel I/O described in Section 4.1 using device specification "K" for the lower screen and "S" for the upper screen. Each character is defined by an 8 X 8 group of pixels. The 8 bytes needed for each of the 133 characters supported by the TS 2068 are located as shown in Figure 4.1.2-1. Note that by constructing your own pixel data and placing (base address-100H) into CHARS, you can define your own character set.

Associated with each character position is an Attribute Byte controlling the background (PAPER) color, the foreground (INK) color, the intensity (BRIGHT), and whether the position is constant or alternates between true and inverse video (FLASH). Two other "attributes", OVER and INVERSE, are implemented by software at the time the character(s) are placed into the display file.

Figure 4.1.2-1 TS2068 Standard Character Tables			
Character Set	Nr Chars	Char Codes	Location
Standard	96	32-127 (20-7FH)	Home ROM (3D00-3FFFH) (Address-100H in CHARS)
Std. Graphics	16	128-143 (80-8FH)	Dynamically Generated by Software
User Defined Graphics	21	144-164 (90-A4H)	Home RAM (Address in UDG)

The screen output routine, SENDTV, is in Module IO 1 of the Home ROM. This routine is used for output to both the screen (upper and lower) and the dot matrix printer. The following sequence illustrates the major operations involved in executing a PRINT "A" statement:

1. Channel 2 is Selected (normal assignment assumed)

loads CURCHL with pointer into CHANS area for Channel 2 (first 2 bytes are address of Output Routine - SENDTV).

clears printer and lower screen flags

sets ATTR T to values based on ATTR_P (current "permanent"*attribute values are transferred to the system variable used by the screen output routine). If the

PRINT statement contained temporary attribute controls, they would override the settings established via Select.

2. The character code for "A" (65/41H) is placed in Register A and a RESTART 16 (10H) is executed (WRCH). This jumps to SENDCH in module EDIT of the Home ROM which passes control to the SENDTV routine based on (CURCHL).
3. The registers are loaded from the System Variables with the current Row/Column position (S_POSN) and Display File address (DF_CC) for the main screen.
4. The character code is determined to be from the standard character set so the registers are loaded with the address from CHARS and the offset to the pixel pattern for "A" is calculated using the character code X 8 (shift left 3 places).
5. The first pixel row (8X1) from the character table is copied to the display file. The character table address is incremented by 1 and the display file address is incremented by 256 (100H). The next pixel row (8X1) is copied to the display file. This process is repeated until the 8 pixel rows have been copied. Masking of the data going into the display file is done based on the flags from P_FLAG thus controlling the OVER and INVERSE attributes.
6. The attribute byte controlling the character position just written is updated based on the value in ATTR_T and other flags.
7. The variables S POSN and DF CC are updated to reflect the nexfscreen position and return is made from the WRCH operation.

In the above sequence, if the print position for the "A" had started a new line following the 22 lines of the main screen, the SCROLL? prompt would have been outputted to the lower screen and, assuming a positive response, the upper screen would be scrolled up 1 line, a blank line inserted at the bottom of the upper screen, and the "A" printed at the start of the new line.

Graphics I/O using pixel coordinates is supported in the primary display file by the PLOT, DRAW and CIRCLE commands. The Home R O M module GRAPHS contains the major routines which implement these commands. They are limited to the 22 lines of the upper screen (256 X 176 pixels).

Figure 4.1.2-2 shows the internal representation used to designate row (line) and column positions. See Section 2.1.10 for details on the organization of the Display Pixel and Attribute Files. See Section 5.2 for details on software support necessary for the advanced video modes.

FIGURE 4.1.2-2
Here

4.1.3 2040 Dot Matrix Printer

Character output to the 2040 Printer is handled by the same routine used for the screen, SENDTV. When the Printer Flag=1, set by initialization for device "P", the pixel data is written into the Print Buffer instead of into the Display File. There is no Attribute Byte. The "attributes" OVER and INVERSE which are software controlled can be active. Since the Print Buffer is always precleared to zeros, OVER has no effect. INVERSE works exactly as it does for the screen, i.e. INK pixels are zero and PAPER pixels are 1.

The Print Buffer is located at 23296 (5B00H) and is 256 (100H) bytes long, the data needed to print one line of 32 characters, each character comprised of 8 bytes (8 X 8 pixels/character). The buffer is cleared to zeros and the flag PRLEFT set to zero at power-on time (or execution of a NEW command). The PRLEFT flag is set to 1 whenever pixel data is written to the buffer. This flag is used when exit is made from a program to print any unprinted data prior to program termination. As the pixel data for a particular character is entered into the buffer, the buffer address is incremented by 32 (20H); the sequential data in the buffer therefore represents 8 complete scan lines of 32 characters. When the Print Buffer is full, or upon processing an End-of-Line (0DH), or at program termination, the contents of the buffer are written to the Printer, the buffer is cleared and the PRLEFT flag is set to zero.

Printer I/O is done via Port 0FBH, but the Printer responds to any I/O Read/Write with Address Bit 7=1 and Address Bit 2=0. Therefore, any Port providing this combination, e.g. Ports 0FA through 0F8 and Ports 0F3 through 0F0 as well as others, will interface to the Printer. See Section 2.1.13.3 for the bit definitions for Printer I/O. The pixel data is written to the device by the routine PRSCAN in module IO_2 of the Home ROM which outputs 1 scan line (32 bytes), one bit at a time on each call to the routine.

There are two controlling routines for output to the printer. DUMPPR is called from SENDTV based on buffer full or End-of-Line control. This routine will call PRSCAN 8 times to output the 256 bytes of the Print Buffer (8 scan lines). The other routine is K DUMP which implements the COPY command. This routine calls PRSCAN 176 times to write the contents of the primary display file for the main screen to the printer (8 X 22). All of the low level print routines are in module IO_2 of the Home ROM.

4.2 Cassette Tape

Tape I/O is done via Port 0FEH. An I/O read of Port 0FEH pulls in the cassette input on Bit 6. An I/O write of Port 0FEH Bit 3 controls the tape output with Bit 3 = 1 generating a high output and Bit 3 = 0 generating a low output. Data is written to the tape under software control creating the following frequencies and format:

Sync Pattern of 4032 cycles at 806.5 Hz. (5 sec.)

Header: 17 bytes of data identifying the following data block as either Program, Number Array, Character Array, or Binary Code and containing other control information.

The header is written as Data, i.e. the Most Significant Bit first in each byte, 1 cycle at 2040 Hz. for a Zero and 1 cycle at 1020 Hz. for a One. The first byte is zero identifying the header. The final byte is a Checksum calculated by XOR of all preceding data bytes.

Software delay of approximately 835 milliseconds.

Sync Pattern of 1612 cycles at 806.5 Hz. (2 secs.)

Transition Pattern of 1 cycle at 2400 Hz.

Data Block: Written as Data (see above) with first byte = -1 (FFH) and a final Checksum byte.

Figure 4.2-1 shows the header formats for the various types of data.

The routines used to actually write and read the tape (W TAPE and R TAPE) are in the TAPE Module of the Extension ROM (see map in Appendix A). They are accessible via the Extension ROM Interface Routine listed in Figure 3.2.2-2. The general flow required to write a header and data block is:

1. Call W TAPE with A=0. IX contains the address of the header and DE contains the length.
2. Delay loop approximately 1 second.
3. Call W TAPE with A=FFH. IX contains the address of the data block and DE contains the length.

The R TAPE routine performs either a LOAD (transfers data from tape to memory) or VERIFY (compare data from tape against data in memory) operation, based on the status at entry: Carry Set for Load and No Carry if Verify. As for the Write, A=Block Type (0 for Header and -1 (FFH) for Data Block). IX contains the memory address.

and DE contains the length of the block to be read (DE = 17 for the header and DE = HDLEN for data). See Fig. 4.2-1 for a definition of HDLEN.

The tape routines return Carry=1 for successful completion and No Carry for error or Break Key detected, Roth W TAPE and R TAPE exit via the routine W BORD which restores the Border color based on bits 3-5 of the system variable BORDCR. If the Break Key is detected during this exit routine, a RESTART 8 (ERROR) is executed.

NOTE: The write to Port 0FEH in the exit routine restoring the Border Color has bit 3 = 0. This creates a final transition on the tape following a write operation. This transition is necessary in order to successfully read back the final data bit from some tape recording devices. If you are calling the W TAPE routine so as to bypass the normal exit path, you must perform this final write to Port 0FEH with Bit 3 = 0 within a similar timeframe.

FIGURE 4.2-1 Tape Header Formats				
Header layout - 17 bytes long				
HDTYPE	HDNAME	HDLEN	HDADD	HDVARS
1 byte	10 bytes	2 bytes LSB/MSB	2 bytes LSB/MSB	2 bytes LSB/MSB
Field values for the tape file types				
Program				
HDTYPE	0			
HDNAME	Up to 10 ASCII characters			
HDLEN	Length of program + variables (E_LINE) - (PROG)			
HDADD	Start line number e.g. 0500 = line 5 or 0080H if no start line number			
HDVARS	Length of program = Offset to Variables (VARS) - (PROG)			
Number array				
HDTYPE	1			
HDNAME	Up to 10 ASCII characters			
HDLEN	Length field from data structure			
HDADD	LSB = 00, MSB = Array ID (100x xxxx binary vale) where x xxxx is (ASCII - 60H)			
HDVARS	N/A = 0			
Character array				
HDTYPE	2			
HDNAME	Up to 10 ASCII characters			
HDLEN	Length field from data structure			
HDADD	LSB = 00, MSB = Array ID (110x xxxx binary vale) where x xxxx is (ASCII - 60H)			
HDVARS	N/A = 0			
Code (Binary)				
HDTYPE	3			
HDNAME	Up to 10 ASCII characters			
HDLEN	Length specified in SAVE			
HDADD	Address specified in SAVE			
HDVARS	N/A = 0			

4.3 Joysticks

The two joysticks are controlled via Register 14 (I/O Port A) of the Programmable Sound Generator Chip (see Sections 2.1.6 and 2.1.7). Address and data are passed via Ports 0F5H and 0F6H respectively. The joysticks are read by first addressing Register 14 in the PSG by writing a 14 (0EH) to Port 0F5H. The data is then read by executing an IN from Port 0F6H, having the port address in 280 Register C and the joystick (player) number in Register B (number = 1 or 2). Note that PSG Register 7, Bit 5 is assumed to be zero, enabling I/O Port A for input. If you ever use I/O Port A for output (R7,B6=1), you will want to clear Bit 6 prior to any input operation,

Sample routine:

```
GETJOY      LD A,0EH          ;Load A = 14
            OUT A,(0F5H)      ;Address the joystick port
            LD B,playerno
            LD C,0F6H         ;Data Port address to C
            IN A,(C)          ;Joystick data to A
            CPL               ;Complement to High Active
            AND 8FH           ;Get significant bits
```

The data read is LOW ACTIVE, i.e. all bits = 1 (byte=FFH) when the stick is at center and the button is not depressed. Figure 4.3-1 shows the interpretation of the data byte.

FIGURE 4.3-1 Joystick Data	
Bit	Function
7	BUTTON DEPRESSED
6	Unused - always '1'
5	Unused - always '1'
4	Unused - always '1'
3	STICK RIGHT
2	STICK LEFT
1	STICK DOWN
0	STICK UP

4.4 Software Generated Sound (BEEP)

The BEEP command produces sound using the speaker by toggling Bit 4 of I/O Port 0FEH to generate a signal of a calculated frequency and duration based on the command parameters. It uses the routine PARP which takes as input two parameters, one defining the period of the signal (HL) and the other defining the number of cycles to be generated (DE) and outputs DE+1 cycles of a tone having the period $8N+236$ to $8N+246$ T-States where $(HL) = N$. Both the BEEP and PARP routines are in the K SCAN module of the Home ROM. The PARP routine is also used to generate the keyboard "click" and the

"raspberry" which can be varied by modifying the values in the system variables PIP (23609/5C39H) and RASP (23608 5C38H).

4.5 Sound Chip

The SOUND command writes the first parameter (register number) to Port 0F5H (address to Programmable Sound Generator) and the second parameter (load data) to Port 0F6H (data to PSG). The program line is scanned for multiple parameter pairs and continues writing address/data pairs to the PSG until the end of the statement is reached. See Section 2.1.6 for details on the hardware of the PSG.

5 Advanced Concepts

5.1 Cartridge Software/Hardware

5.1.1 LROS

An LROS is identified by the following overhead bytes:

Location	Description
0000	Not Used
0001	Cartridge Type 01=LROS
0002/0003	Starting Address (LSB/MSB) Address to be jumped to after Operating System initialization is complete. Order of bytes is as for a JP instruction.
0004	Memory Chunk Specification. Bits 0-7 represent Chunks 0-7 respectively in the Dock Bank in low active format: 0 if in use 1 if not in use
NOTE:	When writing to the Horizontal Select Register (Port F4H), the Chunk Specification is High Active

The Memory Chunk Specification is used to enable the specified chunks in the Dock Bank prior to jumping to the address specified in Location 2 and 3. Control is transferred from the Initialization code in the Extension ROM via the GOTO BANK routine in Home Bank RAM Chunk 3, therefore Bit 3 of the Memory Chunk Specification must be set to 1 in order for the transfer to be accomplished as designed (Chunk 3 also contains the Machine Stack).

CAUTION: If Chunk 3 is marked for use in the Dock Bank, then when the Memory Chunk Spec. is written to Port F4H by the Sank Enable code, execution will continue from that point in Chunk 3 in the Dock Bank with the Stack Pointer addressing ROM.

An LROS is Z80 machine code and is in complete control of the TS 2068 hardware after transfer to the starting address has been made. It can directly implement an application, or it can support multiple applications by implementing a language other than BASIC. An AROS dependent on such an LROS would have to be part of the same cartridge since there is only one cartridge connector.

Interruption Mode 1 has been set by the TS 2068 and interruptions are enabled prior to passing control to the LROS starting address, therefore the LROS must contain appropriate code at location 56 (38H) to cover the case where the interruption occurs after Chunk 0 in the Dock Bank has been enabled, but before any action by the software cartridge to disable the interruption has been taken. Once control is transferred, the LROS may then disable the standard TS 2068 interruption by setting bit 6 of Port FFH, mask the interruption by executing a DI instruction, or set a different Interruption Mode. It may change the location of the Machine Stack. It may also change the memory selection by writing to Port OF4H with each bit set to 1 for the corresponding chunk to be enabled in the Dock Bank (high active format) or 0 to be enabled in the Home Bank. Thus, an LROS may contain code in Chunk 3, but it should be enabled after the OS RAM code has finished execution.

Now that your LROS is in the driver's seat, you are on your own! Some important points to remember when,, mapping your Dock Bank memory and doing bank switching are:

1. The Display RAM is in Home Bank Chunk.2 for the primary display file and Chunk 3 for the second display file. This memory is accessed independently by the video hardware. The software only needs to enable it when actually reading or writing it.
2. The Dock Bank and Extension ROM Bank are mutually exclusive since they share the Horizontal Select Register in Port F4H. You will need a routine in the Home Bank RAM to do any switching between the two. You must also be careful to have the appropriate Home Bank Chunks enabled which are referenced by the Extension ROM code, e.g. the System Variables in Chunk 2 or possibly the bank switching code in Chunk 3.
3. Some interesting switching routines can be constructed by having parallel code in shadowing chunks of memory to take advantage of the "instant" switch in execution from one bank to another when the memory selection is made. E.g., a routine in the Dock Bank ROM in Chunk 6 could push a Home Bank address on the stack, write to Port F4H enabling Chunk 6 and any other desired chunks in the Home Bank (by deselecting them in the Dock), and have code at the next sequential instruction address in Home Bank RAM Chunk 6 to continue the path. A Return instruction, for example, would pass control to the address on the stack. Code to switch memory back to the Dock Bank could be mapped in a similar way.
4. If you plan to use any of the System software routines, unless you know otherwise it is probably necessary to maintain the contents of Home Bank Chunks 2 and 3

intact (and Chunk 7 if the OS RAM routines have been relocated). The system routines rely heavily on the System Variables and assume that any pointers in them are pointing to the Home Bank. See Section 3.3.4.1 for details on using the RAM Interruption Handler and Section 6.0 for known corrections when using System S/W.

5. If you design an LROS implementing a higher-level language and want to support an AROS application, you must design your own initialization code to detect the presence of such an AROS. The TS 2068 will not look for the presence of an AROS if an LROS is present, therefore there will be no entry for the AROS in the System Configuration Table. Note that since there is only one cartridge connector, such an AROS would also have to be integrated with the supporting LROS in a single cartridge or cartridge board.

5.1.2 AROS

An AROS is identified by the following overhead bytes

Location	Description
32768 (8000H)	Language Type 1 = BASIC [and machine code] 2 = Machine code only (Any other value will result in Error S, Missing LROS)
32769 (8001H)	Cartridge Type 2 = AROS
32770/32771 (8002/8003H)	Starting Address(LSB/MSB) BASIC AROS = Address of First Program Line Machine Code AROS = Address of First Z80 Instruction
32772 (8004H)	Memory Chunk Specification Bits 0-7 represent Chunks 0-7 respectively in the Dock Bank in low active format as follows: 0 if in use 1 if not in use NOTE: Bits 0-3 must be set to 1 for proper execution.
32773 (8005H)	Autostart Specification 0 = No Autostart 1 = Autostart

32774/32775 Number of bytes of RAM to be
(8006/8007H) Reserved for Machine Code
 Variables (LSB/MSB - 0100H=1 byte
 Reserved; 0002H=512
 bytes Reserved.

5.1.2.1 BASIC AROS

A BASIC AROS is supported by special code in the System ROM (Section 3.2.1.2). The portion of the cartridge containing BASIC program lines is restricted to the upper half of the memory space beginning at location 32776 (8008H) in the Dock Bank. Support for User-Defined Functions, which requires searching for the definition parameters within the program, is not implemented. Also, because the support code interfaces directly to the bank switching code in Home RAM Chunk 3 (does not allow for it to be relocated to Chunk 7), a BASIC AROS cannot utilize the advanced video modes and also execute BASIC program statements. If the cartridge contained machine code supporting advanced video modes, the TS 2068 would have to be returned to "Normal " video mode with the RAM mapped accordingly (see Figure 1.1-3) if control were to be returned to the BASIC Interpreter USR code.

Since execution of the cartridge BASIC program is done by copying program lines to a buffer in the Home Bank RAM (ARSBUF), the most efficient cartridge execution is obtained by making program lines as large as possible, i.e. making use of the multi-statement feature of the TS 2068. The reverse is true concerning execution of READ commands. An entire DATA statement is copied to the Home Bank RAM, but only the current item is accessed. It therefore will be more efficient to not make DATA statements excessively long. The BASIC program lines appear in the cartridge in exactly the same format used in the RAM, i.e. Line Number (2 bytes), Length (2 bytes), Command Token, etc. terminated by an Enter (0DH). Numerical constants appearing in a program line are followed by the CHR\$(0EH) byte and 5-byte floating point format described in the User Manual (see Appendix C of the TS 2068 User Manual). The Variables area is built in the RAM (address in VARS) exactly as though the program were in the RAM. All variables, including arrays, are built at the time of program execution - there is no provision for copying or accessing pre-defined variables from the cartridge, however, see Section 5.3.2. The last program line must be followed by a terminator byte having the Most Significant Bit set (e.g. 80H), otherwise the Interpreter cannot detect the end of the program.

A BASIC AROS may contain machine code accessed via the USR function. If the machine code address is within the memory designated by the AROS Memory Select Specification as 'in use', the Dock Bank will be enabled, otherwise the machine code address is assumed to be in the Home Bank. (See Section 6.0 for details on known problems in this area of the code.) Obviously, once control is transferred to the machine code in the AROS, the ball is now in your court. You could have additional machine code residing in the lower half of the Dock Bank memory space which you can now switch in.

You only have to know what you're about. If and when you are ready to go back to executing your BASIC program, you must enable Chunks O-3 in the Home Bank and have the stack and other Home Bank RAM in the proper state for return to the USR function code in the BASIC Interpreter, i.e. what it was when the USR function passed control to you.

The Autostart feature begins execution out of the BASIC AROS immediately after system initialization. If the Autostart parameter is zero, control will go to the BASIC Interpreter as if there were no cartridge installed, although internal flags have been set noting that a BASIC AROS is present. The cartridge will be started when you execute a RUN or GOT0 Line Number command.

The final parameter in the overhead bytes allows you to reserve RAM beginning in Chunk 3 at Location 26688 (6840H) for machine code and/or machine code variables. The designated number of bytes are reserved by the AROS support code prior to beginning program execution. The AROS buffer (ARSBUF) begins immediately following this reserved area (see Fig. 1.1-3). Note that this area is part of the RAM that gets relocated if the second display file is opened. Therefore access to your machine code and/or variables should be conditional on the video mode rather than direct if you are going to be using the advanced video modes,. This reserved area begins at 31488 (7B00H) when the second display file is open. Remember -- use of the second display file and execution of BASIC program from the cartridge are mutually exclusive.

The standard technique of reserving space for machine code by modifying RAMTOP could also be used to place machine code/variables at the top of the Home Bank RAM. If you place code above (RAMTOP) which is to be accessed via the BASIC USR function, the affected memory chunk(s) cannot be marked as "in use" in the cartridge in the AROS Memory Selection Specification.

5.1.2.2 Machine Code AROS

A machine code AROS is similar to an LROS with the exception that it is dependent on the System ROM for interruption handling if the interruption is enabled. This implies that Chunks O-3 are enabled in the Home Bank.

The Autostart parameter should be set to 1 since if it is zero, control will be passed to the BASIC Interpreter as if the cartridge were not present. There is no BASIC command to directly start execution of a Machine Code AROS.

Because of a "bug" in the Initialization code handling a Machine Code AROS, the parameter specifying the number of bytes to be reserved for machine code variables must be adjusted by adding 21 (15H) to the actual number of bytes needed. This preserves the 21 byte CHANS area starting at 26688 (6840H). The reserved area then starts at 26709 (6855H) (or 31488 (7B15H) when the second display file is open). Access to the variables should be conditional based on the video mode rather than direct if you plan to

use the advanced video modes. If you do not plan to utilize any of the system software, you can disregard the above and "do your own thing" with the RAM.

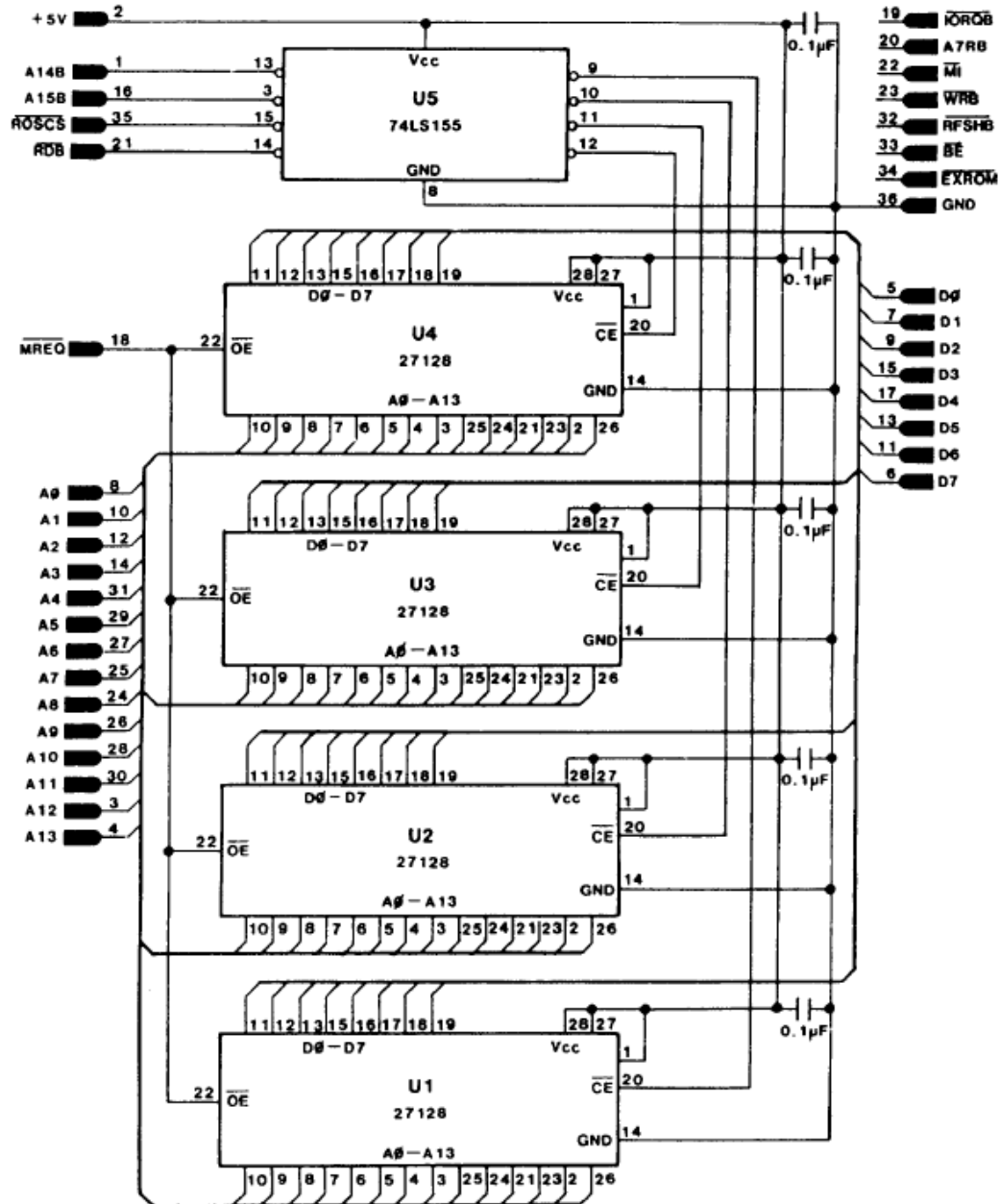
See Section 6.0 for known corrections when using System S/W.

5.1.3 EPROM Cartridge Board Application

Figure 5.1-1 provides the logic diagram for a pluggable EPROM cartridge board capable of configuring up to four 16K-byte (128K-bit) EPROM's of the 27128 type. The artwork for the PC board implementing that logic diagram is provided in Figures 5.1-2, 5.1-3 and 5.1-4 for the Component Side art, the Solder Side art, and the Solder Mask (one common mask for both sides), respectively.

See Section 2.4.2 for mechanical details of the connector portion of the PCB.

FIGURE 5.1-1
PLUGGABLE EPROM CARTRIDGE BOARD
LOGIC DIAGRAM



5.2 Advanced Video Modes

The following sections describe the various video modes available on the TS 2068 and the major software support functions necessary. See Sections 3.2.2.3 and 3.2.2.4 for details on using the Video Mode Change Service. Appendix C contains descriptions and code listings for a number of software packages developed by Timex that support various screen modes and applications. Reference to these packages should aid in gaining an understanding of the software techniques needed to support the video mode hardware. The TS 2068 video mode hardware works out of two areas of RAM, the primary display file at 4000H and the second display file at 6000H. Each area consists of 5912 (1B00H) bytes used for pixel and/or attribute data based on the mode selected via bits 0-5 of Port FFH. The pixel data area divides into three blocks, each supporting 8 contiguous lines on the screen. See Section 2.1.10 for details on organization of the display RAM. Because the two display files occupy the same relative positions within their respective 8K Chunks, by setting/clearing Address Bit 13 a software routine can address the corresponding location in each file:

Address: DF1															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	X	X	X	X	X	X	X	X	X	X	X	X	X
4000H - 5AFFH (Bit 13 = 0)															

Address: DF2															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	X	X	X	X	X	X	X	X	X	X	X	X	X
6000H - 7AFFH (Bit 13 = 1)															

In order to display a character on the screen, 8 bytes of pixel data must be entered into the display file, one for each scan row. For a particular character position, the scan rows are 100H bytes apart. E.g, the 8 bytes of pixel data for position Line 0/Column 0 are located at 4000H, 4100H, 4200H ,.....,4700H. Since this is the first character position on the screen, its Attribute byte, in Normal Mode, is the first byte in the Attribute File which starts at 5800H. The 768 (300H) Attribute Bytes are in sequential order starting at position 0/0 through 0/31, 1/0 through 1/31, and so forth, ending with 23/0 through 23/31.

One method of determining the starting display file address for a particular line/column position is to build a table containing the starting address of each of the 24 lines (2 bytes per entry). Then construct an algorithm that takes the line number and forms an index by multiplying it by 2 (shift left 1), add the index to the base address of the table, and read out the display file address. The column position is then simply an offset added to this address. By testing VIDMOD (23746 - 5CC2H) you can determine whether to set Bit 13 for the second display file, e.g. because you are in an odd column in 64-column mode, or simply because you are using the second display file in dual screen mode.

The following example illustrates this method. The table entries are in Hex:

Line #	Index	Table LSB/MSB	Comment
0	0	00 40 (4000H)	Line 0 (top of screen)
1	2	20 43	Line 1
2	4	40 40	Line 2
***		(+20H)	
7	14 (0EH)	E0 40	Line 7 (end of upper block)
8	16 (10H)	00 48 (4800H)	Line 8 (top of middle block)
9	18 (12H)	20 48	Line 9
***		(+20H)	
15	30 (1EH)	E0 48	Line 15 (end of middle block)
16	32 (20H)	00 50 (5000H)	Line 16 (top of bottom block)
17	34 (22H)	20 50	
***		(+20H)	
23	46 (2EH)	E0 50	Line 23 (end of bottom block)

Line 17, Column 23 (11H/17H) would yield a display file address of 5020H + 17H = 5037H. If VIDMOD indicated the second display file was to be used, setting Bit 13 of the address would yield 7037H. If we were using 64-column mode, because the column is odd (Bit 0 = 1) we would set Bit 13 of the starting line address getting 7020H, then divide the column address by 2 (shift right 1) since there are only 32 columns in each display file. This would give us an offset of 11 (0BH) which added to the starting address results in a display file address of 702BH. Having the display file address, we now insert the 8 bytes of pixel data for the character desired, incrementing the display file address by 100H between each write (this is easily done by simply incrementing the upper register of the register pair containing the address). The following routine is a simplified version illustrating this process. It assumes that Reg. Pair DE contains the address of the desired character in the character table and that HL contains the address of the desired position in the display file.

```

        LD    B, 8
LOOP    LD    A, (DE)
        LD    (HL), A
        INC  DE
        INC  H
        DJNZ LOOP

```

Finally, we must update the Attribute Byte controlling the updated character position. The following sample algorithm will formulate the Attribute File address given the address of any of the scan rows of the character position. We will assume we have saved off the starting display file address and now have it in Register Pair HL.

```

GETATT    LD A,H      ; MSB of DF Address
          RRCA        ; Shift right circular
          RRCA        ; to get Bits 3 & 4 (Block #)
          RRCA        ; to positions 0 & 1
          AND 3       ; Clear other bits
          OR 58H      ; OR in Attribute file base address
          LD H,A      ; Update MSB

```

NOTE: The LSB is the same as for the pixel data.

Using our first example, with a Display File address of 5037H, the Attribute File address would be 5A37H. The second example was using 64-Column Mode which does not require attribute file update (attributes determined by video mode setting).

See Section 5.2.2 for a sample algorithm to formulate the display file address for X,Y pixel coordinates. The above routine for calculating Attribute File address would be substituted for the method used in the example if not working in High Resolution Graphics mode.

In addition to data insertion, two major screen support functions are scrolling and clearing the screen. Scrolling is done in the System ROM by copying the entire display file data and attribute controls up one line position (Line 1 to Line 0, Line 2 to Line 1, etc.) and inserting a blank line at the bottom. Numerous more elaborate scrolling techniques can be implemented using various directions (up, down, left, right) and smaller areas or "windows" of the screen. Similarly, clearing the screen, which consists of writing zeros to the data file and updating the attribute bytes to a uniform value, can be implemented on smaller sections of the screen. The software packages in Appendix C contain examples of such implementations.

5.2.1 Dual Screen Mode

In this mode the second display file is used to provide a second independent screen having the same data and attribute organization as the primary display file. By writing to Port FFH with Bits 0-5 = 1 (Bit 0 set), the second display file is activated at the video screen. Appendix C contains a software package supporting Dual Screen Mode. The software package uses the system variable VIDMOD to determine which display file is the target of the current operation. Special values for VIDMOD have been defined to permit building of one display file while the other is active at the screen so that a complete screen image is ready when the hardware mode is changed. Copy and Exchange routines have been provided to move data within and between the two display files. This enables the BASIC graphics commands like PLOT, CIRCLE and DRAW, which work only in the primary display file, to be used to create screens which are then moved into the second display file.

Because the System ROM works only in the primary display file, you can come up with some unusual situations when you have the second display file active at the screen and you are executing BASIC or using the System ROM routines. If an error occurs, for example, the error message will be placed into the primary display file and the ROM will be waiting for input from the keyboard to direct the next action, but all of this is invisible since you have the other display file active. The machine will appear to be "hung", but it is only doing its normal thing. Be prepared to enter a OUT 255,0 to an invisible command line in order to switch the display back to the standard file!!! Don't forget to also set VIDMOD (POKE 23746,128) to keep things consistent inside the dual screen support code.

5.2.2 High Resolution Graphics Mode

This mode is set by writing to Port OFFH with Bits 0-5=2 (Bit 1 set). In this mode, also called Extended Color Mode, the second display file is used to expand the number of Attribute bytes from one for each 8 X 8 pixel group to one for each 8 X 1 pixel group thus giving 32 X 192 positions within each of which two colors plus Bright and Flash can be defined. Each byte of pixel data entered into the primary display file has its own Attribute byte in the corresponding location in the second display file, e.g. the byte written to Location 4000H has its Attribute byte at Location 6000H, the byte at 47FFH (last byte of last scan row in Line 7) has its Attribute byte at Location 67FFH, the byte at 57FFH (last byte of last scan row in Line 23) has its Attribute byte at Location 77FFH. The routine writing data to the screen would therefore enter the pixel data to the desired location and then set Address Bit 13 of the Primary Display File address and write the desired attribute control byte to the resultant location. If normal characters are being written to the screen in this mode, eight Attribute bytes must also be written, one for each of the bytes defining the character. The same technique would be used for writing to both display files, i.e. for each of the seven bytes entered after the first, the display file address would be incremented by 256 (100H).

The System ROM graphics commands (PLOT, DRAW and CIRCLE) place data into the Primary Display File and update the Attribute File associated with the standard video mode (5800H-5AFFH). In High Resolution Graphics Mode, the hardware does not access this area for attribute control, therefore its contents have no visible effect. If before or immediately following execution of the BASIC graphics operation, you update the attribute control information in the second display file, you could possibly take advantage of the System ROM graphics capability. Admittedly, this is not a simple operation in the case of circles or drawing diagonal lines and it will be more efficient to develop code specifically to support this video mode.

The following sample routine takes as input two single byte binary digits representing the X and Y coordinates of a pixel position on the screen. It formulates the display file address of the byte containing the pixel, creates a pattern or mask byte for the specified bit position, sets the bit in the display file, and updates the attribute byte (High Resolution Graphics Mode assumed). This represents a simplified version of the approach used in the System ROM graphics support routines PLOTBC and SCRMBL.

The two inputs are assumed to be as follows:

Reg. C = X Coordinate 0-255 (0-FFH) going left to right across the screen.

Reg. B = Y Coordinate 0-191 (0-BFH) going from bottom to top of the screen.

NOTE: This covers the full vertical range of 192 positions.

The Y Coordinate is checked for valid range and reversed directionally so that 0 represents the top of the screen and 191 represents the bottom. After this reversal, the two coordinates represent the following values:

X							
7	6	5	4	3	2	1	0
Screen Block (0 - 2)		Line number within block (0 - 7)			Scan Row within line (0 - 7)		

Y							
7	6	5	4	3	2	1	0
Column (0 - 31)					Bit (0 - 7)		

We first formulate the MSB of the display file address using the Block and Scan Line information in the Y Coordinate:

```

PLOTXY    PUSH  BC
          LD    (SAVECO), BC    ; save coordinates
          LD    A, 191          ; test Y within range
          SUB   B
          JP    C, ERROR        ; Y out of range
          LD    B, A            ; Y coordinate not 0 = top
          AND   0C0H            ; get block number
          RRA
          RRA
          RRA
          LD    H, A            ; save block bits
          LD    A, B            ; Y coordinate
          AND   07              ; Get scan row bits
          OR    H                ; combine block and scan row
          OR    40H             ; base address of DF (4000H)
          LD    H, A            ; H = MSB of DF address

```

Next we formulate the LSB of the display file address using the Line information from the Y Coordinate and the Column information from the X Coordinate:

```

LD    A, C          ; get X coordinate
RLCA          ; align to pick up line
RLCA          ; bits from Y
RLCA          ; A= 2 LS bits column/XXX/3 MS
              ; bits column
AND    0C7H        ; clear bits 3-5
LD    L, A          ; save A for later
LD    A, B          ; get Y coordinate
AND    38H          ; get line bits
OR     L            ; combine with column bits
RLCA          ; shift to final position
RLCA          ; A = line #/column
LD    L, A          ; L = LSB display file address

```

Next we get the pixel position within the byte by taking the last 3 bits of the X Coordinate and create a mask byte having all bits zero except the addressed pixel. This mask is then used to set the bit in the Display File. The address is set to Display File 2 to update the Attribute File (High Res. Graphics Mode is assumed to be active), and the routine is finished. The memory locations defined as ATTR and SAVECO are for illustration purposes only:

```

LD    A, C          ; get pixel position
AND    7            ; 0 = leftmost (MSB)
              ; 7 - rightmost (LSB)

LD    B, A          ; use as control count
INC    B            ; B = 1-8
LD    A, 00000001B  ; bit mask
LOOP  RRCA          ; rotate mask bit
      DJNZ  LOOP     ; to proper position
      OR    (HL)      ; OR bit into DF
LD    A, 20H
OR     H            ; set bit 13 for DF2
LD    H, A          ; HL = attribute file
LD    A, (ATTR)     ; get attribute byte
LD    (HL), A       ; update attribute file
POP    BC           ; original X/Y to BC regs
RET

```

Repetitive calls to this routine with the appropriate X/Y Coordinate values will "draw" on the screen. The System ROM routines for drawing lines and circles calculate the

successive X/Y Coordinate values and use common low-level routines similar to the above to place each pixel in the display file.

5.2.3 64 Column Mode

In this mode, set by writing to Port 0FFH with Bits 0-2=6 (Bits 1 and 2 set) and Bits 3-5 selecting ink color (0-7), the pixel data portions of the two display files are merged by the hardware on an alternating column basis to produce 64-columns across the screen. All even columns (0,2,4....62) are derived from the primary display file and all odd columns (1,3,5..... 63) are derived from the second display file. There are still 24 lines vertically from top to bottom. The attributes are controlled by bits 3-5 written to Port FFH selecting one of eight ink/paper combinations. The Bright and Flash attributes are fixed at 0 and the Border is fixed to match the paper color. The Attribute Files in RAM at 5800H-5AFFH (primary display file) and 7800H-7AFFH (second display file) are not utilized in this mode.

Software supporting this mode must set up the display file address for character insertion based on the column position (even=DF1; odd=DF2). When scrolling the screen (or a portion of it), any line of text on the screen requires the same operation to be done at the corresponding locations in each display file. This is also true to clear the screen (or a portion of it). To save a Screen on tape you must save two Code files, one for each display file. The SAVE filename SCREEN\$ will work for the Primary Display File only. You will have to specifically SAVE the second display file via a SAVE filename CODE 24576,6144. Note also that because the Border color is fixed by the video mode, you will not see the usual "stripes" during a tape operation.

Code to support an 80-column mode screen was developed utilizing the 64-column hardware mode and redefining the character size to a 6 X 8 pixel group (there is really room for 84 characters if the full 256 pixel width is used). Since individual characters now can span the two display files (e.g. 2 pixels in DF1 and 4 in DF2) insertion of data into the display files involves masking the 6-bit character (or portion thereof) with the 8 bits of data read/written from/to the display file.

Appendix C contains descriptions and code listings of software packages supporting 64 and 80-Column modes.

5.2.4 Other

Appendix C also contains software packages supporting the following video screen features:

- A. 40-Column Mode - utilizes the 6 X 8 character set defined for 80-Column Mode in "normal" mode. May be combined with the Dual Screen package.
- B. Sprites - supports movement of software-defined objects and multi-directional screen scrolling services in the Primary Display File. You must create the actual bit map defining the shape of your sprite(s), but this package does the rest.

5.3 Other Advanced Concepts

5.3.1 Interruption Fielding

For a machine code program executing in the Home RAM, you can intercept the 17 ms. interruption for your own purposes by permanently enabling Chunk 0 in the Extension ROM Bank (write a 1 to Port 0F4H and always have Bit 7 of Port 0FFH = 1) and inserting at Location 25262 (62AE Hex) a branch to your own interruption handler. (Or if VIDMOD is not zero, insert your branch instruction at Location 64110 (FA6EH).) By doing this you are forcing the interruption to branch to the RAM and then bypassing the OS RAM Interruption Handler - see Sections 3.7.3.1 and 3.3.3.1. Because the Video Mode Change Service automatically updates internal branch addresses in the OS RAM code when it is relocated between Chunk 3 and Chunk 7, you probably do not want to directly overlay the OS RAM Interruption Handler with your own code if you will be using the Video Mode service. Your branch instruction at 62AEH, however, will be copied unmodified to location FA6EH in Chunk 7 and vice versa.

Note that this technique cannot be used if you are using BASIC since then you must have Chunk 0 enabled in the Home Bank. It also cannot be used from a cartridge because the memory selection hardware (Port 0F4H) is common to the Dock and Extension ROM Banks and can only enable one of them at a given time as selected by Bit 7 of Port 0FFH.

5.3.2 BASIC AROS Variables

In order to use pre-defined arrays and/or other BASIC variables, store them in the cartridge (possibly in the lower half of the addressable space which is not usable for BASIC program) and branch to a machine code routine via the USR function at the beginning of your BASIC AROS program. Use this routine to do the necessary memory selection and copy your data from the cartridge to the RAM (address in VARS). Adjust the System Variables E LINE, WORKSP, STKBOT and STKEND to all point to the first free memory following your BASIC variables. Of course, all BASIC variables must conform to the format expected by the BASIC Interpreter. In addition to BASIC structures, you can also store screen images and machine code/variables in the cartridge for transfer to the RAM under your control. Consider using the XFER BYTES service in the OS RAM.

6 Known "Bugs" and corrections

This section describes the known problems in the TS 2068 System Software and gives corrections or workarounds where these have been defined.

6.1 LROS and Autostart Machine Code AROS

6.1.1 Keyboard

If you will be using the System ROM Keyboard routines and accessing the input character code from system variable LAST K (5C08H), you must initialize the TS 2068 to "L" mode by setting the system variable MODE at 23617(5C41H) to zero and setting Bit 3 of FLAGS (23611 -5C3BH) to 1. (The TS 2068 is in "K" mode when control is passed from System Initialization to the Cartridge; Keyword Token codes will be placed in LAST K instead of character codes.

6.1.2 Calculator

If you will be using the System ROM Calculator routines (RESTART 40 (28H)) or any ROM routines that invoke them, you must initialize the System Variable YEM by doing the following:

```
LD HL,5C92H      ;Set HL=MEMBOT
LD (5C68H),HL    ;Initialize MEM
```

6.1.3 Chunk 3

Chunk 3 must not be designated as "in use" by the Cartridge Memory Selection specification byte. This will cause de-selection of the bank switching code prior to completion of the transfer of control to the cartridge starting address. Once control has been transferred, the cartridge code may then enable Chunk 3 in the Dock Bank if desired. (See Section 5.1.)

6.1.4 AROS under LROS

No entry is made in the System Configuration Table for an RROS if an LROS is present. This means that an LROS designed to support either RAM based or cartridge based applications must include code for detection of an AROS.

6.2 Machine Code AROS

When setting the AROS Overhead parameter requesting RAM space for machine code variables, $21 + n$ bytes ($15H + n$) must be requested where n is the number of bytes needed. The machine language variables area then starts at 6 8 5 5 H immediately following the 21-byte CHANS area. (See Section 5.1.2.3.)

NOTE: This does not apply to an AROS that contains both BASIC and machine code.

6.3 BASIC AROS

6.3.1 USR Function

When testing the USR address against the Cartridge Memory Selection byte to determine if the address is in the Home Bank or the Dock Bank, the wrong nibble is tested in the register thus a valid cartridge address could be erroneously processed as a Home Bank address. Since the ROM code cannot be corrected, the machine code in the cartridge would have to be moved to an address that does not cause a problem.

6.3.2 FOR/NEXT

If the limit of the FOR statement has already been passed on its initial execution, (e.g. FOR A=1 TO 10 and A has been set to 12), control is passed to the statement following the corresponding NEXT. In the AROS support code, the address of this statement is lost giving unpredictable results. Since the ROM code cannot be corrected, care must be taken not to use this technique in an AROS Cartridge. Normal usage of FOR/NEXT loops is not affected.

6.3.3 Advanced Video Modes

Because the BASIC AROS support code interfaces directly to the Bank Switching code in Chunk 3 (does not access based on its relocatability), the second display file cannot be open when executing BASIC program from an AROS.

6.4 Video Mode Change Service

6.4.1 Available Memory Test

When the size of memory needed is calculated by adding the size of the second display file (6912 bytes or 1BOOH) to the memory now in use (address in System Variable STKEND), the code fails to check for overflow. Thus if the address in STKEND is greater than 58623 (E4FFH), the fact that there is not enough free memory to open the second display file will not be detected and the system will "crash". If your BASIC program and/or variables area are large, you may want to make this test yourself prior to invoking the Video Mode Change Service in order to avoid this problem. The size of memory needed is subsequently tested against the contents of RAMTOP and if there is not sufficient space (value in RAMTOP is less than size needed), you will get Error 4, Out of Memory.

6.4.2 RAMTOP

When the machine stack and OS RAM code is moved to Chunk 7, the User Defined Graphics area is moved down in RAM by 2112 bytes (840H) to make room for the stack and OS RAM routines at the top of memory. The pointer in UDG is updated, however, the value in RAMTOP is not modified to insure that the relocated UDG area as well as the OS code and stack are protected from expansion of the BASIC program. You can avoid problems by setting RAMTOP via a CLEAR command specifying an address no greater than 63255 (F717H) prior to invoking the Video Mode Change Service. This

reserves space between RAMTOP and the end of memory of 2280 bytes (8E8H) utilized as:

168 bytes (A8H)	User Defined Graphics (21 X 8)
2112 bytes (840H)	Machine Stack and OS Routines
2280 (8E8H)	

Example:	RAMTOP = 63255 (F717H)
+ Reserved Area	2280 (08E8H)
	65535 (FFFFH)

The software packages in Appendix C are written assuming that RAMTOP is set to 57343 (DFFFH) or lower to protect the machine code which is loaded beginning at 57344 (E000H).

6.4.3 New Command

If you have used the Video Mode Change Service to open the second display file and now wish to execute the NEW command, you should first return the computer to "normal" mode by calling the video mode service with A=zero. This returns the User Defined Graphics and other RAM structures to their normal locations. If you don't do this, the UDG area will remain in the alternate location and, if you have not corrected RAMTOP as explained above, part or all of your UDG area could be cleared to zeros by the NEW command.

6.4.4 VIDMOD

When Mode 128 (80H) is designated for activating the Primary Display File in Dual Screen Mode the System Variable VIDMOD at 23746 (5CC2H) is set to zero instead of to 128. This creates a potential problem if the 17 ms. interruption occurs before VIDMOD can be corrected since the interruption fielder will branch to Chunk 3 instead of to Chunk 7 and Chunk 3 is now in use for the second display file. This problem is corrected by disabling the interruption prior to calling the Video Mode Change Service and setting VIDMOD to the correct value prior to re-enabling it. These corrections are included in the Extension ROM Interface Routine in Figure 3.2.2-2.

NOTE: On an initial access changing video mode from normal to Mode 128, the interruption is re-enabled within the Video Mode Change Service itself after copying the stack and other Chunk 3 data to Chunk 7. This cannot be corrected, but has not proven to present a problem in actual use. At the point where the interruption is first enabled, the Chunk 3 code is still intact allowing for correct processing of one interruption, and the path length from there to the point of correcting VIDMOD is apparently less than 17ms. The interruption is also re-enabled within the Video Mode Change Service if you have applied the patches for the BANK ENABLE and RESTORE STATUS routines (Section 6.5.4) which are executed in connection with inserting space into the RAM to open the second display file. Again, this has not proven to be a problem in actual use.

6.4.5 Interrupt Inhibit

By setting Bit 6 of Port 0FFH to a 1, the normal 17 ms. interruption generated from the SCLD to the Z80A CPU will be inhibited. When Port 0FFH is written to by the Video Mode Change Service, Bit 6 is forced to zero. If you wish to inhibit the normal interruption via this mechanism, and also plan to use the Video Mode Change Service, it is recommended that you first invoke the service to remap the RAM and open the second display file, then set Bit 6 of Port 0FFH to inhibit the normal interruption and write your own routine(s) for subsequent changing of the video mode setting that do not involve remapping the RAM. In this way you can maintain the value in Bit 6.

6.5 OS RAM ROUTINES

In patching the OS RAM routines, care must be taken not to relocate CALL and JP instructions since this affects the modification of the code when it is moved between Chunks 3 and 7. All of the code containing actual addresses must be modified to reflect the relocation and this is done using a table in the Extension ROM. Since the table cannot be changed, none of these instructions can be moved. Also, any CALL or JP instructions added must be modified by you when the code is relocated.

6.5.1 Function Dispatcher

For a variety of reasons such as conflict with use of the IX Register, incorrect entries in the ROM Function Dispatcher Jump Table, etc. some Service Codes have been deleted from the Function Dispatcher table (Table 3.3.4-z). In addition, the following correction to the GET STATUS routine' is required in order to successfully utilize the Function Dispatcher from a cartridge.

6.5.2 Get Status

Returns invalid memory selection status for-the Home Bank, ROM Extension and Dock. This results in switching out of either the Home Bank or the Dock when status is "restored". This affects use of the Function Dispatcher and GET WORD routines, and any other code using GET STATUS. Figure 6.5-1 shows the patches and additions necessary to correct this routine.

6.5.3 Put Word

Write data passed in Reg. Pair DE is overwritten prior to use. Figure 6.5-2 shows corrections.

6.5.4 Bank Enable and Restore Status

If the 17ms interruption occurs during update of the memory selection hardware, it can cause the system to hang and RAM to be overwritten. This occurs when the interruption happens in an interval when Port FF Bit 7 is zero (thus selecting the Dock Bank) and Port F4 Bit 0 is one (thus enabling Chunk 0 in the Dock Bank) and there is no memory in Chunk 0 of the Dock Bank. This can be true when there is no cartridge installed, or

if the cartridge installed is an AROS. This problem is corrected by disabling or masking the interruption while updating the memory selection hardware. Figure 6.5-3 shows one implementation of this correction.

6.5.5 Save Status and Restore Status

The value of Port FFH which includes video mode and interruption inhibit as well as Ext. ROM/Dock Select is saved and restored as a full 8-bits. Therefore any modification of this port by code accessed between execution of SAVE STATUS and subsequent execution of RESTORE STATUS (erg. via CALL BANK or use of the Function Dispatcher) is "undone". This is one reason the Video Mode Change Service and some of the bank switching routines such as BANK ENABLE cannot be meaningfully accessed via the Function Dispatcher.

6.5.6 Call Bank

Does not correctly retrieve the stack entry designating the count of parameters being passed. Memory is overwritten in the case where this count is not zero. This is corrected by setting Location 6610H = 9 (POKE 26128,9). You only need to apply the correction once; it will be duplicated in Chunk 7 if the code is relocated.

FIGURE 6.5-1
GET_STATUS CORRECTIONS

```

GET_STATUS  PUSH  AF          ; save regs
            PUSH  DE
            LD    A, B        ; get bank number
            CP    0FEH       ; test if extension (254)
            JR    Z, GS_EXT
            CP    0FFH       ; test if home (255)
            JR    Z, GS_HOME
            AND    A
            JR    Z, GS_DOCK  ; test if dock (0)

***      MORE CODE HERE THAT IS NOT AFFECTED      ***

GS_EXT      LD      C, 0FFH   ; assume none
            IN      A, (0FFH) ; test if selected
            AND     80H
            JR      Z, GS_XT1 ; not active
            JR      GETHS    ; get hsr
GS_DOCK     LD      C, 0FFH   ; assume none
            IN      A, (0FFH) ; test if selected
            AND     80H
            JR      NZ, GS_XT1 ; not active
GETHS       IN      A, (0F4H) ; get hsr
            CPL      ; invert to low active
            JR      GX_XT0    ; exit
GS_HOME     IN      A, (0F4H) ; all bits set are not active in home bank
GS_XT0      LD      C, A      ; memory select C
GS_XT1      POP     DE        ; restore regs
            POP     AF
            RET

```

FIGURE 6.5-1
GET_STATUS CORRECTIONS
(Continued)

From BASIC:
POKE 25610,40 (Location 640AH)
POKE 25611,36
POKE 25614,40 (Location 640EH)
POKE 25615,55
POKE 25617,40 (Location 6411H)
POKE 25618,39
POKE 25648,14 (Location 6430H)
POKE 25649,255
POKE 25650,219
POKE 25651,255
POKE 25652,230
POKE 25653,128
POKE 25654,40
POKE 25655,18
POKE 25656,24
POKE 25657,8
POKE 25658,14
POKE 25659,255
POKE 25660,219
POKE 25661,255
POKE 25662,230
POKE 25663,128
POKE 25664,32
POKE 25665,8
POKE 25666,219
POKE 25667,244
POKE 25668,47
POKE 25669,24
POKE 25670,2
POKE 25671,219
POKE 25672,244
POKE 25673,79

FIGURE 6.5-2
PUT WORD CORRECTIONS

PUT_WORD	PUSH AF ; save regs PUSH BC CALL GET_NUMBER ; bank # of owner PUSH DE ; save data LD D, B ; save target bank # LD B, A ; bank # of owner CALL GET_STATUS ; get bank status PUSH BC ; save it CALL GET_CHUNK ; get bit map CPL ; set active high LD B, D ; target bank # in B LD C, A ; memory select byte CALL BANK_ENABLE ; enable target memory POP BC ; saved bank status POP DE ; saved data LD (HL), E ; write LSB INC HL ; LD (HL), D ; write MSB DEC HL ; restore HL CALL BANK_ENABLE ; restore bank POP BC ; restore registers POP AF RET
----------	---

From BASIC:
 POKE 25408,213
 POKE 25424,193
 POKE 25425,209
 POKE 25426,115
 POKE 25427,35
 POKE 25428,114
 POKE 25429,43

NOTE: The corrections to GET-STATUS and BANK-ENABLE are also required.

FIGURE 6.5-3
BANK- ENABLE AND RESTORE STATUS CORRECTIONS

From BASIC					
BANK	ENABLE:	Location	Object	Code	POKE Address Value
		6499H	00	NOP	25753 0
		649DH	F3	DI	25757 243
		651CH	FB	EI	25884 251
RESTORE -STATUS:					
		654AH	F3	DI	25930 243
		6570H	FB	EI	25968 251

In both cases, the Disable Interrupt and Enable Interrupt are being done by deleting the preservation of the AF Registers (PUSH AF/POP AF). If your code requires AF to be saved, you must do it prior to calling either of these routines or any other system routines that use them. Note also that if you already have the interruption masked when these routines are entered, it will be enabled when they are exited. If this proves to be a problem, replace the Enable Interruption (EI) instruction with a NOP and do the enable at a more appropriate place in your own code.

6.5.7 GET_NUMBER

Always returns the Dock Bank # for any memory enabled in the ROM Extension.
Unlikely to be a problem because of limited use of the ROM Extension.

6.5.8 XFER_BYTES

Improperly passes memory select byte for the case where source and destination are in the same bank. This is corrected by setting Location 676AH = 5FH (POKE 26474,951).

6.6 General

6.6.1 Enter Key Anomalies

Pressing ENTER multiple times with an invalid tape command on the edit line (syntax error) causes the system to reset. This is due to overflowing the Bank Status Stack in RAM Chunk 3/7 due to the multiple calls to and from the Extension ROM via the Call Bank code without normal termination (the error causes-a RESTART 8 to be executed out of Home ROM code called from the ROM Extension). It shouldn't take anybody that many tries to get a tape command right, so this is not a real problem, but you may want to keep it in mind. For any call made through the OS RAM services, you should have a corresponding return to keep the structures clean.

6.6.2 ON_ERR_GOTO

If a non-existent line number is specified, followed by an error, the system will hang.

The ROM code is in an endless loop trying to report the absence of a valid error handler to the non-existent error handler!!! On some errors, you will get an unexpected 0 OK termination showing the line number of your Error Handler. This is because some ROM routines temporarily clear the INTPT Flag (Bit 7 of FLAGS). This flag is set to 0 when checking syntax and set to 1 when executing; if an error is detected while the Flag=0, the error handler code is branched to but is not executed.

6.6.3 Sound Command Parameters

Parameters to the SOUND command are not fully validated, therefore you can specify a number beyond the valid range for a given operation and not get an error, for example, you can write a value greater than 63 to the Enable Register (Reg.7), possibly changing the I/O Port used for reading the joysticks from input to output. If you specify a number larger than 255 (FFH), only the least significant byte will be actually written to the Programmable Sound Generator. Access to PSG Reg. 14 (IO-A) used for the Joysticks is also not precluded via the SOUND command.

If you experience difficulty in reading the joystick(s), do a write to PSG Reg. 7 clearing Bit 6 to 0 to guarantee that the joystick path is enabled for input (see Section 4.3). This write can be done by executing a SOUND 7,63 (or any value less than 63).

The INTEGER function for (-65536) gives an incorrect result of -1, and for other cases where the result should be -65536, it gives -IE-38. Since the ROM code cannot be changed, there is no correction.

6.6.4 Scroll? User Responses

If you respond to the SCROLL? message using multiple keys such as Cap Shift/Z or Cap Shift/Symbol Shift, you will get strange results like dumping of the Edit Line with the "C" or "E cursor, display of ROM data, or multiple scrolls. Stick to single key responses and you won't have any problems!

6.6.5 Delete Key Anomalies

When DELETE (Cap Shift/O) is held down to do deletion of characters in the Edit Line, sometimes it outputs the DELETE Keyword instead (it should not do this in auto-repeat mode). This is especially noticeable when the input line is long. Since the ROM code cannot be corrected, you must try releasing and pressing the DELETE key at differing frequencies and you will be able to get past this "Bug".

Appendix A Home ROM Map

Module	Origin	Length
BLOCK	0000	0000
BASIC	0000	0227
KBSCAN	0227	02D9
IO_1	0500	0502
IO_2	0A02	031B
EDIT	0D1D	0682
CHANS	139F	0142
LIST	14E1	02D4
AROS	17B5	0190
SYNTAX	1945	080A
SYNTWO	214F	04B4
GRAPHS	2603	0251
EXPRN	2854	041C
IDENT	2C70	03E9
INOUT	3059	0301
SUMS	335A	032A
CALC	3684	0437
FUNCTS	3ABB	01CE
TAPEMSG	3C89	0053
CH_SET	3D00	0300

Global	Address	Global	Address
ACS	3C5E	DIGIT?	30D9
ADD	33D3	DIM	2FC0
ALNUM?	3046	DIVIDE	356E
ALPHA?	304B	DRAW	26DB
ANGLE	3B9E	DRAWLN	2813
AROS	18C6	DRAW_L	2810
ARRAY	37C5	DUMPPTR	0A23
AR_LN	17EA	DYADIC	1BDC
AR_NXT	17FF	ECHO	0CA3
ASN	3C4E	EDIT_K	0A82
ATN	3BFD	END?	1B44
ATTBYT	0710	ENDSTT	1AB9
BEEP	0436	ENDTEM	1B4A
BORDER	2436	ERASE	25D4
BREAK?	2009	ERR2	1B91
CAT	25C8	ERR4	1FCF
CHCODE	0371	ERR5	07C1
CHINIT	11AA	ERR6	356C
CHK_SZ	1FBB	ERRB	1F29
CIRCLE	2679	ERRH	237E
CLCHAN	13BE	ERRO	123D
CLEAR	1F36	EXECUTE	1AD8
CLEL	13FF	EXPRN	2854
CLLHS	08A9	EIND_L	16D6
CLOSE	139F	FIND_N	2C70
GLPR	0035	FIX_U	1F23
CLR_BC	1F39	INS_U1	1F1E
CLS	08EA	FLASHA	160D
CLS_B	097F	FLOAT	3656
COLITM	23A6	FOR	1C78
COLOUR	23DE	FORMAT	25CC
CONT	1EE4	FP2A	3193
COS	3BC5	FP2BC	3160
EP_EC	16E8	F_ATTR	28D7
CTRO	37A1	F_INKY	29F2
DATA	1E82	F_PI	29E5
DEF	201D	F_PNT	2624
DELREC	1750	F_SCRN	288E
DELSYM	0B7E	GETAL	17CF
DEL_DE	174D	GET_EL	2D54
DEL_C	0BFD	GET_LN	1324
DESLUG	0D0D	GET_XY	2660
DE_HL	1668	GO_SUB	1F99

Global	Address	Global	Address
GR_COL	238C	NEXTCH	0074
HIFLSH	241D	NEXT_L	165B
INGH	11E1	NOTKB?	2380
ININT	30F9	NXT_HL	2C69
INIT	0D31	OPCHAN	1465
INPUT	222B	OPEN	142A
INSI	12B8	OPTNO	1C49
INSA	0AE7	OUTPUT	31A1
INSERT	12BB	PAEDCB	2E74
INT	3ACA	PARP	03F3
INTDIV	3ABB	PASSEM	25B9
INPT?	2889	PAUSE	1FEB
IN_K	0C0E	PHLAF	004F
I_SEQ	226B	PLOT	2635
JUMP	1EF1	PLOTBC	263E
K_BASE	035C	PLUGIN	0000
K_CLS	08A6	PGPSTR	2FAF
K_DUMP	0A02	PRSCAN	0A4A
K_LIST	1545	PR_CUR	162D
K_LLST	1541	PR_TV2	0776
K_LPR	2155	PSHSTR	2E70
K_NEW	0D1D	PUT	15C9
K_PRIN	2159	PUTDIG	11EA
K_SCAN	02B0	PUTMES	073F
LCU2	132D	PUT_BC	1788
LDMES	3CA8	PUT_LN	1795
LDTVCU	061A	PUT_SR	15A1
LE3	0055	P_LFT	053A
LED18	0E2F	P_NL	0566
LED4	038D	P_RT	0554
LET	2EBD	P_SEQ	217E
LINNG	1768	RAMNO	377F
LIST	14E1	RAND	1ED4
LN	3B2E	RDCH	11CF
LPO	15AC	READ	1D97
LS4	1A44	RECLN	1720
LT22	1BBC	REMGSZ	12CA
MOVE	25D0	RESET	1354
MULT	3468	RESTBC	1ECA
NC_HL	0077	RETURN	1FD4
NEGATE	382D	RND	29B6
NEW	0D82	ROOM?	3768
NEWDEV	24D2	ROOT	3C65
NEXT	1D55	RSET	2454

Global	Address	Global	Address
RSTSTR	13A8	STRITO	220F
R_ATT	0898	STTVCU	05F3
SCRL	0939	SUB	33CE
SCRMBL	2603	SUBLIN	16F0
SEARCH	136B	SUBLIN1	16F3
SELECT	1230	SUMSLD	3379
SEL_HL	1248	SYNERR	1BED
SENDCH	11ED	SYNTAX	1A27
SENDTV	0500	TAN	3BF5
SEPRMT	3C89	TC_HL	0078
SETCUR	0914	TEM1	1B82
STTV	0914	TEM10	1BEF
SET_AT	05B2	TEM6	1BE5
SHIFT	339C	TEMP38	19E0
SIN	3BD0	TEMP39	19E1
SKIP	1D28	TERM?	21E7
SKIPIT	2569	TEST0	3904
SLICER	2E10	TIMES	3489
SINIT	11C1	TOKENS	0098
SOUND	2128	TO_THE	3C6C
SRCHSC	1374	TRUNC	35D3
STBOOL	3926	TVFUL?	0790
STDE_S	314C	TV_COL	23BB
STDE_U	314A	UPD_K	02E1
STKUSN	3059	USRRET	3882
STK_O	1C41	WRCH	0010
STK_A	30E6	XEV	310D
STK_EC	30E9	X_CALC	134E
STK_M	3773	X_T_HL	1363
STOP	1C59		

Extension ROM Map

Module	Origin	Length
XBASIC	0000	0068
TAPE	0068	087F
INIT	08E7	04C9
CHNG_VID	0DB0	0193
PASSING	0F43	0047
BS	0F8A	001E

Global	Address
AKEY	08AA
BLDSCT	09F4
CALL_B	0F99
CHNG_V	0E8E
CLDFIL	0E27
EXINIT	08E7
GOTO_B	0F8A
LOAD	05CC
MERGE	06E5
OPDFIL	0DB0
PASSIN	0F43
RD_BIT	0189
RESSCT	0CRC
R_EDGE	018D
R_TAPE	00FC
SAVE	0851
SLVM	01AB
W_BORD	00E5
W_TAPE	0068

Dispatch table

This module is copied to RAM 6200H (space reserved 6200H-683FH) and relocated to (FC90H-FFFFH) when the second display file is used

Global	Address
BANK_E	6499
BS_MAX	6315
BS_SP	65CE
CALL_B	65D0
CREATE	66E8
DISPAT	6200
GET_CH	6440
GET_NU	645E
GET_ST	6405
GET_WO	6316
GOTO_B	6572
GOTO_E	6815
INT	62AE

7 Home ROM Source Code

```
INCL          "C:\2068\ROM\2068.INC"

;these are probably spurious labels, but I have defined them to
; make the assembler happy
M492A: EQU    $492A
MB3C8: EQU    $B3C8

M6000: EQU    $6000
M6815: EQU    $6815
M65D0: EQU    $65D0
M6499: EQU    $6499
M6722: EQU    $6722
MFD90: EQU    $FD90

M0F09: EQU    $0F09          ;in EXROM

;*****
; Module: BASIC
; Routine: PLUGIN
;*****
PLUGIN:
    DI                      ;disable interrupts
    XOR  A                  ;
    LD   DE,$FFFF          ;physical ramtop
    JP   M0D31              ;go to the initialization routine
;*****
; Module: BASIC
; Routine: ERROR
; Handles errors. top of machine stack has the address of the
; error identifier
;*****
    LD   HL,(CHADD)         ;load HL with character pointer CH_ADD
    LD   (XPTR),HL          ;store in X_PTR - character after ? mark
    JR   GETERNR
;*****
; Module: BASIC
; Routine: WRCH
; Write a character to the current stream
;*****
WRCH:
    JP   M11ED              ;jump to stream output routine
    RST  $38
    RST  $38
    RST  $38
    RST  $38
    RST  $38
;*****
; Module: BASIC
; Routine: GETCURCH
; get the current character pointed to by CH_ADD
;*****
```

```

GETCURCH:
M0018:  LD  HL,(CHADD)          ;get CH_ADD
        LD  A,(HL)             ;get the current character
M001C:  CALL M007D              ;determine type of character
        RET  NC                 ;return if not a token or
                                ; not a space
; if the character was a space or a token whose value was less
; than $1F, get the next character
;
M0020:  CALL NEXTCH             ;get the next character
        JR   M001C
        RST  $38
        RST  $38
        RST  $38
;*****
; Module: BASIC
; Routine: CALCCALL
; Jumps to the calculator interpreter
;*****
        JP   M371A              ;jump to CALC
        RST  $38
        RST  $38
        RST  $38
        RST  $38
        RST  $38
;*****
; Module: BASIC
; Routine: expand the workspace BC bytes
;*****
ALLOCBC: PUSH BC                ;save BC
        LD  HL,(WORKSP)         ;get WORKSP
        PUSH HL                 ;save HL
;*****
; Module: BASIC
; Routine: GLPR
;*****
GLPR:   JP   LCU2                ;insert BC items at (HL)

;
;IM1 interrupt target address.  this is the keyboard interrogation routine
;
INTRPT:
        PUSH AF
        PUSH HL
        LD  HL,(FRAMES)         ;/
        INC HL                  ;|increment FRAMES
        LD  (FRAMES),HL        ;\
        LD  A,H                 ;/is HL = 0?
        OR  L                   ;\
        JR   NZ,M0048
        INC (IY+OFRAMES2)       ;increment high byte of FRAMES
;FRAMES2
M0048:  PUSH BC                  ;/save registers
        PUSH DE                 ;\

```

```

        CALL M02E1          ;go scan the keyboard
        POP DE              ;/restore registers
        POP BC              ;|
;*****
; Module: BASIC
; Routine: PHLAF
; POP HL and AF - WHY???
;*****
PHLAF: POP HL              ;/
        POP AF              ;\
        EI                  ;enable interrupts
        RET                 ;exit

;*****
; Module: BASIC
; Routine: GETERNR
; gets the error number by popping the return address from
; the stack then loading it into L
;*****
GETERNR:
M0053: POP HL              ;get the error address
                                ; (stacked by RST 8 instruction)
        LD L,(HL)           ;stuff error number in L
;*****
; Module: BASIC
; Routine: LE3
; put the error number in L in ERRNR
;*****
LE3:
M0055: LD (IY+OERRNR),L     ;save error number in ERR_NR          ;ERRNR
        LD SP,(ERRSP)       ;load SP with the error routine stack pointer
        JP RESET            ;reset the calculator stack and memory pointers
        RST $38             ;/
        RST $38             ;|
        RST $38             ;|
        RST $38             ;|fill unused bytes
        RST $38             ;|
        RST $38             ;|
        RST $38             ;\
;*****
; Module: BASIC
; Routine: NMI
;*****
        PUSH AF              ;/save registers
        PUSH HL              ;\
        LD HL,(USRNMI)       ;get user-defined NMI routine address
        LD A,H               ;/
        OR L                 ;|if zero exit
        JR NZ,M0070
        JP (HL)              ;jump to NMI routine
M0070: POP HL                ;/restore registers
        POP AF               ;\
        RETN
;*****
; Module: BASIC

```

```

; Routine: NEXTCH
; get the next character after bumping CH_ADD
;*****
NEXTCH:
M0074: LD HL,(CHADD) ;get CH_ADD
;*****
; Module: BASIC
; Routine: NC_HL
; bump the current character pointer by 1
;*****
NC_HL:
M0077: INC HL ;move to next character
;*****
; Module: BASIC
; Routine: TC_HL
; update the character pointer with HL
;*****
TC_HL:
M0078: LD (CHADD),HL ;save HL to CH_ADD
LD A,(HL) ;get the character
RET ;
;*****
; Module: BASIC
; Routine: TEST_CH
; this routine tests the character in A. if it is '!' or later,
; $0D (CR), or $0C (DELETE) it immediately returns with carry
; reset. if the character is less than $10 (but not $0D or $0C)
; or greater than $18 (includes space [$20]) it returns with carry set.
; if the character is $16 (AT) or $17 (TAB) the character pointer in CH_ADD
; is advanced, carry set and return is done. if the character is any
; of the display control tokens (INK, PAPER, FLASH, BRIGHT, INVERSE, or
OVER)
; the CH_ADD is advanced twice.
; enter with char in A and char ptr in HL
; EXIT: NC if char greater than '!'
; Z if char is $0D or $0C
; C if char is $0E, $0F
;
;*****
TEST_CH:
M007D: CP "!" ;/return if '!' or later
RET NC ;\in ASCII collating sequence
CP $0D ;/if it is carriage-return
RET Z ;\exit
CP $0C ;/if is DELETE then exit
RET Z ;\
CP $10 ;/if less than $10 exit
RET C ;\
CP $18 ;/if greater than $17
CCF ;|then exit
RET C ;\
;at this point HL is pointing at a character in the range
; $10 to $17, which are various control tokens
INC HL ;point to next character
CP $16 ;/if the token is a display

```

```

        JR    C,M0093
        INC   HL                      ;point to next char
M0093:   SCF                      ;set carry
        LD    (CHADD),HL             ;save new char ptr in CH_ADD
        RET

;*****
; Module: BASIC
; Routine: TOKENS
;*****
TOKENS:
        DFB {"?"+"$80}
        DFB "RN",{"D"+"$80}
        DFB "INKEY",{"$"+"$80}
        DFB "P",{"I"+"$80}
        DFB "F",{"N"+"$80}
        DFB "POIN",{"T"+"$80}
        DFB "SCREEN",{"$"+"$80}
        DFB "ATT",{"R"+"$80}
        DFB "A",{"T"+"$80}
        DFB "TA",{"B"+"$80}
        DFB "VAL",{"$"+"$80}
        DFB "COD",{"E"+"$80}
        DFB "VA",{"L"+"$80}
        DFB "LE",{"N"+"$80}
        DFB "SI",{"N"+"$80}
        DFB "CO",{"S"+"$80}
        DFB "TA",{"N"+"$80}
        DFB "AS",{"N"+"$80}
        DFB "AC",{"S"+"$80}
        DFB "AT",{"N"+"$80}
        DFB "L",{"N"+"$80}
        DFB "EX",{"P"+"$80}
        DFB "IN",{"T"+"$80}
        DFB "SQ",{"R"+"$80}
        DFB "SG",{"N"+"$80}
        DFB "AB",{"S"+"$80}
        DFB "PEE",{"K"+"$80}
        DFB "I",{"N"+"$80}
        DFB "US",{"R"+"$80}
        DFB "STR",{"$"+"$80}
        DFB "CHR",{"$"+"$80}
        DFB "NO",{"T"+"$80}
        DFB "BI",{"N"+"$80}
        DFB "O",{"R"+"$80}
        DFB "AN",{"D"+"$80}
        DFB "<",{"="+"$80}
        DFB ">",{"="+"$80}
        DFB "<",{">"+"$80}
        DFB "LIN",{"E"+"$80}
        DFB "THE",{"N"+"$80}
        DFB "T",{"O"+"$80}
        DFB "STE",{"P"+"$80}
        DFB "DEF F",{"N"+"$80}
        DFB "CA",{"T"+"$80}
        DFB "FORMA",{"T"+"$80}

```

```

DFB "MOV",{"E"+$80}
DFB "ERAS",{"E"+$80}
DFB "OPEN ",{"#"+$80}
DFB "CLOSE ",{"#"+$80}
DFB "MERG",{"E"+$80}
DFB "VERIF",{"Y"+$80}
DFB "BEE",{"P"+$80}
DFB "CIRCL",{"E"+$80}
DFB "IN",{"K"+$80}
DFB "PAPE",{"R"+$80}
DFB "FLAS",{"H"+$80}
DFB "BRIGH",{"T"+$80}
DFB "INVERS",{"E"+$80}
DFB "OVE",{"R"+$80}
DFB "OU",{"T"+$80}
DFB "LPRIN",{"T"+$80}
DFB "LLIS",{"T"+$80}
DFB "STO",{"P"+$80}
DFB "REA",{"D"+$80}
DFB "DAT",{"A"+$80}
DFB "RESTOR",{"E"+$80}
DFB "NE",{"W"+$80}
DFB "BORDE",{"R"+$80}
DFB "CONTINU",{"E"+$80}
DFB "DI",{"M"+$80}
DFB "RE",{"M"+$80}
DFB "FO",{"R"+$80}
DFB "GO T",{"O"+$80}
DFB "GO SU",{"B"+$80}
DFB "INPU",{"T"+$80}
DFB "LOA",{"D"+$80}
DFB "LIS",{"T"+$80}
DFB "LE",{"T"+$80}
DFB "PAUS",{"E"+$80}
DFB "NEX",{"T"+$80}
DFB "POK",{"E"+$80}
DFB "PRIN",{"T"+$80}
DFB "PLO",{"T"+$80}
DFB "RU",{"N"+$80}
DFB "SAV",{"E"+$80}
DFB "RANDOMIZ",{"E"+$80}
DFB "I",{"F"+$80}
DFB "CL",{"S"+$80}
DFB "DRA",{"W"+$80}
DFB "CLEA",{"R"+$80}
DFB "RETUR",{"N"+$80}
DFB "COP",{"Y"+$80}
DFB "DELET",{"E"+$80}
DFB "ON ER",{"R"+$80}
DFB "STIC",{"K"+$80}
DFB "SOUN",{"D"+$80}
DFB "FRE",{"E"+$80}
DFB "RESE",{"T"+$80}
;*****
; Module: KBSCAN

```

```

; Routine: LCKEYS
;*****
LCKEYS:
;Data table: LCKEYS      module: KBSCAN
;byte mnemonics: B,H,Y,6,5,T,G,V
      DFB $42,$48,$59,$36,$35,$54,$47,$56

;byte mnemonics: N,J,U,7,4,R,F,C
      DFB $4E,$4A,$55,$37,$34,$52,$46,$43

;byte mnemonics: M,K,I,8,3,E,D,X
      DFB $4D,$4B,$49,$38,$33,$45,$44,$58

;byte mnemonics: SLUG,L,0,9,2,W,S,Z
      DFB $0E,$4C,$4F,$39,$32,$57,$53,$5A

;byte mnemonics: SPACE,ENTER,P,0,1,Q,A
      DFB $20,$0D,$50,$30,$31,$51,$41

;*****
; Module: KBSCAN
; Routine: EKEYS
;*****
EKEYS:
;Data table: EKEYS      module: KBSCAN
;byte mnemonics: READ,BIN,LPRINT,DATA,TAN,SGN,ABS,SQR
      DFB $E3,$C4,$E0,$E4,$B4,$BC,$BD,$BB

;byte mnemonics: CODE,VAL,LEN,USR,PI,INKEY$,PEEK,TAB
      DFB $AF,$B0,$B1,$C0,$A7,$A6,$BE,$AD

;byte mnemonics: SIN,INT,RESTORE,RND,CHR$,LLIST,COS,EXP
      DFB $B2,$BA,$E5,$A5,$C2,$E1,$B3,$B9

;byte mnemonics: STR$,LN
      DFB $C1,$B8

;*****
; Module: KBSCAN
; Routine: SEKEYS
;*****
SEKEYS:
;Data table: SEKEYS      module: KBSCAN
;byte mnemonics: FREE,BRIGHT,PAPER,/,ATN,{,},CIRCLE
      DFB $7E,$DC,$DA,$5C,$B7,$7B,$7D,$D8

;byte mnemonics: IN,VAL$,SCREEN$,ATTR,INVERSE,OVER,OUT,COPYRIGHT
      DFB $BF,$AE,$AA,$AB,$DD,$DE,$DF,$7F

;byte mnemonics: ASN,VERIFY,STICK,MERGE,],FLASH,ACS,INK
      DFB $B5,$D6,$7C,$D5,$5D,$DB,$B6,$D9

;byte mnemonics: [,BEEP
      DFB $5B,$D7

```

```

;*****
; Module: KBSCAN
; Routine: NUMFNTBL
;*****
NUMFNTBL:
;Data table: NUMFNTBL      module: KBSCAN
;byte mnemonics: DELETE,EDIT,PRINT,Unused,Unused,CSRLFT,CSRDN,CSRUP
      DFB $0C,$07,$06,$04,$05,$08,$0A,$0B

;byte mnemonics: CSRRT,NOTUSED
      DFB $09,$0F

;*****
; Module: KBSCAN
; Routine: KKEYS
;*****
KKEYS:
;Data table: KKEYS      module: KBSCAN
;byte mnemonics: STOP,*,?,STEP,>=,TO,THEN,^
      DFB $E2,$2A,$3F,$CD,$C8,$CC,$CB,$5E

;byte mnemonics: AT,-,+,=,.,`,`,,;"
      DFB $AC,$2D,$2B,$3D,$2E,$2C,$3B,$22

;byte mnemonics: <=,<,NOT,>,OR,/,<>,POUND
      DFB $C7,$3C,$C3,$3E,$C5,$2F,$C9,$60

;byte mnemonics: AND,:,FORMAT,DEFFN,FN,LINE,OPEN#,CLOSE#
      DFB $C6,$3A,$D0,$CE,$A8,$CA,$D3,$D4

;byte mnemonics: MOVE,ERASE,POINT,CAT
      DFB $D1,$D2,$A9,$CF

;*****
; Module: KBSCAN
; Routine: K_SCAN
;
;          RETURNS          MEANING
;  Z/NZ      D      E
;  Z          $FF      Key      one key pressed
;  Z          Shift    Key      Shift an key pressed
;  Z          $FF      $FF      no key pressed
;  NZ          X      X          too many keys pressed
;*****
K_SCAN:
M02B0:  LD    L,$2F          ;initialize position code
        LD    DE,$FFFF      ;indicate "no key" found
        LD    BC,$FEFE      ;C=keyboard port, B=scan group to test.  the
'0'
                                ; bit is the key group that is scanned

M02B8:  IN    A,(C)          ;get key information, low=key pressed
        CPL                    ;high is now a pressed key
        AND   $1F            ;allow only keyboard bits
        JR    Z,M02CD        ;jump if no keys pressed

```



```

;at least one key has been pressed, so...
    LD    H,A          ;save the key value
    LD    A,L          ;save position code

M02C1:  INC    D              ;/return if too many keys pressed
    RET    NZ          ;\

M02C3:  SUB    $08          ;update position code
    SRL    H            ;check a key bit
    JR     NC,M02C3      ;jump if the key is not pressed
    LD    D,E          ;put previous key value into D
    LD    E,A          ;put present key value into E
    JR     NZ,M02C1      ;check for another key

M02CD:  DEC    L            ;decrement position code
    RLC    B            ;move the keygroup bit
    JR     C,M02B8       ;jump if more groups need to be scanned

;we have scanned all key groups and so far have not returned, so
; there must have been only one key pressed, Shift and key pressed, or
; no keys pressed
    LD    A,D          ;get previous key
    INC    A            ;/return if only one key or no keys
    RET    Z            ;\ pressed
    CP    $28          ;/return if caps shift and another
    RET    Z            ;\ key
    CP    $19          ;/return if symbol shift and another
    RET    Z            ;\ key
    LD    A,E          ;/
    LD    E,D          ;|swap D and E
    LD    D,A          ;\
    CP    $18          ;check for symbol shift
    RET              ;ZF = 1 if there is a symbol shift
                    ;ZF = 0 if two charater keys
;*****
; Module: KBSCAN
; Routine: UPD_K
;*****
UPD_K:
M02E1:  CALL M02B0        ;scan the keyboard
    RET    NZ          ;return if too many keys pressed
    LD    HL,KS_A1      ;get key state

M02E8:  BIT    7,(HL)      ;/jump if no key stored
    JR     NZ,M02F3      ;\
    INC    HL            ;point to debounce count
    DEC    (HL)          ;decrement debounce counter
    DEC    HL            ;point back to KS_A1
    JR     NZ,M02F3      ;jump if debounce counter is not zero
    LD    (HL),$FF       ;force no key pressed for KS_A1

M02F3:  LD    A,L          ;save lower byte of key buffer
    LD    HL,KS_A2       ;"next key" buffer address
    CP    L              ;/jump if we are looking at different

```

```

JR      NZ,M02E8      ;\ buffers

CALL M035C            ;look for key code
RET  NC               ;return if no character key detected

RES  5,(IY+OFLAGS2)   ;reset repeat deletion flag in FLAGS2
LD   HL,KS_A1         ;/
CP   (HL)             ;|jump to handle repeat counters if KEY0 =

KEY2
JR      Z,M0336        ;\

EX  DE,HL             ;save address KS_A1 in DE
LD   HL,KS_A2         ;/
CP   (HL)             ;|jump to handle repeat counters if KEY0 =

KEY2
JR      Z,M0336        ;\

BIT  7,(HL)           ;/jump if KS_A2 indicates no key pressed
JR   NZ,M0317         ;\ (saves A into KS_A2
EX  DE,HL             ;point to KS_A1
BIT  7,(HL)           ;/return if KS_A1 has key hit
RET  Z                ;\
M0317: LD  E,A          ;save A
      LD  (HL),A        ;store A into KS_A1 or KS_A2
      INC HL            ;/initialize the debounce counter
      LD  (HL),$05      ;\
      INC HL            ;/
      LD  A,(REPDEL)    ;|initialize the repeat counter
      LD  (HL),A        ;\
      INC HL            ;point to the character
      LD  C,(IY+OMODE)   ;
      LD  D,(IY+OFLAGS) ;
      PUSH HL           ;save the character pointer
      CALL M0371        ;
      POP  HL           ;
      LD  (HL),A        ;

M032E: LD  (LASTK),A     ;store the keystroke in LASTK
      SET 5,(IY+OFLAGS) ;set KEYHIT flag in FLAGS
      RET              ;done

M0336: INC HL           ;/initialize the debounce counter
      LD  (HL),$05      ;\
      INC HL           ;point to repeat counter

      LD  A,(LASTK)     ;/
      CP  $CE           ;|return if last key value is a token
      RET NC           ;\

      DEC (HL)          ;decrement repeat counter
      RET NZ           ;return if not time to repeat yet

      LD  A,(REPPER)    ;/intitialize repeat counter
      LD  (HL),A        ;\
      INC HL            ;point to character

```

```

        LD    A,(HL)            ;get character
        CP    $0C              ;/jump if character code is not DELETE
        JR     NZ,M032E        ;\
        SET    5,(IY+OFLAGS2)  ;FLAGS2
        PUSH   AF              ;save AF
        LD     BC,$4E20        ;/
M0354:  DEC    BC              ;|delay loop
        LD     A,C             ;|
        OR     B               ;|
        JR     NZ,M0354        ;\
        POP    AF              ;restore AF
        JR     M032E          ;
;*****
; Module: KBSCAN
; Routine: K_BASE
; RETURNS:
;   C/NC      A      MEANS
;   NC        -      NO CHAR KEY
;   C         CODE   CHAR KEY
;*****
K_BASE:
M035C:  LD     B,D              ;"last key" to B
        LD     D,$00          ;
        LD     A,E            ;
        CP     $27            ;/return if key is greater than
        RET    NC              ;\CAPS SHIFT
        CP     $18            ;/jump if not SYMBOL SHIFT
        JR     NZ,M036A        ;\
        BIT    7,B            ;/return if this only one key
        RET    NZ              ;\has been pressed
M036A:  LD     HL,LCKEYS        ;/
        ADD    HL,DE           ;|retrieve the key code from
        LD     A,(HL)          ;\LCKEYS table
        SCF                    ;indicate key was found
        RET                    ;done
;*****
; Module: KBSCAN
; Routine: CHCODE
;   enter with MODE in C, FLAGS in D, and shift state in B
;   and candidate character in E
;*****
CHCODE:
M0371:  LD     A,E            ;get key code to A
        CP     $3A            ;/jump if key code < $3A
        JR     C,M03A5        ;\
        DEC    C              ;/jump if mode is 0
        JP     M,M038D        ;\
        JR     Z,M037F        ;jump if mode is 1
;mode must be 2 - "G" mode
        ADD    A,$4F          ;make graphics character
        RET                    ;done
;
;here for mode 1 - "E" mode
;
M037F:  LD     HL,EKEYS-"A"    ;get base of "E" table

```

```

        INC B                                ;/jump if shifted character
        JR Z,M0388                          ;\
        LD HL,LCKEYS                        ;unshifted "E" mode table
M0388:  LD D,$00                            ;/get "E" mode character from
        ADD HL,DE                          ;|selected table
        LD A,(HL)                          ;\
        RET                                ;
;*****
; Module: KBSCAN
; Routine: LED4
;*****
;mode 0 code
LED4:
M038D:  LD HL,$024B                        ;point to mode 0 keytable
        BIT 0,B                            ;/jump if symbol shift
        JR Z,M0388                        ;\
        BIT 3,D                            ;/jump if "K" mode
        JR Z,M03A2                        ;\
        BIT CAPS_L,(IY+OFLAGS2)           ;/return if CAPS LOCK ;FLAGS2
        RET NZ                            ;\or CAPS SHIFT
        INC B                            ;/return if upper case (B=$FF)
        RET NZ                            ;\
        ADD A,$20                         ;/convert to lower case
        RET                                ;\

M03A2:  ADD A,$A5                          ;convert to keyword
        RET                                ;
;
;keycode < $3a
;
M03A5:  CP $30                            ;/return if key <$30
        RET C                            ;\
        DEC C                            ;
        JP M,M03DB                        ;jump if mode=0
        JR NZ,M03C7                       ;jump if mode=2
;
;mode=1 ("E" mode)
;
        LD HL,$0276                        ;get "E" mode key table
        BIT 5,B                            ;/jump if symbol shift
        JR Z,M0388                        ;\
        CP $38                            ;/jump if digit is 8 or 9
        JR NC,M03C0                       ;\
;
;digit was 0-7
;
        SUB $20                            ;convert to SET FG color
        INC B                            ;/return if no SHIFT
        RET Z                            ;\
        ADD A,$08                         ;convert to SET BG color
        RET                                ;done

M03C0:  SUB $36                            ;convert to BRIGHT/UNBRIGHT
        INC B                            ;/return if no SHIFT
        RET Z                            ;\

```

```

        ADD  A,$FE                ;convert to FLASH/UNFLASH
        RET                               ;
;
;mode=2 ("G" mode)
;
M03C7:  LD   HL,$0252                ;get "G" mode key table
        CP   $39                    ;/
        JR   Z,M0388                ;|jump if character is "0" or "9"
        CP   $30                    ;|
        JR   Z,M0388                ;\
        AND  $07                    ;/
        ADD  A,$80                  ;\make graphics mosaic patterns
        INC  B                      ;/return if no SHIFT
        RET  Z                      ;\
        XOR  $0F                    ;invert pattern if SHIFT
        RET                               ;
;
;mode=0 ("K" or "L" mode)
;
M03DB:  INC  B                      ;/return if no SHIFT
        RET  Z                      ;\
        BIT  5,B                    ;/
        LD   HL,$0252                ;|jump if not SYMBOL SHIFT
        JR   NZ,M0388                ;\
        SUB  $10                    ;convert to symbol
        CP   $22                    ;/jump if ampersand "@"
        JR   Z,M03F0                ;\
        CP   $20                    ;/return for space
        RET  NZ                      ;\
        LD   A,$5F                  ;/make underline "_"
        RET                               ;\
M03F0:  LD   A,$40                    ;/return ampersand "@"
        RET                               ;\
;*****
; Module: KBSCAN
; Routine: PARP
; Enter: DE contains number of cycles - 1
;       HL contains the waveform period (8*HL+236 to 8*HL+246 t-states)
;       (HL = $3CE for 440Hz, 2 cycles)
;*****
PARP:   ;                               A
HL      DE      BC      IX
M03F3:  DI                               ;don't interrupt me      03CE
0002  ----  ----
        LD   A,L                        ;save L                      CE
03CE  0002  ----  ----
        SRL  L                          ;/divide L by 4          CE
0367  0002  ----  ----
        SRL  L                          ;\                      CE 0333
0002  ----  ----
        CPL                               ;/initialize fine tune for the 31
0333  0002  ----  ----
        AND  $03                        ;\requested period      01 0333
0002  ----  ----

```

```

0002      LD    C,A                ;/                01 0333
--01      LD    B,$00            ;\                01 0333
0002 0001      LD    IX,$040F        ;/form the loop address for the
01 0333 0002 0001 040F      ADD    IX,BC        ;\timing loop                01
0333 0002 0001 0410      LD    A,(BORDCR)    ;/
AND    $38                ;|get the border color and make
RRCA                ;| it compatible with the port
RRCA                ;| at $FE
RRCA                ;\
OR    $08                ;set tape bit

;at this point A contains a template value to be output to the speaker
; port. bit 4 (value $10) will be toggled to make the noise

M040F:      NOP                ;/
NOP                ;|the NOPs allow fine tuning of
NOP                ;\the period
INC    B                ;
0333 0002 0101 0410      INC    C                ;
0333 0002 0002 0410      INC    C                ;
M0414:      DEC    C                ;/timing loop based on C
JR      NZ,M0414        ;\
LD    C,$3F            ;/                -- 0333
0002 003F 0410      DEC    B                ;|timing loop based on B
JP      NZ,M0414        ;\
XOR    $10            ;/toggle the speaker output
M041F:      OUT    ($FE),A        ;\
LD    B,H                ;precharge B with upper byte of
0333 0002 0301 0410      ; period value
LD    C,A                ;save C                01
0333 0002 00-- 0410      BIT    4,A        ;/jump if the speaker bit is
JR      NZ,M0430        ;\ a one
LD    A,D                ;/
OR    E                ;|exit if cycle counter is
JR      Z,M0434        ;\ zero
LD    A,C                ;restore C
LD    C,L                ;BC now contains the period value
DEC    DE                ;decrement the cycle counter
JP      (IX)            ;back around for more cycles
M0430:      LD    C,L                ;/BC now again contains the
INC    C                ;|period value, so loop back
JP      (IX)            ;\for more cycles
M0434:      EI                ;I am now interruptable
RET                ;
;*****
; Module: KBSCAN
; Routine: BEEP

```

```

;*****
BEEP:
    RST $28    ;calc entry
    DFB $31                ;DUP
    DFB $27                ;INT          find the integer
portion of the note
    DFB $C0                ;T -> MEM 0 copy to MEM 0
    DFB $03                ;SUB          calc now has the
fractional portion of the note value
    DFB $34,$EC,$6C,$98,$1F,$F5 ;LITERAL  7C6C981FF5 (
5.77622650453122E-2),0441
    DFB $04                ;TIMES
    DFB $A1                ;CONST 1  (256)
    DFB $0F                ;ADD
    DFB $38                ;QUIT          calc now has a factor that
represents the ratio of the
                                ;          requested frequency to a full
step in frequency.
                                ;          e.g. if the note value was
3.73, MEM 0 now contains 3
                                ;          and calc contains
1+0.73*0.57762265e-2 or 1.0042167.
                                ;          when the base period (3 or D#)
is multiplied by this number, the
                                ;          new number will be the period
of the requested frequency
    LD  HL,MEMBOT          ;get the calculator's memory address
(MEM 0)
    LD  A,(HL)              ;/
    AND A                  ;|jump if the number is not an
integer (and reset the carry)
    JR  NZ,M04AA           ;\
    INC HL                  ;/get the sign byte
    LD  C,(HL)              ;\
    INC HL                  ;/get the 1s byte of the number
    LD  B,(HL)              ;\
    LD  A,B                 ;/
    RLA                    ;|check to see if the number is C is less
than
    SBC A,A                 ;|128, and zero the accumulator
    CP  C                   ;|
    JR  NZ,M04AA           ;\
    INC HL                  ;/
    CP  (HL)                ;|jump if high byte is not zero
    JR  NZ,M04AA           ;\
    LD  A,B                 ;A now has the note code
    ADD A,$3C               ;/jump if the note code is 68 or less
    JP  P ,M0463            ;\
    JP  PO ,M04AA           ;jump if overflow, A musta been below
-60

;the note number (-60 to +69 has now been shifted to 0 to $81)
;we now find which octave the note occupies

M0463:  LD  B,$FA           ;B = -6  initialize the octave counter

```

```

M0465:  INC  B                      ;/
        SUB  $0C                    ;|loop until we have subrtacted enough
octaves
        JR   NC,M0465                ;\
        ADD  A,$0C                    ;A now contains the note within the octave
stored in B
        PUSH BC                      ;save the octave number
        LD   HL,$04AC                ;base of the note table
        CALL M37C5                    ;HL = HL + 5 * A
        CALL STK_M                    ;stack the number found above onto the calc
stack
        RST  $28                      ;calc entry
        DFB  $04                      ;TIMES          multiply the base
frequency by the factor computed above
        ;                               this gives
the final frequency
        DFB  $38                      ;QUIT
        POP  AF                      ;get the octave number
        ADD  A,(HL)                    ;/essentially does 2^N*note_number
        LD   (HL),A                    ;\
        RST  $28                      ;calc entry
        DFB  $C0                      ;T -> MEM 0      MEM 0 now contains the
final frequency
        ;                               for the desired note
        DFB  $02                      ;DROP          remove the frequency from
the fp stack
        DFB  $31                      ;DUP           duplicate the duration value
        DFB  $38                      ;QUIT
        CALL INS_U1                    ;convert the fp top of stack to a
value in A
        CP   $0B                      ;
        JR   NC,M04AA                  ;

;translate the duration value to the equivalent number of cycles for PARP

        RST  $28                      ;calc entry
        DFB  $E0                      ;MEM 0 -> T
        get the desired frequency
        DFB  $04                      ;TIMES
multiply by required the duration to give the # of cycles
        DFB  $E0                      ;MEM 0 -> T
again, get the frequency
        DFB  $34,$80,$43,$55,$9F,$80 ;LITERAL  93559F8000 ( 437500),0492
/clock/8
        DFB  $01                      ;SWAP
|
        DFB  $05                      ;DIV
|   convert the frequency into a period value for PARP
        DFB  $34,$35,$71                ;LITERAL  8571000000 ( 30.125),0497
|
        DFB  $03                      ;SUB
\
        DFB  $38                      ;QUIT
        CALL FIX_U                      ;period on FP stack to BC
        PUSH BC                        ;save frequency

```



```

CALL FIX_U          ;convert duration to BC
POP HL              ;restore the frequency
LD D,B              ;/move the duration to DE
LD E,C              ;\
LD A,D              ;/
OR E                ;|if the duration is zero, we are
done
RET Z               ;\
DEC DE              ;adjust the duration for PARP
JP PARP             ;make the noise

M04AA: RST $08       ;/error: INTEGER OUT OF RANGE
      DFB $0A        ;\

;BEEP look-up table
;
TONC:  DFB $89,$02,$D0,$12,$86 ;261.625565290451 Hz
TONCIS: DFB $89,$0A,$97,$60,$75 ;277.182631134987 Hz
TOND:   DFB $89,$12,$D5,$17,$1F ;293.664768099785 Hz
TONDIS: DFB $89,$1B,$90,$41,$02 ;311.126983880997 Hz
TONE:   DFB $89,$24,$D0,$53,$CA ;329.627557039261 Hz
TONF:   DFB $89,$2E,$9D,$36,$B1 ;349.228231549263 Hz
TONFIS: DFB $89,$38,$FF,$49,$3E ;369.994422674179 Hz
TONG:   DFB $89,$43,$FF,$6A,$73 ;391.99543607235 Hz
TONGIS: DFB $89,$4F,$A7,$00,$54 ;415.30469751358 Hz
TONA:   DFB $89,$5C,$00,$00,$00 ;440 Hz
TONB:   DFB $89,$69,$14,$F6,$24 ;466.163761615753 Hz
TONH:   DFB $89,$76,$F1,$10,$05 ;493.883301377296 Hz

;
;name for SAVE, LOAD, MERGE?
;
CALL EXPRN          ;evaluate the current expression
LD A,(FLAGS)        ;/jump if the expression is numerical
ADD A,A              ;|
JP M,M1BED          ;\error - BAD BASIC
POP HL              ;restore HL
RET NC               ;return if syntax checking
PUSH HL             ;save HL
CALL PGPSTR          ;get the string descriptor
LD H,D               ;/string location to DE
LD L,E               ;\
DEC C                ;/return if the string length is zero
RET M                ;\
ADD HL,BC            ;point to last character of string
SET 7,(HL)           ;set bit 7
RET                  ;
;*****
; Module: IO_1
; Routine: SENDTV
; enter with the character to print in A
;*****
SENDTV:
M0500: CALL LDTVCU    ;update DFCC and DFLCC
      CP $20          ;/jump if character is not a control
      JP NC,M05F0     ;\character (output the bit pattern)

```

```

        CP    $0C                ;/jump if character is not DELETE
        JR    NZ,M0513            ;\
        BIT   4,(IY+OFLAGS)      ;/jump if not a listing?
        JP    Z ,M05F0           ;\jump if a listing? (output the bit
pattern)

M0513:   CP    $06                ;/jump if char is less than PRINT COMMA
        JR    C,M0580            ;\ (all of these are not used)

        CP    $18                ;/jump if char is greater than TAB
        JR    NC,M0580          ;\

;
;we are printing a control character so get the jump address for
; the code to handle it
;
        LD    HL,CONTRO-6        ;base of jump table - 6
        LD    E,A                ;/
        LD    D,$00             ;|form the address of the jump table entry
        ADD   HL,DE              ;\
        LD    E,(HL)            ;get the code offset
        ADD   HL,DE              ;make the address of the code handler
        PUSH  HL                 ;save it
        JP    LDTVCU            ;jump to the print routine
                                ; this is effectively a call to the stacked
                                ; address after DFCC and DFLCC have been set

up
;~~~~~
; print control jump table
;
CONTRO:  DFB  QM0A5F-$           ;Print comma
        DFB  M0580-$           ;Edit
        DFB  P_LFT-$           ;Cursor left
        DFB  P_RT-$            ;Cursor right
        DFB  M0580-$           ;Cursor down
        DFB  M0580-$           ;Cursor up
        DFB  M0580-$           ;Delete
        DFB  P_NL-$            ;Enter
        DFB  M0580-$           ;number slug /not print control
        DFB  M0580-$           ;not used      \
        DFB  M0591-$           ;Ink
        DFB  M0591-$           ;Paper
        DFB  M0591-$           ;Flash
        DFB  M0591-$           ;Bright
        DFB  M0591-$           ;Inverse
        DFB  M0591-$           ;Over
        DFB  M058C-$           ;At
        DFB  M058C-$           ;Tab
;~~~~~

;*****
; Module: IO_1
; Routine: P_LFT
; cursor left routine
;*****
P_LFT:

```

```

        INC C                        ;/
        LD A,$22                    ;|
        CP C                        ;|
        JR NZ,M0551                 ;\
        BIT PR,(IY+OFLAGS)          ;/jump if printing to the printer
        JR NZ,M054F                 ;\
        INC B                        ;
        LD C,$02                    ;
        LD A,$19                    ;
        CP B                        ;
        JR NZ,M0551                 ;
        DEC B                        ;
M054F:   LD C,$21                    ;33 lines
M0551:   JP STTVC                    ;
;*****
; Module: IO_1
; Routine: P_RT
; cursor right routine
;*****
P_RT:    LD A,(PFLAG)                ;/get printing flags and
        PUSH AF                      ;\save them
        LD (IY+OPFLAG),01           ;make permanent print flag OVER
        LD A,$20                     ;/print a space to the TV
        CALL M05F0                   ;\ (output the bit pattern)
        POP AF                       ;/restore the print flags
        LD (PFLAG),A                 ;\
        RET
;*****
; Module: IO_1
; Routine: P_NL
; enter with number of lines in B
;*****
P_NL:    BIT PR,(IY+OFLAGS)          ;/jump if printing to printer
        JP NZ,M0A23                 ;\
        LD C,$21                    ;
        CALL TVFUL?                  ;return here if the screen is not full
        DEC B                        ;
        JP STTVC                     ;
QM0A5F:  CALL LDTVCU                 ;get DF pointer (DE is the important
one)
        LD A,C                      ;
        DEC A                        ;
        DEC A                        ;
        AND $10                      ;
        JR M05DA
M0580:   LD A,"?"                    ;/output a question mark bit pattern
        JR M05F0                     ;\
M0584:   LD DE,M059E                 ;
        LD (TVDATA+1),A              ;
        JR M0597                     ;
M058C:   LD DE,M0584                 ;

```

```

        JR      M0594                ;
M0591:  LD      DE,M059E                ;
M0594:  LD      (TVDATA),A            ;
M0597:  LD      HL,(CURCHL)           ;get the address of the current
                                         ; channel

        LD      (HL),E                ;/make it point to the
        INC     HL                    ;|channel handler in DE
        LD      (HL),D                ;\
        RET                               ;

M059E:  LD      DE,SENDTV              ;
        CALL    M0597                ;
        LD      HL,(TVDATA)          ;
        LD      D,A                  ;
        LD      A,L                  ;
        CP      $16                  ;
        JP      C ,M23BB              ;
        JR      NZ,M05D9              ;
        LD      B,H                  ;
        LD      C,D                  ;
;*****
; Module: IO_1
; Routine: SET_AT
;*****
SET_AT:
        LD      A,$1F
        SUB     C
        JR      C,M05C3
        ADD     A,$02
        LD      C,A
        BIT     PR,(IY+OFLAGS)        ;/jump if printing to printer
        JR      NZ,M05D6              ;\
        LD      A,$16
        SUB     B
M05C3:  JP      C ,M1F29
        INC     A
        LD      B,A
        INC     B
        BIT     0,(IY+0TVFLAG)        ;TVFLAG
        JP      NZ,TVFUL?
        CP      (IY+0DFSZ)            ;DFSZ
        JP      C ,M07C1
M05D6:  JP      STTVC
M05D9:  LD      A,H
M05DA:  CALL    LDTVCU
        ADD     A,C
        DEC     A
        AND     $1F
        RET     Z
        LD      D,A
        SET     0,(IY+OFLAGS)        ;FLAGS
M05E7:  LD      A,$20
        CALL    M0776
        DEC     D

```

```

        JR      NZ,M05E7
        RET

M05F0:  CALL M063B
;*****
; Module: IO_1
; Routine: STTVCU
; set the column position and display file pointers for the TV
; based on various flags
;*****
STTVCU:
M05F3:  BIT    PR,(IY+OFLAGS)          ;/jump if printing to
        JR     NZ,M0613                ;\printer
        BIT    LHS,(IY+OTVFLAG)        ;/jump if printing to the
        JR     NZ,M0607                ;\ lower half of the screen
;handle the upper screen parameters
        LD     (SPOSNCOL),BC           ;update the upper screen col pointer
        LD     (DFCC),HL               ;update upper display file character
pointer
        RET                             ;done
;do the lower screen parameters
M0607:  LD     (SPOSNLCOL),BC           ;update lower screen col pointer
        LD     (ECHOE),BC              ;update keyboard buffer pinter
        LD     (DFCCL),HL              ;update lower display file character
pointer
        RET                             ;done

M0613:  LD     (IY+OPPOSN),C            ;update print position
        LD     (PRCC),HL               ;update printer buffer character
pointer
        RET                             ;done
;*****
; Module: IO_1
; Routine: LDTVCU
; loads registers with TV display information
; loads HL with the display file position for a character
; loads BC with the LIN (B) and COL (C) of the current screen position
;*****
LDTVCU:
M061A:  BIT    PR,(IY+OFLAGS)          ;/jump if printing to printer
        JR     NZ,M0634                ;\

        LD     BC,(SPOSNCOL)           ;/get current print column position
        LD     HL,(DFCC)                ;\load into the display file column

        BIT    LHS,(IY+OTVFLAG)        ;/return if printing to the upper half
        RET    Z                        ;\of the screen

        LD     BC,(SPOSNLCOL)          ;/put the lower screen print column
position
        LD     HL,(DFCCL)                ;\into the display column pointer
        RET                             ;done

M0634:  LD     C,(IY+OPPOSN)            ;get the print buffer position
        LD     HL,(PRCC)                ;

```

```

        RET
;
;put a character on the screen
; ENTER:  A contains the character
;         DE contains the character position in the DF
;
M063B:  CP    $0C                ;/jump if char is not DELETE
        JR    NZ,M0643          ;\
        LD    A,$7A             ;
        JR    M0694             ;

M0643:  CP    $7C                ;/jump if char is STICK
        JR    Z,M0694           ;\
        CP    $7E                ;/jump if char is FREE
        JR    Z,M0694           ;\
        CP    $7B                ;/jump if char is printable
        JR    C,M0659           ;\
        CP    $80                ;/jump if char is block graphics 0
        JR    NC,M0659          ;\
        BIT   RETPOS,(IY+OFLAGS) ;/jump if retype is not possible after
        JR    Z,M0694           ;\syntax error (RUN mode?)

M0659:  CP    $80                ;/jump if below block graphics 0
        JR    C,M069A           ;\
        CP    $90                ;/jump if we is a possible UDG
        JR    NC,M0687          ;\

        LD    B,A               ;transfer character to B
        CALL  MKBLKGR           ;generate a block graphic of the
character
                                ; code in B
        CALL  LDTVCU             ;HL now has the DF position
        LD    DE,MEMBOT         ;point to the block graphic bit map
buffer
        JR    M06B4
;
;handle block graphics characters
;this routine dynamically generates the block graphics characters
; each bit of the character code specifies one quadrant of the block
; graphic as follows (the digits refer to bit numbers):
; 11110000 scan line 0
; 11110000 scan line 1
; 11110000 scan line 2
; 11110000 scan line 3
; 33332222 scan line 4
; 33332222 scan line 5
; 33332222 scan line 6
; 33332222 scan line 7
; the character code is in B

MKBLKGR:
M066D:  LD    HL,MEMBOT         ;point to free space, provides a buffer
for
                                ; the character bit map

```

```

CALL M0673                ;generate the first four scan lines (bits 0
and 1)
;
;falling through to here will generate the bit map pattern for bits
; 2 and 4
;
M0673:  RR    B                ;/rotate the lsb into the CF
        SBC   A,A              ;|if lsb is 1, A=$FF else A=$00
        AND   $0F              ;\keep only those needed for the character
        LD    C,A              ;save the character
        RR    B                ;/check the next bit up,
        SBC   A,A              ;|handle as before
        AND   $F0              ;\
        OR    C                ;or in the previously found bit
pattern
        LD    C,$04            ;/
M0681:  LD    (HL),A            ;|save the bit map to the buffer
        INC   HL                ;|
        DEC   C                ;|
        JR    NZ,M0681          ;\
        RET                    ;
;
;handle UDG and above
;
M0687:  SUB    $A5              ;/jump if a token not a UDG
        JR    NC,M0694          ;\
        ADD   A,$15             ;adjust value for UDG bit map table
        PUSH  BC                ;save BC
        LD    BC,(UDG)          ;load BC with the base of the UDG
table
        JR    M069F              ;
;
M0694:  CALL  PRTTOK            ;print the token in A
        JP    LDTVCU            ;HL points to the DF
;
M069A:  PUSH  BC
        LD    BC,(CHARS)
;
;BC contains the base address of the character bit map table
;
M069F:  EX    DE,HL            ;save DE
        LD    HL,FLAGS          ;/force no printing leading spaces
        RES   0,(HL)            ;\
        CP    $20              ;/jump if the character is not a space
        JR    NZ,M06AB          ;\
        SET   0,(HL)            ;force printing leading spaces
;
M06AB:  LD    H,$00             ;/ HL <- character
        LD    L,A                ;\
        ADD   HL,HL              ;/
        ADD   HL,HL              ;| HL <- 8 * HL
        ADD   HL,HL              ;\
        ADD   HL,BC              ;HL now points to the bit map for the
character
        POP   BC                ;restore BC

```

```

        EX    DE,HL                ;DE now points to the bit map
;
;DE points to the character bit map, HL points to the DF or printer buffer
;
M06B4:  LD     A,C                  ;get column position to A
        DEC    A                    ;put in range [0..31]
        LD     A,$21                ;
        JR     NZ,M06C8             ;jump if not
        DEC    B                    ;normalize screen line number
        LD     C,A                  ;
        BIT    PR,(IY+OFLAGS)       ;/jump if printing to TV
        JR     Z,M06C8             ;\
        PUSH   DE                   ;save the bit map buffer address
        CALL   M0A23                ;flush the printer buffer to the printer
        POP    DE                   ;restore the buffer address
        LD     A,C                  ;force check of display

M06C8:  CP     C                    ;
        PUSH   DE                   ;save bit map buffer address
        CALL   Z ,TVFUL?            ;return here if display not full
        POP    DE                   ;restore the bit map buffer address
        PUSH   BC                   ;/save BC and HL
        PUSH   HL                   ;\
        LD     A,(PFLAG)            ;get printing flags
        LD     B,$FF                ;
        RRA                                     ;/jump if XORing new and
        JR     C,M06D9              ;\ old characters
        INC    B                    ;
M06D9:  RRA                                     ;/check invert
        RRA                                     ;\
        SBC    A,A                  ;A=$FF if invert, else A=0
        LD     C,A                  ;save A
        LD     A,$08                ;FOR 8 bytes...
        AND    A                    ;
        BIT    PR,(IY+OFLAGS)       ;/jump if printing to TV
        JR     Z,M06EB             ;\
        SET    PRLEFT,(IY+OFLAGS2) ;force print buffer not empty
        SCF                          ;printer flag
;
;I still haven't determined the magic required to get the display file
; address into the right register
;
M06EB:  EX     DE,HL                ;HL contains the bit map address, DE
contains DF pointer
M06EC:  EX     AF,AF'               ;to alt regs
        LD     A,(DE)               ;get the first byte of DF character
        AND    B                    ;clear all bits if not XORing
characters
        XOR    (HL)                 ;XOR the bit map
        XOR    C                    ;invert, if called for
        LD     (DE),A               ;save back to DF
        EX     AF,AF'               ;back to normal regs
        JR     C,M0708              ;
        INC    D                    ;point to next byte in DF
M06F6:  INC    HL                   ;point to next bit map character

```



```

        DEC A                                ;decrement byte counter
        JR NZ,M06EC                          ;jump back if more are needed
        EX DE,HL                             ;HL=DF pointer, DE=bit map pointer
        DEC H                                ;point back to last modified byte in
DF
        BIT PR,(IY+OFLAGS)                   ;/update the attribute for this
character
        CALL Z ,ATTBYT                       ;\
        POP HL                               ;restore HL
        POP BC                               ;restore BC
        DEC C                                ;
        INC HL                               ;
        RET                                  ;

M0708:  EX AF,AF'                            ;alt regs
        LD A,$20                             ;/
        ADD A,E                             ;|adjust the address for a print
        LD E,A                               ;|buffer rather than the DF
        EX AF,AF'                            ;|
        JR M06F6                             ;\
;*****
; Module: IO_1
; Routine: ATTBYT
;*****
ATTBYT:
M0710:  LD A,H                               ;get the high byte of the DF address
        RRCA                                ;/
        RRCA                                ;|divide by 8
        RRCA                                ;\
        AND $03                             ;use only the lower 2 bits
        OR $58                              ;/generate the MS byte of the
        LD H,A                               ;\attribute address
        LD DE,(ATTRT)                       ;get the temporary attributes
        LD A,(HL)                           ;get the current attribute
        XOR E                               ;/for every bit in D that is a 0,
        AND D                               ;|the corresponding bit in A will
        XOR E                               ;\come from E, else it will come from
A
        BIT B_CF,(IY+OPFLAG)                ;/jump if colors OK
        JR Z,M072F                          ;\
        AND $C7                             ;keep flash, hilite, and fgnd colors
        BIT 2,A                             ;/jump if green is active
        JR NZ,M072F                          ;\
        XOR $38                              ;invert fgnd colors
M072F:  BIT F_CB,(IY+OPFLAG)                 ;/jump if colors still OK
        JR Z,M073D                          ;\
        AND $F8                             ;keep flash, hilite, and bgnd colors
        BIT 5,A                             ;/jump if green is active
        JR NZ,M073D                          ;\
        XOR $07                              ;invert bgnd colors
M073D:  LD (HL),A                            ;store the attribute
        RET
;*****
; Module: IO_1
; Routine: PUTMES

```

```

;
;*****
PUTMES:
M073F:  PUSH HL                ;save HL
        LD  H,$00              ;/push value to prevent trashing
        EX  (SP),HL            ;\return address???
        JR  M074F              ;go find string in buffer

;
;print the token whose value is in A
;
PRTTOK:
M0745:  LD  DE,$0098            ;base of token table
        CP  $5B                ;/adjust token number, if necessary
        JR  C,M074E            ;|
        SUB $1F                ;\
M074E:  PUSH AF                ;save token value

M074F:  CALL M077C              ;find the message number A in table
        ; at (DE)
        JR  C,M075D            ;no leading space needed, so jump ahead

        LD  A,$20              ;/print a leading space, if needed
        BIT 0,(IY+OFLAGS)      ;|                ;FLAGS
        CALL Z ,M0776          ;\

M075D:  LD  A,(DE)              ;get the character
        AND $7F                ;mask off the high bit
        CALL M0776             ;output the char to the current stream

        LD  A,(DE)              ;re-get the character
        INC DE                  ;bump the message pointer
        ADD A,A                 ;/back around for more if the
        JR  NC,M075D           ;\high bit of the character is not set

        POP DE                  ;trash return address unless entered via
PUTMES

        CP  $48                ;/jump if the character is "$"
        JR  Z,M0770            ;\

        CP  $82                ;/return if char is less than "A"
        RET C                  ;\

M0770:  LD  A,D                 ;/return if message is below page 3
        CP  $03                ;|
        RET C                  ;\
        LD  A,$20              ;output a space
;*****
; Module: IO_1
; Routine: PR_TV2
;*****
PR_TV2:
M0776:  PUSH DE                ;save current character pointer
        EXX                    ;to alt registers

```

```

RST $10                ;WRCHAR - output the character in A to the
                        ; current stream
EXX                    ;to regular registers
POP DE                ;restore character pointer
RET                    ;done

;
;find the message with index "A" located in table at (DE).  if
; return with CF=0 if the current character is greater than "@".
; this will force printing a leading space if BIT 0,(IY+OFLAGS) is "0"
;FLAGS
;
FINDMSG:
M077C:  PUSH AF                ;save message number index
        EX  DE,HL              ;HL = buffer pointer
        INC A                  ;bump message number passed
M077F:  BIT  7,(HL)            ;/skip over characters until
        INC HL                  ;|bit 7 of the current byte is
        JR   Z,M077F           ;\set
        DEC A                  ;decrement msg number
        JR   NZ,M077F          ;not the message we want so,
                                ; back around
        EX  DE,HL              ;put the buffer pointer back in DE
        POP AF                 ;restore the message number index
        CP  $20                ;/return if the message pointer is less than
        RET C                  ;\ $20
        LD  A,(DE)              ;get current character
        SUB $41                ;compare to '@'
        RET                    ;
;*****
; Module: IO_1
; Routine: TVFUL?
; determine if the VIDEO display is full.  if it is exit via
; error 5 - OUT OF SCREEN
; enter with the line number in B
;*****
TVFUL?:
M0790:  BIT  PR,(IY+OFLAGS)      ;/return if printing to printer;
        RET NZ                  ;\nothing more to be done
        LD  DE,STTV             ;/jump address
        PUSH DE                  ;\
        LD  A,B                  ;put line number in A
        BIT LHS,(IY+OTVFLAG)     ;/jump if printing to the lower
        JP  NZ,M083D             ;\half of the screen

        CP  (IY+ODFSZ)           ;/if
        JR  C,M07C1              ;\exit via error 5

        RET NZ                  ;if we are not on the last line, return
        BIT TVLIST,(IY+OTVFLAG)  ;/jump if not an automatic listing
        JR  Z,M07C3              ;\

        LD  E,(IY+OBREG)         ;BREG
        DEC E                    ;
        JR  Z,M080D              ;

```

```

        LD  A,$00                ;/force lower screen stream
        CALL SELECT                ;\
        LD  SP,(LISTSP)           ;get the auto list return address pointer
        RES  TVLIST,(IY+OTVFLAG)  ;force no auto listing
        RET                        ;
;*****
; Module: IO_1
; Routine: ERR5
;*****
ERR5:
M07C1:  RST  $08                ;error: OUT OF SCREEN
        DFB  $04                ;
;*****
; Module:
; Routine:
; ENTRY: B contains the number of lines to scroll
;*****
M07C3:  DEC  (IY+OSCRCT)          ;decrement scroll count
        JR   NZ,M080D            ;jump if scroll count not zero (proceed)
        LD  A,$18                ;24 possible lines
        SUB  B                    ;subtract screen line number
        LD  (SCRCT),A            ;store the number of lines below B
back
; into scroll count
        LD  HL,(ATTRT)           ;/get the temporary printing attributes
        PUSH HL                  ;\ and save them
        LD  A,(PFLAG)            ;/save the printing flags
        PUSH AF                  ;\
        LD  A,$FD                ;/stream -3 (keyboard/lower screen)
        CALL SELECT                ;\
        XOR  A                    ;/
        LD  DE,SCROLLMSG         ;|print the "SCROLL?" message
        CALL PUTMES                ;\
        SET  CLHS,(IY+OTVFLAG)    ;force clearing the lower half screen
        LD  HL,FLAGS              ;
        SET  LMODE2,(HL)          ;force "L" mode at cursor
        RES  KEYHIT,(HL)          ;force no keyboard input
        EXX                        ;/
        CALL RDCH                 ;|read the keyboard
        EXX                        ;\
        CP   " "                  ;/jump if a space
        JR   Z,M083B              ;\
        CP   $E2                  ;/jump if "STOP" token
        JR   Z,M083B              ;\
        OR   $20                  ;force lower case
        CP   $6E                  ;/jump to BREAK if "n" is input
        JR   Z,M083B              ;\
        LD  A,$FE                ;/stream -1 (main screen)
        CALL SELECT                ;\ (select the main screen)
        POP  AF                    ;/restore the printing flags
        LD  (PFLAG),A            ;\
        POP  HL                    ;/restore the temporary attributes
        LD  (ATTRT),HL           ;\

```

```

M080D:  CALL SCRL1                ;scroll the screen one line
        LD  B,(IY+ODFSZ)          ;get display file line counter
        INC B                    ;bump by one
        LD  C,$21                ;start of line
        PUSH BC                  ;save BC
        CALL M09D6               ;get the display file address for line "B"
        LD  A,H                  ;/
        RRCA                     ;|form attribute address for
        RRCA                     ;|the line number in "B"
        RRCA                     ;|
        AND  $03                 ;|
        OR   $58                 ;|
        LD  H,A                  ;\
        LD  DE,$5AE0             ;line 23 attribute address
        LD  A,(DE)               ;A = line 23 attribute
        LD  C,(HL)               ;C = line "B" attribute
        LD  B,$20                ;FOR 32 bytes
        EX  DE,HL                ;HL -> line 23, DE -> line "B"
M082B:  LD  (DE),A                ;/
        LD  (HL),C               ;|swap attributes
        INC DE                   ;|
        INC HL                   ;|
        DJNZ M082B              ;\
        POP BC                  ;restore BC
        RET                     ;
;~~~~~
SCROLLMSG:
        DFB $80
        DFB "scroll",{"?"+$80}
;~~~~~
M083B:  RST  $08                ;/error: BREAK - CONT REPEATS
        DFB $0C                 ;\

M083D:  CP   $02                ;/jump if the requested lower screen line
        JR   C,M07C1            ;\is 1
        ADD  A,(IY+ODFSZ)        ;add the present lower screen size
        SUB  $19                ;/return if more than 24 lines requested
        RET  NC                 ;\

        NEG                     ;
        PUSH BC                 ;
        LD   B,A                ;
        LD   HL,(ATTRT)         ;/save the temporary attributes
        PUSH HL                 ;\
        ;
        LD   HL,(PFLAG)         ;
        PUSH HL                 ;
        CALL DO_ATTS            ;
        LD   A,B                ;
M0857:  PUSH AF                 ;
        LD   HL,DFSZ            ;
        LD   B,(HL)             ;
        LD   A,B                ;
        INC  A                  ;

```

```

        LD    (HL),A                ;
        LD    HL,SPOSNLIN          ;
        CP    (HL)                  ;
        JR    C,M0868              ;
        INC   (HL)                  ;
        LD    B,$18                 ;
M0868:  CALL  SCRLB                 ;
        POP   AF                    ;
        DEC   A                     ;
        JR    NZ,M0857             ;
        POP   HL                    ;
        LD    (IY+OPFLAG),L        ;PFLAG
        POP   HL                    ;
        LD    (ATTRT),HL           ;
        LD    BC,(SPOSNCOL)        ;
        RES   0,(IY+OTVFLAG)       ;TVFLAG
        CALL  STTVC                 ;
        SET   0,(IY+OTVFLAG)       ;TVFLAG
        POP   BC                    ;
        RET                          ;

;*****
; Module: IO_1
; Routine: DO_ATTS
;
;  setup attributes.  if printing to the upper screen, zero MASKT and
;  transfer the border color to ATTRT, else transfer ATTRP to ATTRT.
;  also, if printing to the upper screen, transfer the perm attributes
;  to the temp attributes via R_ATTS.  if printing to the LHS, do
;  put the temporary attributed into the permanent attributes.
;*****
DO_ATTS:
M0888:  XOR    A                    ;zero A
        LD     HL,(ATTRP)           ;get the permanent attributes and mask
        BIT    0,(IY+OTVFLAG)       ;/jump if not printing to LHS
        JR     Z,M0896              ;\
        LD     H,A                  ;load attribute mask (take attrib from
ATTRP)
        LD     L,(IY+OBORDCR)       ;get the border color
M0896:  LD     (ATTRT),HL           ;store into the temp attributes
;*****
; Module: IO_1
; Routine: R_ATTS
;  if ZF=1, transfer the permanent attributes to the temporary attributes
;  if ZF=0, update the temporary attributes
;  ENTER:  A with new attribute flags
;          ZF = 0:  use the flags in PFLAG else use passed flags
;*****
R_ATTS:  LD     HL,PFLAG            ;get the print flag address
        JR     NZ,M08A0            ;
        LD     A,(HL)              ;get the print attribute flags
        RRCA                       ;make the perm flags the temp flags

M08A0:  XOR     (HL)                ;
        AND     $55                 ;save the temporary flags,

```

```

                                ;zero the permanent flags
XOR  (HL)                      ;this transfers in the permanent flags
                                ;into A and puts the temp flags into A
LD   (HL),A                    ;save the flags
RET
;*****
; Module: IO_1
; Routine: K_CLS
;*****
K_CLS:
M08A6:  CALL CLS                ;clear the screen
;*****
; Module: IO_1
; Routine: CLLHS
;*****
CLLHS:
M08A9:  LD   HL,TVFLAG           ;/
RES  5,(HL)                    ;|force no clear LHS when key pressed
SET  0,(HL)                    ;\force printing to LHS
CALL DO_ATT                    ;set up the attributes. effectively
                                ; transfers the permanent attributes to the
                                ; temporary attributes
LD   B,(IY+ODFSZ)              ;get the size of the LHS
CALL CLS_B                     ;clear the LHS
LD   HL,$5AC0                  ;address of end of LHS attributes+1
LD   A,(ATTRP)                 ;get the permanent attributes
DEC  B                          ;normalize line counter for 1 base
JR   M08C9                     ;
M08C2:  LD   C,$20               ;for 32 characters
M08C4:  DEC  HL                  ;point to an attribute byte
LD   (HL),A                    ;store the attribute
DEC  C                          ;/jump around until all 32 characters
JR   NZ,M08C4                  ;\ have been cleared
M08C9:  DJNZ M08C2               ;jump back around for the next line
LD   (IY+ODFSZ),02             ;force 2 lines for the LHS

M08CF:  LD   A,$FD               ;/stream -3 (keyboard/lower screen)
CALL SELECT                     ;\
LD   HL,(CURCHL)                ;point to the current channel address
; store
LD   DE,SENDTV                  ;
AND  A                          ;reset the carry flag
M08DB:  LD   (HL),E              ;/
INC  HL                         ;|update the current channel output
LD   (HL),D                     ;| routine address, then the input
INC  HL                         ;\ routine address
LD   DE,IN_K                    ;
CCF                                ;/"loop" counter
JR   C,M08DB                    ;\
LD   BC,$1721                   ;line 2, start of line
JR   STTV                      ;update the printing parameters
;*****
; Module: IO_1
; Routine: CLS
;*****

```

```

CLS:
M08EA:  LD  HL,$0000                ;/zero the X and Y print
        LD  (XCOORD),HL            ;\coordinates
        RES  ALOS,(IY+OFLAGS2)     ;force no autolisting
        CALL M08CF                  ;
        LD  A,$FE                  ;/stream -2 (main screen)
        CALL SELECT                  ;\
        CALL DO_ATTTS               ;update the atributes
        LD  B,$18                  ;FOR 24 lines
        CALL CLS_B                  ;clear the screen
        LD  HL,(CURCHL)             ;get the address of the current
channel

                                ; output routine
        LD  DE,SENDTV               ;/
        LD  (HL),E                 ;|make the current output channel
        INC  HL                     ;| the TV
        LD  (HL),D                 ;\
        LD  (IY+OSCRCT),01          ;set scroll count to 1
        LD  BC,$1821               ;get ready to set the cursor pointers
                                ; to the top left corner of the
                                ; screen
;*****
; Module: IO_1A
; Routine: STTVC
; set the TV current print position pointers
; ENTER: D contains (24-line number)
;       E contains (33-char number)
;*****
STTVC:
M0914:  LD  HL,PRBUF                ;point to printer buffer
        BIT  PR,(IY+OFLAGS)         ;/jump if printing to
        JR   NZ,M092F               ;\ the printer
        LD  A,B                     ;get the line number into A
        BIT  LHS,(IY+OTVFLAG)       ;/jump if not printing to
        JR   Z,M0929                ;\ the lower half of the screen
        ADD  A,(IY+ODFSZ)            ;add the number of lines in the lower
                                ; screen
        SUB  $18                    ;
M0929:  PUSH BC                     ;briefly save BC
        LD  B,A                     ;/
        CALL M09D6                  ;|find the display file address for the
                                ;\ screen line in "B"
        POP  BC                     ;restore BC

M092F:  LD  A,$21                   ;/form the char position number for
        SUB  C                       ;\ the char position in C
        LD  E,A                     ;/
        LD  D,$00                   ;|form the DF address for the char in
        ADD  HL,DE                  ;\ C
        JP   STTVCU                 ;update character pointers

;*****
; Module: IO_1A
; Routine: SCRL
; SCRL1: entry point to scroll one line

```



```

; SCRLB: entry point to scroll "B" lines
;*****
SCRL1:
M0939:  LD  B,$17          ;second line
SCRLB:
M093B:  CALL M09D6         ;get the line's address in DF1

;
;fill the screen bit map with
;
      LD  C,$08           ;FOR 8 scan lines
M0940:  PUSH BC            ;save scan line counter
      PUSH HL            ;save the display file "FROM" address
      LD  A,B             ;save character lines
      AND $07            ;test if this is the last scan line
                        ; in the character line
      LD  A,B             ;save the character line counter
      JR  NZ,M0954        ;jump if the not the last scan line
                        ; in the character line
M0948:  EX  DE,HL          ;/make DE point to the previous
      LD  HL,$F8E0        ;|(-720)
      ADD HL,DE           ;\scan line
      EX  DE,HL          ;HL now points to the line whose number was in B
                        ;DE points to the line before it
      LD  BC,$0020        ;32 bytes
      DEC A               ;decrement the character line counter
      LDIR               ;move the bytes

M0954:  EX  DE,HL          ;/
      LD  HL,$FFE0        ;|-$20 (32)
      ADD HL,DE           ;|move HL to point back one character line
      EX  DE,HL          ;\
      LD  B,A             ;
      AND $07            ;
      RRCA                ;
      RRCA                ;
      RRCA                ;
      LD  C,A             ;
      LD  A,B             ;
      LD  B,$00           ;
      LDIR               ;

      LD  B,$07           ;
      ADD HL,BC           ;
      AND $F8            ;
      JR  NZ,M0948        ;
      POP HL              ;
      INC H               ;
      POP BC              ;
      DEC C               ;
      JR  NZ,M0940        ;

      CALL M09C3          ;form the address for the attributes
      LD  HL,$FFE0        ;subtract 32 from DE
      ADD HL,DE           ;

```

```

        EX    DE,HL        ;
        LDIR                     ;
        LD    B,$01        ;clear the bottom line
                           ; of the display
;*****
; Module: IO_1A
; Routine: CLS_B
; clear lower B lines of the display
;*****
CLS_B:
M097F:  PUSH BC            ;save the # of lines to clear
        CALL M09D6        ;HL=location in DF1 where the line
                           ; in B resides
        LD    C,$08       ;FOR 8 scan lines per character line
M0985:  PUSH BC            ;save the scan line counter
        PUSH HL           ;save the scan line address
        LD    A,B         ;get the character line number
M0988:  AND    $07         ;number of character lines to clear in this
                           ; block
        RRCA              ;/
        RRCA              ;|*32 bytes/character line
        RRCA              ;\
        LD    C,A         ;initialize the byte counter
        LD    A,B         ;save the # of character lines
        LD    B,$00       ;zero the MS byte of the byte counter
        DEC   C           ;decrement the scan line counter
        LD    D,H         ;/copy the first location to clear
        LD    E,L         ;\ to DE
        LD    (HL),$00    ;zero the first location
        INC   DE          ;bump the "to" address to the next
                           ; byte
        LDIR              ;clear the scan line
        LD    DE,$0701    ;/point to the start of the next
        ADD   HL,DE       ;\ scan line in the next block
        DEC   A           ;decrement the character line counter
        AND   $F8         ;force clearing 8 character lines in the
                           ; next block
        LD    B,A         ;save the character line counter
        JR    NZ,M0988    ;back around for more scan lines
        POP   HL          ;restore the display file pointer
        INC   H           ;move to the next scan line
        POP   BC          ;restore the scan line counter
        DEC   C           ;/back around for more scan lines
        JR    NZ,M0985    ;\
;
;now, do the attributes
;
        CALL M09C3        ;get the start address of the attributes
        LD    H,D         ;/copy the attribute start address to HL
        LD    L,E         ;\
        INC   DE          ;again, bump the "to" pointer
        LD    A,(ATTRP)   ;get the permanent attributes
        BIT   LHS,(IY+0TVFLAG) ;/jump if doing the upper half screen
        JR    Z,M09BB     ;\
        LD    A,(BORDCR)  ;the border color is used for the LHS

```

```

M09BB:  LD  (HL),A           ;change the first attribute
        DEC BC             ;adjust counter
        LDIR              ;change the rest of the attributes
        POP BC            ;restore the number of lines to clear (B)
        LD  C,$21         ;???
        RET               ;done

M09C3:  LD  A,H           ;/
        RRCA              ;|
        RRCA              ;|form the address for the attribute
        RRCA              ;|RAM
        DEC A             ;|
        OR  $50           ;|
        LD  H,A           ;\
        EX  DE,HL         ;DE=start of attributes to change
        LD  H,C           ;/
        LD  L,B           ;\get the number of character lines
        ADD HL,HL         ;*2
        ADD HL,HL         ;*4
        ADD HL,HL         ;*8
        ADD HL,HL         ;*16
        ADD HL,HL         ;*32
        LD  B,H           ;/number of attribute bytes
        LD  C,L           ;\ to change
        RET

;
;form the base address in DF1 of the line number in B
; ENTRY: B contains the line to access
; EXIT: HL points to the location in DF1 where the
;       line resides. basically, perform the computation
;       $4000+$100*INT(line#/2)+(32*(line# AND 7)), where line# = 24-B
;
M09D6:  LD  A,$18         ;max 24 lines to the screen
        SUB B             ;number of lines down from top of
                           ; screen B is
        LD  D,A           ;save A

;32*(line# AND 7)
        RRCA              ;/
        RRCA              ;|LS byte of base address of
        RRCA              ;|line to clear
        AND $E0           ;|
        LD  L,A           ;\
; $100*INT(line#/2)
        LD  A,D           ;/
        AND $18           ;|form the MS byte of the line
        OR  $40           ;|base address
        LD  H,A           ;\
        RET               ;

;
;wait for a key, then clear the lower half screen
; (LHS)
;
        PUSH AF           ;/
        PUSH BC           ;|save registers

```

```

        PUSH DE                ;\
        LD BC,$9C40            ;
M09ED:  DEC BC                  ;/
        LD A,C                  ;|wait until BC=0
        OR B                    ;|
        JR NZ,M09ED            ;\
M09F2:  XOR A                    ;/
        IN A,($FE)              ;|loop until no keys are pressed
        AND $1F                 ;|
        CP $1F                  ;|
        JR Z,M09F2              ;\
        CALL CLLHS              ;clear the lower half screen
        POP DE                  ;/restore registers
        POP BC                  ;|
        POP AF                  ;\
        RET                     ;
;*****
; Module: IO_2
; Routine: K_DUMP
; dump the screen to the printer
;*****
K_DUMP:
        DI                     ;don't interrupt us
        LD B,$B0                ;22 character lines * 8 scanlines per character
        LD HL,DF1               ;HL points to display file #1
M0A08:  PUSH HL                 ;/save the pointers
        PUSH BC                 ;\
        CALL PRSCAN             ;output (HL) to the printer
        POP BC                  ;/restore the registers
        POP HL                  ;\
        INC H                   ;point to the next scan line
        LD A,H                  ;/jump if we have not finished
        AND $07                 ;| 8 scan lines
        JR NZ,M0A1F             ;\
        LD A,L                  ;/bump to the next character
        ADD A,$20               ;|line
        LD L,A                  ;\

;we now update the high byte of the DF address
; if we have not finished a section of the screen
;(lines 0..7, 8..15, or 16..23) we must set H back to
; $40, $48, or $50 depending on the section being printed
;
; at this point H will contain $48, $50, or $58 for the
; first, second, or third segment respectively
;
        CCF                    ;carry will be set when transitioning
                                ; to the next DF segment
        SBC A,A                 ;A=$FF if in the same DF segment or
                                ;A=$00 if going to the next segment
        AND $F8                 ;-8
        ADD A,H                 ;/subtract 8 to put H back to where
        LD H,A                  ;\ it should be
M0A1F:  DJNZ M0A08              ;decrement scan line counter
        JR M0A30                ;jump to finish up

```

```

;*****
; Module: IO_2
; Routine: DUMPPTR
;*****
DUMPPTR:
M0A23:  DI                      ;leave us alone
        LD  HL,PRBUF           ;point to the printer buffer
        LD  B,$08              ;eight scan lines
M0A29:  PUSH BC                ;save scan line counter
        CALL PRSCAN            ;output the buffer to the printer
        POP  BC                ;restore the scan line counter
        DJNZ M0A29            ;back around for all of the scan lines

M0A30:  LD  A,$04              ;/motor off
        OUT ($FB),A           ;\
        EI                    ;interrupts now allowed

CLPRBUF:
M0A35:  LD  HL,PRBUF           ;point to the printer buffer
        LD  (IY+OPRCC),L       ;the printer "cursor" now points
                                ; to the first location of the
                                ; printer buffer
        XOR  A                  ;zero the accum
        LD  B,A                ;FOR 256 bytes
M0A3D:  LD  (HL),A             ;/
        INC  HL                ;|zero the printer buffer
        DJNZ M0A3D            ;\
        RES  PRLEFT,(IY+OFLAGS2) ;force printer buffer empty indication
        LD  C,$21              ;
        JP  STTVVC            ;

;*****
; Module: IO_2
; Routine: PRSCAN
; output the bytes at (HL) to the printer. generally, this will be
; one scan line of character data
; ENTRY: HL points to a 32 byte buffer
; B contains a printer config byte.
;*****
PRSCAN:
M0A4A:  LD  A,B                ;/if B<3 set motor speed slow,
        CP  $03                ;|else set high
        SBC A,A                ;\
        AND $02                ;save the motor speed bit
        OUT ($FB),A           ;turn motor on at selected speed
        LD  D,A                ;save the printer configuration
M0A53:  CALL BREAK?            ;/jump if break not pressed
        JR  C,M0A62            ;\
        LD  A,$04              ;/turn printer motor off
        OUT ($FB),A           ;\
        EI                    ;enable interrupts
        CALL CLPRBUF           ;clear the printer buffer and
                                ; force buffer flag to EMPTY

        RST $08                ;error: BREAK - CONT REPEATS
        DFB $0C                ;

```

```

M0A62:  IN  A,($FB)                ;read printer port
        ADD A,A                  ;/return if printer "NOT CONFIGURED"
        RET M                    ;\
        JR  NC,M0A53              ;jump if not "START OF PAPER"
                                   ; (paper out?)
        LD  C,$20                ;FOR 32 characters
M0A6A:  LD  E,(HL)                ;get the first byte to output
        INC HL                   ;point to the next
        LD  B,$08                ;FOR 8 bits
M0A6E:  RL  D                    ;/rotate a bit into bit 7 of
        RL  E                    ;| D
        RR  D                    ;\
M0A74:  IN  A,($FB)              ;/wait until printer is ready
        RRA                      ;| for the next pixel
        JR  NC,M0A74              ;\
        LD  A,D                  ;/output the pixel to the
        OUT ($FB),A              ;\ printer
        DJNZ M0A6E               ;loop until the bit counter is zero
        DEC C                    ;/loop until the byte counter is
        JR  NZ,M0A6A              ;\ zero
        RET                      ;done
;*****
; Module: IO_2
; Routine: EDIT_K
;*****
EDIT_K:
M0A82:  LD  HL,(ERRSP)            ;/push the error stack pointer onto
        PUSH HL                  ;\the stack
M0A86:  LD  HL,$0BE5              ;/stack $0BE5
        PUSH HL                  ;\
        LD  (ERRSP),SP          ;put the current stack pointer int ERRSP
                                   ; will be $0E3A on entry to BASIC
M0A8E:  CALL RDCH                 ;get a character from the current channel
        PUSH AF                  ;save the char
        LD  D,$00                ;/
        LD  E,(IY+{- $01})       ;|keyboard click
        LD  HL,$00C8             ;|
        CALL PARP                 ;\
        POP  AF                  ;get the character
        LD  HL,$0A8E              ;/return address
        PUSH HL                  ;\
        CP  $0C                  ;/jump if not DELETE
        JR  NZ,M0AB2              ;\
        BIT 5,(IY+OFLAGS2)        ;/jump if DELETE key
        JR  NZ,M0AB2              ;\held down
        BIT 3,(IY+OFLAGS)         ;/jump if not "L" mode
        JR  Z,M0AE7               ;\
M0AB2:  CP  $18                   ;/jump if possibly a letter
        JR  NC,M0AE7              ;\ or token
        CP  $07                   ;/jump if not an editing
        JR  C,M0AE7               ;\key

        CP  $10                   ;/jump if an editing key
        JR  C,M0AF8               ;\

```

```

        LD    BC,$0002                ;
        LD    D,A                    ;save A
        CP    $16                    ;
        JR    C,M0AD2                ;
;AT,TAB
        INC    BC                    ;
        BIT    7,(IY+OFLAGX)        ;
        JP    Z ,M0B84                ;
        CALL    RDCH                ;
        LD    E,A                    ;
;
;INK,PAPER,FLASH,BRIGHT,INVERT,OVER
;
M0AD2:   CALL    RDCH                ;
        PUSH    DE                    ;
        LD    HL,(KCUR)                ;
        RES    0,(IY+OMODE)        ;MODE
        CALL    INSERT                ;
        POP    BC                    ;
        INC    HL                    ;
        LD    (HL),B                ;
        INC    HL                    ;
        LD    (HL),C                ;
        JR    M0AF1                ;
;*****
; Module: IO_2
; Routine: INSA
;*****
INSA:
M0AE7:   RES    0,(IY+OMODE)        ;forces either K/L mode or G mode
        LD    HL,(KCUR)                ;point current character in input
buffer
        CALL    M12B8                ;move character into free RAM
M0AF1:   LD    (DE),A
        INC    DE
        LD    (KCUR),DE
        RET
;
;EDIT,CSR_LT,CSR_RT,CSR_DN,CSR_UP,DELETE,CR,SLUG,UNUSED
;
M0AF8:   LD    E,A                    ;/
        LD    D,$00                ;|form the address into the
        LD    HL,M0B06-EDITKEY        ;|lookup table at M0B06
        ADD    HL,DE                ;\
        LD    E,(HL)                ;/form the address of the
        ADD    HL,DE                ;\edit key handler
        PUSH    HL                ;stack it
        LD    HL,(KCUR)                ;point to the char in the input
buffer
        RET                        ;jump to the edit key handler
;~~~~~
M0B06:   DFB    EDITCMD-$            ;EDIT            $09
        DFB    CSR_LTCMD-$          ;CSR_LT            $66

```

DFB CSR_RTCMD-\$	;CSR_RT	\$6A
DFB CSR_DNCMD-\$	;CSR_DN	\$50
DFB CSR_UPCMD-\$	;CSR_UP	\$B5
DFB DELETECMD-\$	;DELETE	\$70
DFB CRCMD-\$	;CR	\$7E
DFB SLUGCMD-\$	;SLUG	\$CF
DFB UNUSEDKEYCMD-\$	;UNUSED	\$D4

EDITCMD:

```

LD HL,(EPPC)          ;
BIT 5,(IY+OFLAGX)     ;FLAGX
JP NZ,DEL_C           ;
CALL FIND_L           ;
CALL GET_LN           ;
LD A,D                ;
OR E                  ;
JP Z ,DEL_C           ;
PUSH HL               ;
INC HL                ;
LD C,(HL)             ;
INC HL                ;
LD B,(HL)             ;
LD HL,$000A           ;
ADD HL,BC             ;
LD B,H                ;
LD C,L                ;
CALL CHK_SZ           ;
CALL DEL_C            ;
LD HL,(CURCHL)        ;
EX (SP),HL            ;
PUSH HL               ;
LD A,$FF              ;/stream -1 (RAM write)
CALL SELECT           ;\
POP HL
DEC HL
DEC (IY+OEPPC)        ;EPPC
CALL M15AC
INC (IY+OEPPC)        ;EPPC
LD HL,(ELINE)
INC HL
INC HL
INC HL
INC HL
LD (KCUR),HL
POP HL
CALL M1248
RET

```

CSR_DNCMD:

```

BIT 5,(IY+OFLAGX)     ;/jump if expecting user input rather
JR NZ,M0B67           ;\ a program line
LD HL,EPPC            ;HL now contains the pointer to the
                      ; current line in a program listing
CALL NEXT_L           ;point to the next line
JR M0BD4              ;

```



```

M0B67: LD (IY+OERRNR),$10 ;
JR CRCMD

CSR_LTCMD:
CALL M0B97
JR M0B77

CSR_RTCMD:
LD A,(HL)
CP $0D
RET Z
INC HL
M0B77: LD (KCUR),HL
RET

DELETECMD:
CALL M0B97
;*****
; Module: IO_2
; Routine: DELSYM
;
; ENTER: HL contains the address of the item to be deleted
;*****
DELSYM:
LD BC,$0001 ;/delete one byte at (HL)
JP DELREC ;\
M0B84: CALL RDCH ;/read two characters from
CALL RDCH ;\ the current channel
CRCMD: POP HL ;
POP HL ;
M0B8C: POP HL ;
LD (ERRSP),HL ;
BIT 7,(IY+OERRNR) ;/return if error 0/0:0
RET NZ ;\
LD SP,HL ;
RET ;

M0B97: SCF ;force input from input line
CALL M0CFB ;DE points to WORKSP
SBC HL,DE ;
ADD HL,DE ;
INC HL ;
POP BC ;
RET C ;
PUSH BC ;
LD B,H ;
LD C,L ;
M0BA4: LD H,D ;
LD L,E ;
INC HL ;
LD A,(DE) ;
AND $F0 ;
CP $10 ;
JR NZ,M0BB7 ;
INC HL ;

```

```

        LD    A,(DE)                ;
        SUB   $17                    ;
        ADC   A,$00                  ;
        JR    NZ,M0BB7              ;
        INC   HL                     ;
M0BB7:  AND    A                      ;
        SBC   HL,BC                  ;
        ADD   HL,BC                  ;
        EX    DE,HL                 ;
        JR    C,M0BA4                ;
        RET                                ;

CSR_UPCMD:
        BIT   INPLN,(IY+OFLAGX)      ;/return if expecting input from
        RET   NZ                      ;\ the input line
        LD    HL,(EPPC)               ;get listing "current" line number
        CALL  FIND_L                 ;find the line
        EX    DE,HL                  ;
        CALL  GET_LN                 ;DE=line number
        LD    HL,EPPC+1              ;
        CALL  DE_HL                  ;
M0BD4:  CALL  TSLIST                 ;list the line
        LD    A,$00                  ;/stream 0 (lower screen)
        JP    SELECT                 ;\

SLUGCMD:
        BIT   7,(IY+OFLAGX)          ;FLAGX
        JR    Z,CRCMD

UNUSEDKEYCMD:
        JP    M0AE7

M0BE5:  BIT   RETPOS,(IY+OFLAGX2)     ;/return if retype is not possible
after
        JR    Z,M0B8C                ;\ an error
        LD    (IY+OERRNR),$FF         ;error 0
        LD    D,$00                   ;/
        LD    E,(IY+{- $02})          ;|
        LD    HL,$1A90                ;\
        CALL  PARP                    ;make the noise
        JP    M0A86                   ;

;*****
; Module: IO_2
; Routine: DEL_C
;*****
DEL_C:
M0BFD:  PUSH  HL                      ;
        CALL  M0CF6                  ;
        DEC   HL                     ;points to the ENTER in the edit buffer
        CALL  DEL_DE                 ;deletes the bytes between (HL)
                                           ; and (DE)
        LD    (KCUR),HL              ;
        LD    (IY+OMODE),00          ;force "K" mode
        POP   HL                     ;restore HL
        RET                          ;

```

```

;*****
; Module: IO_2
; Routine: IN_K
;*****
IN_K:
    BIT  ECHREQ,(IY+OTVFLAG)      ;/call if echo to screen requested
    CALL NZ,M0C83                ;\
    AND  A                        ;
    BIT  KEYHIT,(IY+OFLAGS) ;
    RET  Z                        ;
    LD   A,(LASTK)                ;
    RES  KEYHIT,(IY+OFLAGS) ;
    PUSH AF                      ;
    BIT  CLHS,(IY+OTVFLAG)        ;
    CALL NZ,M08A9                ;
    POP  AF                      ;
    CP   $20                      ;
    JR   NC,M0C81                ;
    CP   $10                      ;
    JR   NC,M0C60                ;
    CP   $06                      ;
    JR   NC,M0C41                ;
    LD   B,A                      ;
    AND  $01                      ;
    LD   C,A
    LD   A,B
    RRA
    ADD  A,$12
    JR   M0C6B
M0C41:  JR   NZ,M0C4C
    LD   HL,FLAGS2
    LD   A,$08
    XOR  (HL)
    LD   (HL),A
    JR   M0C5A
M0C4C:  CP   $0E
    RET  C
    SUB  $0D
    LD   HL,MODE
    CP   (HL)
    LD   (HL),A
    JR   NZ,M0C5A
    LD   (HL),$00
M0C5A:  SET  3,(IY+OTVFLAG)        ;TVFLAG
    CP   A
    RET
M0C60:  LD   B,A
    AND  $07
    LD   C,A
    LD   A,$10
    BIT  3,B
    JR   NZ,M0C6B
    INC  A
M0C6B:  LD   (IY+{- $2D}),C        ;KDATA
    LD   DE,$0C73

```

```

        JR    M0C79
        LD    A,(KDATA)
        LD    DE,$0C0E
M0C79:  LD    HL,(CHANS)
        INC   HL
        INC   HL
        LD    (HL),E
        INC   HL
        LD    (HL),D
M0C81:  SCF
        RET

M0C83:  CALL DO_ATTTS                ;set up the attributes
        RES   ECHREQ,(IY+0TVFLAG)   ;force no echo
        RES   CLHS,(IY+0TVFLAG)     ;force no clearing of lower half screen
        LD    HL,(SPOSNLCOL)        ;/get and save the print line and
column
        PUSH  HL                    ;\
        LD    HL,(ERRSP)            ;/get and save the error stack pointer
        PUSH  HL                    ;\
        LD    HL,M0CCD              ;/new error stack pointer
        PUSH  HL                    ;|
        LD    (ERRSP),SP            ;\
        LD    HL,(ECHOE)            ;/"jump" address
        PUSH  HL                    ;\
        SCF                          ;
;*****
; Module: IO_2
; Routine: ECHO
;*****
ECHO:
        CALL M0CFB                  ;returns with DE pointing to the WORKSP
                                      ; or ELINE if expecting user input
        EX    DE,HL                  ;
        CALL M15C9                   ;
        EX    DE,HL                  ;
        CALL M162D                   ;
        LD    HL,(SPOSNLCOL)         ;
        EX    (SP),HL                ;
        EX    DE,HL                  ;
        CALL DO_ATTTS                ;
M0CB6:  LD    A,(SPOSNLLIN)           ;
        SUB   D                      ;
        JR    C,M0CE2                ;
        JR    NZ,M0CC4               ;
        LD    A,E
        SUB   (IY+OSPOSNLCOL)        ;SPOSNLCOL
        JR    NC,M0CE2
M0CC4:  LD    A,$20
        PUSH  DE
        CALL SENDTV
        POP   DE
        JR    M0CB6
M0CCD:  LD    D,$00
        LD    E,(IY+{- $02})         ;RASP

```

```

        LD    HL,$1A90
        CALL  PARP
        LD    (IY+OERRNR),$FF          ;ERRNR
        LD    DE,(SPOSNLCOL)
        JR    M0CE4
M0CE2:  POP    DE
        POP    HL
M0CE4:  POP    HL
        LD    (ERRSP),HL
        POP    BC
        PUSH  DE
        CALL  STTVC
        POP    HL
        LD    (ECHOE),HL
        LD    (IY+{OXPTR+1}),00        ;
        RET

M0CF6:  LD    HL,(WORKSP)              ;point to the workspace
        DEC    HL                      ;points to the $80 byte
        AND    A                      ;

M0CFB:  LD    DE,(ELINE)               ;point to the edit buffer
        BIT    INPLN,(IY+OFLAGX)      ;/return if expecting program line
        RET    Z                      ;\
        LD    DE,(WORKSP)             ;/input from input line
        RET    C                      ;\
        LD    HL,(STKBOT)             ;/???
        RET                          ;\
;*****
; Module: IO_2
; Routine: DESLUG
;  deslugs a line until it reaches $0D
;*****
DESLUG:
M0D0D:  LD    A,(HL)                  ;get the current char
        CP    $0E                    ;is it a slug
        LD    BC,$0006                ;number of bytes to skip
        CALL  Z,DELREC                ;delete the fp number
        LD    A,(HL)                  ;get the next char
        INC    HL                     ;bump pointer
        CP    $0D                    ;end of line?
        JR    NZ,M0D0D                ;back around until the end of line
        RET
;*****
; Module: EDIT
; Routine: K_NEW
;*****
K_NEW:  DI                          ;no interrupts
        LD    A,$FF                  ;NEW flag
        LD    DE,(RAMTOP)             ;get sys variables to be saved
        EXX                          ;
        LD    BC,(PRAMT)              ;
        LD    DE,(RASP)               ;
        LD    HL,(UDG)               ;

```

```

        EXX                ;
;*****
; Module: EDIT
; Routine: INIT
; System initialization. Enter with A=0 and DE=FFFF
; and interrupts disabled.
;*****
INIT:
M0D31:  LD    B,A           ;zero out B
        LD    A,$07        ;/set border to
        OUT   ($FE),A      ;\white
        LD    A,$3F        ;/initialize I?
        LD    I,A          ;|and set A to bottom of ram
                                ;\3FFF
        NOP                ;
        NOP                ;
        NOP                ;
        NOP                ;
        NOP                ;
        NOP                ;
;start memory test
        LD    H,D           ;/HL=FFFF
        LD    L,E           ;\
M0D42:  LD    (HL),$02      ;/
        DEC   HL            ;|fill ram from $FFFF
        CP    H             ;|to $4000 with $02
        JR    NZ,M0D42
M0D48:  AND    A            ;reset carry for subtract
        SBC   HL,DE         ;/
        ADD   HL,DE         ;|step through addresses
        INC   HL            ;|
        JR    NC,M0D55
        DEC   (HL)          ;decrement memory location
        JR    Z,M0D55
                                ;set RAMTOP
        DEC   (HL)          ;decrement memory again
        JR    Z,M0D48
                                ;loop again
M0D55:  DEC   HL            ;point to the last valid memory address
;
;fill system variables with their old values via K_NEW
; these will be zero on power-on
;
        EXX                ;swap to the alternate registers
        LD    (PRAMT),BC    ;
        LD    (RASP),DE     ;
        LD    (UDG),HL      ;
        EXX                ;to normal registers

        INC   B             ;/jump if NEW
        JR    Z,M0D7F        ;\

        LD    (PRAMT),HL    ;set P_RAMTOP to discovered value
        LD    DE,$3EAF      ;/initialize the UDG area

```

```

$3F57      LD    BC,$00A8          ;|transfer the character data at $3EAF -
;
EX    DE,HL          ;|to the top of RAM
LDDR           ;\
EX    DE,HL          ;
INC    HL            ;
LD    (UDG),HL       ;set UDG to point to transferred data
DEC    HL            ;point just below UDG
LD    BC,$0040       ;
LD    (RASP),BC      ;set RASP=40, PIP=0
M0D7F:  LD    (RAMTOP),HL        ;set BASIC's RAMTOP to just below the
; UDG area
;*****
; Module: EDIT
; Routine: NEW
;*****
NEW:
LD    HL,$3C00        ;/set CHARS
LD    (CHARS),HL      ;\
LD    HL,$6200        ;/set MSBOT - address of
LD    (MSTBOT),HL     ;\location above machine stack
DEC    HL             ;point to machine stack
LD    (HL),$3E        ;initialize to $3E
DEC    HL             ;/we finally have a stack
LD    SP,HL           ;\
DEC    HL             ;/
DEC    HL             ;|set ERR_SP
LD    (ERRSP),HL      ;\
IM    1               ;set interrupt mode 1
NOP                ;wait a bit
LD    IY,ERRNR        ;initialize IY to address of
; ERR_NR
LD    HL,$6840        ;/set CHANS to point to
LD    (CHANS),HL      ;\channels buffer
LD    DE,$11AA        ;/
LD    BC,$0015        ;|transfer channel data
EX    DE,HL           ;|to RAM buffer
LDIR           ;\
EX    DE,HL           ;
LD    A,$38           ;/
LD    (ATTRP),A        ;|set ATTR_P
LD    (ATTRT),A        ;|set ATTR_T
LD    (BORDCR),A       ;\set BORDCR - border color
LD    HL,$0523        ;/
LD    (REPDEL),HL     ;\set REPDEL
DEC    (IY+{- $3A})    ;set KSTATE to FF? assumed because
;KS_A1
; RAM test should have left it 0
; this should indicate no key pressed
DEC    (IY+{- $36})    ;set $5C40(23556) to FF          ;KS_A2
LD    HL,$11C1        ;/
LD    DE,STRMS        ;|initialize the stream table
LD    BC,$000E        ;|
LDIR           ;\
XOR    A              ;/1. enable dock bank selection

```

```

        OUT  ($FF),A                ;|2. enable 16mS interrupt
                                      ;|3. set normal video mode
                                      ;|4. set attrib to black on white
        SET  PR,(IY+OFLAGS)        ;force printing to printer
        CALL CLPRBUF                ;
        LD   (IY+ODFSZ),02         ;set DF_SZ to 2 lines                ;DFSZ
        CALL M08A6                 ;clear the screen?
        XOR  A                     ;zero A
        SET  4,(IY+OFLAGS)        ;set bit in FLAGS                ;FLAGS
        LD   DE,$1117              ;??? DE gets munged early on in PUTMES
        CALL PUTMES                ;print banner message?
        SET  5,(IY+OTVFLAG)       ;set flag in TV_FLAG            ;TVFLAG
        LD   HL,$0E0B              ;/
        LD   DE,$6000              ;|tranfer the routine that
        LD   BC,$001D              ;|moves the function dispatcher
        LDIR                                ;\code from EXROM to RAM
        CALL M6000                 ;call the transfer routine
        LD   HL,$65CE              ;/initialize the bank stack pointer
        LD   ($65CE),HL            ;\in the function dispatcher
        LD   HL,$08E7              ;/jumps to $08E7 in the EXROM
        CALL M6815                 ;|the return address is popped into
                                      ;\IX and then...

;*****
; these instructions are transferred to 6000 by init @ 0DED
; then called. this routine transfers the function dispatcher
; from the EXROM to its initial RAM location
        LD   A,$01                ;/enable the dock bank
        OUT  ($F4),A              ;\in the HSR
        IN   A,($FF)              ;/
        SET  7,A                  ;|now enable the EXROM
        OUT  ($FF),A              ;\
        LD   HL,$1000              ;/
        LD   DE,$6200              ;|transfer the function
        LD   BC,$0630              ;|dispatcher code
        LDIR                                ;\
        RES  7,A                  ;/disable the EXROM
        OUT  ($FF),A              ;\
        XOR  A                    ;/back to home bank
        OUT  ($F4),A              ;\
        RET                        ;done
;*****

M0E28:  LD   (IY+ODFSZ),02         ;DFSZ
        CALL TSLIST

;*****
; Module: EDIT
; Routine: LED18
; BASIC entry point?
;*****
LED18:
M0E2F:  CALL M133F                ;initialize ELINE,KCUR, and WORKSP pointers

M0E32:  LD   A,$00                ;/stream 0 (lower screen)

```



```

CALL SELECT                                ;\

CALL EDIT_K
CALL M1A27
BIT 7,(IY+OERRNR)                        ;ERRNR
JR    NZ,M0E55
BIT 4,(IY+OFLAGS2)                      ;FLAGS2
JR    Z,M0E8D
LD    HL,(ELINE)
CALL M0D0D
LD    (IY+OERRNR),$FF                    ;ERRNR
JR    M0E32
M0E55: LD    HL,(ELINE)
LD    (CHADD),HL
CALL M1768
LD    A,B
OR    C
JP    NZ,M1158
RST $18                                ;get current char
CP    $0D
JR    Z,M0E28
BIT 0,(IY+OFLAGS2)                      ;FLAGS2
CALL NZ,CLS
CALL M08A9
LD    A,$19
SUB (IY+OSPOSNLIN)                      ;SPOSNLIN
LD    (SCRCT),A
SET 7,(IY+OFLAGS)                      ;FLAGS
LD    (IY+OERRNR),$FF                    ;ERRNR
LD    (IY+ONSPPC),01                    ;NSPPC
LD    (IY+OERRLN),00                    ;ERRLN
CALL M1AD8
M0E8D: HALT
LD    A,(IY+OERRNR)                      ;ERRNR
CP    $FF
JR    Z,M0EC8
BIT 7,(IY+{OERRLN+1})                    ;
JR    Z,M0EC8
SET 6,(IY+{OERRLN+1})                    ;
INC A
LD    (ERRT),A
LD    (IY+OERRNR),$FF                    ;ERRNR
LD    HL,(PPC)
LD    (ERRC),HL
LD    A,(SUBPPC)
LD    (ERRS),A
LD    HL,(ERRLN)
RES 7,H
RES 6,H
LD    (NEWPPC),HL
LD    (IY+ONSPPC),01                    ;NSPPC
LD    HL,$0E8D
PUSH HL
JP    M1AB9
M0EC8: LD    A,$07

```

```

        OUT  ($F5),A
        LD   A,$FF
        OUT  ($F6),A
        RES  3,(IY+0TVFLAG)          ;TVFLAG
        RES  5,(IY+0FLAGS)           ;FLAGS
        BIT  1,(IY+0FLAGS2)          ;FLAGS2
        CALL NZ,M0A23
        LD   A,(ERRNR)
        INC  A

M0EE3:  PUSH AF
        LD   HL,$0000
        LD   (IY+0FLAGX),H           ;FLAGX
        LD   (IY+{0XPTR+1}),H       ;
        LD   (DEFADD),HL
        LD   HL,$0001
        LD   (CH_0),HL
        CALL M133F
        RES  5,(IY+0FLAGX)           ;FLAGX
        CALL M08A9
        SET  5,(IY+0TVFLAG)          ;TVFLAG
        POP  AF
        LD   B,A
        CP   $0A
        JR   C,M0F0C
        ADD  A,$07
M0F0C:  CALL M11EA
        LD   A,$20
        RST  $10
        LD   A,B
        LD   DE,$0F65
        CALL PUTMES
        XOR  A
        LD   DE,$1115
        CALL PUTMES
        LD   BC,(PPC)
        CALL M1788
        LD   A,$3A
        RST  $10
        LD   C,(IY+0SUBPPC)          ;SUBPPC
        LD   B,$00
        CALL M1788
        CALL DEL_C
        LD   A,(ERRNR)
        INC  A
        JR   Z,M0F56
        CP   $09
        JR   Z,M0F43
        CP   $15
        JR   NZ,M0F46
M0F43:  INC  (IY+0SUBPPC)              ;SUBPPC
M0F46:  LD   BC,$0003
        LD   DE,OSPCC
        LD   HL,NSPPC
        BIT  7,(HL)

```

```

        JR      Z,M0F54
        ADD     HL,BC
M0F54:  LDDR
M0F56:  LD      (IY+ONSPPC),$FF          ;NSPPC
        RES     3,(IY+OFLAGS)          ;FLAGS
        RES     3,(IY+OTVFLAG)         ;TVFLAG
        JP      M0E32
        ADD     A,B
        LD      C,A

;*****
; Module: EDIT
; Routine: ERRMSG
;*****
ERRMSG:
        DFB     $CB
        DFB     "NEXT without FO",{"R"+$80}
        DFB     "Variable not foun",{"d"+$80}
        DFB     "Subscript wron",{"g"+$80}
        DFB     "Out of memor",{"y"+$80}
        DFB     "Out of scree",{"n"+$80}
        DFB     "Number too bi",{"g"+$80}
        DFB     "RETURN without GOSU",{"B"+$80}
        DFB     "End of fil",{"e"+$80}
        DFB     "STOP statemen",{"t"+$80}
        DFB     "Invalid argumen",{"t"+$80}
        DFB     "Integer out of rang",{"e"+$80}
        DFB     "Nonsense in BASI",{"C"+$80}
        DFB     "BREAK - CONT repeat",{"s"+$80}
        DFB     "Out of DAT",{"A"+$80}
        DFB     "Invalid file nam",{"e"+$80}
        DFB     "No room for lin",{"e"+$80}
        DFB     "STOP in INPU",{"T"+$80}
        DFB     "FOR without NEX",{"T"+$80}
        DFB     "Invalid I/O devic",{"e"+$80}
        DFB     "Invalid colo",{"r"+$80}
        DFB     "BREAK into progra",{"m"+$80}
        DFB     "RAMTOP no goo",{"d"+$80}
        DFB     "Statement los",{"t"+$80}
        DFB     "Invalid strea",{"m"+$80}
        DFB     "FN without DE",{"F"+$80}
        DFB     "Parameter erro",{"r"+$80}
        DFB     "Tape loading erro",{"r"+$80}
        DFB     "Missing LRO",{"S"+$80}
        DFB     ",",{" "+$80}
        DFB     $7F," 1982 Sinclair Research Ltd", $0D,$0D
        DFB     $7F," 1983 Timex Computer Cor",{"p"+$80}
;*****
; Module: EDIT
; Routine: UNKEDIT
;*****
UNKEDIT:
        LD      A,$10
        LD      BC,$0000
        JP      M0EE3

```

```

M1158:  LD  (EPPC),BC
        LD  HL,(CHADD)
        EX  DE,HL
        LD  HL,$1150
        PUSH HL
        LD  HL,(WORKSP)
        SCF
        SBC HL,DE
        PUSH HL
        LD  H,B
        LD  L,C
        CALL FIND_L
        JR  NZ,M1178
        CALL RECLEN
        CALL DELREC
M1178:  POP  BC
        LD  A,C
        DEC A
        OR  B
        JR  Z,M11A6
        PUSH BC
        INC BC
        INC BC
        INC BC
        INC BC
        DEC HL
        LD  DE,(PROG)
        PUSH DE
        CALL INSERT
        POP  HL
        LD  (PROG),HL
        POP  BC
        PUSH BC
        INC DE
        LD  HL,(WORKSP)
        DEC HL
        DEC HL
        LDDR
        LD  HL,(EPPC)
        EX  DE,HL
        POP  BC
        LD  (HL),B
        DEC HL
        LD  (HL),C
        DEC HL
        LD  (HL),E
        DEC HL
        LD  (HL),D
M11A6:  POP  AF
        JP  M0E28
;~~~~~
;*****
; Module: EDIT
; Routine: CHINIT

```

```

; channels initialization data
;*****
CHINIT:
    DFB $00
    DFB $05
    DFB $0E
    DFB $0C
    DFB $4B
    DFB $00
    DFB $05
    DFB $BF
    DFB $11
    DFB $53
    DFB $E7
    DFB $0A
    DFB $BF
    DFB $11
    DFB $52
    DFB $00
    DFB $05
    DFB $BF
    DFB $11
    DFB $50
    DFB $80

INVI:  RST $08
       DFB 12H ;'INVALID I/O'
;*****
; Module: EDIT
; Routine: SMINIT
;  streams initialization data
;*****
SMINIT:
    DFB $01
    DFB $00
    DFB $06
    DFB $00
    DFB $0B
    DFB $00
    DFB $01
    DFB $00
    DFB $01
    DFB $00
    DFB $06
    DFB $00
    DFB $10
    DFB $00

;*****
; Module: EDIT
; Routine: RDCH
;  read a character from the current channel,
;  deposit it in A
;*****
RDCH:

```

```

M11CF:  BIT   CLHS,(IY+OTVFLAG)           ;/jump if not clearing LHS
        JR    NZ,M11D9                   ;\ on key press
        SET   ECHREQ,(IY+OTVFLAG)        ;force echo keyboard input
M11D9:  CALL  INCH                       ;input a character from the current
                                           ; channel

        RET   C                          ;return with the character
        JR    Z,M11D9                    ;back around for the next
                                           ; character

        RST   $08                        ;/error 8: END OF FILE
        DFB   $07                        ;\
;*****
; Module: EDIT
; Routine: INCH
;  input a character from the current channel in CURCHL (into A?)
;*****
INCH:
M11E1:  EXX                                ;to alt registers
        PUSH  HL                         ;briefly save HL
        LD    HL,(CURCHL)                ;get current channel routine address
        INC   HL                         ;/point to the input routine
        INC   HL                         ;\
        JR    M11F2                      ;get the character
;*****
; Module: EDIT
; Routine: PUTDIG
;  send a decimal digit whose binary value is in A to the current
;  channel
;*****
PUTDIG:
M11EA:  LD    E,$30                      ;/make the digit ASCII
        ADD   A,E                        ;\
;*****
; Module: EDIT
; Routine: SENDCH
;  send a character in A to the current stream
;*****
SENDCH:
M11ED:  EXX                                ;to the alt registers
        PUSH  HL                         ;save HL
        LD    HL,(CURCHL)                ;get CURCHL - the current channel
;
;this entry point outputs to the channel whose output routine address
; is in H'L' if CRCBN is 0 or 1.
;on entry registers MUST point to the alt reg set
;
M11F2:  EX    AF,AF'                     ;to alt accum
        LD    A,(CRCBN)                  ;get the current channel bank number
        CP    $02                        ;
        JR    NC,M1205                    ;
        EX    AF,AF'                     ;back to normal accum
        LD    E,(HL)                     ;/
        INC   HL                         ;|get output routine address
        LD    D,(HL)                     ;\for the current channel

```

```

EX    DE,HL                ;output rtn adr to HL, CURCHL to DE
CALL  M1264                ;"CALL" channel handler
POP   HL                   ;restore HL
EXX                                ;back to the normal regs
RET

M1205:  EX    AF,AF'        ;back to normal accum
        LD    HL,(CURCHL)   ;get the current channel pointer
        LD    B,(HL)        ;
        LD    C,$88        ;

;this trashes A, the character we wish to send
; bug?
        LD    A,(ARSFLG)    ;cartridge variable, zero on power up
        BIT   0,A           ;/jump if AROS is running
        JR    NZ,M1215      ;\
        INC   HL            ;/get the
        INC   HL            ;\
M1215:  LD    A,(STRMN)      ;get current stream number
        LD    E,A           ;/put in DE to save it
        LD    D,$00         ;\
        PUSH  DE            ;save the stream pointer
        LD    DE,$0007      ;
        ADD   HL,DE         ;
        PUSH  HL            ;call address
        PUSH  BC            ;C=horiz sel, B=bank
        LD    BC,$0002      ;two parameters out
        PUSH  BC            ;
        LD    BC,$0000      ;no parameters in
        PUSH  BC            ;
        CALL  M65D0         ;CALL_BANK
        POP   HL            ;restore the current stream number
        EXX                                ;
        RET

;*****
; Module: EDIT
; Routine: SELECT
;  select a stream
;  select the stream number in A
;*****
SELECT:
M1230:  ADD   A,A            ;double stream number
        ADD   A,$16         ;add offset into stream table
        LD    L,A           ;/form address in HL
        LD    H,$5C         ;\
        LD    E,(HL)        ;/get pointer to the stream
        INC   HL            ;|routine
        LD    D,(HL)        ;\
        LD    A,D           ;/is entry zero?
        OR    E             ;\
        JR    NZ,M123F      ;jump if stream entry is not zero
;*****
; Module: EDIT
; Routine: ERRO
;*****

```

```

ERRO:
M123D:  RST  $08          ;/error: INVALID STREAM
        DFB  $17          ;\

M123F:  CP   $80          ;/jump if stream number is negative
        JR   NC,M1265      ;\
        DEC  DE           ;
        LD   HL,(CHANS)    ;get CHANS
        ADD  HL,DE         ;
;*****
; Module: EDIT
; Routine: SEL_HL
; select the channel whose address is in HL
;*****
SEL_HL:
M1248:  LD   (CURCHL),HL    ;make the channel address in HL the
                                ; current channel
        LD   A,$00         ;/force bank number of current channel
        LD   (CRCBN),A     ;\ to be zero
        RES  4,(IY+OFLAGS2) ;force no retype possible after syntax error
        INC  HL            ;
        INC  HL            ;
        INC  HL            ;
        INC  HL            ;
        LD   C,(HL)        ;get channel type (K,S,P)
        LD   HL,KLOOK      ;/find in the table
        CALL SEARCH        ;\
        RET  NC            ;return if not found (not K,S,P)
        LD   D,$00         ;/
        LD   E,(HL)        ;|form the address to the routine
        ADD  HL,DE         ;\

M1264:  JP   (HL)          ;jump to the output routine

M1265:  LD   HL,(SYSCON)    ;get SYSCON
        SUB  $80           ;
        LD   D,A           ;
        ADD  HL,DE         ;
        LD   (CURCHL),HL    ;get CURCHL
        LD   A,(HL)
        LD   (CRCBN),A     ;store in CURCBN
        RES  4,(IY+OFLAGS2) ;FLAGS2
        INC  HL
        INC  HL
        INC  HL
        INC  HL
        INC  HL
        INC  HL
        LD   A,(CRCBN)     ;get CURCBN
        LD   B,A           ;
        LD   C,$88         ;
        LD   D,(HL)        ;
        INC  HL            ;
        LD   E,(HL)        ;
        LD   H,D           ;

```



```

        LD    L,E            ;
        PUSH HL              ;call adr
        PUSH BC              ;C=horiz sel, B=bank
        LD    BC,$0000      ;no params
        PUSH BC              ;
        PUSH BC              ;
        CALL M65D0          ;CALL_BANK
        RET

;~~~~~
KLOOK:
        DFB "K"
        DFB KANALK-$
        DFB "S"
        DFB KANALS-$
        DFB "P"
        DFB KANALP-$
        DFB 0                ;end of table

KANALK: SET LHS,(IY+OTVFLAG)    ;force printing to the lower half of
the screen
        RES KEYHIT,(IY+OFLAGS) ;force no keyhit
        SET RETPOS,(IY+OFLAGS2) ;no retype possible after syntax error
        JR KANS1

KANALS: RES LHS,(IY+OTVFLAG)    ;print to the top half of the screen
KANS1:  RES PR,(IY+OFLAGS)      ;print to TV
        JP DO_ATTS             ;jump to set the attributes

KANALP: SET PR,(IY+OFLAGS)      ;force print to printer
        RET

;*****
; Module: EDIT
; Routine: INSI
; insert 1 byte at (HL)
;*****
INSI:
M12B8: LD    BC,$0001
;*****
; Module: EDIT
; Routine: INSERT
; make BC bytes available
; move BC items at (HL), into spare RAM if available
; ENTRY: BC - # of items to move
;        HL - start address of area to move
;
; EXIT:  HL - original address-1
;        DE - HL+BC
;*****
INSERT:
M12BB: PUSH HL                ;save insertion address
        CALL CHK_SZ           ;check for sufficient RAM
                                ; error 4 if not
        POP HL                ;restore insertion address

```

```

        CALL REMGSZ                ;adjust various pointers in the system
variables

;DE now points to the old STKEND (just before the free RAM) and HL points to
; the fence value (will be discarded). BC will contain the length of the
; memory block to be moved. this will usually be different from the entry
; value
        LD    HL,(STKEND)          ;get STKEND (updated start of spare space)
        EX    DE,HL                ;HL = entry value, DE = HL + BC
        LDDR                     ;move the bytes up to open buffer
        RET                        ;
;*****
; Module: EDIT
; Routine: REMGSZ
; this routine updates various pointers in the system variables.
; ENTRY: HL = memory address
;        BC = bytes to move
; on entry HL contains a memory address that is compared to pointers
; starting with ARSBUF. if any pointer points to an address below
; the value stored in HL, the pointer is incremented by BC and restored
; to its original place.
;
; EXIT: DE = contains old value of STKEND
;        HL = the fence
;        BC = updated to the block size to move. this will, in general,
;        be different than the entry value because the system will
;        need to move large numbers of bytes to allow insertion of
;        program lines.
;*****
REMGSZ:
M12CA:  PUSH AF                    ;/save registers
        PUSH HL                    ;\
;
;this section of code adjusts the pointer to the AROS buffer stored
; in ARSBUF, if needed
;
        LD    HL,ARSBUF            ;get AROS buffer pointer
        LD    E,(HL)               ;/copy to DE, DE now points to the
        INC    HL                  ;|AROS buffer
        LD    D,(HL)               ;\
        EX    (SP),HL              ;save ARSBUF and get old HL
        AND    A                   ;/set carry if HL<DE
        SBC    HL,DE               ;|(if address in HL is lower than the
        ADD    HL,DE               ;\AROS buffer) (does CP HL,DE)
        EX    (SP),HL              ;get ARSBUF back and save HL
        JR    NC,M12E0             ;jump if the address in HL is after the AROS
        EX    DE,HL                ;DE now has ARSBUF
; and HL has the address of the AROS buffer
        ADD    HL,BC               ;adjust the address of the AROS buffer
; by BC bytes
        EX    DE,HL                ;HL now has ARSBUF and DE
; has the new address of the AROS buffer
        LD    (HL),D               ;/
        DEC    HL                  ;|store the value in ARSBUF
        LD    (HL),E               ;\

```

```

;
STACK
;this section adjusts the system variables VARS through STKEND
;
M12E0:  LD    HL,VARS                ;get the address for VARS
POINTER          LIMIT
        LD    A,$0E                ;number of variables to update

M12E5:  CP    $09                  ;\NXTLIN - needs to be tested against ARSFLG
        JR    Z,M12ED              ;\because it may be in the AROS buffer
        CP    $08                  ;\DATADD - data statement address,
        JR    NZ,M12FA             ;\same situation as NXTLIN

;
;test the AROS present flag, if an AROS is present, skip the pointer update
;
M12ED:  PUSH HL                    ;briefly save HL
POINTER
        POINTER,LIMIT
        LD    HL,ARSFLG            ;/
ARSFLG
        POINTER,LIMIT
        LD    L,(HL)               ;|get the AROS flag and test
        BIT   7,L                  ;\to see if an AROS is present
        POP   HL                   ;restore HL
POINTER
LIMIT
        JR    Z,M12FA              ;jump if AROS not present
        INC   HL                   ;point to the high byte of the pointer
        JR    M130E                ;skip the addition routine

;
;update a pointer by determining if it points above the fence address in HL.
if it does,
; add BC to the pointer and then stuff it back into the system variables
;
M12FA:  LD    E,(HL)               ;/get address pointer at (HL) into DE
POINTER (POINTER)  LIMIT
        INC   HL                   ;| perhaps VARS from M12E0
        LD    D,(HL)               ;\
        EX    (SP),HL              ;save VARS and get HL
LIMIT (POINTER)  POINTER
        AND   A                    ;/
        SBC   HL,DE                ;|jump if HL >= (POINTER);
        ADD   HL,DE                ;|(doesn't neede updataing since we are
        EX    (SP),HL              ;| moving stuff above where it points)
POINTER (POINTER)  LIMIT
        JR    NC,M130E             ;\

;
;adjust the pointer by adding BC to it, then save it back to the system
variables
;
        PUSH DE                    ;save variables area address
POINTER (POINTER)  (POINTER),LIMIT
        EX    DE,HL                ;HL=(VARS), DE=address
(POINTER) POINTER  (POINTER),LIMIT
        ADD   HL,BC                ;/adjust the variables pointer
(POINTER+) POINTER  (POINTER),LIMIT

```

```

        EX   DE,HL           ;\by BC
POINTER (POINTER+) (POINTER),LIMIT
        LD   (HL),D          ;/
        DEC  HL              ;|
        LD   (HL),E          ;\
        INC  HL              ;point to MSB of just adjusted pointer
        POP  DE              ;
                                POINTER
(POINTER) LIMIT
;
;now look at the next pointer and decrement the counter in A that tells us
; how many pointers are left to update.  if there are more, loop back again
;
M130E:  INC  HL              ;bump pointer pointer to next pointer
        DEC  A              ;decrement item counter
        JR   NZ,M12E5       ;back around to update more pointers

        EX   DE,HL          ;
                                (POINTER)
POINTER+2 LIMIT
        POP  DE              ;
                                (POINTER)
LIMIT   AF
        POP  AF              ;
                                (POINTER)
LIMIT   RETURN ADR
        AND  A              ;
        SBC  HL,DE          ;/find the number of bytes between the fence
        LD   B,H            ;|and where old STKEND pointed and copy it to
        LD   C,L            ;|BC. this sets the size of the memory block to
                                ;\be moved
        INC  BC              ;bump BC
        ADD  HL,DE          ;restore HL
        EX   DE,HL          ;HL again contains the fence, DE contains old
STKEND
        RET                  ;
                                ;weebiedun

DUMMYLINE:
        DWL  $0000

M1320:  EX   DE,HL          ;point to the previous line
        LD   DE,DUMMYLINE   ;allow an exit by pointing to
                                ; a dummy line number
;*****
; Module: EDIT
; Routine: GET_LN
; get the line number into DE from (HL)
; ENTRY: HL points to the program line number
;        DE points to the previous program line number
; EXIT:  HL points to the low byte of the line number
;        DE contains the line number
;*****
GET_LN:
M1324:  LD   A,(HL)          ;get the high byte of the line number
        AND  $C0            ;/jump if not a line number
        JR   NZ,M1320       ;\
        LD   D,(HL)         ;/

```

```

        INC HL          ;|load the line number into DE
        LD E,(HL)       ;\
        RET             ;exit
;*****
; Module: EDIT
; Routine: LCU2
; insert BC items at (STKBOT) (the temporary work space).
; moves the calc stack up BC bytes.
; ENTRY: BC - number of items to insert
; STACK: (tos) new WORKSP pointer, value for BC
; EXIT: DE points to the first byte of of the reserved area
; HL = points two bytes into into the moved memory block
; BC updated from stack
;*****
LCU2:
M132D: LD HL,(STKBOT)    ;get STKBOT
        DEC HL          ;point to address just below
                        ;calc stack
        CALL INSERT      ;insert BC items at (HL), updataing system
variables
        INC HL          ;
        INC HL          ;
        POP BC          ;/get saved value for WORKSP and put it there
        LD (WORKSP),BC  ;\
        POP BC          ;pop number of items to insert
        EX DE,HL        ;
        INC HL          ;HL = 2 bytes into the moved memory block
                        ;DE = the first byte of the new block
        RET             ;

M133F: LD HL,(ELINE)    ;point to the edit line
        LD (HL), $0D     ;drop a newline there
        LD (KCUR),HL     ;current character pointer
        INC HL          ;/put sentinal byte in ELINE buffer
        LD (HL), $80     ;\
        INC HL          ;/initialize the workspace pointer
        LD (WORKSP),HL ;\
;*****
; Module: EDIT
; Routine: X_CALC
;*****
X_CALC:
M134E: LD HL,(WORKSP)
        LD (STKBOT),HL
;*****
; Module: EDIT
; Routine: RESET
; reset the floating point stack and floating point memory pointers
; EXIT with STKBOT in HL
;*****
RESET:
M1354: LD HL,(STKBOT)   ;/collapse the fp stack
        LD (STKEND),HL ;\
        PUSH HL         ;save STKBOT
        LD HL, MEMBOT   ;/

```

```

        LD    (MEM),HL                ;\reset fp memory pointer
        POP   HL                      ;restore STKBOT
        RET                                ;done
;*****
; Module: EDIT
; Routine: X_T_HL
;*****
X_T_HL:
        LD    DE,(ELINE)
        JP    M174D

;*****
; Module: EDIT
; Routine: SEARCH
; searches for an item in C starting at (HL). terminates when item
; found or when 0 found.
; on exit HL points one past the item if found or to the 0 byte if
; 0 was found. CF=0 if zero found, else CF=1 if item in C was found
;*****
M136A:   INC    HL
SEARCH:
M136B:   LD     A,(HL)                ;get byte
        AND    A                      ;test for zero
        RET    Z                      ;return if zero (CF reset)
        CP     C                      ;compare with C
        INC    HL                     ;bump the pointer
        JR     NZ,M136A               ;dint find it, so back around
        SCF                                ;found the byte, so set C
        RET                                ;done
;*****
; Module: EDIT
; Routine: SRCHSC
; search the system configuration table for the item in C
;*****
SRCHSC:
M1374:   LD     HL,(SYSCON)            ;get SYSCON
        LD     DE,$000C                ;\step past the AROS and LROS
        ADD    HL,DE                  ;\entries
M137B:   LD     A,(HL)                ;get syscon table item
        CP     $80                    ;end of table?
        JR     Z,M139A                ; desired entry
        INC    HL                     ;/
        INC    HL                     ;\
        CP     $01                    ;/bank not in use, so check
        JR     NZ,M138A               ;/check for bank signature,
        LD     A,(HL)                ;|if found exit routine
        CP     C
        JR     Z,M139C
M138A:   PUSH   HL                    ;save pointer
        EX     DE,HL                  ;/
        LD     DE,$0018                ;|skip to next syscon table
        ADD    HL,DE                  ;\entry+2
        EX     DE,HL                  ;DE points to new entry
        POP    HL                     ;get old pointer

```

```

        PUSH DE                ;save new pointer
        LD  DE,$0016          ;/make old pointer point
        ADD HL,DE              ;\to new syscon entry
        POP DE                 ;restore DE
        JR  M137B
M139A:  AND  A                  ;adjust flags
        RET                   ;exit
                                ;found the item
M139C:  DEC  HL                 ;go back to point to ??
        SCF                     ;set the carry
        RET                     ;exit
;*****
; Module: CHANS
; Routine: CLOSE
;*****
CLOSE:  CALL M140F              ;get the stream whose number is on the
                                ; top of the calculator stack
        LD  A,B                ;/
        OR  C                   ;|return if the stream routine
        RET Z                   ;\address is zero (invalid stream)
        CALL M13BE
;*****
; Module: CHANS
; Routine: RSTSTR
; reset stream?
;*****
RSTSTR:
M13A8:  LD   BC,$0000          ;
        LD   DE,$A3E2          ;
        EX   DE,HL              ;
        ADD  HL,DE              ;
        JR   C,M13B9           ;
        LD   BC,$11CF          ;address of RD_CH
        ADD  HL,BC              ;
        LD   C,(HL)             ;
        INC  HL                 ;
        LD   B,(HL)             ;
M13B9:  EX   DE,HL              ;
        LD   (HL),C             ;
        INC  HL                 ;
        LD   (HL),B             ;
        RET                     ;
;*****
; Module: CHANS
; Routine: CLCHAN
; close a channel
; ENTER: HL points to stream address pointer
; BC is the address of the stream routine
;*****
CLCHAN:
M13BE:  PUSH HL                ;save stream address pointer
        LD   A,B                ;/jump if the stream is negative
        CP   $80                ;|
        JR   NC,M13D8           ;\
        LD   HL,(CHANS)         ;get the address of the CHANS area

```

```

        ADD HL,BC          ;add
        INC HL             ;
        INC HL             ;
        INC HL             ;
        LD C,(HL)          ;get the stream type "K","S", or "P"
        EX DE,HL           ;save the pointer in HL
        LD HL,CSTRTA       ;point to the close stream table
        CALL SEARCH        ;search for the stream type
        LD C,(HL)          ;/get the offset and add it to
        LD B,$00           ;|HL to give the address of the
        ADD HL,BC          ;\close stream routine
        JP (HL)            ;close the stream

M13D8:  SUB $80            ;normalize the number
        LD B,A             ;
        LD HL,(SYSCON)
        ADD HL,BC
        LD A,(HL)
        CP $00
        RET Z
        CP $80
        RET Z
        INC HL
        LD B,(HL)
        INC HL
        INC HL
        INC HL
        INC HL
        LD E,(HL)
        INC HL
        LD D,(HL)
        LD H,D
        LD L,E
        LD A,(STRMN)
        LD E,A
        LD D,$00
        PUSH DE
        PUSH HL
        PUSH BC            ;C=horiz sel,B=bank
        LD BC,$0002        ;/PARAM_OUT
        PUSH BC            ;\
        LD BC,$0000        ;no PARAM_IN
;*****
; Module: CHANS
; Routine: CLEL
; when entered the caller MUST have set up the stack with all
; but the PARAM_IN parameter for a CALL_BANK. BC must contain
; the PARAM_IN parameter
;*****
CLEL:   PUSH BC             ;push PARAM_IN parameter
        CALL M65D0         ;CALL_BANK
        POP HL
        RET
;~~~~~
; close stream table

```



```

;
CSTRTA:      DFB "K"
             DFB CLOSTR-$
             DFB "S"
             DFB CLOSTR-$
             DFB "P"
             DFB CLOSTR-$

CLOSTR: POP HL
        RET
;~~~~~

M140F:  CALL INS_U1                ;convert the top of the calculator stack
                                           ; to a byte in A.  if the number is greater
                                           ; than 255, we won't return here
        LD  (STRMN),A              ;update current stream number
        CP  $10                    ;/jump if stream number is valid,
        JR   C,M141B                ;\

M1419:  RST  $08                    ;error: INVALID STREAM
        DFB  $17                    ;

M141B:  ADD  A,$03                  ;first 3 streams cannot be messed with
        RLCA                        ;/
        LD  HL,STRMS                ;|point to entry in streams table
        LD  C,A                     ;|
        LD  B,$00                    ;|
        ADD HL,BC                    ;\
        LD  C,(HL)                  ;/get the address for the
        INC HL                       ;| stream routine to BC
        LD  B,(HL)                  ;\
        DEC HL                       ;restore HL
        RET                          ;

;*****
; Module: CHANS
; Routine: OPEN
; enter with ??? in A
;*****
OPEN:   CP  ","                      ;/jump if current character is a comma
        JR  Z,M1433
        CALL END?                    ;will return if interpreting, else
                                           ; will exit via ENDTEM

        JR  M143E

M1433:  CALL INTPT?                  ;/jump if interpreting
        JR  NZ,M143E                ;\
        CALL SKIPIT                  ;skip to the end of a statement
        CALL END?                    ;

M143E:  RST $28                      ;calc entry
        DFB $01                      ;SWAP  ;
        DFB $38                      ;QUIT  ;
        CALL M140F                    ;jump to error
        LD  A,B                      ;
        OR  C                        ;
        JR  Z,M145E
        EX  DE,HL                    ;

```

```

        LD    HL,(CHANS)          ;
        ADD   HL,BC               ;
        INC   HL                  ;
        INC   HL                  ;
        INC   HL                  ;
        LD    A,(HL)              ;
        EX    DE,HL              ;
        CP    $4B                 ;
        JR    Z,M145E             ;
        CP    $53                 ;
        JR    Z,M145E             ;
        CP    $50                 ;
        JR    NZ,M1419            ;
M145E:  CALL  M1465                ;
        LD    (HL),E              ;
        INC   HL                  ;
        LD    (HL),D              ;
        RET                      ;
;*****
; Module: CHANS
; Routine: OPCHAN
;*****
OPCHAN:
M1465:  PUSH  HL
        CALL  PGPSTR              ;get a string descriptor
        DEC   BC                  ;/
        LD    A,B                 ;|jump if the string length
        OR    C                   ;| is one
        JR    Z,M1472             ;\

M146E:  RST   $08                 ;/error: INVALID IO DEVICE
        DFB   $12                 ;\

M1470:  RST   $08                 ;/error: INVALID SAVE NAME
        DFB   $0E                 ;\no name or name more than 10 chars

M1472:  INC   BC                  ;restore the channel designator string length
        PUSH  BC                  ;briefly save it
        LD    A,(DE)              ;get the channel designator character
        AND   $DF                 ;force upper case
        LD    C,A                 ;/find the channel type in the channel open
        LD    HL,OPTAB            ;|table
        CALL  SEARCH              ;\
        JR    NC,M1486            ;not in the table to jump to INVALID IO
DEVICE
        LD    C,(HL)              ;/
        LD    B,$00               ;|form the address of the needed open routine
        ADD   HL,BC               ;\
        POP   BC                  ;restore the string length
        JP    (HL)                ;jump to the open routine
M1486:  JR    M146E

M1488:  CALL  M1374
        JR    NC,M146E
        POP   BC

```

```

DEC BC
LD A,B
OR C
JR NZ,M146E
PUSH DE
EX DE,HL
CALL M25B9
EX DE,HL
LD B,(HL) ;get bank
LD C,$88 ;all chunks but chunk 0 assigned to new bank
INC HL
INC HL
LD E,(HL)
INC HL
LD D,(HL)
LD H,D
LD L,E
LD A,(STRMN)
LD E,A
LD D,$00
PUSH DE ;
PUSH HL ;call address
PUSH BC ;B=horiz sel, C=bank
LD HL,(STKEND) ;/
LD C,(HL) ;\get required number of output parameters
DEC HL ;trash the parameter
LD (STKEND),HL ;resave STKEND
LD B,$00 ;
INC BC ;/bump output params count by 2
INC BC ;\
PUSH BC ;push output parameters
LD BC,$0000 ;/
PUSH BC ;\no input parameters
CALL M65D0 ;CALL_BANK
POP DE
LD A,D
ADD A,$80
LD D,A
POP HL
RET
;~~~~~
;Table for opening streams
OPTAB: DFB "K"
      DFB OPENK-$

      DFB "S"
      DFB OPENS-$

      DFB "P"
      DFB OPENP-$
      DFB 0

OPENK: LD E,1
      JR M14DB

```

```
OPENS:  LD E,6
        JR M14DB
```

```
OPENP:  LD E,10H
M14DB:  DEC BC
        LD A,B
        OR C
        JP NZ,M1470
        LD D,A
        POP HL
        RET
```

```
;*****
; Module: TSLIST
; Routine: TSLIST
;*****
TSLIST:
M14E1:  LD  (LISTSP),SP      ;save the stack pointer for the listing
        LD  (IY+OTVFLAG),$10 ;force output automatic listing

;clear the upper screen
        CALL CLS           ;clear the upper screen
;clear the lower screen
        SET  LHS,(IY+OTVFLAG) ;force printing to lower half of screen
        LD   B,(IY+ODFSZ)     ;get lower screen size
        CALL CLS_B           ;clear the lower screen for B lines
        RES  LHS,(IY+OTVFLAG) ;release writing to the lower screen

        SET  ALOS,(IY+OFLAGS2) ;force auto listing on screen
        LD   HL,(EPPC)        ;get the current listing line number
        LD   DE,(STOP)       ;get the line listed at top of screen
        AND  A                ;/
        SBC  HL,DE            ;|CP HL,DE
        ADD  HL,DE            ;\
        JR   C,M152D          ;jump if top line is greater than
                                ; the current listing line
        PUSH DE                ;save the top line
        CALL FIND_L           ;find the current line
        LD   DE,$02C0        ;
        EX   DE,HL            ;
        SBC  HL,DE            ;
        EX   (SP),HL          ;
        CALL FIND_L           ;
        POP  BC
M151A:  PUSH BC
        CALL RECLLEN
        POP  BC
        ADD  HL,BC
        JR   C,M1530
        EX   DE,HL
        LD   D,(HL)
        INC  HL
        LD   E,(HL)
        DEC  HL
        LD   (STOP),DE
```

```

        JR      M151A
M152D:  LD      (STOP),HL
M1530:  LD      HL,(STOP)
        CALL   FIND_L
        JR      Z,M1539
        EX     DE,HL
M1539:  CALL   M157F
        RES    4,(IY+0TVFLAG)          ;TVFLAG
        RET
;*****
; Module: LIST
; Routine: K_LLST
;*****
K_LLST:
        LD      A,$03
        JR      M1547
;*****
; Module: LIST
; Routine: K_LIST
;*****
K_LIST:
        LD      A,$02                  ;stream 2 (print/list)
M1547:  LD      (IY+0TVFLAG),00          ; printing to upper half of screen
                                         ; not outputting line for edit or number for
string
                                         ; no keyboard echo
                                         ; no automatic listing
                                         ; do not clear lower half screen when key
pressed
        CALL   INTPT?                  ;select stream 2 if interpreting
        CALL   NZ,SELECT                ;\
        RST    $18                      ;get current character
        CALL   STRITO                   ;update the current stream for "bank"
devices
        JR      C,M156B                 ;jump if not valid instruction

        RST    $18                      ;get current character
        CP     "; "                     ;/
        JR      Z,M1560                 ;\
        CP     ", "                     ;
        JR      NZ,M1566                ;

M1560:  RST    $20                      ;get next char
        CALL   TEM6                     ;get the first line to be listed
        JR      M156E                   ;

M1566:  CALL   STKFP0                   ;stack fp zero if not syntax checking
        JR      M156E                   ;

M156B:  CALL   M1C49                    ;list from line "zero"

M156E:  CALL   END?                     ;we will return here if executing
        CALL   FIX_U                    ;get the first line number to be listed
        LD      A,B                     ;force the number to be a valid
        AND     $3F                     ;\ line number

```

```

        LD    H,A                ;/load the line number into HL
        LD    L,C                ;\
        LD    (EPPC),HL         ;update the current line number in
the listing
        CALL  FIND_L            ;
M157F:   LD    E,$01             ;
M1581:   CALL  M15A1             ;
        RST   $10               ;
        BIT   4,(IY+0TVFLAG)    ;TVFLAG
        JR    Z,M1581
        LD    A,(DFSZ)
        SUB   (IY+0SPOSNLIN)    ;SPOSNLIN
        JR    NZ,M1581
        XOR   E
        RET   Z
        PUSH  HL
        PUSH  DE
        LD    HL,STOP
        CALL  NEXT_L
        POP   DE
        POP   HL
        JR    M1581
;*****
; Module: LIST
; Routine: PUT_SR
;*****
PUT_SR:
M15A1:   LD    BC,(EPPC)
        CALL  M16E8
        LD    D,$3E
        JR    Z,M15B1
;*****
; Module: LIST
; Routine: LPO
;*****
LPO:
M15AC:   LD    DE,$0000
        RL    E
M15B1:   LD    (IY+0BREG),E      ;BREG
        LD    A,(HL)
        CP    $40
        POP   BC
        RET   NC
        PUSH  BC
        CALL  M1795
        INC   HL
        INC   HL
        INC   HL
        RES   0,(IY+0FLAGS)     ;FLAGS
        LD    A,D
        AND   A
        JR    Z,M15CD
        RST   $10
;*****
; Module: LIST

```

```

; Routine: PUT
;*****
PUT:
M15C9:  SET  SPC,(IY+OFLAGS)          ;force space before tokens
M15CD:  PUSH DE                      ;
      EX  DE,HL                      ;
      RES  L_STR,(IY+OFLAGS2) ;
      LD   HL,FLAGS                  ;/force "K" mode at current
      RES  LMODE1,(HL)                ;\ character
      BIT  INPLN,(IY+OFLAGX)          ;/jump if expecting program
      JR   Z,M15E0                    ;\ line
      SET  LMODE1,(HL)                ;force "L" mode at current
                                           ;character
M15E0:  LD   HL,(XPTR)                ;get pointer to 1st char that in not
syntactically
                                           ; ok
      AND  A                          ;/SUB HL,DE
      SBC  HL,DE                      ;\
      JR   NZ,M15ED                    ;
      LD   A,$3F                      ;
M15ED:  CALL M160D                      ;
      CALL M162D                      ;
      EX  DE,HL                      ;
      LD   A,(HL)                      ;
      CALL M1602                      ;
      INC  HL                          ;
      CP   $0D                        ;
      JR   Z,M1600                    ;
      EX  DE,HL                      ;
      CALL M1683                      ;
      JR   M15E0                      ;
M1600:  POP  DE                      ;
      RET                               ;
M1602:  CP   $0E                      ;
      RET  NZ                          ;
      INC  HL                          ;
      INC  HL                          ;
      INC  HL                          ;
      INC  HL                          ;
      INC  HL                          ;
      INC  HL                          ;
      INC  HL                          ;
      LD   A,(HL)                      ;
      RET                               ;
;*****
; Module: LIST
; Routine: FLASHA
;*****
FLASHA:
M160D:  EXX
      LD   HL,(ATTRT)
      PUSH HL
      RES  7,H
      SET  7,L
      LD   (ATTRT),HL
      LD   HL,PFLAG

```

```

        LD    D,(HL)
        PUSH DE
        LD    (HL),$00
        CALL SENDTV
        POP  HL
        LD    (IY+OPFLAG),H          ;PFLAG
        POP  HL
        LD    (ATTRT),HL
        EXX
        RET
;*****
; Module: LIST
; Routine: PR_CUR
;*****
PR_CUR:
M162D:  LD    HL,(KCUR)
        AND   A
        SBC   HL,DE
        RET   NZ
        LD    A,(MODE)
        RLC   A
        JR    Z,M163F
        ADD   A,$43
        JR    M1655
M163F:  LD    HL,FLAGS
        RES   3,(HL)
        LD    A,$4B
        BIT   2,(HL)
        JR    Z,M1655
        SET   3,(HL)
        INC   A
        BIT   3,(IY+OFLAGS2)          ;FLAGS2
        JR    Z,M1655
        LD    A,$43
M1655:  PUSH DE
        CALL M160D
        POP  DE
        RET
;*****
; Module: LIST
; Routine: NEXT_L
; find the next line number
; ENTRY: HL: pointer to the line number to be found
; EXIT:
;*****
NEXT_L:
M165B:  LD    E,(HL)                  ;/get the number of the line
        INC   HL                      ;| to be found
        LD    D,(HL)                  ;\
        PUSH HL                      ;save the pointer
        EX    DE,HL                   ;HL now contains the line number to be found
        INC   HL                      ;look for the next line number
        CALL FIND_L                   ;find the line number in BC at (HL)
        CALL GET_LN                   ;get the line number into DE
        POP   HL                      ;restore the program line number

```



```

;*****
; Module: LIST
; Routine: DE_HL
;
; ENTRY: HL points to the program line line number location
;        DE contains the line number
; EXIT:  HL points to the low byte of the program line number
;        DE contains the line number
;*****
DE_HL:
M1668:  BIT   INPLN,(IY+OFLAGX)          ;/return if looking for input rather
than
        RET   NZ                        ;\a program line
        LD    (HL),D                    ;/store the line number at
        DEC   HL                        ;| its location
        LD    (HL),E                    ;\
        RET                               ;

M1671:  LD     A,E
        AND   A
        RET   M
        JR    M1683
M1676:  XOR    A
M1677:  ADD    HL,BC
        INC   A
        JR    C,M1677
        SBC   HL,BC
        DEC   A
        JR    Z,M1671
        JP    M11EA
M1683:  RES    4,(IY+OFLAGS)              ;
        BIT   2,(IY+OFLAGS)              ;
        JR    Z,M1691
        SET   4,(IY+OFLAGS)              ;
M1691:  CALL   DIGIT?
        JR    NC,M16D4
        CP    $0C
        JR    Z,M16D0
        CP    $21
        JR    C,M16D4
        RES   2,(IY+OFLAGS)              ;
        CP    $7B
        JR    NZ,M16AC
        BIT   4,(IY+OFLAGS)              ;
        JR    Z,M16D4
M16AC:  CP     $CB
        JR    Z,M16D4
        CP    $3A
        JR    NZ,M16C2
        BIT   5,(IY+OFLAGX)              ;/jump if expecting input rather than
        JR    NZ,M16D0                    ;\ program line
        BIT   L_STR,(IY+OFLAGS2) ;/jump if not inside of a string
        JR    Z,M16D4                    ;\
        JR    M16D0
M16C2:  CP     $22

```

```

        JR      NZ,M16D0
        PUSH AF
        LD      A,(FLAGS2)
        XOR     $04
        LD      (FLAGS2),A
        POP     AF
M16D0:   SET     2,(IY+0FLAGS)          ;FLAGS
M16D4:   RST     $10
        RET
;*****
; Module: LIST
; Routine: FIND_L
; find the position of the line whose number is in HL.
; This routine returns with HL pointing to the line
; whose number is greater than or equal to the line number in BC
; ENTRY: HL contains the line number to be found
; EXIT: HL points to the line whose number is greater than or equal
;       to the requested line number
;       DE points to the previous line
;       BC contains the line number
;*****
FIND_L:
M16D6:   PUSH HL          ;save the line number
        LD      HL,(PROG)      ;point to the start of the program text
        LD      D,H          ;/copy it to DE
        LD      E,L          ;\
M16DC:   POP     BC         ;pop the line number
        CALL CP_BC          ;compare the line number in BC to (HL)
        RET     NC          ;return if the current line number is
                           ; greater than or equal to the number
                           ; in BC
        PUSH BC          ;save the desired line number
        CALL RECLen        ;get the length of the record
        EX      DE,HL       ;HL now points to the next line
                           ;DE points to the previous line
        JR      M16DC       ;back around for the next line
;*****
; Module: LIST
; Routine: CP_BC
; compare the line number at (HL) with the number in BC
; EXIT: ZF set if a match
;       CF set if BC < (HL)
;*****
CP_BC:
M16E8:   LD      A,(HL)      ;/compare the first byte of the
        CP      B           ;\line number to the desired line number
        RET     NZ          ;return if no match
        INC     HL          ;/get the next byte of the line number
        LD      A,(HL)      ;\
        DEC     HL          ;restore the pointer
        CP      C           ;compare to the low byte of the requested
                           ; number
        RET                ;return
;*****
; Module: LIST

```

```

; Routine: SUBLIN
;*****
SUBLIN:
    INC HL
    INC HL
    INC HL
;*****
; Module: LIST
; Routine: SUBLIN1
; find the sub-line whose value is in D
; ENTRY: D has the subline number (starting with 1)
;*****
SUBLIN1:
M16F3: LD (CHADD),HL ;
        LD C,$00 ;
M16F8: DEC D ;/return if the first sub-line -
        RET Z ;\ we are already there
        RST $20 ;get next char
        CP E ;/
        JR NZ,M1702 ;\
        AND A
        RET
M1700: INC HL
        LD A,(HL)
M1702: CALL M1602
        LD (CHADD),HL
        CP $22 ;/not a quote, so continue
        JR NZ,M170D ;\
        DEC C
M170D: CP $3A
        JR Z,M1715
        CP $CB
        JR NZ,M1719
M1715: BIT 0,C
        JR Z,M16F8
M1719: CP $0D
        JR NZ,M1700
        DEC D
        SCF
        RET
;*****
; Module: LIST
; Routine: RECLen
; find the next BASIC line or variable starting at HL
; ENTRY: HL = point from which to start looking
; EXIT: HL same as entry
; DE = address of next record
; BC = length record
;*****
RECLen:
M1720: PUSH HL ;save the character pointer
        LD A,(HL) ;get the next byte
        CP $40 ;/jump if this is a BASIC line
        JR C,M173D ;\ (line numbers are less than $40)

```

```

        BIT 5,A          ;/jump if and array or a string
        JR  Z,M173E      ;\

        ADD A,A          ;/jump if single letter or
        JP  M ,M172F      ;\FOR-NEXT
        CCF              ;
M172F:   LD  BC,$0005      ;assume single letter variable name
        JR  NC,M1736      ;jump if indeed single letter
        LD  C,$12         ;length of FOR-NEXT variable
M1736:   RLA              ;/
        INC HL           ;|gobble characters until bit 7
        LD  A,(HL)        ;|is set (name of FOR-NEXT or last
        JR  NC,M1736      ;\letter of multi-letter name)
        JR  M1743         ;got all of name, so move on to
                        ; adjust memory pointer

;handle a BASIC line
M173D:   INC HL           ;bump for a BASIC line
M173E:   INC HL           ;2nd bump for BASIC line or bump to
                        ; access length for number arrays or
                        ; string entities
        LD  C,(HL)        ;/
        INC HL           ;|get the length into BC
        LD  B,(HL)        ;\
        INC HL           ;point to the next byte
M1743:   ADD HL,BC        ;form the address for the next record
        POP DE           ;restore the character pointer

;put the difference between HL and DE into BC
M1745:   AND  A           ;
        SBC  HL,DE        ;
        LD  B,H           ;
        LD  C,L           ;
        ADD  HL,DE        ;
        EX  DE,HL         ;
        RET              ;
;*****
; Module: LIST
; Routine: DEL_DE
; delete the bytes between HL and DE
;*****
DEL_DE:
M174D:   CALL M1745
;*****
; Module: LIST
; Routine: DELREC
; delete BC bytes from memory. move memory and adjust system
; variables.
; ENTER HL = start address of block to delete
; BC = number of bytes to delete
;*****
DELREC:
M1750:   PUSH BC          ;save BC
        LD  A,B           ;/
        CPL              ;|

```

```

        LD    B,A                ;|form the 2's complement
        LD    A,C                ;|of BC
        CPL                      ;|
        LD    C,A                ;|
        INC   BC                 ;\
        PUSH  BC                 ;save BC
        CALL  REMGSZ             ;adjust system variable pointers down BC
bytes
        EX    (SP),HL            ;get BC back and save HL
        ADD   HL,BC              ;
        LD    C,L                ;
        LD    B,H                ;
        POP   DE                 ;
        POP   HL                 ;
        ADD   HL,DE              ;
        PUSH  DE                 ;
        LDIR                      ;
        POP   HL                 ;
        RET                      ;
;*****
; Module: LIST
; Routine: LINNG
;*****
LINNG:
M1768:  LD    HL,(ELINE)
        DEC   HL
        LD    (CHADD),HL
        RST   $20                ;get next char
        LD    HL,MEMBOT
        LD    (STKEND),HL
        CALL  ININT
        CALL  FP2BC
        JR    C,M1782
        LD    HL,$D8F0
        ADD   HL,BC
M1782:  JP    C ,M1BED            ;error - BAD BASIC
        JP    RESET
;*****
; Module: LIST
; Routine: PUT_BC
;*****
PUT_BC:
M1788:  PUSH  DE
        PUSH  HL
        XOR   A
        BIT   7,B
        JR    NZ,M17AF
        LD    H,B
        LD    L,C
        LD    E,$FF
        JR    M179D
;*****
; Module: LIST
; Routine: PUT_LN
;*****

```

```

PUT_LN:
M1795:  PUSH DE
        LD  D,(HL)
        INC HL
        LD  E,(HL)
        PUSH HL
        EX  DE,HL
        LD  E,$20
M179D:  LD  BC,$FC18
        CALL M1676
        LD  BC,$FF9C
        CALL M1676
        LD  C,$F6
        CALL M1676
        LD  A,L
M17AF:  CALL M11EA
        POP HL
        POP DE
        RET
M17B5:  PUSH BC
        LD  BC,$FF00
        CALL M6499
        POP BC
        CALL INSERT
        LD  HL,(SYSCON)
        LD  DE,$0004
        ADD HL,DE
        LD  A,(HL)
        LD  B,$00
        LD  C,A
        CALL M6499
        RET

```

```

;*****
; Module: AROS
; Routine: GETAL
; find the line number in BC in the AROS cartridge
; ENTRY:  BC contains the line number
; EXIT:   HL points to the first byte of the line number
;*****
GETAL:
M17CF:  LD  HL,(SYSCON)      ;/get the address of the
        INC HL              ;| code in the AROS
        INC HL              ;\ cartridge
        LD  E,(HL)          ;/
        INC HL              ;|DE now points to the code
        LD  D,(HL)          ;\ in the cart
        EX  DE,HL           ;... make that HL
;
;search for the AROS line number
;
M17D8:  LD  A,(HL)           ;/we didn't find it so
        CP  B                ;| move on
        JR  NZ,M17E0         ;\
        INC HL              ;bump the pointer

```

```

        LD  A,(HL)                ;get the next byte
        DEC HL                    ;point back to where we were
        CP  C                      ;
M17E0:  RET  NC                    ;return if we have passed the required
                                   ; line number - that exact number does not
                                   ; exist, but we are past it

        INC HL                    ;/get past the line number
        INC HL                    ;\
        LD  E,(HL)                ;/
        INC HL                    ;|DE now has the line length
        LD  D,(HL)                ;\
        INC HL                    ;HL points to the first byte of
                                   ; code in the line
        ADD HL,DE                  ;HL now points to the start of the next
                                   ; line
        JR   M17D8                ;back around for the next line
;*****
; Module: AROS
; Routine: AR_LN
;*****
AR_LN:
M17EA:  PUSH HL                    ;save the line pointer
        LD  HL,(SYSCON)            ;/
        LD  DE,$0004                ;|get the address of the
        ADD HL,DE                  ;\
        LD  A,(HL)                ;/get the chunk map
        LD  C,A                    ;\pass to C
        LD  B,$00                  ;specify dock bank
        CALL BANK_ENABLE           ;
        POP  BC                    ;
        CALL M17CF                  ;
        JR   M1818                ;
;*****
; Module: AROS
; Routine: AR_NXT
;*****
AR_NXT:
M17FF:  CALL INTPT?                ;/return if syntax checking
        RET  Z                      ;\
        LD  HL,(SYSCON)            ;
        LD  DE,$0004                ;
        ADD HL,DE                  ;
        LD  A,(HL)                ;/get the chunk map
        LD  C,A                    ;\pass to C
        LD  B,$00                  ;dock bank
        CALL BANK_ENABLE           ;
        LD  HL,(NXTLIN)            ;
        LD  (IY+ONSPPC),00         ;NSPPC
M1818:  LD  A,(HL)                  ;
        AND  $C0                    ;
        JR   Z,M1824                ;
        LD  BC,$FF00                ;home bank, all chunks
        CALL BANK_ENABLE           ;
        RET                        ;

```

```

M1824:  LD  D,(HL)                ;
        INC HL                    ;
        LD  E,(HL)                ;
        LD  (PPC),DE              ;
        INC HL                    ;
        LD  E,(HL)                ;
        INC HL                    ;
        LD  D,(HL)                ;
        INC HL                    ;
        PUSH HL                   ;
        ADD HL,DE                 ;
        LD  (NXTLIN),HL          ;
        PUSH DE                   ;
        LD  HL,(CHANS)           ;
        DEC HL                    ;
        LD  DE,(ARSBUF)          ;
        AND A                     ;
        SBC HL,DE                 ;
        LD  DE,$00D0             ;
        EX  DE,HL                ;
        AND A                     ;
        SBC HL,DE                 ;
        JR   NC,M186E            ;
        LD  A,L                   ;
        CPL                       ;
        LD  C,A                   ;
        LD  A,H                   ;
        CPL                       ;
        LD  B,A                   ;
        INC BC                    ;
        INC BC                    ;
        LD  HL,(ARSBUF)          ;
        PUSH BC                   ;
        LD  BC,$FF00              ;home bank, all chunks
        CALL BANK_ENABLE         ;
        POP BC                    ;
        CALL DELREC              ;
        LD  HL,(SYSCON)          ;
        LD  DE,$0004             ;
        ADD HL,DE                 ;
        LD  A,(HL)               ;get the chunk map
        LD  B,$00                 ;dock bank
        LD  C,A                   ;chunk map to C
        CALL BANK_ENABLE         ;
M186E:  POP HL                    ;
        PUSH HL                   ;
        LD  DE,$00CF             ;
        DEC HL                    ;
        AND A                     ;
        SBC HL,DE                 ;
        JR   C,M1883             ;
        LD  C,L                   ;
        LD  B,H                   ;
        INC BC                    ;
        LD  HL,(CHANS)           ;

```



```

DEC HL ;
CALL M17B5 ;
M1883: POP BC ;
POP DE ;
LD HL,$00FF ;
PUSH HL ;
PUSH DE ;
LD HL,(ARSBUF) ;
LD (HL),$0D ;
LD (CHADD),HL ;
INC HL ;
PUSH HL ;
PUSH BC ;
LD BC,$0001 ;
PUSH BC ;
CALL M6722 ;
LD A,(IY+ONSPPC) ;NSPPC
LD (IY+ONSPPC),$FF ;NSPPC
CP $01 ;
ADC A,$00 ;
DEC A ;
PUSH AF ;
LD (SUBPPC),A ;
LD (IY+OERRNR),$FF ;ERRNR
LD BC,$FF00 ;home bank, all chunks
CALL BANK_ENABLE ;
POP AF ;
JP Z ,LS4 ;
INC A ;
LD D,A ;
LD E,$00 ;
CALL SUBLIN1 ;
JP Z ,M1B4A ;

RST $08 ;/error: STATEMENT LOST
DFB $16 ;\
;*****
; Module: AROS
; Routine: AROS
;*****
AROS:
LD HL,ARSFLG
LD (HL),$80
LD BC,$00D0
LD HL,$6840
DEC HL
CALL INSERT
LD HL,$6840
LD (ARSBUF),HL
LD HL,(SYSCON)
LD DE,$0006
ADD HL,DE
LD C,(HL)
INC HL
LD B,(HL)

```

```

LD    HL,$6840
DEC   HL
CALL  INSERT
LD    HL,(SYSCON)
LD    DE,$0004
ADD   HL,DE
LD    A,(HL)                ;get chunk map
LD    B,$00                ;dock bank
LD    C,A                  ;chunk map to C
CALL  BANK_ENABLE          ;
LD    HL,(SYSCON)
INC   HL
INC   HL
LD    E,(HL)
INC   HL
LD    D,(HL)
EX    DE,HL
LD    D,(HL)
INC   HL
LD    E,(HL)
LD    BC,$FF00            ;home bank, all chunks
CALL  BANK_ENABLE          ;
LD    HL,(SYSCON)
LD    BC,$0005
ADD   HL,BC
LD    A,(HL)
CP    $00
JR    Z,M1941
LD    (NEWPPC),DE
CALL  M08A6
LD    HL,(SYSCON)
INC   HL
INC   HL
LD    E,(HL)
INC   HL
LD    D,(HL)
EX    DE,HL
DEC   HL
LD    (DATADD),HL
LD    (IY+OERRNR),$FF      ;ERRNR
SET   7,(IY+OFLAGS)        ;FLAGS
LD    (IY+ONSPPC),00       ;NSPPC
LD    HL,$0E8D
PUSH  HL
LD    HL,$1AB9
EI
JP    (HL)                ;

M1941:  EI
        JP    M0E2F
;~~~~~

KEWTBL: DFB PADEFN-$
        DFB PACAT-$
        DFB PAFORM-$

```

DFB PAMOVE-\$
 DFB PAERA-\$
 DFB PAOPEN-\$
 DFB PACLOS-\$
 DFB PAMERG-\$
 DFB PAVERI-\$
 DFB PABEEP-\$
 DFB PACIRC-\$
 DFB PAINK-\$
 DFB PAPAPE-\$
 DFB PAFLAS-\$
 DFB PABRIG-\$
 DFB PAINVE-\$
 DFB PAOVER-\$
 DFB PAOUT-\$
 DFB PALPRI-\$
 DFB PALLIS-\$
 DFB PASTOP-\$
 DFB PAREAD-\$
 DFB PADATA-\$
 DFB PAREST-\$
 DFB PANEW-\$
 DFB PABORD-\$
 DFB PACONT-\$
 DFB PADIM-\$
 DFB PAREM-\$
 DFB PAFOR-\$
 DFB PAGOTO-\$
 DFB PAGOSU-\$
 DFB PAINPU-\$
 DFB PALOAD-\$
 DFB PALIST-\$
 DFB PALET-\$
 DFB PAPAUS-\$
 DFB PANEXT-\$
 DFB PAPOKE-\$
 DFB PAPRIN-\$
 DFB PAPLOT-\$
 DFB PARUN-\$
 DFB PASAVE-\$
 DFB PARAND-\$
 DFB PAIF-\$
 DFB PACLS-\$
 DFB PADRAW-\$
 DFB PACLEA-\$
 DFB PARETU-\$
 DFB PACOPY-\$

KEWTBL2:

DFB PADEL-\$;DELETE
 DFB PAONERR-\$;ONERR
 DFB PARESET-\$;RESET
 DFB PASND-\$;SOUND

PALET: DFB 1 ;class 1

```

        DFB "="          ;3D
        DFB 2            ;class 2

PAGOTO: DFB 6            ;class 6
        DFB 0            ;class 0
        DWL M1EF1        ;

PAIF:   DFB 6            ;class 6
        DFB $CB          ;"THEN"
        DFB 5            ;class 5
        DWL TSIF         ;

PAGOSU: DFB 6            ;class 6
        DFB 0            ;class 0
        DWL GO_SUB       ;

PASTOP: DFB 0            ;class 0
        DWL TSSTOP       ;

PARETU: DFB 0            ;class 0
        DWL RETURN       ;

PAFOR:  DFB 4            ;class 4
        DFB "="          ;
        DFB 6            ;class 6
        DFB $CC          ;"TO"
        DFB 6            ;class 6
        DFB 5            ;class 5
        DWL FOR          ;

PANEXT: DFB 4            ;class 4
        DFB 0            ;class 0
        DWL NEXT         ;

PAPRIN: DFB 5            ;class 5
        DWL K_PRIN       ;

PAINPU: DFB 5            ;class 5
        DWL INPUT        ;

PADIM:  DFB 5            ;class 5
        DWL DIM          ;

PAREM:  DFB 5            ;class 5
        DWL M1B00        ;

PANEW:  DFB 0            ;class 0
        DWL K_NEW        ;1D0D

PARUN:  DFB 3            ;class 3
        DWL RUN          ;

PALIST: DFB 5            ;class 5
        DWL K_LIST       ;

```

```

PAPOKE: DFB 8          ;class 8
        DFB 0          ;class 0
        DWL POKE       ;

PARAND: DFB 3          ;class 3
        DWL RAND       ;

PACONT: DFB 0          ;class 0
        DWL CONT       ;

PACLEA: DFB 3          ;class 3
        DWL CLEAR      ;

PACLS:  DFB 0          ;class 0
        DWL K_CLS      ;

PAPLOT: DFB 9          ;class 9
        DFB 0          ;class 0
        DWL PLOT       ;

PAPAUS: DFB 6          ;class 6
        DFB 0          ;class 0
        DWL PAUSE      ;

PAREAD: DFB 5          ;class 5
        DWL READ       ;

PADATA: DFB 5          ;class 5
        DWL DATA      ;

PAREST: DFB 3          ;class 3
        DWL M1E9D      ;

PADRAW: DFB 9          ;class 9
        DFB 5          ;class 5
        DWL DRAW       ;

PACOPY: DFB 0          ;class 0
        DWL K_DUMP     ;

PALPRI: DFB 5          ;class 5
        DWL K_LPR      ;

PALLIS: DFB 5          ;class 5
        DWL K_LLST     ;

PASAVE: DFB 11         ;class B
PALOAD: DFB 11         ;class B
PAVERI: DFB 11         ;class B
PAMERG: DFB 11         ;class B

PABEEP: DFB 8          ;class 8
        DFB 0          ;class 0
        DWL BEEP       ;

```

```

PACIRC: DFB 9          ;class 9
        DFB 5          ;class 5
        DWL CIRCLE     ;

PAINK:  DFB 7          ;class 7
PAPAPE: DFB 7          ;class 7
PAFLAS: DFB 7          ;class 7
PABRIG: DFB 7          ;class 7
PAINVE: DFB 7          ;class 7
PAOVER: DFB 7          ;class 7

PAOUT:  DFB 8          ;class 8
        DFB 0          ;class 0
        DWL K_OUTPUT   ;

PABORD: DFB 6          ;class 6
        DFB 0          ;class 0
        DWL M243E      ;

PADEFN: DFB 5          ;class 5
        DWL DEF        ;

PAOPEN: DFB 6          ;class 6
        DFB ", "       ;
        DFB 10         ;class 10
        DFB 05         ;class 5
        DWL OPEN       ;

PACLOS: DFB 6          ;class 6
        DFB 0          ;class 0
        DWL CLOSE      ;

PAFORM: DFB 10         ;class 10
        DFB ", "       ;
        DFB 05         ;class 5
        DWL FORMAT     ;CC25

PAMOVE: DFB $0A        ;class A
        DFB ", "       ;
        DFB $05        ;class 5
        DWL MOVE       ;D025

PAERA:  DFB $0A        ;class A
        DFB ", "       ;
        DFB $05        ;class 5
        DWL ERASE      ;

PACAT:  DFB 10         ;class 10
        DFB ", "       ;
        DFB $05        ;class 5
        DWL CAT        ;

PADEL:  DFB $05        ;class 5
        DWL $20D1      ;

```

```

PAONERR:
    DFB $05          ;class 5
    DWL $2080        ;

PARESET:
    DFB $05          ;class 5
    DWL RSET         ;

PASND:  DFB $05          ;class 5
        DWL SOUND      ;
;*****
; Module: SYNTAX
; Routine: SYNTAX
;*****
SYNTAX:
M1A27:  RES  7,(IY+OFLAGS)      ;FLAGS
        CALL M1768
        LD   A,B
        OR   C
        JR   Z,M1A3A
        LD   A,(ARSFLG)
        BIT  7,A
        JP   NZ,M1BED          ;error - BAD BASIC
M1A3A:  XOR   A
        LD   (SUBPPC),A
        DEC  A
        LD   (ERRNR),A
        JR   M1A45
;*****
; Module: SYNTAX
; Routine: LS4
;*****
M1A44:
LS4:    RST   $20              ;get next char to A
M1A45:  CALL  X_CALC           ;reset pointers to STKBOT and STKEND to
(WORKSP)
M1A48:  INC   (IY+OSUBPPC)     ;/bump the substatement and jump if too many
        JP   M,M1BED          ;\error - BAD BASIC

        RST   $18              ;get current character
        LD   B,$00            ;set up B for later
        CP   $0D              ;/jump if the end of a line
        JP   Z,M1B09          ;\
        CP   ":"              ;/jump if the end of a sub line
        JR   Z,LS4            ;\
        LD   HL,$1AB9         ;/this address leads to
        PUSH HL               ;\EXECUTE
        LD   C,A              ;save the present character
        RST   $20             ;gobble the next char (point to the next
char)
        LD   A,C              ;restore the last character

        CP   $0C              ;/jump if token is DELETE
        JR   Z,M1A7F          ;\

```

```

        CP    $7B                ;/jump if char can't be a keyword token
        JR    C,M1A71            ;\

        CP    $80                ;/jump if char might be a keyword token
        JR    NC,M1A71           ;\
        BIT    0,A               ;/jump if token is for keywords
        JR    NZ,M1A7F           ;\ONERR, SOUND, RESET

M1A71:   SUB    $CE                ;/error if not a keyword (not function etc.)
        JP    C ,M1BED           ;\error - BAD BASIC

        LD    C,A               ;/make the address to look into the
        LD    HL,KEWTBL         ;|keyword table offset table
M1A7A:   ADD    HL,BC             ;\
        LD    C,(HL)            ;/make the address to look into the
        ADD    HL,BC            ;\syntax table
        JR    M1A98             ;

M1A7F:   CP    $0C                ;/jump if token is not DELETE
        JR    NZ,M1A87           ;\
        LD    A,$00             ;/force A to zero and bypass
        JR    M1A8F             ;\ next test

M1A87:   SUB    $7A                ;A=1 for ONERR, 3 for SOUND, 5 for RESET
        CP    $05                ;/jump if token not RESET
        JR    NZ,M1A8F           ;\
        LD    A,$02             ;force A (RESET) to 2

M1A8F:   LD    HL,KEWTBL2        ;second half of keyword table
        LD    C,A               ;move the massaged token to C
        JR    M1A7A             ;do the table look-up thang

M1A95:   LD    HL,(TADDR)        ;the class handler returns here from the
                                   ; address pushed below

M1A98:   LD    A,(HL)            ;get the first (or next) byte of the
syntax table -
                                   ; the instruction's "class" (first byte) or
the required
                                   ; token or class (succeeding bytes)
        INC    HL               ;point to the next byte in the syntax table
        LD    (TADDR),HL        ;store its address in TADDR
        LD    BC,M1A95          ;/"return" address from class handler
        PUSH    BC              ;\
        LD    C,A               ;save the "class"
        CP    $20               ;/jump if the "class" is not in the range
$00..$11
        JR    NC,M1AB2           ;| this means it could be the next
value in the syntax
                                   ;\ table.

        LD    HL,CLASTBL        ;/form address of offset into the expression
        LD    B,$00             ;| class routine offset table
        ADD    HL,BC            ;\
        LD    C,(HL)           ;/get offset from the table

```



```

        ADD HL,BC                ;|and form addrss of the actual "class"
                                   ;\handler ...
        PUSH HL                 ;... and push it onto the stack
        RST $18                 ;get current character
        DEC B                   ;B=$FF ???
        RET                     ;jump to the class handler

M1AB2:  RST $18                 ;get current character
        CP C                   ;check the current character in A
against
                                   ; the value from the syntax table.
                                   ; declare an error if they don't match -
                                   ; the syntax is wrong
        JP NZ,M1BED             ;error - BAD BASIC
        RST $20                 ;get next char
        RET                     ;either done or back around for the
                                   ; next byte in then syntax table
;*****
; Module: SYNTAX
; Routine: ENDBTT
;*****
ENDBTT:
M1AB9:  CALL BREAK?             ;/jump if BREAK was not
        JR C,M1AC0              ;\ pressed

        RST $08                 ;error: BREAK
        DFB $14                 ;

M1AC0:  BIT 7,(IY+ONSPPC)        ;/
        JP NZ,M1B4A             ;\
        LD HL,(NEWPPC)          ;
        BIT 7,H                 ;
        JR NZ,M1AD8             ;
        LD A,(ARSFLG)           ;
        BIT 7,A                 ;
        JP NZ,M17EA             ;
        JR M1AEC                ;
;*****
; Module: SYNTAX
; Routine: EXECUTE
;*****
EXECUTE:
M1AD8:  LD HL,$FFFE
        LD (PPC),HL
        LD HL,(WORKSP)
        DEC HL
        LD DE,(ELINE)
        DEC DE
        LD A,(NSPPC)
        JR M1B27
M1AEC:  CALL FIND_L
        LD A,(NSPPC)
        JR Z,M1B15
        AND A
        JR NZ,M1B42

```

```

        LD    B,A
        LD    A,(HL)
        AND   $C0
        LD    A,B
        JR    Z,M1B15

        RST   $08                ;/????
        RST   $38                ;\

M1B00:   POP   BC
        LD    A,(ARSFLG)
        BIT   7,A
        JP    NZ,M17FF

M1B09:   CALL  INTPT?            ;/return if syntax checking
        RET   Z                  ;\
        LD    HL,(NXTLIN)        ;|get the address of the next line
        LD    A,$C0              ;/
        AND   (HL)               ;|return if not a valid line number
        RET   NZ                  ;\
        XOR   A                  ;|force first subline, no forced jump

M1B15:   CP    $01                ;/will give 1 for A=0, else will
        ADC   A,$00              ;\give A
        LD    D,(HL)             ;/
        INC   HL                 ;|put the new line number into the
        LD    E,(HL)             ;| "line currently being interpreted"
        LD    (PPC),DE           ;\ pointer
        INC   HL                 ;/
        LD    E,(HL)             ;|get the line length
        INC   HL                 ;|
        LD    D,(HL)             ;\
        EX    DE,HL              ;/point to the next line and put the pointer
        ADD   HL,DE              ;| into the "next line to be
interpreted"
        INC   HL                 ;| pointer
M1B27:   LD    (NXTLIN),HL        ;\
        EX    DE,HL              ;/put the new line's address into the
        LD    (CHADD),HL         ;\ interpreter code pointer
        LD    D,A                ;/save the subline in D
        LD    E,$00              ;\
        LD    (IY+ONSPPC),$FF    ;|load a value that will not force a
jump to the new subline
        DEC   D                  ;/put the desired sub-line into its
pointer
        LD    (IY+OSUBPPC),D     ;\
        JP    Z ,LS4             ;|we don't need to find a sub-line, so
move on
        INC   D                  ;|normalize the value in D
        CALL  SUBLIN1            ;
        JR    Z,M1B4A            ;

M1B42:   RST   $08                ;/error: STATEMENT LOST
        DFB   $16                ;\

```

```

;*****
; Module: SYNTAX
; Routine: END?
; checks the interpret flag (bit 7 of FLAGS) and returns to
; the caller if the machine is running code, else ???
;*****
END?:
M1B44:  CALL INTPT?          ;will return NZ if we are interpreting
        RET  NZ             ;interpreting, so return to execute
                                ; the code
                                ;syntax checking, so move on
        POP  BC             ;trash caller's return address
        POP  BC             ;
;*****
; Module: SYNTAX
; Routine: ENDTEM
;*****
ENDTEM:
M1B4A:  RST  $18             ;get current character
        CP   $0D             ;/jump if not the end
        JR   NZ,M1B5C        ;\ of the a line
        LD   HL,(NXTLIN)     ;get the pointer to the next BASIC line
        LD   A,(ARSFLG)      ;/
        BIT  7,A             ;|jump if we is in an AROS
        JP   NZ,M17FF        ;\
        JR   M1B09           ;normal line execution

M1B5C:  CP   ":"             ;
        JP   Z ,LS4          ;
        JP   M1BED           ;error - BAD BASIC

CLASTBL:
        DFB  TEM0-$
        DFB  TEM1-$
        DFB  TEM2-$
        DFB  TEM3-$
        DFB  TEM4-$
        DFB  TEM5-$
        DFB  TEM6-$
        DFB  TEM7-$
        DFB  TEM8-$
        DFB  TEM9-$
        DFB  TEM10-$
        DFB  TEM11-$

TEM3:   CALL M1C49
TEM0:   CP   A
TEM5:   POP  BC
        CALL Z ,END?
        EX  DE,HL
        LD  HL,(TADDR)
        LD  C,(HL)
        INC HL
        LD  B,(HL)
        EX  DE,HL

```

```

        PUSH BC
        RET
;*****
; Module: SYNTAX
; Routine: TEM1
;*****
TEM1:
M1B82:  CALL FIND_N
M1B85:  LD   (IY+OFLAGX),00          ;FLAGX
        JR   NC,M1B93
        SET  1,(IY+OFLAGX)          ;FLAGX
        JR   NZ,M1BA9
;*****
; Module: SYNTAX
; Routine: ERR2
;*****
ERR2:
M1B91:  RST  $08                    ;/error: VARIABLE NOT FOUND
        DFB  $01                    ;\

M1B93:  CALL Z,M2D54
        BIT  6,(IY+OFLAGS)          ;FLAGS
        JR   NZ,M1BA9
        XOR  A
        CALL INTPT?                 ;/return if interpreting
        CALL NZ,PGPSTR              ;\
        LD  HL,FLAGX
        OR  (HL)
        LD  (HL),A
        EX  DE,HL
M1BA9:  LD  (STRLEN),BC
        LD  (DEST),HL
        RET
TEM2:   POP  BC
        CALL M1BB9
        CALL END?
        RET
M1BB9:  LD  A,(FLAGS)
;*****
; Module: SYNTAX
; Routine: LT22
;*****
LT22:
M1BBC:  PUSH AF
        CALL EXPRN
        POP  AF
        LD  D,(IY+OFLAGS)          ;FLAGS
        XOR  D
        AND  $40
        JR   NZ,M1BED              ;error - BAD BASIC
        BIT  7,D
        JP   NZ,M2EBD
        RET
TEM4:   CALL FIND_N
        PUSH AF

```

```

        LD    A,C
        OR    $9F
        INC   A
        JR     NZ,M1BED          ;error - BAD BASIC
        POP   AF
        JR     M1B85
;*****
; Module: SYNTAX
; Routine: DYADIC
;*****
DYADIC:
M1BDC:  RST   $20                ;get next char
TEM8:   CALL  TEM6              ;evaluate the next expression
        CP    ","                ;/jump if another argument is
        JR     NZ,M1BED          ;\not available
        RST   $20                ;get next char, fall through to
                                   ; evaluate the next expression
;*****
; Module: SYNTAX
; Routine: TEM6
;  evaluate the next expression and return if it is numeric
;*****
TEM6:
M1BE5:  CALL  EXPRN              ;evaluate an expression
        BIT   NUM,(IY+OFLAGS)    ;/return if the expression was an
integer
        RET   NZ                ;\
;*****
; Module: SYNTAX
; Routine: SYNERR
;*****
SYNERR:
M1BED:  RST   $08                ;/error: NONSENSE IN BASIC
        DFB   $0B                ;\
;*****
; Module: SYNTAX
; Routine: TEM10
;  evaluate an expression.  if the expression is not a string
;  bounce to ERROR C
;*****
TEM10:
M1BEF:  CALL  EXPRN              ;evaluate the next expression
        BIT   NUM,(IY+OFLAGS)    ;/return if the last expression
        RET   Z                  ;\ was a number
        JR     M1BED            ;error - BAD BASIC
;*****
; Module: SYNTAX
; Routine: TEM7
;  evaluate an expression.  if the expression is not a string
;  bounce to ERROR C
;*****
TEM7:   BIT   7,(IY+OFLAGS)      ;FLAGS
        RES   0,(IY+OTVFLAG)    ;TVFLAG
        CALL  NZ,DO_ATTS

```

```

        POP    AF
        LD     A,(TADDR)
        LD     HL,(TADDR)
        LD     DE,$1914
        AND    A
        SBC    HL,DE
        LD     A,L
        CALL   M23A6
        CALL   END?
        LD     HL,(ATTRT)
        LD     (ATTRP),HL
        LD     HL,PFLAG
        LD     A,(HL)
        RLCA
        XOR    (HL)
        AND    $AA
        XOR    (HL)
        LD     (HL),A
        RET

TEM9:    CALL   INTPT?                ;/jump if syntax checking
        JR     Z,M1C41                ;\
        RES    0,(IY+OTVFLAG)         ;TVFLAG
        CALL   DO_ATTS
        LD     HL,MASKT
        LD     A,(HL)
        OR     $F8
        LD     (HL),A
        RES    6,(IY+OPFLAG)          ;PFLAG
        RST    $18                    ;get current character

;*****
; Module: SYNTAX
; Routine: STK_0
;*****
STK_0:
M1C41:    CALL   M238C
        JR     TEM8                    ;evaluate the next two comma separated
expressions

TEM11:    JP     NEWDEV
;*****
; Module: SYNTAX
; Routine: OPTNO
; stack a fp zero unless we are syntax checking and at the end of
; a line or statement
;*****
OPTNO:
M1C49:    CP     $0D                    ;/jump if the end of line
        JR     Z,M1C51                ;\
        CP     ":"                     ;/jump to evaluate a numeric
        JR     NZ,TEM6                 ;| expression if not the end of
; \ statement

STKFP0:
M1C51:    CALL   INTPT?                ;/return if syntax checking

```

```

        RET    Z            ;\

;stack a zero to the fp stack

        RST $28            ;calc entry
        DFB $A0            ;CONST 0 (0)
        DFB $38            ;QUIT
        RET

;*****
; Module: SYNTAX
; Routine: STOP
;*****
TSSTOP:
        RST $08            ;error: STOP
        DFB $08            ;

TSIF:
        POP BC
        CALL INTPT?        ;/jump if syntax checking
        JR     Z,M1C75      ;\
        RST $28            ;calc entry
        DFB $02            ;DROP
        DFB $38            ;QUIT
        EX DE,HL
        CALL M3904
        JR     NC,M1C75
        LD A,(ARSFLG)
        BIT 7,A
        JP NZ,M17FF
        JP M1B09
M1C75:   JP M1A45
;*****
; Module: SYNTAX
; Routine: FOR
;*****
FOR:
        CP $CD
        JR     NZ,M1C85
        RST $20            ;get next char
        CALL TEM6
        CALL END?
        JR M1C8B
M1C85:   CALL END?
        RST $28            ;calc entry
        DFB $A1            ;CONST 1 (256)
        DFB $38            ;QUIT
M1C8B:   RST $28            ;calc entry
        DFB $C0            ;T -> MEM 0
        DFB $02            ;DROP
        DFB $01            ;SWAP
        DFB $E0            ;MEM 0 -> T
        DFB $01            ;SWAP
        DFB $38            ;QUIT
        CALL M2EBD
        LD (MEM),HL

```

```

DEC HL
LD A,(HL)
SET 7,(HL)
LD BC,$0006
ADD HL,BC
RLCA
JR C,M1CA9
LD C,$0D
CALL INSERT
INC HL
M1CA9: PUSH HL
RST $28 ;calc entry
DFB $02 ;DROP
DFB $02 ;DROP
DFB $38 ;QUIT
POP HL
EX DE,HL
LD C,$0A
LDIR
LD HL,(PPC)
EX DE,HL
LD (HL),E
INC HL
LD (HL),D
LD D,(IY+OSUBPPC) ;SUBPPC
INC D
INC HL
LD (HL),D
CALL M1D84
RET NC
LD HL,(PPC)
LD (NEWPPC),HL
LD A,(SUBPPC)
NEG
LD D,A
LD HL,(SYSCON)
INC HL
LD A,(HL)
CP $02
JR NZ,M1CF2
INC HL
INC HL
INC HL
LD A,(HL) ;get the chunk map
AND $0F ;only accept the lower half of the chunk
; spec
LD C,A ;chunks map to C
LD B,$00 ;specify dock bank
CALL BANK_ENABLE ;
LD BC,(PPC)
CALL M17CF
LD H,B
LD L,C
DEC HL
JR M1CF5

```



```

M1CF2:  LD   HL,(CHADD)
M1CF5:  LD   E,$F3
M1CF7:  LD   BC,(NXTLIN)
        CALL M1D28
        LD   (NXTLIN),BC
        LD   B,(IY+OSTRLEN)          ;STRLEN
        JR   C,M1D26
        RST  $20                      ;get next char
        OR   $20
        CP   B
        JR   Z,M1D10
        RST  $20                      ;get next char
        JR   M1CF7
M1D10:  RST  $20                      ;get next char
        LD   A,$01
        SUB  D
        LD   (NSPPC),A
        LD   HL,ARSFLG
        LD   L,(HL)
        BIT  7,L
        JR   Z,M1D25
        LD   BC,$FF00                ;home bank, all chunks
        CALL BANK_ENABLE             ;
M1D25:  RET

M1D26:  RST  $08                      ;error: FOR WITHOUT NEXT
        DFB  $11                      ;

```

```

;*****
; Module: SYNTAX
; Routine: SKIP
;*****

```

```

SKIP:
M1D28:  LD   A,(HL)
        CP   $3A
        JR   Z,M1D4D
M1D2D:  INC  HL
        LD   A,(HL)
        AND  $C0
        SCF
        RET  NZ
        LD   A,E
        CP   $E4
        JR   NZ,M1D3B
        LD   (ADATLN),HL
M1D3B:  LD   B,(HL)
        INC  HL
        LD   C,(HL)
        LD   (NEWPPC),BC
        INC  HL
        LD   C,(HL)
        INC  HL
        LD   B,(HL)
        PUSH HL
        ADD  HL,BC

```

```

        LD    B,H
        LD    C,L
        POP   HL
        LD    D,$00
M1D4D:  PUSH  BC
        CALL SUBLIN1
        POP   BC
        RET   NC
        JR    M1D2D
;*****
; Module: SYNTAX
; Routine: NEXT
;*****
NEXT:
        BIT   1,(IY+OFLAGX)          ;FLAGX
        JP    NZ,M1B91
        LD    HL,(DEST)
        BIT   7,(HL)
        JR    Z,M1D82
        INC   HL
        LD    (MEM),HL
        RST   $28    ;calc entry
        DFB   $E0    ;MEM 0 -> T
        DFB   $E2    ;MEM 2 -> T
        DFB   $0F    ;ADD
        DFB   $C0    ;T -> MEM 0
        DFB   $02    ;DROP
        DFB   $38    ;QUIT
        CALL  M1D84
        RET    C
        LD    HL,(MEM)
        LD    DE,$000F
        ADD   HL,DE
        LD    E,(HL)
        INC   HL
        LD    D,(HL)
        INC   HL
        LD    H,(HL)
        EX    DE,HL
        JP    M1EFD

M1D82:  RST   $08          ;error: NEXT WITHOUT FOR
        DFB   $00          ;

M1D84:  RST   $28    ;calc entry
        DFB   $E1    ;MEM 1 -> T
        DFB   $E0    ;MEM 0 -> T
        DFB   $E2    ;MEM 2 -> T
        DFB   $36    ;MINUSQ
        DFB   $00,$02    ;IFJUMP    1D8C
        DFB   $01    ;SWAP
        DFB   $03    ;SUB
        DFB   $37    ;PLUSQ
        DFB   $00,$04    ;IFJUMP    1D93
        DFB   $38    ;QUIT

```

```

        AND    A
        RET
        JR     C,M1DCC
        RET
M1D96:  RST    $20                ;get next char
;*****
; Module: SYNTAX
; Routine: READ
;*****
READ:
        CALL M1B82
        CALL INTPT?                ;/jump if syntax checking
        JP    Z ,M1E78              ;\
        RST    $18                  ;get current character
        LD     (XPTR),HL
        LD     HL,ARSFLG
        LD     L,(HL)
        BIT    7,L
        JP    Z ,M1E52
        LD     HL,(SYSCON)
        LD     DE,$0004
        ADD    HL,DE
        LD     A,(HL)                ;get the chunk map
        AND    $0F                  ;only accept the lower half of the chunk map
        LD     B,$00                ;dock bank
        LD     C,A                  ;chunk map to C
        CALL  BANK_ENABLE          ;
        LD     HL,(DATADD)
        LD     A,(HL)
        CP     $2C
        JR     Z,M1DD8
        LD     E,$E4
        CALL  M1D28
M1DCC:  JR     NC,M1DD5
        LD     BC,$FF00              ;home bank, all chunks
        CALL  BANK_ENABLE          ;
        JP    M1E62
M1DD5:  LD     (DATADD),HL
M1DD8:  LD     HL,(ADATLN)
        INC    HL
        INC    HL
        LD     C,(HL)
        INC    HL
        LD     B,(HL)
        LD     (DTLNLN+1),BC
        LD     BC,$FF00              ;home bank, all chunks
        CALL  BANK_ENABLE          ;
        LD     BC,(DTLNLN+1)
        LD     HL,(CHANS)
        PUSH  HL
        DEC    HL
        CALL  INSERT
        POP   DE
        LD     HL,$00FF
        PUSH  HL

```

```

LD HL,(ADATLN)
INC HL
INC HL
INC HL
INC HL
PUSH HL
PUSH DE
LD BC,(DTLNLN+1)
PUSH BC
LD BC,$0001
PUSH BC
CALL M6722
LD HL,(ADATLN)
LD DE,(DTLNLN+1)
ADD HL,DE
LD DE,$0004
ADD HL,DE
LD BC,(DATADD)
AND A
SBC HL,BC
LD B,H
LD C,L
LD HL,(CHANS)
AND A
SBC HL,BC
PUSH HL
INC HL
LD (CHADD),HL
CALL M1BB9
POP DE
LD HL,(CHADD)
AND A
SBC HL,DE
LD DE,(DATADD)
ADD HL,DE
LD (DATADD),HL
LD HL,(CHANS)
LD BC,(DTLNLN+1)
AND A
SBC HL,BC
CALL DELREC
JP M1E6E
M1E52: LD HL,(DATADD)
LD A,(HL)
CP $2C
JP Z,M1E64
LD E,$E4
CALL M1D28
JR NC,M1E64

M1E62: RST $08 ;/error: OUT OF DATA
DFB $0D ;\

M1E64: CALL M0077
CALL M1BB9

```

```

        RST $18                                ;get current character
        LD (DATADD),HL
M1E6E:  LD HL,(XPTR)
        LD (IY+{0XPTR+1}),00
        CALL M0078
M1E78:  RST $18                                ;get current character
        CP $2C
        JP Z,M1D96
        CALL END?
        RET
;*****
; Module: SYNTAX
; Routine: DATA
;*****
DATA:
        CALL INTPT?                            ;/jump if interpreting
        JR NZ,M1E92                            ;\
M1E87:  CALL EXPRN
        CP $2C
        CALL NZ,END?
        RST $20                                ;get next char
        JR M1E87
M1E92:  LD A,$E4

M1E94:  LD B,A                                ;
        CPDR                                    ;
        LD DE,$0200                            ;
        JP SUBLIN1                             ;
M1E9D:  CALL FIX_U                            ;
        LD HL,(SYSCON)
        INC HL
        LD A,(HL)
        CP $02
        JR NZ,M1ECA
        INC HL
        INC HL
        INC HL
        LD A,(HL)
        AND $0F
        PUSH BC                                ;briefly save BC
        LD C,A                                ;chunk map to C
        LD B,$00                                ;dock bank
        CALL BANK_ENABLE                       ;
        POP BC                                ;restore BC
        CALL M17CF
        LD BC,$FF00                            ;home bank, all chunks
        CALL BANK_ENABLE                       ;
        JR M1EC5
        CALL FIND_L
M1EC5:  DEC HL
        LD (DATADD),HL
        RET
;*****
; Module: SYNTAX
; Routine: RESTBC

```

```

;*****
RESTBC:
M1ECA:  LD  H,B
        LD  L,C
        CALL FIND_L
        DEC HL
        LD  (DATADD),HL
        RET
;*****
; Module: SYNTAX
; Routine: RAND
;*****
RAND:
        CALL FIX_U
        LD  A,B
        OR  C
        JR  NZ,M1EDF
        LD  BC,(FRAMES)
M1EDF:  LD  (SEED),BC
        RET
;*****
; Module: SYNTAX
; Routine: CONT
;*****
CONT:
        LD  HL,(OLDPPC)
        INC H
        JP  Z ,M1B42
        DEC H
        LD  D,(IY+00SPCC)          ;OSPCC
        JR  M1EFD
;*****
; Module: SYNTAX
; Routine: JUMP
; GOTO
;*****
JUMP:
M1EF1:  CALL FIX_U                ;top of FP stack to BC
        LD  H,B                  ;/transfer BC to HL
        LD  L,C                  ;\
        LD  D,$00                ;force sub line 0
        LD  A,H                  ;/jump if line number is
        CP  $F0                  ;|too large
        JR  NC,M1F29             ;\
M1EFD:  LD  (NEWPPC),HL          ;put new line number into BASIC's
                                           ; program jump instruction pointer
        LD  (IY+0NSPPC),D        ;force jump subline pointer to 0
        RET                      ;return to the interpreter

K_OUTPUT:
        CALL M1F0F
        OUT (C),A
        RET

POKE:
        CALL M1F0F

```

```

        LD    (BC),A
        RET

M1F0F:  CALL FP2A
        JR    C,M1F29
        JR    Z,M1F18
        NEG
M1F18:  PUSH AF
        CALL FIX_U
        POP  AF
        RET
;*****
; Module: SYNTAX
; Routine: INS_U1
;*****
INS_U1:
M1F1E:  CALL FP2A          ;to of fp stack to A
        JR    M1F26        ;to error checking
;*****
; Module: SYNTAX
; Routine: FIX_U
; put the top of the fp stack onto BC.  declare error if number
; is too big
;*****
FIX_U:
M1F23:  CALL FP2BC        ;top of calculator stack to BC
M1F26:  JR    C,M1F29      ;error - number too large
        RET  Z            ;return if no
;*****
; Module: SYNTAX
; Routine: ERRB
;*****
ERRB:
M1F29:  RST  $08          ;/error: INTEGER OUT OF RANGE
        DFB  $0A          ;\

RUN:
        CALL JUMP          ;call GOTO (JUMP)
        LD    BC,$0000
        CALL M1ECA
        JR    M1F39
;*****
; Module: SYNTAX
; Routine: CLEAR
;*****
CLEAR:
        CALL FIX_U
;*****
; Module: SYNTAX
; Routine: CLR_BC
;*****
CLR_BC:
M1F39:  LD    A,B
        OR    C
        JR    NZ,M1F41

```

```

M1F41:  LD  BC,(RAMTOP)
        PUSH BC
        LD  DE,(VARS)
        LD  HL,(ELINE)
        DEC HL
        CALL M174D
        CALL M08A6
        LD  HL,ARSFLG
        LD  L,(HL)
        BIT  7,L
        JR   Z,M1F67
        LD  HL,(SYSCON)
        INC HL
        INC HL
        LD  E,(HL)
        INC HL
        LD  D,(HL)
        EX  DE,HL
        DEC HL
        LD  (DATADD),HL
        JR  M1F6E
M1F67:  LD  HL,(PROG)
        DEC HL
        LD  (DATADD),HL
M1F6E:  LD  HL,(STKEND)
        LD  DE,$0032
        ADD HL,DE
        POP DE
        SBC HL,DE
        JR   NC,M1F82
        LD  HL,(PRAMT)
        AND  A
        SBC HL,DE
        JR   NC,M1F84

M1F82:  RST  $08          ;error: RAMTOP NO GOOD
        DFB $15          ;

M1F84:  EX  DE,HL
        LD  (RAMTOP),HL
        POP DE
        POP BC
        LD  HL,(MSTBOT)
        DEC HL
        LD  (HL),$3E
        DEC HL
        LD  SP,HL
        PUSH BC
        LD  (ERRSP),SP
        EX  DE,HL
        JP  (HL)
;*****
; Module: SYNTAX
; Routine: GO_SUB
;  this routine saves PPC and SUBPPC on the machine stack

```



```

; then uses GOTO to put the target address into NEWPPC and NSPPC
;*****
GO_SUB: POP DE ;get return adr off stack
        LD H,(IY+OSUBPPC) ;get sub line that is executing
        INC H ;point to next subline
        EX (SP),HL ;/stack it and save return address
        INC SP ;\ (8 bit PUSH)
        LD BC,(PPC) ;get current line number
        PUSH BC ;save it
        PUSH HL ;put return address back onto the stack
        LD (ERRSP),SP ;save the error stack pointer
        PUSH DE ;put ret adr back on stack
        CALL JUMP ;call GOTO to set up NEWPPC and NSPPC
        LD HL,(MSTBOT) ;get address above machine stack
        DEC H ;one page down
        LD DE,$0010 ;need 16 bytes
        ADD HL,DE ;point to address
        SBC HL,SP ;/stack pointer is still above the
        RET C ;required end address, so proceed
        JR M1FCF ;out of space
;*****
; Module: SYNTAX
; Routine: CHK_SZ
; checks to see if sufficient RAM is available below RAMTOP
; to give caller BC bytes of room. if not exits to error 4
;*****
CHK_SZ:
M1FBB: LD HL,(STKEND) ;get STKEND (start of spare space)
        ADD HL,BC ;add the number of bytes requested
        JR C,M1FCF ;
        EX DE,HL ;DE points to final address
        LD HL,$0050 ;/add safety margin for machine stack
        ADD HL,DE ;|point to > $FFFF, so error 4
        JR C,M1FCF
        LD DE,(RAMTOP) ;get RAMTOP - top of BASIC's memory
        SBC HL,DE ;/exit if requested < RAMTOP
        RET C ;\
;*****
; Module: SYNTAX
; Routine: ERR4
;*****
ERR4:
M1FCF: LD L,$03 ;/give error 4
        JP M0055 ;\ OUT OF MEMORY
;*****
; Module: SYNTAX
; Routine: RETURN
; removes the line number and sub-line number from the machine stack
; then jumps into the GOTO routine to put them into NEWPPC and NSPPC
;*****
RETURN:
        POP BC ;save the return address
        POP HL ;
        POP DE ;pop return line number
        LD A,D ;/jump if not a valid line number

```

```

CP    $3E                ;|
JR     Z,M1FE7           ;\
DEC    SP                ;adjust SP to get return sub-line number
EX     (SP),HL            ;get sub-line number and save HL
EX     DE,HL             ;sub-line number in D, line number in HL
LD     (ERRSP),SP        ;
PUSH   BC                ;save the machine return address
JP     M1EFD             ;GOTO the GOSUB return address
M1FE7: PUSH DE            ;save DE
        PUSH HL          ;save HL

        RST $08          ;/error - RETURN WITHOUT GOSUB
        DFB $06          ;\
;*****
; Module: SYNTAX
; Routine: PAUSE
;*****
PAUSE:  RES    KEYHIT,(IY+OFLAGS) ;reset keyhit flag
        CALL  FIX_U          ;put top of fp stack into BC
M1FF2:  HALT                ;wait for IM1
        DEC   BC             ;decrement the PAUSE counter
        LD    A,B            ;/
        OR    C              ;|jump if BC = 0, PAUSE timed
        JR    Z,M2004        ;\ out
        LD    A,B            ;/
        AND   C              ;|if BC <> $FFFF jump to examine
        INC   A              ;|the key hit
        JR    NZ,M1FFE       ;\
        INC   BC             ;BC = $FFFF, therefore no timeout
M1FFE:  BIT    KEYHIT,(IY+OFLAGS) ;/jump back to wait if a key has
        JR    Z,M1FF2        ;\ not been hit
M2004:  RES    KEYHIT,(IY+OFLAGS) ;force no KEYHIT
        RET
;*****
; Module: SYNTAX
; Routine: BREAK?
; test BREAK key. Return NC if break pressed
;*****
BREAK?:
M2009:  LD     A,$7F          ;/
        IN     A,($FE)        ;|read the keyboard
        RRA                ;\
        RET    C             ;return if BREAK not pressed

        BIT    6,(IY+{OERRLN+1}) ;/if we running a progrm
        JR    Z,M2017        ;\ check for the CAPS SHIFT key too
        SCF                ;force no BREAK
        RET                ;

M2017:  LD     A,$FE          ;/carry will be set if
        IN     A,($FE)        ;|CAPS SHIFT is depressed
        RRA                ;\
        RET                ;
;*****
; Module: SYNTAX

```

```

; Routine: DEF
;*****
DEF:
    CALL INTPT?                ;/jump if syntax checking
    JR    Z,M2027              ;\
    LD    A,$CE                ;load A with DEF FN token
    JP    M1E94

M2027:  SET    NUM,(IY+OFLAGS)   ;force a numerical result
    CALL    ALPHA?

    JR      NC,M2046
    RST     $20                 ;get next char
    CP      $24
    JR      NZ,M203A
    RES     6,(IY+OFLAGS)      ;FLAGS
    RST     $20                 ;get next char
M203A:  CP      $28
    JR      NZ,M207A
    RST     $20                 ;get next char
    CP      $29
    JR      Z,M2063
M2043:  CALL    ALPHA?
M2046:  JP      NC,M1BED         ;error - BAD BASIC
    EX      DE,HL
    RST     $20                 ;get next char
    CP      $24
    JR      NZ,M2051
    EX      DE,HL
    RST     $20                 ;get next char
M2051:  EX      DE,HL
    LD      BC,$0006
    CALL    INSERT
    INC     HL
    INC     HL
    LD      (HL),$0E
    CP      $2C
    JR      NZ,M2063
    RST     $20                 ;get next char
    JR      M2043
M2063:  CP      $29
    JR      NZ,M207A
    RST     $20                 ;get next char
    CP      $3D
    JR      NZ,M207A
    RST     $20                 ;get next char
    LD      A,(FLAGS)
    PUSH    AF
    CALL    EXPRN
    POP     AF
    XOR     (IY+OFLAGS)        ;FLAGS
    AND     $40
M207A:  JP      NZ,M1BED         ;error - BAD BASIC
    CALL    END?
    RST     $18                 ;get current character

```

```

CP    $7F
JR     Z,M20AE
CP    $EC
JR     Z,M20BC
CP    $E8
JP    NZ,M1BED                      ;error - BAD BASIC
RST    $20                          ;get next char
CALL    END?
BIT    7,(IY+{OERRLN+1})           ;
RET    Z
LD     HL,(ERRC)
LD     (NEWPPC),HL
LD     A,(ERRS)
LD     (NSPPC),A
RES    6,(IY+{OERRLN+1})           ;
M20A7: POP    HL
LD     DE,$0007
ADD    HL,DE
PUSH    HL
RET
M20AE: RST    $20                    ;get next char
CALL    END?
RES    7,(IY+{OERRLN+1})           ;
RES    6,(IY+{OERRLN+1})           ;
JR     M20A7
M20BC: RST    $20                    ;get next char
CALL    TEM6
CALL    END?
CALL    FP2BC
LD     A,B
AND    $3F
OR     $80
LD     B,A
LD     (ERRLN),BC
RET
RST    $18                          ;get current character
CP     $2C
JR     NZ,M20E0
CALL    INTPT?                      ;/jump if syntax checking
JR     Z,M20E7                      ;\

RST    $28      ;calc entry
DFB    $A1      ;CONST 1  (256)
DFB    $38      ;QUIT
JR     M20E7
M20E0: CALL    TEM6
CP     $2C
JR     NZ,M211C
M20E7: RST    $20                    ;get next char
CP     $0D
JR     Z,M20F5
CP     $3A
JR     Z,M20F5
CALL    TEM6
JR     M20FE

```

```

M20F5:  LD    BC,$270F
        CALL INTPT?                ;/call if interpreting
        CALL NZ,STK_BC             ;\
M20FE:  CALL END?
        CALL M211E
        INC  HL
        CALL FIND_L
        PUSH HL
        CALL M211E
        CALL FIND_L
        EX   DE,HL
        POP  HL
        PUSH HL
        SCF
        SBC  HL,DE
        JR   C,M211C
        POP  HL
        CALL M174D
        RET

M211C:   RST   $08                  ;error: NONSENSE IN BASIC
        DFB  $0B                    ;

M211E:   CALL FP2BC
        LD   A,B
        AND  $3F
        LD   H,A
        LD   L,C
        RET

M2127:   RST   $20                  ;get next char
;*****
; Module: SYNTAX
; Routine: SOUND
;*****
SOUND:
        CALL TEM8                  ;evaluate the next two comma separated
expressions
        CALL INTPT?                ;/jump if syntax checking
        JR   Z,M2146              ;\
        CALL FP2A                  ;/convert the top of fp stack to A
        PUSH AF                    ;\and save the result (data value)
        CALL FP2A                  ;convert the next argument to A (register
number)
        CP   $11                  ;/jump if the register number is too
large
        JP   NC,M1BED              ;\error - BAD BASIC
        DEC  A                     ;/jump if A >= $80
        INC  A                     ;|
        JP   M ,M1BED              ;\error - BAD BASIC
        OUT  ($F5),A               ;output register number to AY8910
address register
        POP  AF                    ;get the data value
        OUT  ($F6),A               ;output to the AY8910 data register
M2146:   RST   $18                  ;get current character
        CP   ";"                   ;/jump back around for the next

```

```

        JR      Z,M2127                ;\set of arguments
        CALL END?                      ;will return if interpreting
        RET

M214F:  CALL INTPT?                    ;check mode (syntax checking or
interpreting
        POP HL                        ;
        RET Z                          ;return if syntax checking
        JP (HL)
;*****
; Module: SYNTWO
; Routine: K_LPR
;*****
K_LPR:
        LD A,$03
        JR M2163
;*****
; Module: SYNTWO
; Routine: K_PRIN
;*****
K_PRIN:
        LD A,(ARSFLG)                ;
        RES 0,A                      ;
        LD (ARSFLG),A                ;
        LD A,$02                    ;stream 2 (print/list)
M2163:  CALL INTPT?                    ;/call if interpreting
        CALL NZ,SELECT                ;\
        CALL INTPT?                    ;/call if interpreting
        CALL NZ,M2179                ;\
        CALL DO_ATT5                  ;
        CALL M217E                    ;
        CALL END?                    ;
        RET                          ;
M2179:  SET 4,(IY+OFLAGS)              ;
        RET                          ;
;*****
; Module: SYNTWO
; Routine: P_SEQ
;*****
P_SEQ:
M217E:  RST $18                      ;get current character
        CALL M21E4
        JR Z,M2191
M2184:  CALL M21ED
        JR Z,M2184
        CALL M219B
        CALL M21ED
        JR Z,M2184
M2191:  CP $29
        RET Z
M2194:  CALL M214F
        LD A,$0D
        RST $10
        RET
M219B:  RST $18                      ;get current character

```

```

        CP    $AC
        JR     NZ,M21AD
        CALL  DYADIC
        CALL  M214F
        CALL  M2660
        LD    A,$16
        JR     M21BD
M21AD:   CP    $AD
        JR     NZ,M21C3
        RST   $20                ;get next char
        CALL  TEM6
        CALL  M214F
        CALL  FIX_U
        LD    A,$17
M21BD:   RST   $10
        LD    A,C
        RST   $10
        LD    A,B
        RST   $10
        RET
M21C3:   CALL  M239C
        RET   NC
        CALL  STRITO
        RET   NC
        CALL  EXPRN
        CALL  M214F
        BIT   6,(IY+OFLAGS)      ;FLAGS
        CALL  Z ,PGPSTR
        JP    NZ,M31A1
M21DB:   LD    A,B
        OR    C
        DEC   BC
        RET   Z
        LD    A,(DE)
        INC   DE
        RST   $10
        JR     M21DB
M21E4:   CP    $29                ;/return if close paren
        RET   Z                  ;\
;*****
; Module: SYNTWO
; Routine: TERM?
; checks for termination of a line
; if end of statement (newline or ":") return with ZF=1
;*****
TERM?:   CP    $0D                ;/return if end of line
        RET   Z                  ;\
        CP    $3A                ;/return if colon
        RET   Z                  ;\

M21ED:   RST   $18                ;get current char
        CP    $3B                ;/jump if semi-colon
        JR     Z,M2206            ;\
        CP    $2C                ;/jump if comma

```

```

        JR      NZ,M2200  ;\
        CALL INTPT?      ;/jump if syntax checking
        JR      Z,M2206   ;\
        LD      A,$06     ;
        RST     $10       ;WRITECHAR
        JR      M2206     ;
M2200:   CP      $27       ;/return if not apostrophe
        RET     NZ        ;\
        CALL M2194        ;
M2206:   RST     $20       ;get next char
        CALL M21E4        ;recursive call to check character
        JR      NZ,M220D  ;
        POP     BC        ;
M220D:   CP      A        ;
        RET     ;
;*****
; Module: SYNTWO
; Routine: STRITO
; update the current stream for "bank" devices
;*****
STRITO:
M220F:   CP      "#"      ;is the character a number sign?
        SCF          ;declare error
        RET     NZ      ;return if not #
        RST     $20     ;get next char
        CALL TEM6       ;evaluate a numeric expression
        AND     A        ;
        CALL M214F      ;"CALL" to (HL) if interpreting
        CALL INS_U1     ;convert the fp top of stack to a value in A
        LD      (STRMN),A ;put into stream number for "bank" devices
        CP      $10     ;/jump to invalid stream if
        JP      NC,M123D ;\requested stream # > 16
        CALL SELECT     ;select the stream in A
        AND     A        ;
        RET
;*****
; Module: SYNTWO
; Routine: INPUT
;*****
INPUT:
        LD      A,(ARSFLG)
        SET     0,A
        LD      (ARSFLG),A
        CALL INTPT?     ;/jump if syntax checking
        JR      Z,M2240  ;\
        LD      A,$01    ;/stream 1 (input command)
        CALL SELECT     ;\
        CALL M08A9
M2240:   LD      (IY+OTVFLAG),01 ;TVFLAG
        CALL M226B
        CALL END?
        LD      BC,(SPOSNCOL)
        LD      A,(DFSZ)
        CP      B
        JR      C,M2257

```



```

        LD    C,$21
        LD    B,A
M2257:  LD    (SPOSNCOL),BC
        LD    A,$19
        SUB   B
        LD    (SCRCT),A
        RES   0,(IY+0TVFLAG)          ;TVFLAG
        CALL  STTVC
        JP    M08A9
;*****
; Module: SYNTWO
; Routine: I_SEQ
;*****
I_SEQ:
M226B:  CALL  M21ED
        JR    Z,M226B
        CP    $28
        JR    NZ,M2282
        RST   $20                      ;get next char
        CALL  M217E
        RST   $18                      ;get current character
        CP    $29
        JP    NZ,M1BED                  ;error - BAD BASIC
        RST   $20                      ;get next char
        JP    M235C
M2282:  CP    $CA
        JR    NZ,M2297
        RST   $20                      ;get next char
        CALL  M1B82
        SET   7,(IY+0FLAGX)            ;FLAGX
        BIT   6,(IY+0FLAGS)            ;FLAGS
        JP    NZ,M1BED                  ;error - BAD BASIC
        JR    M22A4
M2297:  CALL  ALPHA?
        JP    NC,M2359
        CALL  M1B82
        RES   7,(IY+0FLAGX)            ;FLAGX
M22A4:  CALL  INTPT?                    ;/jump if syntax checking
        JP    Z ,M235C                  ;\
        CALL  M134E
        LD    HL,FLAGX
        RES   6,(HL)
        SET   5,(HL)
        LD    BC,$0001
        BIT   7,(HL)
        JR    NZ,M22C6
        LD    A,(FLAGS)
        AND   $40
        JR    NZ,M22C4
        LD    C,$03
M22C4:  OR    (HL)
        LD    (HL),A
M22C6:  RST   $30
        LD    (HL),$0D
        LD    A,C

```

```

RRCA
RRCA
JR      NC,M22D3
LD      A,$22
LD      (DE),A
DEC     HL
LD      (HL),A
M22D3:  LD      (KCUR),HL
        BIT     7,(IY+OFLAGX)          ;FLAGX
        JR      NZ,M2308
        LD      HL,(CHADD)
        PUSH    HL
        LD      HL,(ERRSP)
        PUSH    HL
        LD      HL,$22E4
        PUSH    HL
        BIT     4,(IY+OFLAGS2)        ;FLAGS2
        JR      Z,M22F2
        LD      (ERRSP),SP
M22F2:  LD      HL,(WORKSP)
        CALL    M0D0D
        LD      (IY+OERRNR),$FF        ;ERRNR
        CALL    EDIT_K
        RES     7,(IY+OFLAGS)          ;FLAGS
        CALL    M2363
        JR      M230B
M2308:  CALL    EDIT_K
M230B:  LD      (IY+{OKCUR+1}),00        ;
        CALL    M2380
        JR      NZ,M231E
        CALL    M0C83
        LD      BC,(ECHOE)
        CALL    STTVC
M231E:  LD      HL,FLAGX
        RES     5,(HL)
        BIT     7,(HL)
        RES     7,(HL)
        JR      NZ,M2345
        POP     HL
        POP     HL
        LD      (ERRSP),HL
        POP     HL
        LD      (XPTR),HL
        SET     7,(IY+OFLAGS)          ;FLAGS
        CALL    M2363
        LD      HL,(XPTR)
        LD      (IY+{OXPTR+1}),00        ;
        LD      (CHADD),HL
        JR      M235C
M2345:  LD      HL,(STKBOT)
        LD      DE,(WORKSP)
        SCF
        SBC     HL,DE
        LD      B,H
        LD      C,L

```

```

        CALL M2E70
        CALL M2EBD
        JR    M235C
M2359:  CALL M219B
M235C:  CALL M21ED
        JP    Z ,M226B
        RET
M2363:  LD     HL,(WORKSP)
        LD     (CHADD),HL
        RST    $18                      ;get current character
        CP     $E2
        JR     Z,M237A
        LD     A,(FLAGX)
        CALL M1BBC
        RST    $18                      ;get current character
        CP     $0D
        RET    Z

        RST    $08                      ;error: NONSENSE IN BASIC
        DFB    $0B                      ;

M237A:  CALL INTPT?                      ;/return if syntax checking
        RET    Z                      ;\
;*****
; Module: SYNTWO
; Routine: ERRH
;*****
ERRH:
        RST    $08                      ;error: STOP IN INPUT
        DFB    $10                      ;
;*****
; Module: SYNTWO
; Routine: NOTKB?
;*****
NOTKB?:
M2380:  LD     HL,(CURCHL)
        INC    HL
        INC    HL
        INC    HL
        INC    HL
        LD     A,(HL)
        CP     $4B
        RET
M238B:  RST    $20                      ;get next char
;*****
; Module: SYNTWO
; Routine: GR_COL
;*****
GR_COL:
M238C:  CALL M239C
        RET    C
        RST    $18                      ;get current character
        CP     $2C
        JR     Z,M238B
        CP     $3B

```

```

        JR      Z,M238B
        JP      M1BED                      ;error - BAD BASIC
M239C:  CP      $D9
        RET     C
        CP      $DF
        CCF
        RET     C
        PUSH    AF
        RST     $20                      ;get next char
        POP     AF
;*****
; Module: SYNTWO
; Routine: COLITM
;*****
COLITM:
M23A6:  SUB     $C9
        PUSH    AF
        CALL    TEM6
        POP     AF
        AND     A
        CALL    M214F
        PUSH    AF
        CALL    INS_U1
        LD      D,A
        POP     AF
        RST     $10
        LD      A,D
        RST     $10
        RET
;*****
; Module: SYNTWO
; Routine: TV_COL
;*****
TV_COL:
M23BB:  SUB     $11
        ADC     A,$00
        JR      Z,M23DE
        SUB     $02
        ADC     A,$00
        JR      Z,M241D
        CP      $01
        LD      A,D
        LD      B,$01
        JR      NZ,M23D2
        RLCA
        RLCA
        LD      B,$04
M23D2:  LD      C,A
        LD      A,D
        CP      $02
        JR      NC,M23EE
        LD      A,C
        LD      HL,PFLAG
        JR      M2416
;*****

```

```

; Module: SYNTWO
; Routine: COLOUR
;*****
COLOUR:
M23DE:  LD  A,D
        LD  B,$07
        JR  C,M23E8
        RLCA
        RLCA
        RLCA
        LD  B,$38
M23E8:  LD  C,A
        LD  A,D
        CP  $0A
        JR  C,M23F0

M23EE:  RST  $08          ;error: INVALID COLOR
        DFB $13          ;

M23F0:  LD  HL,ATTRT
        CP  $08
        JR  C,M2402
        LD  A,(HL)
        JR  Z,M2401
        OR  B
        CPL
        AND $24
        JR  Z,M2401
        LD  A,B
M2401:  LD  C,A
M2402:  LD  A,C
        CALL M2416
        LD  A,$07
        CP  D
        SBC A,A
        CALL M2416
        RLCA
        RLCA
        AND $50
        LD  B,A
        LD  A,$08
        CP  D
        SBC A,A
M2416:  XOR  (HL)
        AND B
        XOR  (HL)
        LD  (HL),A
        INC HL
        LD  A,B
        RET
;*****
; Module: SYNTWO
; Routine: HIFLSH
;*****
HIFLSH:

```

```

M241D:  SBC  A,A
        LD   A,D
        RRCA
        LD   B,$80
        JR   NZ,M2427
        RRCA
        LD   B,$40
M2427:  LD   C,A
        LD   A,D
        CP   $08
        JR   Z,M2431
        CP   $02
        JR   NC,M23EE
M2431:  LD   A,C
        LD   HL,ATTRT
        CALL M2416
;*****
; Module: SYNTWO
; Routine: BORDER
;*****
BORDER:
;
        LD   D,$24
        LD   A,C
        RRCA
        RRCA
        RRCA
        JR   M2416
M243E:  CALL INS_U1
        CP   $08
        JR   NC,M23EE
        OUT  ($FE),A
        RLCA
        RLCA
        RLCA
        BIT  5,A
        JR   NZ,M2450
        XOR  $07
M2450:  LD   (BORDCR),A
        RET
;*****
; Module: SYNTWO
; Routine: RSET
;*****
RSET:
        RST  $18                                ;get current character
        CP   $2A
        JR   NZ,M247F
        CALL M0020
        CALL END?
        RET
        LD   A,$10
        LD   HL,CH_0
M2465:  CALL M13A8
        INC  HL
        INC  HL

```

```

DEC A
JR NZ,M2465
LD HL,$09F4
PUSH HL
LD B,$FE
M2473: LD C,$88
PUSH BC
LD BC,$0000
PUSH BC
PUSH BC
CALL M65D0
RET
M247F: CP $23
JR Z,M2498
CALL END?
RET
LD HL,$0C4C
PUSH HL
LD BC,$FEFE
PUSH BC
LD BC,$0000
PUSH BC
PUSH BC
CALL M65D0
RET
M2498: RST $20 ;get next char
CALL TEM6
CALL END?
CALL INS_U1
CP $11
JR NC,M24B7
AND A
JP M ,M24B7
ADD A,A
ADD A,$16
LD L,A
LD H,$5C
LD E,(HL)
INC HL
LD D,(HL)
LD A,D
OR E
JR NZ,M24B9

M24B7: RST $08 ;error INVALID STREAM
DFB $17 ;

M24B9: LD A,D
CP $80
RET C
JP M2567
SUB $80
LD D,A
LD DE,(SYSCON)
ADD HL,DE

```

```

        INC HL
        LD B,(HL)
        LD D,$00
        LD E,$12
        ADD HL,DE
        PUSH HL
        JR M2473
;*****
; Module: SYNTWO
; Routine: NEWDEV
;*****
NEWDEV:
M24D2:  RST $18                      ;get current character
        CP "*"                      ;
        JP NZ,DOSAVE                ;save the program

        RST $20                      ;get next char
        CALL M1BEF
        CP ","
        JP NZ,M1BED                  ;error - BAD BASIC
        CALL INTPT?                  ;/jump if interpreting
        JR NZ,M24EC                  ;\
        CALL SKIPIT
        CALL END?

M24EC:  JR M2567
        CALL PGPSTR
        DEC BC
        LD A,B
        OR C
        JR NZ,M2567
        LD A,(DE)
        AND $DF
        LD C,A
        CALL M1374
        JP NC,M2567
        PUSH HL
        LD DE,$0014
        ADD HL,DE
        LD A,(HL)
        BIT 1,A
        JP Z,M2567
        POP HL
        EX DE,HL
        CALL M25B9
        EX DE,HL
        LD A,(TADDR)
        AND A
        CP $00
        JR C,M253F
        JR Z,M2543
        ADD A,$D4
        LD C,A
M251E:  PUSH BC
        LD D,(HL)

```



```

        LD     E,$88
        LD     BC,$000C
        ADD    HL,BC
        LD     C,(HL)
        INC    HL
        LD     B,(HL)
        PUSH   BC
        PUSH   DE
        LD     HL,(STKEND)
        DEC    HL
        LD     C,(HL)
        INC    C
        LD     (STKEND),HL
        LD     B,$00
        PUSH   BC
        LD     BC,$0000
        PUSH   BC
        CALL   M65D0
        RET

M253F:  LD     C,$F8
        JR     M251E
M2543:  LD     C,$EF
        JR     M251E

;
;jump to the save command in the EXROM
;
DOSAVE: POP    AF
        LD     BC,SLVM                ;
        PUSH   BC                    ;
        LD     BC,$FEFE                ;dock bank, chunk 0 active
        PUSH   BC                    ;
        LD     BC,$0000                ;no in or out params
        PUSH   BC                    ;
        PUSH   BC                    ;
        LD     A,(VIDMOD) ;/determine where the function
        AND    A                      ;\ dispatcher is
        JR     NZ,M2562 ;has been moved to high memory, so
                        ; call there
        CALL   M65D0                  ;CALL_BANK

M255E:  CALL   END?
        RET

M2562:  CALL   MFD90                  ;high mem function dispatcher call
        JR     M255E

M2567:  RST    $08                    ;/error INVALID IO DEVICE
        DFB    $12                    ;\
;*****
; Module: SYNTWO
; Routine: SKIPIT
; skip the rest of a line. gobble characters until a newline ($0D)
; a colon (:) or quote mark is found.
;*****
SKIPIT:

```

```

M2569:  LD  A,(ARSFLG)      ;/reset string quote
        RES 1,A           ;|flag
        LD  (ARSFLG),A    ;\
        PUSH BC           ;briefly save BC
        RST $18           ;get current character

;loop until the end of a subline, end of line or quote
; is found
M2573:  CP   $22           ;/jump if a quote
        JR   Z,M2582       ;\
        CP   ":"          ;/jump if colon (subline marker)
        JR   Z,M2582       ;\
        CP   $0D          ;/jump if newline
        JR   Z,M2582       ;\
        RST $20           ;/get next char
        JR   M2573         ;\and loop back around
M2582:  CP   ":"          ;/jump if not end of a subline
        JR   NZ,M258D      ;\
        LD  A,(ARSFLG)    ;/jump if still in a string literal
        BIT 1,A           ;|delimited by quote marks
        JR   NZ,M25B6      ;\
;
;look back as many as five characters to determine whether or not
; the quote, :, or newline are actually components of a floating
; point number
M258D:  PUSH HL           ;save current character pointer
        LD  B,$05         ;set up to look for "number slug"
M2590:  DEC  HL           ;point back
        LD  A,(HL)        ;/
        CP  $0E           ;|look back 5 characters to determine
        JR  Z,M25B5       ;|if the character was part of a
        DJNZ M2590        ;\floating point number
        POP HL           ;restore the character pointer
        RST $18           ;get current character
        CP  $22           ;/jump if not a quote
        JR  NZ,M25B3      ;\
;at this point we have found a quote in a literal string.
; we check the quote flag to determine whether this is the
; first or second quote found.  if the first, set the quote
; flag, else reset it
        LD  A,(ARSFLG)    ;/jump to reset the
        BIT 1,A           ;|quote flag
        JR  NZ,M25AC      ;\
        SET 1,A           ;/no previous quote, so
        LD  (ARSFLG),A    ;|set the flag and move on
        JR  M25B6         ;\to the next characters
M25AC:  RES 1,A           ;/reset the quote flag
        LD  (ARSFLG),A    ;|and move on
        JR  M25B6         ;\
M25B3:  POP  BC           ;restore BC
        RET              ;return to caller. HL and
                        ;CHADD point to the next character
                        ;to be interpreted
M25B5:  POP  HL           ;restore the character pointer
M25B6:  RST $20           ;get next char

```

```

        JR    M2573                ;back around to gobble more characters
;*****
; Module: SYNTWO
; Routine: PASSEM
;*****
PASSEM:
M25B9:  LD    BC,$FEFE            ;enable the EXROM for chunk 0
        CALL BANK_ENABLE        ;BANK_ENABLE
        CALL M0F09
        LD    BC,$FF00          ;enable the home ROM in all chunks
        CALL BANK_ENABLE        ;BANK_ENABLE
;*****
; Module: SYNTWO
; Routine: CAT
;*****
CAT:
        LD    B,$CF             ;load B with "CAT" token
        JR    M25D6            ;jumps to error if interpreting
;*****
; Module: SYNTWO
; Routine: FORMAT
;*****
FORMAT:
        LD    B,$D0             ;load B with "FORMAT" token
        JR    M25D6            ;jumps to error if interpreting
;*****
; Module: SYNTWO
; Routine: MOVE
;*****
MOVE:
        LD    B,$D1             ;load B with "MOVE" token
        JR    M25D6            ;jumps to error if interpreting
;*****
; Module: SYNTWO
; Routine: ERASE
;*****
ERASE:
        LD    B,$D2             ;load B with "ERASE" token
M25D6:  CALL INTPT?              ;/jump if interpreting
        JR    NZ,M25E1          ;\goes to error routine
        CALL SKIPIT             ;skip the rest of the line
        CALL END?               ;returns here if interpreting
M25E1:  JP    M2567              ;error - INVALID IO DEVICE

        LD    BC,$000C          ;offset into jump table

;CALL_BANK with HL pointing to a jump table, BC containing
; an offset into it and DE containing the bank and horiz sel
; respectively. the top of the FP stack at (STKEND) contains
; a byte with the number of parameters out expected
;
        ADD    HL,BC            ;/
        LD     C,(HL)           ;|get jump addresss
        INC    HL               ;|

```

```

LD B,(HL) ;\
PUSH BC ;push address onto stack
PUSH DE ;push horiz sel and bank
LD HL,(STKEND) ;/
DEC HL ;|get the number of params out
LD C,(HL) ;|from CALL_BANK from the
INC C ;|free space
LD (STKEND),HL ;\
LD B,$00 ;/number of parameters out
PUSH BC ;\
LD BC,$0000 ;/no parameters in
PUSH BC ;\
CALL M65D0 ;CALL_BANK
RET

RST $08 ;/error - INVALID IO DEVICE
DFB $12 ;\
;*****
; Module: GRAPHS
; Routine: SCRMBL
;*****
SCRMBL:
M2603: LD A,$AF
SUB B
JP C ,M2852
LD B,A
AND A
RRA
SCF
RRA
AND A
RRA
XOR B
AND $F8
XOR B
LD H,A
LD A,C
RLCA
RLCA
RLCA
XOR B
AND $C7
XOR B
RLCA
RLCA
LD L,A
LD A,C
AND $07
RET
;*****
; Module: GRAPHS
; Routine: F_PNT
;*****
F_PNT:
M2624: CALL M2660

```

```

        CALL M2603
        LD  B,A
        INC B
        LD  A,(HL)
M262D:  RLCA
        DJNZ M262D
        AND $01
        JP  M30E6          ;stack A on calc stack
;*****
; Module: GRAPHS
; Routine: PLOT
;*****
PLOT:
M2635:  CALL M2660
        CALL M263E
        JP  DO_ATTS
;*****
; Module: GRAPHS
; Routine: PLOTBC
;*****
PLOTBC:
M263E:  LD  (XCOORD),BC
        CALL M2603
        LD  B,A
        INC B
        LD  A,$FE
M2649:  RRCA
        DJNZ M2649
        LD  B,A
        LD  A,(HL)
        LD  C,(IY+OPFLAG)      ;PFLAG
        BIT 0,C
        JR   NZ,M2656
        AND B
M2656:  BIT 2,C
        JR   NZ,M265C
        XOR B
        CPL
M265C:  LD  (HL),A
        JP  M0710
;*****
; Module: GRAPHS
; Routine: GET_XY
; return with X and Y in D and E and the sign of the numbers in
; B and C ($01 signifies positive, $FF signifies negative)
;*****
GET_XY:
M2660:  CALL M266D          ;get the first parameter
        LD  B,A            ;save the number
        PUSH BC           ;save number (B) and the sign info (C)
        CALL M266D        ;get the second parameter
        LD  E,C           ;load the sign info into E
        POP BC            ;get the first parameter back
        LD  D,C           ;put the first parameter sign in D
        LD  C,A           ;put the second parameter in C

```

```

RET
M266D: CALL FP2A          ;get the top of the fp stack to A
      JP  C ,M2852        ;error B
      LD  C,$01          ;/return if if the number was positive
      RET  Z              ;\
      LD  C,$FF          ;indicate a negative value
      RET
;*****
; Module: GRAPHS
; Routine: CIRCLE
;*****
CIRCLE:
      RST  $18            ;get current character
      CP   $2C
      JP  NZ,M1BED        ;error - BAD BASIC
      RST  $20            ;get next char
      CALL TEM6
      CALL END?
      RST $28 ;calc entry
      DFB $2A ;ABS
      DFB $3D ;FLOAT
      DFB $38 ;QUIT
      LD  A,(HL)
      CP  $81
      JR  NC,M2694
      RST $28 ;calc entry
      DFB $02 ;DROP
      DFB $38 ;QUIT
      JR  M2635
M2694: RST $28 ;calc entry
      DFB $A3 ;CONST 3 (PI/2)
      DFB $38 ;QUIT
      LD  (HL),$83
      RST $28 ;calc entry
      DFB $C5 ;T -> MEM 5
      DFB $02 ;DROP
      DFB $38 ;QUIT
      CALL M27D6
      PUSH BC
      RST $28 ;calc entry
      DFB $31 ;DUP
      DFB $E1 ;MEM 1 -> T
      DFB $04 ;TIMES
      DFB $38 ;QUIT
      LD  A,(HL)
      CP  $80
      JR  NC,M26B3
      RST $28 ;calc entry
      DFB $02 ;DROP
      DFB $02 ;DROP
      DFB $38 ;QUIT
      POP  BC
      JP  M2635
M26B3: RST $28 ;calc entry
      DFB $C2 ;T -> MEM 2

```

```

    DFB $01          ;SWAP
    DFB $C0          ;T -> MEM 0
    DFB $02          ;DROP
    DFB $03          ;SUB
    DFB $01          ;SWAP
    DFB $E0          ;MEM 0 -> T
    DFB $0F          ;ADD
    DFB $C0          ;T -> MEM 0
    DFB $01          ;SWAP
    DFB $31          ;DUP
    DFB $E0          ;MEM 0 -> T
    DFB $01          ;SWAP
    DFB $31          ;DUP
    DFB $E0          ;MEM 0 -> T
    DFB $A0          ;CONST 0 (0)
    DFB $C1          ;T -> MEM 1
    DFB $02          ;DROP
    DFB $38          ;QUIT
    INC (IY+{MEM2-Y}) ;
    CALL INS_U1
    LD L,A
    PUSH HL
    CALL INS_U1
    POP HL
    LD H,A
    LD (XCOORD),HL
    POP BC
    JP M2779
;*****
; Module: GRAPHS
; Routine: DRAW
;*****
DRAW:
    RST $18          ;get current character
    CP $2C
    JR Z,M26E6
    CALL END?
    JP M27D0
M26E6: RST $20          ;get next char
    CALL TEM6
    CALL END?
    RST $28          ;calc entry
    DFB $C5          ;T -> MEM 5
    DFB $A2          ;CONST 1 (0.5)
    DFB $04          ;TIMES
    DFB $1F          ;SIN
    DFB $31          ;DUP
    DFB $30          ;NOT
    DFB $30          ;NOT
    DFB $00,$06      ;IFJUMP 26FC
    DFB $02          ;DROP
    DFB $38          ;QUIT
    JP M27D0
    RET NZ
    LD (BC),A

```

```

POP BC
LD (BC),A
LD SP,$E12A
LD BC,$2AE1
RRCA
RET PO
DEC B
LD HL,($01E0)
DEC A
JR C,M278D
CP $81
JR NC,M271A
RST $28 ;calc entry
DFB $02 ;DROP
DFB $02 ;DROP
DFB $38 ;QUIT
JP M27D0
M271A: CALL M27D6
PUSH BC
RST $28 ;calc entry
DFB $02 ;DROP
DFB $E1 ;MEM 1 -> T
DFB $01 ;SWAP
DFB $05 ;DIV
DFB $C1 ;T -> MEM 1
DFB $02 ;DROP
DFB $01 ;SWAP
DFB $31 ;DUP
DFB $E1 ;MEM 1 -> T
DFB $04 ;TIMES
DFB $C2 ;T -> MEM 2
DFB $02 ;DROP
DFB $01 ;SWAP
DFB $31 ;DUP
DFB $E1 ;MEM 1 -> T
DFB $04 ;TIMES
DFB $E2 ;MEM 2 -> T
DFB $E5 ;MEM 5 -> T
DFB $E0 ;MEM 0 -> T
DFB $03 ;SUB
DFB $A2 ;CONST 1 (0.5)
DFB $04 ;TIMES
DFB $31 ;DUP
DFB $1F ;SIN
DFB $C5 ;T -> MEM 5
DFB $02 ;DROP
DFB $20 ;COS
DFB $C0 ;T -> MEM 0
DFB $02 ;DROP
DFB $C2 ;T -> MEM 2
DFB $02 ;DROP
DFB $C1 ;T -> MEM 1
DFB $E5 ;MEM 5 -> T
DFB $04 ;TIMES
DFB $E0 ;MEM 0 -> T

```



```

DFB $E2          ;MEM 2 -> T
DFB $04          ;TIMES
DFB $0F          ;ADD
DFB $E1          ;MEM 1 -> T
DFB $01          ;SWAP
DFB $C1          ;T -> MEM 1
DFB $02          ;DROP
DFB $E0          ;MEM 0 -> T
DFB $04          ;TIMES
DFB $E2          ;MEM 2 -> T
DFB $E5          ;MEM 5 -> T
DFB $04          ;TIMES
DFB $03          ;SUB
DFB $C2          ;T -> MEM 2
DFB $2A          ;ABS
DFB $E1          ;MEM 1 -> T
DFB $2A          ;ABS
DFB $0F          ;ADD
DFB $02          ;DROP
DFB $38          ;QUIT
LD  A,(DE)
CP  $81
POP BC
JP  C ,M27D0
PUSH BC
RST $28 ;calc entry
DFB $01          ;SWAP
DFB $38          ;QUIT
LD  A,(XC00RD)
CALL M30E6 ;stack A on calc stack
RST $28 ;calc entry
DFB $C0          ;T -> MEM 0
DFB $0F          ;ADD
DFB $01          ;SWAP
DFB $38          ;QUIT
LD  A,(YC00RD)
CALL M30E6 ;stack A on calc stack
RST $28 ;calc entry
DFB $C5          ;T -> MEM 5
DFB $0F          ;ADD
DFB $E0          ;MEM 0 -> T
DFB $E5          ;MEM 5 -> T
DFB $38          ;QUIT
POP BC
M2779: DEC B
JR    Z,M27B8
JR    M2792
M277E: RST $28 ;calc entry
DFB $E1          ;MEM 1 -> T
DFB $31          ;DUP
DFB $E3          ;MEM 3 -> T
DFB $04          ;TIMES
DFB $E2          ;MEM 2 -> T
DFB $E4          ;MEM 4 -> T
DFB $04          ;TIMES

```

```

        DFB $03          ;SUB
        DFB $C1          ;T -> MEM 1
        DFB $02          ;DROP
        DFB $E4          ;MEM 4 -> T
        DFB $04          ;TIMES
        DFB $E2          ;MEM 2 -> T
        DFB $E3          ;MEM 3 -> T
M278D:  DFB $04          ;TIMES
        DFB $0F          ;ADD
        DFB $C2          ;T -> MEM 2
        DFB $02          ;DROP
        DFB $38          ;QUIT
M2792:  PUSH BC
        RST $28          ;calc entry
        DFB $C0          ;T -> MEM 0
        DFB $02          ;DROP
        DFB $E1          ;MEM 1 -> T
        DFB $0F          ;ADD
        DFB $31          ;DUP
        DFB $38          ;QUIT
        LD  A,(XCOORD)
        CALL M30E6        ;stack A on calc stack
        RST $28          ;calc entry
        DFB $03          ;SUB
        DFB $E0          ;MEM 0 -> T
        DFB $E2          ;MEM 2 -> T
        DFB $0F          ;ADD
        DFB $C0          ;T -> MEM 0
        DFB $01          ;SWAP
        DFB $E0          ;MEM 0 -> T
        DFB $38          ;QUIT
        LD  A,(YCOORD)
        CALL M30E6        ;stack A on calc stack
        RST $28          ;calc entry
        DFB $03          ;SUB
        DFB $38          ;QUIT
        CALL M2810
        POP  BC
        DJNZ M277E
M27B8:  RST $28          ;calc entry
        DFB $02          ;DROP
        DFB $02          ;DROP
        DFB $01          ;SWAP
        DFB $38          ;QUIT
        LD  A,(XCOORD)
        CALL M30E6        ;stack A on calc stack
        RST $28          ;calc entry
        DFB $03          ;SUB
        DFB $01          ;SWAP
        DFB $38          ;QUIT
        LD  A,(YCOORD)
        CALL M30E6        ;stack A on calc stack
        RST $28          ;calc entry
        DFB $03          ;SUB
        DFB $38          ;QUIT

```

```

M27D0:  CALL M2810
        JP   DO_ATT5
M27D6:  RST $28      ;calc entry
        DFB $31          ;DUP
        DFB $28          ;ROOT
        DFB $34,$32,$00  ;LITERAL 8200000000 ( 2),27DC
        DFB $01          ;SWAP
        DFB $05          ;DIV
        DFB $E5          ;MEM 5 -> T
        DFB $01          ;SWAP
        DFB $05          ;DIV
        DFB $2A          ;ABS
        DFB $38          ;QUIT
        CALL FP2A
        JR    C,M27EE
        AND  $FC
        ADD  A,$04
        JR    NC,M27F0
M27EE:  LD   A,$FC
M27F0:  PUSH AF
        CALL M30E6      ;stack A on calc stack
        RST $28      ;calc entry
        DFB $E5          ;MEM 5 -> T
        DFB $01          ;SWAP
        DFB $05          ;DIV
        DFB $31          ;DUP
        DFB $1F          ;SIN
        DFB $C4          ;T -> MEM 4
        DFB $02          ;DROP
        DFB $31          ;DUP
        DFB $A2          ;CONST 1 (0.5)
        DFB $04          ;TIMES
        DFB $1F          ;SIN
        DFB $C1          ;T -> MEM 1
        DFB $01          ;SWAP
        DFB $C0          ;T -> MEM 0
        DFB $02          ;DROP
        DFB $31          ;DUP
        DFB $04          ;TIMES
        DFB $31          ;DUP
        DFB $0F          ;ADD
        DFB $A1          ;CONST 1 (256)
        DFB $03          ;SUB
        DFB $1B          ;NEGATE
        DFB $C3          ;T -> MEM 3
        DFB $02          ;DROP
        DFB $38          ;QUIT
        POP  BC
        RET
;*****
; Module: GRAPHS
; Routine: DRAW_L
;*****
DRAW_L:
M2810:  CALL M2660

```

```

;*****
; Module: GRAPHS
; Routine: DRAWLN
;*****
DRAWLN:
    LD    A,C
    CP    B
    JR     NC,M281D
    LD    L,C
    PUSH  DE
    XOR   A
    LD    E,A
    JR     M2824
M281D:  OR    C
    RET   Z
    LD    L,B
    LD    B,C
    PUSH  DE
    LD    D,$00
M2824:  LD    H,B
    LD    A,B
    RRA
M2827:  ADD   A,L
    JR     C,M282D
    CP    H
    JR     C,M2834
M282D:  SUB   H
    LD    C,A
    EXX
    POP   BC
    PUSH  BC
    JR     M2838
M2834:  LD    C,A
    PUSH  DE
    EXX
    POP   BC
M2838:  LD    HL,(XCOORD)
    LD    A,B
    ADD   A,H
    LD    B,A
    LD    A,C
    INC   A
    ADD   A,L
    JR     C,M2850
    JR     Z,M2852
M2845:  DEC   A
    LD    C,A
    CALL  M263E
    EXX
    LD    A,C
    DJNZ  M2827
    POP   DE
    RET
M2850:  JR     Z,M2845

```

```

M2852:  RST  $08                      ;error: INTEGER OUT OF RANGE
        DFB  $0A                      ;
;*****
; Module: EXPRN
; Routine: EXPRN
;  evaluate an expression
;*****
EXPRN:
M2854:  RST  $18                      ;get current character into A
        LD   B,$00                    ;/save C
        PUSH BC                       ;\

;this will search the table at M294C for the character in A.  if
; the operator or token is found, the routine will jump to the
; handler for it.
M2858:  LD   C,A                      ;/
        LD   HL,M294C                  ;|find C at (HL)
        CALL SEARCH                    ;\
        LD   A,C                      ;restore the character to A
        JP   NC,M2A42                  ;jump if item not found
        LD   B,$00                    ;/get offset value
        LD   C,(HL)                   ;\
        ADD  HL,BC                    ;/form target address and
        JP   (HL)                     ;\jump to it

M2868:  CALL NEXTCH                    ;get the next character
        INC  BC                        ;
        CP   $0D                      ;/can't be newline
        JP   Z ,M1BED                  ;\error - BAD BASIC
        CP   $22                      ;/jump if not quote
        JR   NZ,M2868                  ;\
        CALL NEXTCH                    ;get the next character
        CP   $22                      ;compare to quote
        RET                             ;

M287B:  RST  $20                      ;get next char with filtering
        CP   "("                      ;/jump if not open paren
        JR   NZ,M2886                  ;\
        CALL DYADIC                    ;evaluate the next comma delimited
pair
                                           ; or parameters
        RST  $18                      ;get current character
        CP   ")"                      ;/jump if not close paren
M2886:  JP   NZ,M1BED                  ;\error - BAD BASIC
;*****
; Module: EXPRN
; Routine: INPT?
;  checks the interpret flag
;  returns ZF=1 if syntax check, ZF = 0 if interpreting statement
;  (condition NZ if interpreting)
;*****
INTPT?:
M2889:  BIT  7,(IY+OFLAGS)             ;FLAGS
        RET
;*****

```

```

; Module: EXPRN
; Routine: F_SCRN
;*****
F_SCRN:
M288E:  CALL M2660      ;get the x and y screen coords
        LD  HL,(CHARS) ;get the pointer to the character bit map table
        LD  DE,$0100   ;point to the bit map
        ADD HL,DE      ;point to the position in the
        LD  A,C        ;
        RRCA          ;
        RRCA          ;
        RRCA          ;
        AND $E0        ;
        XOR  B         ;
        LD  E,A        ;
        LD  A,C        ;
        AND $18        ;
        XOR  $40
        LD  D,A
        LD  B,$60
M28A8:  PUSH BC
        PUSH DE
        PUSH HL
        LD  A,(DE)
        XOR (HL)
        JR   Z,M28B3
        INC A
        JR   NZ,M28CC
        DEC A
M28B3:  LD  C,A
        LD  B,$07
M28B6:  INC D
        INC HL
        LD  A,(DE)
        XOR (HL)
        XOR C
        JR   NZ,M28CC
        DJNZ M28B6
        POP BC
        POP BC
        POP BC
        LD  A,$80
        SUB B
        LD  BC,$0001
        RST $30
        LD  (DE),A
        JR  M28D6
M28CC:  POP HL
        LD  DE,$0008
        ADD HL,DE
        POP DE
        POP BC
        DJNZ M28A8
        LD  C,B
M28D6:  RET

```

```

;*****
; Module: EXPRN
; Routine: F_ATTR
;*****
F_ATTR:
M28D7:  CALL M2660
        LD  A,C
        RRCA
        RRCA
        RRCA
        LD  C,A
        AND $E0
        XOR B
        LD  L,A
        LD  A,C
        AND $03
        XOR $58
        LD  H,A
        LD  A,(HL)
        JP  M30E6      ;stack A on calc stack
        CALL INTPT?    ;/jump if interpreting
        JR   Z,M28F5    ;\
        RST $28      ;calc entry
        DFB $A3      ;CONST 3 (PI/2)
        DFB $38      ;QUIT
M28F5:  JP  M2A81
M28F8:  CALL M287B
        CALL NZ,M2902
        RST $20      ;get next char
M28FF:  JP  M2A81
M2902:  CALL M2660
        LD  A,B
        CALL M292B
        LD  A,C
        CALL M292B
        LD  D,C
        LD  A,$0E
        OUT ($F5),A
        LD  C,$F6
        IN  A,(C)
        CPL
        LD  B,D
        DJNZ M2926
        AND $0F
        CP  $0F
        JR   C,M2922
        AND $00
M2922:  CALL M30E6      ;stack A on calc stack
        RET
M2926:  RLCA
        AND $01
M2929:  JR  M2922
M292B:  SUB $02
        ADC A,$00
        JR   NZ,M2932

```

```

RET

M2932:  RST  $08          ;error: INVALID ARGUMENT
        DFB  $09          ;

M2934:  CALL INTPT?        ;/jump if syntax checking
        JR    Z,M2948      ;\
M2939:  LD    HL,(RAMTOP)
        LD    DE,(STKEND)
        AND  A
        SBC  HL,DE
        LD   C,L
        LD   B,H
        CALL STK_BC
M2948:  RST  $20          ;get next char
        JP   M2A81

M294C:  DFB  $22          ;QUOTE
        DFB  $24          ;{2971-$} AND $00FF *
        DFB  "("          ;
        DFB  $57          ;{29A6-$} AND $00FF *
        DFB  "."          ;
        DFB  $FA          ;{2A4B-$} AND $00FF *
        DFB  "+"          ;
        DFB  $1A          ;{296D-$} AND $00FF *
        DFB  $7C          ;SOUND or }
        DFB  $16          ;{296B-$} AND $00FF *
        DFB  $7E          ;FREE
        DFB  $12          ;{2969-$} AND $00FF *
        DFB  $A8          ;FN
        DFB  $5A          ;{29B3-$} AND $00FF *
        DFB  $A5          ;RND
        DFB  $5B          ;{29B6-$} AND $00FF *
        DFB  $A7          ;PI
        DFB  $88          ;{F_PI-$} AND $00FF *
        DFB  $A6          ;INKEY$
        DFB  $93          ;{F_INKEY-$} AND $00FF *
        DFB  $C4          ;BIN
        DFB  $EA          ;{2A4B-$} AND $00FF *
        DFB  $AA          ;SCREEN$
        DFB  $C3          ;{2A26-$} AND $00FF *
        DFB  $AB          ;ATTR
        DFB  $CB          ;{2A30-$} AND $00FF *
        DFB  $A9          ;POINT
        DFB  $D2          ;{2A39-$} AND $00FF *
        DFB  $00          ;end of table

M2969:  JR    M2934
M296B:  JR    M28F8
M296D:  RST  $20          ;get next char
M296E:  JP    M2858
M2971:  RST  $18          ;get current character
        INC  HL
        PUSH HL

```



```

M2975:  LD  BC,$0000
        CALL M2868
;      DFB $CD,$68,$28
        JR   NZ,M2997
M297C:  CALL M2868
        JR   Z,M297C
        CALL INTPT?           ;/jump if syntax checking
        JR   Z,M2997           ;\
        RST  $30
        POP  HL
        PUSH DE
M2989:  LD  A,(HL)
        INC  HL
        LD  (DE),A
        INC  DE
        CP  $22
        JR   NZ,M2989
        LD  A,(HL)
        INC  HL
        CP  $22
        JR   Z,M2989
M2997:  DEC  BC
        POP  DE
M2999:  LD  HL,FLAGS
        RES  6,(HL)
        BIT  7,(HL)
        CALL NZ,M2E70
        JP  M2AD0

M29A6:  RST  $20                ;get next char
        CALL EXPRN
        CP  $29
        JP  NZ,M1BED           ;error - BAD BASIC
        RST  $20                ;get next char
        JP  M2AD0
;      JP  M2B7B
M29B3:  DFB $C3,$7B,$2B

;*****
; Module: EXPRN
; Routine: RND
;*****
RND:
        CALL INTPT?           ;/jump if syntax checking
        JR   Z,M29E3           ;\
        LD  BC,(SEED)
        CALL STK_BC
        RST $28                ;calc entry
        DFB $A1                ;CONST 1 (256)
        DFB $0F                ;ADD
        DFB $34,$37,$16        ;LITERAL 8716000000 ( 75),29C8
        DFB $04                ;TIMES
        DFB $34,$80,$41,$00,$00,$80 ;LITERAL 9100008000 ( 65537),29CF
        DFB $32                ;INTDIV
        DFB $02                ;DROP

```

```

        DFB $A1                                ;CONST 1 (256)
        DFB $03                                ;SUB
        DFB $31                                ;DUP
    DFB $38                                ;QUIT
        CALL FP2BC
        LD (SEED),BC
        LD A,(HL)
        AND A
        JR Z,M29E3
        SUB $10
        LD (HL),A
M29E3:   JR M29EE
;*****
; Module: EXPRN
; Routine: F_PI
;*****
F_PI:
        CALL INTPT?                            ;/jump if syntax checking
        JR Z,M29EE                            ;\
        RST $28 ;calc entry
        DFB $A3                                ;CONST 3 (PI/2)
        DFB $38                                ;QUIT
        INC (HL)
M29EE:   RST $20                                ;get next char
        JP M2A81
;*****
; Module: EXPRN
; Routine: F_INKY
;*****
F_INKY:
        LD BC,$105A
        RST $20                                ;get next char
        CP $23
        JP Z ,M2ACB
        LD HL,FLAGS
        RES 6,(HL)
        BIT 7,(HL)
        JR Z,M2A23
        CALL M02B0
        LD C,$00
        JR NZ,M2A1E
        CALL M035C
        JR NC,M2A1E
        DEC D
        LD E,A
        CALL M0371
        PUSH AF
        LD BC,$0001
        RST $30
        POP AF
        LD (DE),A
        LD C,$01
M2A1E:   LD B,$00
        CALL M2E70
M2A23:   JP M2AD0

```

```

M2A26:  CALL M287B
        CALL NZ,M288E
        RST $20                      ;get next char
        JP  M2999
M2A30:  CALL M287B
        CALL NZ,M28D7
        RST $20                      ;get next char
        JR  M2A81
M2A39:  CALL M287B
        CALL NZ,M2624
        RST $20                      ;get next char
        JR  M2A81

M2A42:  CALL ALNUM?                  ;/jump if A was not
        JR  NC,M2A9D                 ;\ alphanumeric
        CP  "A"                      ;/jump if a letter
        JR  NC,M2A87                 ;\

M2A4B:  CALL INTPT?                  ;/jump if interpreting
        JR  NZ,M2A73                 ;\
        CALL M3059

        RST $18                      ;get current character
        LD  BC,$0006                 ;/open a space for a floating point
        CALL INSERT                  ;\ number
        INC HL                      ;/drop in the number slug
        LD  (HL),$0E                 ;\
        INC HL                      ;/
        EX  DE,HL                   ;|
        LD  HL,(STKEND)              ;|transfer in the fp number
        LD  C,$05                    ;|
        AND A                        ;|
        SBC HL,BC                    ;|
        LD  (STKEND),HL              ;|
        LDIR                          ;\
        EX  DE,HL
        DEC HL                      ;point back to the last byte of the fp number
        CALL M0077                   ;get the byte at ++HL
        JR  M2A81                   ;jump to indicate a numeric result
;
;step over the ASCII digits of a number until the
; slug character is reached
M2A73:  RST $18                      ;get current character
                                           ;(RST $18 puts CHADD into HL)

M2A74:  INC  HL                      ;/
        LD  A,(HL)                   ;|loop until then number
        CP  $0E                      ;|slug is found
        JR  NZ,M2A74                 ;\
        INC HL                      ;point to the fp number
        CALL STK_M                   ;stack the fp value at (HL)
        LD  (CHADD),HL              ;update CHADD
M2A81:  SET  NUM,(IY+OFLAGS)          ;indicate a number result
        JR  M2A9B

```

```

M2A87:  CALL FIND_N                ;find a variable in the variables area
        JP   C ,M1B91             ;
        CALL Z ,M2D54
        LD   A,(FLAGS)
        CP   $C0
        JR   C,M2A9B
        INC  HL
        CALL STK_M
M2A9B:  JR   M2AD0
M2A9D:  LD   BC,$09DB
        CP   $2D
        JR   Z,M2ACB
        LD   BC,$1018
        CP   $AE
        JR   Z,M2ACB
        SUB  $AF
        JP   C ,M1BED             ;error - BAD BASIC
        LD   BC,$04F0
        CP   $14
        JR   Z,M2ACB
        JP   NC,M1BED            ;error - BAD BASIC
        LD   B,$10
        ADD  A,$DC
        LD   C,A
        CP   $DF
        JR   NC,M2AC5
        RES  6,C
M2AC5:  CP   $EE
        JR   C,M2ACB
        RES  7,C
M2ACB:  PUSH BC
        RST  $20                 ;get next char
        JP   M2858
M2AD0:  RST  $18                 ;get current character
M2AD1:  CP   $28
        JR   NZ,M2AE1
        BIT  6,(IY+OFLAGS)       ;FLAGS
        JR   NZ,M2AF2
        CALL M2E10
        RST  $20                 ;get next char
        JR   M2AD1
M2AE1:  LD   B,$00
        LD   C,A
        LD   HL,OPTBL
        CALL SEARCH
        JR   NC,M2AF2
        LD   C,(HL)
        LD   HL,OPPRI-$C3
        ADD  HL,BC
        LD   B,(HL)
M2AF2:  POP  DE
        LD   A,D
        CP   B
        JR   C,M2B31

```

```

AND A
JP Z ,M0018
PUSH BC
LD HL,FLAGS
LD A,E
CP $ED
JR NZ,M2B0A
BIT 6,(HL)
JR NZ,M2B0A
LD E,$99
M2B0A: PUSH DE
CALL INTPT? ;/jump if syntax checking
JR Z,M2B19 ;\
LD A,E
AND $3F
LD B,A
RST $28 ;calc entry
DFB $3B ;XEQTb
DFB $38 ;QUIT
JR M2B22
M2B19: LD A,E
XOR (IY+OFLAGS) ;FLAGS
AND $40
M2B1F: JP NZ,M1BED ;error - BAD BASIC
M2B22: POP DE
LD HL,FLAGS
SET 6,(HL)
BIT 7,E
JR NZ,M2B2E
RES 6,(HL)
M2B2E: POP BC
JR M2AF2
M2B31: PUSH DE
LD A,C
BIT 6,(IY+OFLAGS) ;FLAGS
JR NZ,M2B4E
AND $3F
ADD A,$08
LD C,A
CP $10
JR NZ,M2B46
SET 6,C
JR M2B4E
M2B46: JR C,M2B1F
CP $17
JR Z,M2B4E
SET 7,C
M2B4E: PUSH BC
RST $20 ;get next char
JP M2858
;~~~~~
OPTBL: DFB "+" ;ADD
DFB $CF ;2B7A
DFB "-" ;SUBTRACT
DFB $C3 ;2B6E

```

```

DFB "*"          ;MULTIPLY
DFB $C4          ;2B6F
DFB "/"          ;DIVIDE
DFB $C5          ;2B70
DFB "^"          ;POWER
DFB $C6          ;2B71
DFB "="          ;EQUALS
DFB $CE          ;2B79
DFB ">"          ;GREATER THAN
DFB $CC          ;2B77
DFB "<"          ;LESS THAN
DFB $CD          ;2B78
DFB $C7          ;<=
DFB $C9          ;2B74
DFB $C8          ;>=
DFB $CA          ;2B75
DFB $C9          ;<>
DFB $CB          ;2B76
DFB $C5          ;OR
DFB $C7          ;2B72
DFB $C6          ;AND
DFB $C8          ;2B73
DFB $00

```

;operation priority table

```

OPPRI:  DFB $06
        DFB $08
        DFB $08
        DFB $0A
        DFB $02
        DFB $03
        DFB $05
        DFB $05
        DFB $05
        DFB $05
        DFB $05
        DFB $05
        DFB $06

```

;was M29B3

```

M2B7B:  CALL INTPT?          ;/jump if not syntax checking
        JR      NZ,M2BB5      ;\

```

;~~~~~

```

M2B80:  RST  $20              ;get next char
        CALL ALPHA?
        JP   NC,M1BED          ;error - BAD BASIC
        RST  $20              ;get next char
        CP   $24
        PUSH AF
        JR   NZ,M2B8E
M2B8E:  RST  $20              ;get next char
        CP   $28
        JR   NZ,M2BA4
        RST  $20              ;get next char
        CP   $29

```

```

        JR      Z,M2BA7
M2B97:  CALL  EXPRN
        RST   $18                      ;get current character
        CP    $2C
        JR     NZ,M2BA2
M2B9F:  RST   $20                      ;get next char
        JR     M2B97
M2BA2:  CP    $29
M2BA4:  JP    NZ,M1BED                  ;error - BAD BASIC
M2BA7:  RST   $20                      ;get next char
        LD    HL,FLAGS
M2BAB:  RES   6,(HL)
        POP   AF
        JR     Z,M2BB2
        SET   6,(HL)
M2BB2:  JP    M2AD0
M2BB5:  RST   $20                      ;get next char
        AND   $DF
M2BB8:  LD    B,A
        RST   $20                      ;get next char
        SUB   $24
        LD    C,A
        JR     NZ,M2BC0
        RST   $20                      ;get next char
M2BC0:  RST   $20                      ;get next char
        PUSH  HL
        LD    HL,(PROG)
        DEC   HL
M2BC6:  LD    DE,$00CE
        PUSH  BC
        CALL  M1D28
        POP   BC
        JR     NC,M2BD2

        RST   $08                      ;error: FN WITHOUT DEF FN
        DFB   $18                      ;

M2BD2:  PUSH  HL
        CALL  M2C69
        AND   $DF
        CP    B
        JR     NZ,M2BE3
        CALL  M2C69
        SUB   $24
        CP    C
        JR     Z,M2BEF
M2BE3:  POP   HL
        DEC   HL
        LD    DE,$0200
        PUSH  BC
        CALL  SUBLIN1
        POP   BC
        JR     M2BC6
M2BEF:  AND   A
        CALL  Z ,M2C69

```

```

        POP DE
        POP DE
        LD (CHADD),DE
        CALL M2C69
        PUSH HL
        CP $29
        JR Z,M2C43
M2C01:  INC HL
        LD A,(HL)
        CP $0E
        LD D,$40
        JR Z,M2C10
        DEC HL
        CALL M2C69
        INC HL
        LD D,$00
M2C10:  INC HL
        PUSH HL
        PUSH DE
        CALL EXPRN
        POP AF
        XOR (IY+OFLAGS) ;FLAGS
        AND $40
        JR NZ,M2C49
        POP HL
        EX DE,HL
        LD HL,(STKEND)
        LD BC,$0005
        SBC HL,BC
        LD (STKEND),HL
        LDIR
        EX DE,HL
        DEC HL
        CALL M2C69
        CP $29
        JR Z,M2C43
        PUSH HL
        RST $18 ;get current character
        CP $2C
        JR NZ,M2C49
        RST $20 ;get next char
        POP HL
        CALL M2C69
        JR M2C01
M2C43:  PUSH HL
        RST $18 ;get current character
        CP $29
        JR Z,M2C4B

M2C49:  RST $08 ;error: PARAMETER ERROR
        DFB $19 ;

M2C4B:  POP DE
        EX DE,HL
        LD (CHADD),HL

```



```

        LD    HL,(DEFADD)
        EX    (SP),HL
        LD    (DEFADD),HL
        PUSH DE
        RST   $20                ;get next char
        RST   $20                ;get next char
        CALL EXPRN
        POP   HL
        LD    (CHADD),HL
        POP   HL
        LD    (DEFADD),HL
        RST   $20                ;get next char
        JP    M2AD0
;*****
; Module: EXPRN
; Routine: NXT_HL
;*****
NXT_HL:
M2C69:   INC    HL
        LD     A,(HL)
        CP    $21
        JR     C,M2C69
        RET
;*****
; Module: IDENT
; Routine: FIND_N
;   find a number in the FP variables area
;   ENTER:  HL = pointer to the first character of the variable name
;
;   EXIT:   CHADD points to the first character after the variable
;   name as it occurs in the BASIC line.
;   if no matching name was found:
;       1. the carry flag is set
;       2. the zero flag is set when the search was for an array variable
;       3. HL points to the first letter of the variable name
;   if the search found a match:
;       1. the carry flag is reset
;       2. the zero flag is set for both simple string variables and
;          all array variables.
;       3. HL points to the letter of a single letter variable name or
;          the last character of a multi variable name as it appears
;          in the variable list
;   bit 6 of the C register is reset when dealing with an array of
;   numbers and set when dealing with an array of strings
;   bit 7 of C is reset during line execution and set during syntax
;   checking
;
;*****
FIND_N:
M2C70:   SET    6,(IY+OFLAGS)      ;FLAGS
        RST    $18                ;get current character
        CALL  ALPHA?              ;/jump if not a letter
        JP    NC,M1BED            ;\error - BAD BASIC
        PUSH  HL                  ;save the pointer to the first letter
        AND   $1F                 ;make it lower case

```

```

LD C,A ;put letter in c
RST $20 ;get next char
PUSH HL ;save the pointer to the second char
CP "(" ;/jump if an array
JR Z,M2CAD ;\
SET 6,C ;set bit 6
CP "$" ;/jump if a string
JR Z,M2C9C ;\
SET 5,C ;set bit 5
CALL ALNUM? ;/jump if not alphanumeric
JR NC,M2CA1 ;\
M2C92: CALL ALNUM? ;/
JR NC,M2CAD ;\
RES 6,C ;
RST $20 ;get next char
JR M2C92

;start handling strings here
M2C9C: RST $20 ;get next char
RES NUM,(IY+OFLAGS) ;indicate a string result

M2CA1: LD A,(DEFADD+1) ;get high byte of DEF FN argument
AND A ;/jump if no argument present
JR Z,M2CAD ;\

CALL INTPT? ;/jump if interpreting
JP NZ,M2D0F ;\

;start looking for the variable
M2CAD: LD B,C ;put character in B
CALL INTPT? ;/jump if interpreting
JR NZ,M2CBB ;\

LD A,C ;put the var name into A
AND $E0 ;keep the type bits
SET 7,A ;ensure bit 7 is set
LD C,A ;put it back into C
JR M2CF2 ;jump fwd for syntax checking

M2CBB: LD HL,(VARS) ;point to the variables region
M2CBE: LD A,(HL) ;get the first byte of the variable
; name
AND $7F ;/jump if we found the $80
JR Z,M2CF0 ;\sentinal at the end of the vars
region
CP C ;/jump if this is not the name
JR NZ,M2CE8 ;\ for which we are looking for
RLA ;/rotate and double to test
ADD A,A ;\for different variable types
JP P,M2CFD ;strings and arrays
JR C,M2CFD ;simple numeric and FOR-NEXT
POP DE ;get the pointer to the 2nd char
PUSH DE ;save it again
PUSH HL ;save the variable pointer
M2CD0: INC HL ;look at the next character

```

```

M2CD1:  LD  A,(DE)                ;point to the next character
        INC DE                  ;and get it
        CP  " "                 ;/ignore spaces
        JR   Z,M2CD1            ;\
        OR  $20                 ;ensure lower case (cheap and nasty, but
works)
        CP  (HL)                ;/we matched, so keep looking
        JR   Z,M2CD0            ;\ at the name
        OR  $80                 ;/see if maybe it is the last letter
        CP  (HL)                ;\of a multi-letter name...
        JR   NZ,M2CE7           ;nope, so move on
        LD  A,(DE)              ;get the next character
        CALL ALNUM?             ;/jump if not alpha-num
        JR   NC,M2CFC           ;\
M2CE7:  POP  HL                  ;fetch the variable pointer
M2CE8:  PUSH BC                 ;briefly save BC
        CALL RECLEN             ;move past the present variable
        EX  DE,HL               ;get the pointer back into HL
        POP  BC                 ;restore BC
        JR   M2CBE             ;back around for another

M2CF0:  SET  7,B                ;tell the world no variable was found
M2CF2:  POP  DE                 ;trash the pointer to the 2nd character
        RST  $18                ;get current character
        CP  "("                 ;/
        JR   Z,M2D01            ;\
        SET  5,B                ;indicate not an array
        JR   M2D09              ;

M2CFC:  POP  DE                 ;trash the saved variable pointer
M2CFD:  POP  DE                 ;drop the second char pointer
        POP  DE                 ;drop the first letter pointer
        PUSH HL                 ;save the last letter pointer
        RST  $18                ;get current character

;move past the characters in a multi-character name
M2D01:  CALL ALNUM?             ;/jump when we have reached the
        JR   NC,M2D09           ;\last character (bit 7 set)
        RST  $20                ;get next char
        JR   M2D01              ;back to test it

M2D09:  POP  HL                 ;HL now points to the first or
        ; last character
        RL  B                   ;
        BIT  6,B                ;
        RET                    ;done

M2D0F:  LD  HL,(DEFADD)
        LD  A,(HL)
        CP  $29
        JP  Z ,M2CAD
M2D18:  LD  A,(HL)
        OR  $60
        LD  B,A
        INC HL

```

```

        LD    A,(HL)
        CP    $0E
        JR     Z,M2D29
        DEC   HL
        CALL  M2C69
        INC   HL
        RES   5,B
M2D29:  LD    A,B
        CP    C
        JR     Z,M2D3F
        INC   HL
        INC   HL
        INC   HL
        INC   HL
        INC   HL
        CALL  M2C69
        CP    $29
        JP     Z ,M2CAD
        CALL  M2C69
        JR     M2D18
M2D3F:  BIT    5,C
        JR     NZ,M2D4F
        INC   HL
        LD    DE,(STKEND)
        CALL  M377F
        EX    DE,HL
        LD    (STKEND),HL
M2D4F:  POP    DE
        POP    DE
        XOR    A
        INC    A
        RET
;*****
; Module: IDENT
; Routine: GET_EL
;*****
GET_EL:
M2D54:  XOR    A
        LD    B,A
        BIT    7,C
        JR     NZ,M2DA5
        BIT    7,(HL)
        JR     NZ,M2D6C
        INC    A
M2D5F:  INC    HL
        LD    C,(HL)
        INC    HL
        LD    B,(HL)
        INC    HL
        EX     DE,HL
        CALL  M2E70
        RST    $18
        JP     M2E07
M2D6C:  INC    HL
        INC    HL

```

;get current character

```

        INC    HL
        LD     B,(HL)
        BIT    6,C
        JR     Z,M2D7E
        DEC    B
        JR     Z,M2D5F
        EX     DE,HL
        RST    $18                ;get current character
        CP     $28
        JR     NZ,M2DDE
M2D7E:  EX     DE,HL
        JR     M2DA5
M2D81:  PUSH   HL
        RST    $18                ;get current character
        POP    HL
        CP     $2C
        JR     Z,M2DA8
        BIT    7,C
        JR     Z,M2DDE
        BIT    6,C
        JR     NZ,M2D96
        CP     $29
        JR     NZ,M2DD0
        RST    $20                ;get next char
        RET
M2D96:  CP     $29
        JR     Z,M2E06
        CP     $CC
        JR     NZ,M2DD0
M2D9E:  RST    $18                ;get current character
        DEC    HL
        LD     (CHADD),HL
        JR     M2E03
M2DA5:  LD     HL,$0000
M2DA8:  PUSH   HL
        RST    $20                ;get next char
        POP    HL
        LD     A,C
        CP     $C0
        JR     NZ,M2DB9
        RST    $18                ;get current character
        CP     $29
        JR     Z,M2E06
        CP     $CC
        JR     Z,M2D9E
M2DB9:  PUSH   BC
        PUSH   HL
        CALL   M2EAC
        EX     (SP),HL
        EX     DE,HL
        CALL   M2E8A
        JR     C,M2DDE
        DEC    BC
        CALL   M2EB2

```

```

        ADD HL,BC
        POP DE
        POP BC
        DJNZ M2D81
        BIT 7,C
M2DD0:   JR      NZ,M2E38
        PUSH HL
        BIT 6,C
        JR      NZ,M2DEA
        LD      B,D
        LD      C,E
        RST     $18                      ;get current character
        CP      $29
        JR      Z,M2DE0

M2DDE:   RST     $08                      ;error: SUBSCRIPT WRONG
        DFB     $02                      ;

M2DE0:   RST     $20                      ;get next char
        POP     HL
        LD      DE,$0005
        CALL    M2EB2
        ADD     HL,BC
        RET

M2DEA:   CALL    M2EAC
        EX      (SP),HL
        CALL    M2EB2
        POP     BC
        ADD     HL,BC
        INC     HL
        LD      B,D
        LD      C,E
        EX      DE,HL
        CALL    M2E6F
        RST     $18                      ;get current character
        CP      $29
        JR      Z,M2E06
        CP      $2C
        JR      NZ,M2DDE
M2E03:   CALL    M2E10
M2E06:   RST     $20                      ;get next char
M2E07:   CP      $28
        JR      Z,M2E03
        RES     6,(IY+0FLAGS)           ;FLAGS
        RET
;*****
; Module: IDENT
; Routine: SLICER
;*****
SLICER:
M2E10:   CALL    INTPT?                  ;/call if interpreting
        CALL    NZ,PGPSTR                ;\
        RST     $20                      ;get next char
        CP      $29
        JR      Z,M2E6B

```

```

PUSH DE
XOR A
PUSH AF
PUSH BC
LD DE,$0001
RST $18 ;get current character
POP HL
CP $CC
JR Z,M2E3F
POP AF
CALL M2E8B
PUSH AF
LD D,B
LD E,C
PUSH HL
RST $18 ;get current character
POP HL
CP $CC
JR Z,M2E3F
CP $29
M2E38: JP NZ,M1BED ;error - BAD BASIC
LD H,D
LD L,E
JR M2E52
M2E3F: PUSH HL
RST $20 ;get next char
POP HL
CP $29
JR Z,M2E52
POP AF
CALL M2E8B
PUSH AF
RST $18 ;get current character
LD H,B
LD L,C
CP $29
JR NZ,M2E38
M2E52: POP AF
EX (SP),HL
ADD HL,DE
DEC HL
EX (SP),HL
AND A
SBC HL,DE
LD BC,$0000
JR C,M2E66
INC HL
AND A
JP M,M2DDE
LD B,H
LD C,L
M2E66: POP DE
RES 6,(IY+OFLAGS) ;FLAGS
M2E6B: CALL INTPT? ;/return if syntax checking
RET Z ;\

```

```

M2E6F:  XOR  A
;*****
; Module: IDENT
; Routine: PSHSTR
;*****
PSHSTR:
M2E70:  RES  6,(IY+OFLAGS)          ;FLAGS
;*****
; Module: IDENT
; Routine: PAEDCB
;  stack AEDCB onto the fp stack
;*****
PAEDCB:
M2E74:  PUSH BC          ;save BC briefly
        CALL ROOM?      ;determine if enough memory for one fp number
        POP  BC         ;restore BC
        LD   HL,(STKEND) ;get pointer to the top of the FP stack
        LD   (HL),A      ;/
        INC  HL          ;|
        LD   (HL),E      ;|
        INC  HL          ;|put the registers on the
        LD   (HL),D      ;|fp stack
        INC  HL          ;|
        LD   (HL),C      ;|
        INC  HL          ;|
        LD   (HL),B      ;|
        INC  HL          ;\
        LD   (STKEND),HL ;update fp stack pointer
M2E89:  RET              ;done
M2E8A:  XOR  A           ;
M2E8B:  PUSH DE
        PUSH HL
        PUSH AF
        CALL TEM6
        POP  AF
        CALL INTPT?     ;/jump if syntax checking
        JR   Z,M2EA9     ;\
        PUSH AF
        CALL FIX_U
        POP  DE
        LD   A,B
        OR   C
        SCF
        JR   Z,M2EA6
        POP  HL
        PUSH HL
        AND  A
        SBC  HL,BC
M2EA6:  LD   A,D
        SBC  A,$00
M2EA9:  POP  HL
        POP  DE
        RET
M2EAC:  EX   DE,HL
        INC  HL

```



```

        LD  E,(HL)
        INC HL
        LD  D,(HL)
        RET
M2EB2:  CALL INTPT?                ;/return if syntax checking
        RET Z                      ;\
        CALL M3468
        JP  C ,M1FCF
        RET
;*****
; Module: IDENT
; Routine: LET
;*****
LET:
M2EBD:  LD  HL,(DEST)
        BIT 1,(IY+OFLAGX)          ;FLAGX
        JR  Z,M2F24
        LD  BC,$0005
M2EC9:  INC BC
M2ECA:  INC HL
        LD  A,(HL)
        CP  $20
        JR  Z,M2ECA
        JR  NC,M2EDD
        CP  $10
        JR  C,M2EE7
        CP  $16
        JR  NC,M2EE7
        INC HL
        JR  M2ECA
M2EDD:  CALL ALNUM?
        JR  C,M2EC9
        CP  $24
        JP  Z ,M2F7E
M2EE7:  LD  A,C
        LD  HL,(ELINE)
        DEC HL
        CALL INSERT
        INC HL
        INC HL
        EX  DE,HL
        PUSH DE
        LD  HL,(DEST)
        DEC DE
        SUB $06
        LD  B,A
        JR  Z,M2F0D
M2EFC:  INC HL
        LD  A,(HL)
        CP  $21
        JR  C,M2EFC
        OR  $20
        INC DE
        LD  (DE),A
        DJNZ M2EFC

```

```

        OR    $80
        LD    (DE),A
        LD    A,$C0
M2F0D:  LD    HL,(DEST)
        XOR   (HL)
        OR    $20
        POP   HL
        CALL  M2FA8
M2F17:  PUSH  HL
        RST   $28    ;calc entry
        DFB   $02          ;DROP
        DFB   $38          ;QUIT
        POP   HL
        LD    BC,$0005
        AND   A
        SBC   HL,BC
        JR    M2F64
M2F24:  BIT    6,(IY+OFLAGS)          ;FLAGS
        JR    Z,M2F30
        LD    DE,$0006
        ADD   HL,DE
        JR    M2F17
M2F30:  LD    HL,(DEST)
        LD    BC,(STRLEN)
        BIT   0,(IY+OFLAGX)          ;FLAGX
        JR    NZ,M2F6D
        LD    A,B
        OR    C
        RET   Z
        PUSH  HL
        RST   $30
        PUSH  DE
        PUSH  BC
        LD    D,H
        LD    E,L
        INC   HL
        LD    (HL),$20
        LDDR
        PUSH  HL
        CALL  PGPSTR
        POP   HL
        EX    (SP),HL
        AND   A
        SBC   HL,BC
        ADD   HL,BC
        JR    NC,M2F59
        LD    B,H
        LD    C,L
M2F59:  EX    (SP),HL
        EX    DE,HL
        LD    A,B
        OR    C
        JR    Z,M2F61
        LDIR
M2F61:  POP   BC

```

```

        POP  DE
        POP  HL
M2F64:  EX   DE,HL
        LD   A,B
        OR   C
        RET  Z
        PUSH DE
        LDIR
        POP  HL
        RET
M2F6D:  DEC  HL
        DEC  HL
        DEC  HL
        LD   A,(HL)
        PUSH HL
        PUSH BC
        CALL M2F84
        POP  BC
        POP  HL
        INC  BC
        INC  BC
        INC  BC
        JP   DELREC
M2F7E:  LD   A,$DF
        LD   HL,(DEST)
        AND  (HL)
M2F84:  PUSH AF
        CALL PGPSTR
        EX   DE,HL
        ADD  HL,BC
        PUSH BC
        DEC  HL
        LD   (DEST),HL
        INC  BC
        INC  BC
        INC  BC
        LD   HL,(ELINE)
        DEC  HL
        CALL INSERT
        LD   HL,(DEST)
        POP  BC
        PUSH BC
        INC  BC
        LDDR
        EX   DE,HL
        INC  HL
        POP  BC
        LD   (HL),B
        DEC  HL
        LD   (HL),C
        POP  AF
M2FA8:  DEC  HL
        LD   (HL),A
        LD   HL,(ELINE)
        DEC  HL

```

```

        RET
;*****
; Module: IDENT
; Routine: PGPSTR
; pop BC, DE, and A from the FP stack
;*****
PGPSTR:
M2FAF:  LD   HL,(STKEND)      ;get end of FP stack
        DEC  HL              ;point to the byte
        LD   B,(HL)          ;/
        DEC  HL              ;|get a string length
        LD   C,(HL)          ;\
        DEC  HL              ;/
        LD   D,(HL)          ;|get a string's location
        DEC  HL              ;|
        LD   E,(HL)          ;\
        DEC  HL              ;/unused byte for strings
        LD   A,(HL)          ;\
        LD   (STKEND),HL     ;save the new STKEND
        RET                  ;
;*****
; Module: IDENT
; Routine: DIM
;*****
DIM:
        CALL FIND_N          ;/jump if array not found
M2FC3:  JP   NZ,M1BED         ;\error - BAD BASIC
        CALL INTPT?         ;/jump if interpreting
        JR   NZ,M2FD3        ;\
        RES  6,C
        CALL M2D54
        CALL END?
M2FD3:  JR   C,M2FDD
        PUSH BC
        CALL RECLEN
        CALL DELREC
        POP  BC
M2FDD:  SET  7,C
        LD   B,$00
        PUSH BC
        LD   HL,$0001
        BIT  6,C
        JR   NZ,M2FEB
M2FEB:  EX  DE,HL
M2FEC:  RST  $20              ;get next char
        LD   H,$FF
        CALL M2E8A
        JP   C ,M2DDE
        POP  HL
        PUSH BC
        INC  H
        PUSH HL
        LD   H,B
        LD   L,C

```

```

CALL M2EB2
EX DE,HL
RST $18 ;get current character
CP $2C
JR Z,M2FEC
CP $29
JR NZ,M2FC3
RST $20 ;get next char
POP BC
LD A,C
LD L,B
LD H,$00
INC HL
INC HL
ADD HL,HL
ADD HL,DE
JP C,M1FCF
PUSH DE
PUSH BC
PUSH HL
LD B,H
LD C,L
LD HL,(ELINE)
DEC HL
CALL INSERT
INC HL
LD (HL),A
POP BC
DEC BC
DEC BC
DEC BC
INC HL
LD (HL),C
INC HL
LD (HL),B
POP BC
LD A,B
INC HL
LD (HL),A
LD H,D
LD L,E
DEC DE
LD (HL),$00
BIT 6,C
JR Z,M303A
LD (HL),$20
M303A: POP BC
LDDR
M303D: POP BC
LD (HL),B
DEC HL
LD (HL),C
DEC HL
DEC A
JR NZ,M303D

```

```

        RET
;*****
; Module: IDENT
; Routine: ALNUM?
; return with carry set if the character in A is
; an digit or a letter.
;*****
ALNUM?:
M3046:  CALL DIGIT?          ;/return if char was a
        CCF                  ;|digit
        RET C                ;\
;*****
; Module: IDENT
; Routine: ALPHA?
; return with carry set if an alpha character
;*****
ALPHA?:
M304B:  CP    "A"            ;/return with carry
        CCF                  ;|reset - not a digit
        RET NC               ;\
        CP    "["            ;/character was in
        RET C                ;\A..Z so return with carry set
        CP    "a"            ;/below "a" so return
        CCF                  ;|
        RET NC               ;\
        CP    "{"            ;/character was in
        RET                  ;\a..z so return
;*****
; Module: INOUT
; Routine: STKUSN
; stack a binary number onto the fp stack
;*****
STKUSN:
M3059:  CP    $C4            ;/jump if not BINary number
        JR    NZ,M3076       ;\
        LD    DE,$0000       ;initialize the accumulator
M3060:  RST    $20            ;get next char
        SUB    "1"           ;/this forces A to be zero for a "1"
        ADC    A,$00          ;\ and one for a "0"
        JR    NZ,M3071       ;jump if character was not a "1" or "0"
        EX    DE,HL          ;/
        CCF                  ;|carry will be 1 for 1 and 0 for 0
        ADC    HL,HL          ;\transfer the "1" into the accumulator
        JP    C ,ERR6         ;number too big
        EX    DE,HL          ;put accum back into DE and char ptr into HL
        JR    M3060          ;back around for the next digit
M3071:  LD    B,D             ;/
        LD    C,E             ;|transfer DE to BC then stack BC
        JP    STK_BC         ;\ to the fp stack

;*****
; Module: INOUT
; Routine: STKNUM
; stack an ASCII number onto the fp stack

```

```

;*****
STKNUM:
M3076:  CP  "."          ;/jump if character is a decimal point
        JR  Z,M3089      ;\

        CALL ININT       ;stack the integer portion of the number

        CP  "."          ;/jump if not decimal point and check
        JR  NZ,M30A9     ;\ for an exponent
;
;handle the decimal part of the number
;
        RST  $20         ;get next char
        CALL DIGIT?      ;/not a digit, so see if
        JR  C,M30A9      ;\ it is an "E" or "e" for an exponent
        JR  M3093        ;jump to assemble the fractional portion of
the number
;
;handle a lone fractional part
;
M3089:  RST  $20         ;get next char
        CALL DIGIT?      ;/jump if A is not a decimal digit
M308D:  JP  C ,M1BED     ;\error - BAD BASIC
;
;stack an accumulator on the fp stack
;
        RST  $28         ;calc entry
        DFB  $A0         ;CONST 0 (0)
        DFB  $38         ;QUIT

M3093:  RST  $28         ;calc entry
        DFB  $A1         ;CONST 1 (256)
        DFB  $C0         ;T -> MEM 0 (MEM 0 contains fp 1)
        DFB  $02         ;DROP
        DFB  $38         ;QUIT

M3098:  RST  $18         ;get current character
        CALL ASC2BIN     ;/jump if the character was not
        JR  C,M30A9      ;\ a decimal digit
        RST  $28         ;calc entry
        DFB  $E0         ;MEM 0 -> T
        DFB  $A4         ;CONST 4 (2560)
        DFB  $05         ;DIV
        DFB  $C0         ;T -> MEM 0
        DFB  $04         ;TIMES
        DFB  $0F         ;ADD
        DFB  $38         ;QUIT
        RST  $20         ;get next char
        JR  M3098

M30A9:  CP  "E"          ;/jump if an exponent is present
        JR  Z,M30B0      ;\
        CP  "e"          ;/return if no exponent
        RET  NZ          ;\
M30B0:  LD  B,$FF        ;

```

```

        RST $20          ;get next char
        CP $2B           ;
        JR Z,M30BC       ;
        CP $2D           ;
        JR NZ,M30BD      ;
        INC B            ;
M30BC:   RST $20          ;get next char
M30BD:   CALL DIGIT?      ;/jump if A is not a decimal digit
        JR C,M308D       ;\

        PUSH BC
        CALL ININT
        CALL FP2A
        POP BC
        JP C ,ERR6
        AND A
        JP M ,ERR6
        INC B
        JR Z,M30D6
        NEG
M30D6:   JP M310D
;*****
; Module: INOUT
; Routine: DIGIT?
; checks to see if a character is between '0' and '9'
; will return with carry set if not a digit
; ENTRY: A = character to be checked
;
; EXIT:      CF = 0 if a digit
;           CF = 1 if not a digit
;*****
DIGIT?:
M30D9:   CP $30          ;/return if less than '0'
        RET C            ;\
        CP $3A          ;/
        CCF              ;|make carry correct for return
        RET              ;\

;*****
; Module: INOUT
; Routine: ASC2BIN
; makes an ASCII character for a decimal digit into its binary
; representation.
; ENTRY: A = character to be checked
;
; EXIT: A = binary representation of a valid ASCII digit, else
;       no change
;       CF = 1 if not a digit
;*****
ASC2BIN:
M30E0:   CALL DIGIT?      ;/return if A is not a digit
        RET C            ;\
        SUB $30          ;make A the binary value of the digit
;*****

```



```

; Module: INOUT
; Routine: STK_A
; put A onto the floating point stack
; munges BC
;*****
STK_A:
M30E6: LD C,A ;/save A to BC
      LD B,$00 ;\
;*****
; Module: INOUT
; Routine: STK_BC
; puts BC onto the fp stack
;*****
STK_BC:
M30E9: LD IY,ERRNR ;???
      XOR A ;/zero the A and E registers
      LD E,A ;\ (exp and sign byte of the integer form
      LD D,C ;/fill in the active bytes
      LD C,B ;\
      LD B,A ;zero the pad byte
      CALL PAEDCB ;place the byte on the fp stack
      RST $28 ;/update the calculator with the
      DFB $38 ;\new value
      AND A
      RET
;*****
; Module: INOUT
; Routine: ININT
; places an ASCII integer number on the fp stack as a fp number
; ENTER A = first digit of the number to be stacked
;
; EXIT CF set if character is not a decimal digit
;*****
ININT:
M30F9: PUSH AF ;save the character
      RST $28 ;calc entry
      DFB $A0 ;CONST 0 (0) stack a fp 0 (zero the accumulator)
      DFB $38 ;QUIT
      POP AF ;restore the character

M30FE: CALL ASC2BIN ;put the binary value in A onto the fp stack
      RET C ;return if the character was not a digit

      RST $28 ;calc entry
      DFB $01 ;SWAP digit to the top of the
stack
      DFB $A4 ;CONST 4 (2560)
      DFB $04 ;TIMES
      DFB $0F ;ADD
      DFB $38 ;QUIT
      CALL NEXTCH ;get the next character
      JR M30FE ;jump around for the next digit
;*****
; Module: INOUT
; Routine: XEY

```

```
;*****
FP_XEY:
M310D:  RLCA
        RRCA
        JR    NC,M3113
        CPL
        INC  A
M3113:  PUSH AF
        LD   HL,MEMBOT
        CALL M3926
        RST $28    ;calc entry
        DFB $A4          ;CONST 4  (2560)
        DFB $38          ;QUIT
        POP  AF
M311E:  SRL   A
        JR    NC,M312F
        PUSH AF
        RST $28    ;calc entry
        DFB $C1          ;T -> MEM 1
        DFB $E0          ;MEM 0 -> T
        DFB $00,$04      ;IFJUMP  312B
        DFB $04          ;TIMES
        DFB $33,$02      ;JUMP    312C
        DFB $05          ;DIV
        DFB $E1          ;MEM 1 -> T
        DFB $38          ;QUIT
        POP  AF
M312F:  JR    Z,M3139
        PUSH AF
        RST $28    ;calc entry
        DFB $31          ;DUP
        DFB $04          ;TIMES
        DFB $38          ;QUIT
        POP  AF
        JR   M311E
M3139:  RST $28    ;calc entry
        DFB $02          ;DROP
        DFB $38          ;QUIT
        RET

;Entry point from FLOAT routine
;*****
; Module: INOUT
; Routine: INT2COMPL
; This routine forms the twos complement of the integer form
; at (HL) and places it in DE. HL points to the sign byte of
; the integer value. C has the sign byte.
;*****
INT2COMPL:
M313D:  INC  HL                ;point to the sign byte
        LD   C,(HL)           ;get integer form sign byte to C
        INC  HL                ;/get LSByte of integer
        LD   A,(HL)           ;\
        XOR  C                ;/generate the two's complement
        SUB  C                ;|of the LSByte and store to
```

```

        LD    E,A                ;\E
        INC   HL                 ;/
        LD    A,(HL)             ;|generate the two's complement
        ADC   A,C                ;|of the MSByte and store to
        XOR   C                  ;|E
        LD    D,A                ;\
        RET

;*****
; Module: INOUT
; Routine: STDE_U
;  store the number in DE as an unsigned integer
;*****
STDE_U:
        LD    C,$00             ;zero the sign byte and press on
;*****
; Module: INOUT
; Routine: STDE_S
;  store the integer number in CDE to (HL). tank the 2s complement
;  of the number prior to storage
;*****
STDE_S:
M314C:  PUSH HL                 ;save number pointer
        LD    (HL),$00          ;save integer exponent
        INC   HL                ;/save integer sign
        LD    (HL),C            ;\
        INC   HL                ;/
        LD    A,E               ;|
        XOR   C                 ;|
        SUB   C                 ;|
        LD    (HL),A            ;|
        INC   HL                ;|form and save 2's complement
        LD    A,D               ;|of DE
        ADC   A,C               ;|
        XOR   C                 ;|
        LD    (HL),A            ;\
        INC   HL                ;/zero last byte of
        LD    (HL),$00          ;\integer value
        POP   HL                ;restore the number pointer
        RET
;*****
; Module: INOUT
; Routine: FP2BC
;  makes the top of the calculator stack an integer by rounding
;  if it's a floating point value.
;  RETURNS:
;    DE = integer value of calculator (bombs out via calculator's
;    integer routine if the number is too large)
;    A  = copy of E
;    ZF = 1 if the result was an integer
;    CF = 1 if the sign byte was not zero
;*****
FP2BC:
M3160:  RST $28                 ;calc entry  /this construct loads HL with the
        DFB $38                 ;QUIT      \address of the "current value"
        LD    A,(HL)            ;/

```

```

already AND A ;|jump if the top of the calc stack is
JR Z,M316B ;\an integer

stack RST $28 ;calc entry round the value on the calc
DFB $A2 ;CONST 1 (0.5) (0.5)
DFB $0F ;ADD
DFB $27 ;INT
DFB $38 ;QUIT

M316B: RST $28 ;calc entry /drop the top of the calc stack
DFB $02 ;DROP |since it will now be put in
the DFB $38 ;QUIT \BC register
PUSH HL ;/save the registers
PUSH DE ;\DE contains the address of the dropped value
EX DE,HL ;HL now points to the dropped value
LD B,(HL) ;save the sign byte
CALL INT2COMPL ;form the 2s complement of the value at (HL)
and put it ; into DE
XOR A ;clear the accumulator
SUB B ;this will set the carry if the result is
not an integer
BIT 7,C ;this will set the zero flag if the sign is
negative LD B,D ;/transfer the newly minted value to BC
LD C,E ;\
LD A,E ;copy E to A
POP DE ;/restore registers
POP HL ;\
RET ;done

M317F: LD D,A ;
RLA ;
SBC A,A ;
LD E,A ;
LD C,A ;
XOR A ;
LD B,A ;
CALL PAEDCB
RST $28 ;calc entry
DFB $34,$EF,$1A,$20,$9A,$85 ;LITERAL 7F1A209A85 (
.301029995665886),3190
DFB $04 ;TIMES
DFB $27 ;INT
DFB $38 ;QUIT
;*****
; Module: INOUT
; Routine: FP2A
; convert the top of the fp stack to an integer in A
; B will contain the sign byte of the original value
; carry will be set if the value was negative
; zero will be set if the value was positive

```

```

;*****
FP2A:
M3193:  CALL FP2BC      ;convert the top of fp stack to BC
        RET  C          ;return if number was greater than 255
        PUSH AF         ;save a
        DEC  B           ;/
        INC  B           ;|jump if the sign byte is 0
        JR   Z,M319F     ;\
        POP  AF          ;restore the value
        SCF              ;set carry as value was negative
        RET              ;return
M319F:  POP  AF          ;restore the value
        RET              ;restore the value
;*****
; Module: INOUT
; Routine: OUTPUT
;*****
OUTPUT:
M31A1:  RST $28      ;calc entry
        DFB $31      ;DUP
        DFB $36      ;MINUSQ
        DFB $00,$0B      ;IFJUMP 31B0
        DFB $31      ;DUP
        DFB $37      ;PLUSQ
        DFB $00,$0D      ;IFJUMP 31B6
        DFB $02      ;DROP
        DFB $38      ;QUIT
        LD  A,$30
        RST $10
        RET
        LD  HL,($3E38)
        DEC L
        RST $10
        RST $28      ;calc entry
        DFB $A0      ;CONST 0 (0)
        DFB $C3      ;T -> MEM 3
        DFB $C4      ;T -> MEM 4
        DFB $C5      ;T -> MEM 5
        DFB $02      ;DROP
        DFB $38      ;QUIT
        EXX
        PUSH HL
        EXX
M31BF:  RST $28      ;calc entry
        DFB $31      ;DUP
        DFB $27      ;INT
        DFB $C2      ;T -> MEM 2
        DFB $03      ;SUB
        DFB $E2      ;MEM 2 -> T
        DFB $01      ;SWAP
        DFB $C2      ;T -> MEM 2
        DFB $02      ;DROP
        DFB $38      ;QUIT
        LD  A,(HL)
        AND  A

```

```

        JR      NZ,M3215
        CALL INT2COMPL
        LD      B,$10
        LD      A,D
        AND     A
        JR      NZ,M31DC
        OR      E
        JR      Z,M31E2
        LD      D,E
        LD      B,$08
M31DC:  PUSH    DE
        EXX
        POP     DE
        EXX
        JR      M323A
M31E2:  RST $28    ;calc entry
        DFB $02          ;DROP
        DFB $E2          ;MEM 2 -> T
        DFB $38          ;QUIT
        LD      A,(HL)
        SUB     $7E
        CALL    M317F
        LD      D,A
        LD      A,(MEM5+1)
        SUB     D
        LD      (MEM5+1),A
        LD      A,D
        CALL    M310D
        RST $28    ;calc entry
        DFB $31          ;DUP
        DFB $27          ;INT
        DFB $C1          ;T -> MEM 1
        DFB $03          ;SUB
        DFB $E1          ;MEM 1 -> T
        DFB $38          ;QUIT
        CALL    FP2A
        PUSH    HL
        LD      (MEM3),A
        DEC     A
        RLA
        SBC     A,A
        INC     A
        LD      HL,MEM5
        LD      (HL),A
        INC     HL
        ADD     A,(HL)
        LD      (HL),A
        POP     HL
        JP      M328E
M3215:  SUB     $80
        CP      $1C
        JR      C,M322E
        CALL    M317F
        SUB     $07
        LD      B,A

```

```

LD HL, MEM5+1
ADD A, (HL)
LD (HL), A
LD A, B
NEG
CALL M310D
JR M31BF
M322E: EX DE, HL
CALL M3379
EXX
SET 7, D
LD A, L
EXX
SUB $80
LD B, A
M323A: SLA E
RL D
EXX
RL E
RL D
EXX
LD HL, MEM4+4
LD C, $05
M3249: LD A, (HL)
ADC A, A
DAA
LD (HL), A
DEC HL
DEC C
JR NZ, M3249
DJNZ M323A
XOR A
LD HL, MEM4
LD DE, MEM3
LD B, $09
RLD
LD C, $FF
M3260: RLD
JR NZ, M3268
DEC C
INC C
JR NZ, M3272
M3268: LD (DE), A
INC DE
INC (IY+{MEM5-Y}) ;
INC (IY+{MEM5-Y+1}) ;
LD C, $00
M3272: BIT 0, B
JR Z, M3277
INC HL
M3277: DJNZ M3260
LD A, (MEM5)
SUB $09
JR C, M328A
DEC (IY+{MEM5-Y}) ;

```

```

        LD    A,$04
        CP    (IY+{MEM4-Y+3})    ;
        JR    M32CB
M328A:  RST $28    ;calc entry
        DFB $02            ;DROP
        DFB $E2            ;MEM 2 -> T
        DFB $38            ;QUIT
M328E:  EX    DE,HL
        CALL M3379
        EXX
        LD    A,$80
        SUB   L
        LD    L,$00
        SET   7,D
        EXX
        CALL M339C
M329E:  LD    A,(IY+{MEM5-Y})    ;
        CP    $08
        JR    C,M32AB
        EXX
        RL    D
        EXX
        JR    M32CB
M32AB:  LD    BC,$0200
M32AE:  LD    A,E
        CALL M334A
        LD    E,A
        LD    A,D
        CALL M334A
        LD    D,A
        PUSH  BC
        EXX
        POP   BC
        DJNZ  M32AE
        LD    HL,MEM3
        LD    A,C
        LD    C,(IY+{MEM5-Y})    ;
        ADD   HL,BC
        LD    (HL),A
        INC   (IY+{MEM5-Y})    ;
        JR    M329E
M32CB:  PUSH  AF
        LD    HL,MEM3
        LD    C,(IY+{MEM5-Y})    ;
        LD    B,$00
        ADD   HL,BC
        LD    B,C
        POP   AF
M32D7:  DEC   HL
        LD    A,(HL)
        ADC   A,$00
        LD    (HL),A
        AND   A
        JR    Z,M32E4
        CP    $0A

```



```

      CCF
      JR      NC,M32EC
M32E4: DJNZ M32D7
      LD      (HL), $01
      INC     B
      INC     (IY+{MEM5-Y+1})      ;
M32EC: LD      (IY+{MEM5-Y}),B      ;
      RST     $28      ;calc entry
      DFB     $02      ;DROP
      DFB     $38      ;QUIT
      EXX
      POP     HL
      EXX
      LD      BC,(MEM5)
      LD      HL, MEM3
      LD      A,B
      CP      $09
      JR      C,M3305
      CP      $FC
      JR      C,M332B
M3305: AND     A
      CALL    Z ,M11EA
M3309: XOR     A
      SUB     B
      JP      M ,M3311
      LD      B,A
      JR      M331D
M3311: LD      A,C
      AND     A
      JR      Z,M3318
      LD      A,(HL)
      INC     HL
      DEC     C
M3318: CALL    M11EA
      DJNZ    M3311
M331D: LD      A,C
      AND     A
      RET     Z
      INC     B
      LD      A,$2E
M3323: RST     $10
      LD      A,$30
      DJNZ    M3323
      LD      B,C
      JR      M3311
M332B: LD      D,B
      DEC     D
      LD      B,$01
      CALL    M3309
      LD      A,$45
      RST     $10
      LD      C,D
      LD      A,C
      AND     A
      JP      P ,M3342

```

```

        NEG
        LD  C,A
        LD  A,$2D
        JR  M3344
M3342:  LD  A,$2B
M3344:  RST  $10
        LD  B,$00
        JP  M1788
M334A:  PUSH DE
        LD  L,A
        LD  H,$00
        LD  E,L
        LD  D,H
        ADD HL,HL
        ADD HL,HL
        ADD HL,DE
        ADD HL,HL
        LD  E,C
        ADD HL,DE
        LD  C,H
        LD  A,L
        POP DE
        RET

M335A:  LD  A,(HL)
        LD  (HL),$00
        AND A
        RET Z
        INC HL
        BIT 7,(HL)
        SET 7,(HL)
        DEC HL
        RET Z
        PUSH BC
        LD  BC,$0005
        ADD HL,BC
        LD  B,C
        LD  C,A
        SCF
M336E:  DEC HL
        LD  A,(HL)
        CPL
        ADC A,$00
        LD  (HL),A
        DJNZ M336E
        LD  A,C
        POP BC
        RET

```

```

;*****
; Most of the explanations in this file come from
; "The Complete Timex TS1000/Sinclair ZX81 ROM Disassembly"
; by Dr. Ian Logan and Dr. Frank O'Hara
; (C) 1982 by the authors

```

```

; Published by Melbourne House Publishers
; ISBN 0 86161 113 6
;
;*****
; Module: SUMS
; Routine: SUMSLD
; This routine is called by the addition, multiplication, and
; division routines to fetch two floating point numbers to the
; main and alternate register sets. When called from the
; multiplication and division routines, the sign of the result
; is stored in the second byte of the first number (M2)
; On entry, HL points to one number M1,M2,M3,M4,M5 and
; DE points to another number N1,N2,N3,N4,N5
; On exit, HL points to the first number
; H'B'C'CB = M1 M2 M3 M4 M5
; L'D'E'DE = N1 N2 N3 N4 N5
;*****
SUMSLD:
M3379:  PUSH HL          ;/save HL and AF
        PUSH AF         ;\
        LD C,(HL)        ;M1 to C
        INC HL           ;point to M2
        LD B,(HL)        ;M2 to B
        LD (HL),A        ;save the sign of the result
        INC HL           ;point to M3
        LD A,C           ;M1 to A
        LD C,(HL)        ;M3 to C
        PUSH BC          ;save M2 & M3
        INC HL           ;point to M4
        LD C,(HL)        ;M4 to C
        INC HL           ;point to M5
        LD B,(HL)        ;M5 to B
        EX DE,HL         ;HL now points to N1
        LD D,A           ;M1 to D
        LD E,(HL)        ;N1 to E
        PUSH DE          ;Save M1 & N1
        INC HL           ;point to N2
        LD D,(HL)        ;N2 to D
        INC HL           ;point to N3
        LD E,(HL)        ;N3 to E
        PUSH DE          ;save N2 & N3
        EXX              ;to alternate register set
        POP DE           ;D' = N2, E' = N3
        POP HL           ;H' = M1, L' = N1
        POP BC           ;B' = M2, C' = M3
        EXX              ;back to normal reg set
        INC HL           ;point to N4
        LD D,(HL)        ;D = N4
        INC HL           ;point to N5
        LD E,(HL)        ;E = N5
        POP AF           ;restore AF
        POP HL           ;/restore the pointer to the first
                        ;\number
        RET              ;done
;*****

```

```

; Module: SUMS
; Routine: SHIFT
;
; This subroutine will shift a number up to 32 places to the right
; to properly align it for addition. Prior to this routine, the number
; with the small exponent has been put in the addend position. Any
; overflow to the right, into the carry, is ripped right back to the
; beginning of the number then the number is set to zero so that addition
; will not alter the other number (augend)
;*****
SHIFT:
M339C:  AND  A           ;/if shift argument is zero no
        RET  Z           ;|shift is needed
                        ;\

        CP   $21         ;/if we shift more than 32 bits
        JR   NC,M33B8

        PUSH BC           ;save BC briefly
        LD   B,A         ;load the DJNZ counter
M33A4:  EXX              ;
        SRA  L           ;/
        RR   D           ;|execute one 40 bit right
        RR   E           ;|shift L'D'E'DE ->
        EXX              ;|
        RR   D           ;|
        RR   E           ;\
        DJNZ M33A4
        POP  BC           ;restore the BC register
        RET  NC           ;done if no carry to retrieve
        CALL M33C3        ;contend with carry
        RET  NZ           ;return unless carry rippled
                        ;right back. if so, zero the result

M33B8:  EXX              ;/
        XOR  A           ;|clear the accum
M33BA:  LD   L,$00         ;|zero out the FP
        LD   D,A         ;|value
        LD   E,L         ;|
        EXX              ;|
        LD   DE,$0000     ;\
        RET              ;done

M33C3:  INC  E           ;/
        RET  NZ           ;|
        INC  D           ;|propagate the carry through
        RET  NZ           ;| D'E'DE
        EXX              ;|
        INC  E           ;|
        JR   NZ,M33CC
        INC  D           ;|
M33CC:  EXX              ;|
        RET              ;\
;*****
; Module: SUMS

```

```

; Routine: SUB
;*****
FP_SUB:
M33CE:  EX  DE,HL          ;swap the number pointers
        CALL M382D        ;negate the subtrahend
        EX  DE,HL          ;restore the number pointers
                                ;continue on with addition
;*****
; Module: SUMS
; Routine: ADD
;
;*****
FP_ADD:
        LD  A,(DE)          ;/if both numbers are not
        OR  (HL)            ;|internal integer format,
        JR   NZ,M33FD
        PUSH DE              ;save the second number pointer
        INC HL               ;/point to the sign byte of the
        PUSH HL              ;\first number and save it
        INC HL               ;point to the LSByte of M
        LD  E,(HL)          ;get it
        INC HL               ;point to MSByte of M
        LD  D,(HL)          ;DE = M
        INC HL               ;points to unused byte of M
        INC HL               ;point to exponent byte of N
        INC HL               ;point to sign of N
        LD  A,(HL)          ;save N sign
        INC HL               ;point to N LSByte
        LD  C,(HL)          ;get it
        INC HL               ;point to N MSByte
        LD  B,(HL)          ;BC = N
        POP HL               ;Point to sign of N1
        EX  DE,HL            ;HL = N, DE points to N sign
        ADD HL,BC             ;HL = M + N
        EX  DE,HL            ;DE = M + N, HL points to N1 sign
        ADC A,(HL)           ;/if the signs were the same and if
        RRCA                  ;| the result generated a carry,
        ADC A,$00             ;| we have overflowed 32 bits so
        JR   NZ,M33FB
                                ;at this point CF=1 if the result is
                                ; negative, CF=0 if positive
        SBC A,A               ;generate $FF for negative $00 for
                                ; positive
        LD  (HL),A            ;store sign byte
        INC HL                ;
        LD  (HL),E            ;store LSByte
        INC HL                ;
        LD  (HL),D            ;store MSByte
        DEC HL                ;/
        DEC HL                ;|HL now points to M exponent byte
        DEC HL                ;\
        POP DE                ;restore pointer to N
        RET                  ;

M33FB:  DEC  HL              ;/restore number pointers

```

```

        POP    DE                ;\

M33FD:  CALL   M3652             ;convert operand to FP form suitable
                                           ; for further math

        EXX                    ;/save next literal address
        PUSH   HL                ;|
        EXX                    ;\
        PUSH   DE                ;save addend pointer
        PUSH   HL                ;save augend pointer
        CALL   M335A             ;prepare the augend
        LD     B,A               ;save its exponent
        EX     DE,HL             ;/prepare the addend
        CALL   M335A             ;\
        LD     C,A               ;save addend exponent
        CP     B                 ;/
        JR     NC,M3414

        LD     A,B               ;|smaller exponent, swap the
        LD     B,C               ;|numbers
        EX     DE,HL             ;\
M3414:  PUSH   AF                ;save the larger exponent
        SUB    B                 ;get the difference in exponents
        CALL   M3379             ;get the numbers into the machine
                                           ;registers from the calc stack
        CALL   M339C             ;shift the addend right
        POP    AF                ;restore the larger exponent
        POP    HL                ;HL points to the result
        LD     (HL),A            ;save the exponent of the result
        PUSH   HL                ;save the result address
        LD     L,B               ;/M4 to H and M5 to L
        LD     H,C               ;\
        ADD    HL,DE             ;add the two right bytes
        EXX                    ;/
        EX     DE,HL             ;|add the two left bytes with
        ADC    HL,BC             ;\carry
        EX     DE,HL             ;/
        LD     A,H               ;|add H'L' and the carry; the
        ADC    A,L               ;|result will ensure that a
        LD     L,A               ;|single shift right is called
        RRA                     ;|if two positive numbers overflowed
        XOR    L                 ;|or two negs has not overflowed left
        EXX                    ;\
        EX     DE,HL             ;the result is now in D'E'DE
        POP    HL                ;get the result pointer
        RRA                     ;the test for shift. (H'L' were
                                           ; $00 for positive numbers and $FF

        JR     NC,M343B

        LD     A,$01             ;do a single right shift
        CALL   M339C             ;perform the shift
        INC    (HL)              ;add one to the exponent
        JR     Z,M345E

M343B:  EXX                    ;/
        LD     A,L               ;|test for negative result
        AND    $80               ;|
        EXX                    ;\

```

```

INC HL ;/store the sign
LD (HL),A ;\
DEC HL ;point back to the exponent
JR Z,M3464
LD A,E ;/
NEG ;|
CCF ;|
LD E,A ;|
LD A,D ;|
CPL ;|perform the 2's compl
ADC A,$00 ;|
LD D,A ;|
EXX ;|
LD A,E ;|
CPL ;|
ADC A,$00 ;|
LD E,A ;|
LD A,D ;|
CPL ;|
ADC A,$00 ;\
JR NC,M3462
RRA ;/get .5 into the mantissa and
EXX ;|increment the exponent. This
INC (HL) ;|will be necessary when two
;|negative numbers add to a power of
;|\2
M345E: JP Z ,ERR6 ;report overflow
EXX ;/store the last byte
M3462: LD D,A ;|
EXX ;\
M3464: XOR A ;clear the carry
JP M3514 ;normalize the result
;*****
; Module: SUMS
; Routine: MULT
;*****
MULT:
M3468: PUSH BC ;save the result sign byte
LD B,$10 ;for 16 bits
LD A,H
LD C,L
LD HL,$0000 ;zero accumulator
M3470: ADD HL,HL ;left shift accumulator
JR C,M347D
RL C ;/shift multiplier 1 bit
RLA ;\
JR NC,M347B
ADD HL,DE ;bit was 1 so add
JR C,M347D
M347B: DJNZ M3470
M347D: POP BC ;restore sign info
RET ;end

M347F: CALL M3904
RET C

```

```

        INC    HL
        XOR    (HL)
        SET    7,(HL)
        DEC    HL
        RET
;*****
; Module: SUMS
; Routine: TIMES
;*****
FP_TIMES:
        LD     A,(DE)           ;/if N's and M's exponents
        OR     (HL)           ;|not both zero to to
        JR     NZ,M34AF
        PUSH  DE               ;save N's pointer
        PUSH  HL               ;save M's pointer
        PUSH  DE               ;working copy of N's pointer
        CALL  INT2COMPL        ;form the 2' complement of (HL) in DE
        EX    DE,HL            ;M to DE
        EX    (SP),HL          ;get N's pointer, saving M
        LD     B,C              ;save M's sign byte
        CALL  INT2COMPL        ;get the 2's complement
        LD     A,B              ;save N's sign byte
        XOR    C                ;get the result of the sign
        LD     C,A              ;save the sign
        POP   HL                ;get M
        CALL  M3468             ;perform a 32 bit multiply
        EX    DE,HL            ;transfer product to DE
        POP   HL                ;get N's pointer
        JR     C,M34AE          ;promote to FP multiply
        LD     A,D              ;/
        OR     E                ;|if product <> 0 jump
        JR     NZ,M34A9
        LD     C,A              ;zero for exponent
M34A9:   CALL  M314C             ;form 2's compl and save product
        POP   DE                ;restore N's pointer
        RET                    ;done

M34AE:   POP   DE                ;get M's pointer
M34AF:   CALL  M3652             ;convert operand to FP form suitable
        ; for further math
        XOR    A                ;set A to zero for sign of first number
        CALL  M347F             ;prepare the first number
        RET    C                ;zero, so nothing need be done
        EXX                    ;/
        PUSH  HL                ;|save next literal pointer
        EXX                    ;\
        PUSH  DE                ;save N's pointer
        EX    DE,HL            ;/
        CALL  M347F             ;|prepare N for multiplication
        EX    DE,HL            ;\
        JR     C,M351C
        PUSH  HL                ;save M's pointer
        CALL  M3379             ;get the numbers into the machine
        ;registers from the calc stack

```



```

LD    A,B                ;M5 to A
AND   A                  ;reset carry flag
SBC   HL,HL              ;/initialize accumulator
EXX                      ;|
PUSH  HL                 ;|save M1 and N1
SBC   HL,HL              ;|
EXX                      ;\
LD    B,$21              ;for 33 bits
JR    M34E4
M34D3: JR    NC,M34DA
      ADD   HL,DE          ;/add multiplicand
      EXX                      ;|in D'E'DE to
      ADC   HL,DE          ;|accumulator in H'L'HL
      EXX                      ;\
M34DA: EXX                      ;/
      RR    H              ;|
      RR    L              ;|shift multiplicand to
      EXX                      ;|test for addition
      RR    H              ;|
      RR    L              ;|
                        ;|
M34E4: EXX                      ;|
      RR    B              ;|
      RR    C              ;|
      EXX                      ;|
      RR    C              ;|
      RRA                      ;\
      DJNZ  M34D3
      EX    DE,HL          ;/
      EXX                      ;|move H'L'HL to D'E'de
      EX    DE,HL          ;|
      EXX                      ;\
      POP   BC              ;restore the exponents
      POP   HL              ;restore the pointer to the result
      LD    A,B            ;/add the exponents
      ADD   A,C            ;\
      JR    NZ,M34FA
M34FA: AND   A              ;\carry, else leave unchanged
      DEC   A              ;/prepare to increase exponent
      CCF                      ;\by $80
M34FC: RLA                  ;/
      CCF                      ;|"add" $80 to exponent
      RRA                      ;\
      JP    P ,M3505        ;no overflow
      JR    NC,ERR6
      AND   A              ;clear carry
M3505: INC   A              ;the exponent is complete
      JR    NZ,M3510
      JR    C,M3510
      EXX                      ;if there is not carry set and the
      BIT   7,D            ;result is already in normal form
      EXX                      ;(bit 7 of D' set) then there is no
      JR    NZ,ERR6
                        ;the result is just in range, i.e. just
                        ;under 2^127

```

```

M3510:  LD   (HL),A           ;store the exponent
        EXX                  ;pass the fifth result byte to A for
        LD   A,B             ;the normalization sequence, i.e.
        EXX                  ;the overflow from L to B'
;*****
; Module: SUMS
; Routine: TESTNORML
;*****
TESTNORML:
M3514:  JR    NC,M352B
        LD   A,(HL)          ;else, deal with underflow (zero)
        AND  A               ;or near underflow
M3518:  LD   A,$80            ;(result 2^-128)
        JR    Z,M351D
M351C:  XOR  A               ;zero (case 2^-128) and if so
M351D:  EXX                  ;produce 2^-128 if number is normal;
        AND  D               ;otherwise rocude zero.
        CALL M33BA           ;the exponent must then be set to
        RLCA                 ;zero (for zero) or 1 (for 2^-128)
        LD   (HL),A          ;restore the exponent byte
        JR    C,M3554
        INC  HL              ;otherwise, put zero into second
        LD   (HL),A          ;byte of result on calculator
        DEC  HL              ;stack
        JR    M3554
;*****
; Module: SUMS
; Routine: NORML
;*****
NORML:
M352B:  LD   B,$20           ;normaise the result by up to 32
M352D:  EXX                  ;decimal shifts left of
        BIT  7,D             ;D'E'DE (with A adjoined), until bit
        EXX                  ;7 of D' is set. A hold zero after
        JR    NZ,M3545
        RLCA                 ;gained or lost. A hold the fifth
        RL  E               ;byte fro B' after multiplication or
        RL  D               ;division; but as only about 32
        EXX                  ;bits can be correct, no precision
        RL  E               ;lost. Note that A is rotated
        RL  D               ;circularly with branch at carry ...
        EXX                  ;...eventually a random process
        DEC  (HL)           ;The exponent is decremented
                           ;on each shift
        JR    Z,M3518
                           ;numbers from 2^-129 are rounded
                           ;up to 2^-128
        DJNZ M352D
        JR    M351C
                           ;whole result is zero
M3545:  RLA                  ;after normalization, add back any
        JR    NC,M3554
        CALL M33C3           ;jump forward if the carry does not
        JR    NZ,M3554
        EXX                  ;if it should ripple back, then

```

```

        LD    D,$80          ;set the mantissa to 0.5 and increment
        EXX                    ;the exponent
        INC   (HL)           ;
        JR    Z,ERR6
M3554:  PUSH  HL              ;save the result pointer
        INC   HL              ;point to the sign byte of the result
        EXX                    ;/the result is moved from
        PUSH  DE              ;|D'E'DE to BCDE
        EXX                    ;|then to ACDE
        POP   BC              ;\
        LD    A,B             ;retrieve the sign bit
        RLA                    ;and put into bit 7 of the
        RL    (HL)            ;mantissa
        RRA                    ;/
        LD    (HL),A          ;|
        INC   HL              ;|store the result
        LD    (HL),C          ;|
        INC   HL              ;|
        LD    (HL),D          ;|
        INC   HL              ;|
        LD    (HL),E          ;\
        POP   HL              ;restore the result pointer
        POP   DE              ;restore the pointer to N
        EXX                    ;/
        POP   HL              ;|restore the next literal
        EXX                    ;\pointer
        RET                    ;done
;*****
; Module: SUMS
; Routine: ERR6
;*****
ERR6:
M356C:  RST $08              ;/error: NUMBER TOO BIG
        DFB $05              ;\
;*****
; Module: SUMS
; Routine: DIVIDE
;*****
FP_DIVIDE:
        CALL M3652            ;convert operand to FP form suitable
        EX    DE,HL           ;exchange N and M pointers
        XOR   A               ;clear the exponent register
        CALL M347F            ;prepare the divisor
        JR    C,ERR6
        EX    DE,HL           ;restore the pointers
        CALL M347F            ;prepare the dividend
        RET   C               ;return if 0, already done
        EXX                    ;/
        PUSH  HL              ;|save next literal pointer
        EXX                    ;\
        PUSH  DE              ;/save number pointers
        PUSH  HL              ;\
        CALL M3379            ;fetch the numbers to the machine registers
        EXX                    ;
        PUSH  HL              ;save M1 and N1 on the stack

```

```

LD H,B ;/
LD L,C ;|copy the dividend from
EXX ;|B'C'CB to H'L'HL
LD H,C ;|
LD L,B ;\
XOR A ;clear the carry
LD B,$DF ;count from -33 to -1
JR M35A1

M3591: RLA ;/
RL C ;|
EXX ;|rotate the quotient register
RL C ;|collecting the result from
RL B ;|the last subtract via the
EXX ;\RLA instruction
M359A: ADD HL,HL ;/
EXX ;|rotate the dividend
ADC HL,HL ;|one bit
EXX ;\
JR C,M35B1

M35A1: SBC HL,DE ;/
EXX ;|trial subtract
SBC HL,DE ;|
EXX ;\
JR NC,M35B8

ADD HL,DE ;/
EXX ;|the subtract didn't go,
ADC HL,DE ;|so restore HL
EXX ;\
AND A ;no bit to the quotient
JR M35B9

M35B1: AND A ;/
SBC HL,DE ;|subtract without restore
EXX ;|to pick up the bit lost at
SBC HL,DE ;|$359F
EXX ;\
M35B8: SCF ;force a bit into the quotient
M35B9: INC B ;step the loop count
JP M,M3591 ;loop 32 times
PUSH AF ;save any 33 bit
JR Z,M359A

;34th bit PUSH AF above saves this
;bit too
LD E,A ;/move the mantissa from
LD D,C ;|B'C'CA to D'E'DE
EXX ;|
LD E,C ;|
LD D,B ;\
POP AF ;/
RR B ;|put any 33rd or 34th
POP AF ;|bits into the B' to
RR B ;|be used in normalization
EXX ;\
POP BC ;restore exponents
POP HL ;restore result pointer
LD A,B ;/compute exponent difference

```

```

        SUB C                ;\
        JP  M34FC            ;exit via the exponent routine
                                ;in the multiplication routine
;*****
; Module: SUMS
; Routine: TRUNC
;*****
FP_TRUNC:
        LD  A,(HL)           ;get the exponent byte
        AND A                ;/result is an integer
        RET Z                ;\so return
        CP  $81              ;/jump if exponent is >= $81
        JR  NC,M35E0
        LD  (HL),$00          ;/number is <1, so prepare
        LD  A,$20             ;\to zero out all 32 bits
        JR  M3631

M35E0:   CP  $91              ;/if the number does not
        JR  NZ,M35FE

;the following tests to see if the number is
; 9180000000 (-65536). if so, it changes the number to
; 00FF000000 then proceed to the normal routine
; I believe this attempts to fix a problem with the original TS1000/ZX81
; ROM routine but creates its own at 35F5
        INC HL                ;/
        INC HL                ;|test the 2nd and 3rd
        INC HL                ;|mantissa bytes for
        LD  A,$80             ;|1's anywhere
        AND (HL)              ;|
        DEC HL                ;|
        OR  (HL)              ;\
        DEC HL                ;point to the 1st mantissa byte
        JR  NZ,M35F2

                                ;the number now must be
                                ;91xx000000
        LD  A,$80             ;/test the 1st mantissa byte
        XOR (HL)              ;|for the value $80
                                ;\
M35F2:   DEC HL                ;point to the exponent

        JR  NZ,M362B

                                ;9180000000 (-65536)
;~~~~~
;here starts something real bad. I get -1 (8180000000) when I take
; INT(-65536). seems like all that need be done is to clean up
; the pointers in HL and DE and exit this routine.
;~~~~~
        LD  (HL),A            ;put zero into the exponent?
        INC HL                ;/set all of the bits in the
        LD  (HL),$FF          ;|1st mantissa byte to 1'S
                                ;\
        DEC HL                ;point back to the exponent
        LD  A,$18             ;we gonna reset 24 bits
        JR  M3631

```

```

M35FE:  JR      NC,M362C
;or more integer bits
;the number has only 16 integer bits so convert to internal
; integer format
        PUSH DE          ;save STKEND
        CPL              ;/effectively subtracts
        ADD A,$91         ;\A-1 from $91
        INC HL            ;point to 1st mantissa byte
        LD D,(HL)         ;store in D
        INC HL            ;point to 2nd mantissa byte
        LD E,(HL)         ;store in E
        DEC HL            ;/point back to exponent
        DEC HL            ;\
        LD C,$00          ;assume positive number
        BIT 7,D           ;check for high bit set in DE
        JR      Z,M3611
        DEC C             ;was negative, so make negative
;integer sign byte
M3611:  SET 7,D            ;set DE[7] to make true binary
        LD B,$08          ;/
        SUB B             ;|test to see how many bits
        ADD A,B           ;\to shift
        JR      C,M361D
        LD E,D            ;/8 bits or less, so
        LD D,$00          ;\adjust DE
        SUB B             ;get residual number of bits
M361D:  JR      Z,M3626
; the operation
        LD B,A            ;set up the shift counter
M3620:  SRL D             ;/shift DE right to provide the
        RR E              ;|needed number of bits
        DJNZ M3620
M3626:  CALL M314C         ;form the 2's complement of CDE
; and save at (HL)
        POP DE           ;restore STKEND
        RET              ;done

M362B:  LD A,(HL)         ;get the exponent to A

;if and when we get here, we have either a valid floating point value
; that is not a 16 bit or less number
M362C:  SUB $A0           ;if the number's exponent is
        RET P            ;at least $A0,there are no fractional
;bits: the number is already a (large)
;integer so return as done
        NEG              ;the number of bits to

;entry point if number was 9180000000 (-65536). number is now
; 00FF000000 with A set to $18
M3631:  PUSH DE          ;save STKEND
        EX DE,HL         ;HL now points one past the 5th byte of X
        DEC HL           ;point to the 5th byte of X
        LD B,A           ;get the number of bits to be 0
        SRL B            ;/divide by 8 to determine

```

```

        SRL    B            ;|the number of whole bytes
        SRL    B            ;\to change
        JR     Z,M3642
M363D:  LD      (HL),$00      ;/
        DEC    HL            ;|reset the whole bytes
        DJNZ   M363D
M3642:  AND     $07           ;determine the number of bits
        JR     Z,M364F
        LD     B,A           ;initialize the bit counter
        LD     A,$FF        ;set up the mask
M3649:  SLA     A            ;/shift 0's into the mask
        DJNZ   M3649
        AND    (HL)         ;/zero the unwanted bits
        LD     (HL),A        ;\and store
M364F:  EX      DE,HL        ;restore HL
        POP    DE           ;restore STKEND
        RET                ;done

M3652:  CALL    M3655        ;tricky way to convert (DE) and
                                ; (HL) to floating point
M3655:  EX      DE,HL        ;exit via FLOAT
;*****
; Module: SUMS
; Routine: FLOAT
;*****
FP_FLOAT:
M3656:  LD      A,(HL)       ;get exponent
        AND     A           ;if not zero is already FP
        RET     NZ          ;\number
        PUSH    DE          ;save pointer to number N
        CALL    INT2COMPL   ;get sign-magnitude of (HL)
                                ;from the 2's complement to DE
        XOR     A           ;zero the accum
        INC     HL          ;point to M5
        LD      (HL),A      ;zero it out
        DEC     HL          ;point to M4
        LD      (HL),A      ;zero it
        LD      B,$91       ;exponent for 16 bit number
        LD      A,D         ;/check to see if number>255,
        AND     A           ;|if so, continue
        JR      NZ,M3670
        OR      E           ;get the LSByte of the number
        LD      B,D         ;stage the MSByte
        JR      Z,M367C
                                ;we are done converting
        LD      D,E         ;transfer 8 bit LSByte to D
        LD      E,B         ;zero out the LSByte in E
        LD      B,$89       ;exponent for 8 bit number

M3670:  EX      DE,HL        ;HL now has number, DE has pointer
M3671:  DEC     B           ;/for every left shift
        ADD     HL,HL        ;|decrement the exponent once
        JR      NC,M3671
        RRC     C           ;/put the MSBit back
        RR      H           ;|

```

```

        RR    L                ;\
        EX    DE,HL            ;swap the pointer and value

M367C:  DEC    HL                ;/fill in M3
        LD    (HL),E            ;\
        DEC    HL                ;/fill in M2
        LD    (HL),D            ;\
        DEC    HL                ;/fill in M1 (exponent)
        LD    (HL),B            ;\
        POP    DE                ;restore the pointer to N
        RET                    ;exit
;*****
; Module: CALC
; Routine: FPCONST
;*****
FPCONST:
        DFB    $00,$B0,$00            ;0000000000 ( 0),3687
        DFB    $40,$B0,$00,$01        ;0000010000 ( 256),368B
        DFB    $30,$00                ;8000000000 ( .5),368D
        DFB    $F1,$49,$0F,$DA,$A2    ;81490FDA A2 ( 1.57079632673413),3692
        DFB    $40,$B0,$00,$0A        ;00000A0000 ( 2560),3696
;*****
; Module: CALC
; Routine: FPJMPTBL
;*****
FPJMPTBL:
        DWL    $3AAA ;IFJUMP          FP_IFJUMP
        DWL    $37FB ;SWAP            FP_SWAP
        DWL    $3760 ;DROP            FP_DROP
        DWL    $33CE ;SUB             FP_SUB
        DWL    $3489 ;TIMES           FP_TIMES
        DWL    $356E ;DIV             FP_DIVIDE
        DWL    $3C6C ;POWER           FP_TO_THE
        DWL    $3936 ;OR              FP_OR
        DWL    $393F ;AND             FP_AND
        DWL    $3956 ;LE              FP_LE
        DWL    $3956 ;GE              FP_GE
        DWL    $3956 ;NE              FP_NE
        DWL    $3956 ;GT              FP_GT
        DWL    $3956 ;LT              FP_LT
        DWL    $3956 ;EQU             FP_EQU
        DWL    $33D3 ;ADD             FP_ADD
        DWL    $3948 ;STGAND          FP_STGAND
        DWL    $3956 ;STLE            FP_STLE
        DWL    $3956 ;STGE            FP_STGE
        DWL    $3956 ;STNE            FP_STNE
        DWL    $3956 ;STGT            FP_STGT
        DWL    $3956 ;STLT            FP_STLT
        DWL    $3956 ;ST EQU          FP_ST EQU
        DWL    $39B7 ;CONCAT          FP_CONCAT
        DWL    $39F9 ;VALS            FP_VALS
        DWL    $38D7 ;USRS            FP_USRS
        DWL    $3A60 ;INKEY           FP_INKEY
        DWL    $382D ;NEGATE          FP_NEGATE
        DWL    $3A84 ;CODE            FP_CODE

```



```

DWL $39F9 ;VAL      FP_VALS
DWL $3A8F ;LEN      FP_LEN
DWL $3BD0 ;SIN      FP_SIN
DWL $3BC5 ;COS      FP_COS
DWL $3BF5 ;TAN      FP_TAN
DWL $3C4E ;ASN      FP_ASN
DWL $3C5E ;ACS      FP_ACS
DWL $3BFD ;ATN      FP_ATN
DWL $3B2E ;LN       FP_LN
DWL $3ADF ;EXP      FP_EXP
DWL $3ACA ;INT      FP_INT
DWL $3C65 ;ROOT     FP_ROOT
DWL $3851 ;SGN      FP_SGN
DWL $3829 ;ABS      FP_ABS
DWL $386B ;PEEK     FP_PEEK
DWL $3864 ;IN       FP_IN
DWL $3872 ;USR      FP_USR
DWL $3A3A ;STR      FP_STR
DWL $39E4 ;CHR      FP_CHR
DWL $391C ;NOT      FP_NOT
DWL $377F ;DUP      FP_DUP
DWL $3ABB ;INTDIV   FP_INTDIV
DWL $3AA1 ;JUMP     FP_JUMP
DWL $3785 ;LITERAL  FP_LIT
DWL $3A95 ;LOOP     FP_LOOP
DWL $3921 ;MINUSQ   FP_MINUSQ
DWL $3914 ;PLUSQ    FP_PLUSQ
DWL $3AB6 ;QUIT     FP_QUIT
DWL $3B9E ;ANGLE    FP_ANGLE
DWL $35D3 ;TRUNC    FP_TRUNC
DWL $3761 ;XEQT     FP_XEQT
DWL $310D ;XEY      FP_XEY
DWL $3656 ;CBSV     FP_FLOAT
DWL $3808 ;GET CONST
DWL $37DA ;TO MEM   FP_TO_MEM
DWL $37EC ;FROM MEM FP_FROM_MEM
DWL $37CE ;??      FP_UNKNOWN
;*****
; Module: CALC
; Routine: UNKNOWN
;*****
UNKNOWN:
M371A:  CALL M39DA
M371D:  LD   A,B
        LD   (BREG),A
M3721:  EXX
        EX   (SP),HL
        EXX
        LD   (STKEND),DE
        EXX
        LD   A,(HL)
        INC  HL
M372B:  PUSH HL
        AND  A
        JP   P ,M373F

```

```

        LD    D,A
        AND   $60
        RRCA
        RRCA
        RRCA
        RRCA
        ADD   A,$7C
        LD    L,A
        LD    A,D
        AND   $1F
        JR    M374D
M373F:  CP    $18
        JR    NC,M374B
        EXX
        LD    BC,$FFFB
        LD    D,H
        LD    E,L
        ADD   HL,BC
        EXX
M374B:  RLCA
        LD    L,A
M374D:  LD    DE,$3696
        LD    H,$00
        ADD   HL,DE
        LD    E,(HL)
        INC   HL
        LD    D,(HL)
        LD    HL,$3724
        EX    (SP),HL
        PUSH  DE
        EXX
        LD    BC,(STKEND+1)
FP_DROP:
        RET

FP_XEQTB:
        POP   AF
        LD    A,(BREG)
        EXX
        JR    M372B
;*****
; Module: CALC
; Routine: ROOM?
;  determine if there is room enough for one floating point value
;  bombs via error 4
;*****
ROOM?:
M3768:  PUSH  DE           ;/save registers
        PUSH  HL           ;\
        LD    BC,$0005      ;one fp number size
        CALL  CHK_SZ        ;check for room
;if we returned, there was room
        POP   HL           ;/retstore the registers
        POP   DE           ;\
        RET

```

```

;*****
; Module: CALC
; Routine: STK_M
;  stack a number at (HL) onto the calc stack
;*****
STK_M:
M3773:  LD  DE,(STKEND)      ;destination is the calc stack
        CALL M377F          ;move the number
        LD  (STKEND),DE     ;update the calc stack pointer
        RET                 ;done
;*****
; Module: CALC
; Routine: RAMNO
;  move BC bytes from (HL) to (DE)
;*****
RAMNO:
FP_DUP:
M377F:  CALL ROOM?          ;find out if there is room
        LDIR                ;do the move
        RET                 ;
;*****
; Module: CALC
; Routine: FPLIT
;  stack a floating point literal
;*****
FP_LIT: LD  H,D              ;
        LD  L,E              ;
M3787:  CALL ROOM?          ;determine if there is room
        EXX                  ;
        PUSH HL              ;
        EXX                  ;
        EX  (SP),HL          ;
        PUSH BC              ;
        LD  A,(HL)           ;
        AND $C0              ;
        RLCA                 ;
        RLCA                 ;
        LD  C,A              ;
        INC C                 ;
        LD  A,(HL)           ;
        AND $3F              ;
        JR   NZ,M379D        ;
        INC HL               ;
        LD  A,(HL)           ;
M379D:  ADD  A,$50            ;
        LD  (DE),A           ;
        LD  A,$05            ;
;*****
; Module: CALC
; Routine: CTRO
;*****
CTRO:
;          DEC  B

```

```

SUB C
INC HL
INC DE
LD B,$00
LDIR
POP BC
EX (SP),HL
EXX
POP HL
EXX
LD B,A
XOR A
M37B0: DEC B
RET Z
LD (DE),A
INC DE
JR M37B0
M37B6: AND A
M37B7: RET Z
PUSH AF
PUSH DE
LD DE,$0000
CALL M3787
POP DE
POP AF
DEC A
JR M37B7
;*****
; Module: CALC
; Routine: ARRAY
; form the address of the FP item with index in A at address (HL)
; BC is munged
;*****
ARRAY:
M37C5: LD C,A ;/
RLCA ;| A = 5 * A
RLCA ;|
ADD A,C ;\
LD C,A ;|
LD B,$00 ;/form the address in HL
ADD HL,BC ;\
RET ;

FP_UNKNOWN:
PUSH DE
LD HL,(MEM)
CALL M37C5
CALL M377F
POP HL
RET

FP_TO_MEM:
LD H,D
LD L,E
EXX

```

```

PUSH HL
LD HL,$3684
EXX
CALL M37B6
CALL M3787
EXX
POP HL
EXX
RET

```

FP_FROM_MEM:

```

PUSH HL
EX DE,HL
LD HL,(MEM)
CALL M37C5
EX DE,HL
CALL M377F
EX DE,HL
POP HL
RET

```

FP_SWAP:

```

M37FB: LD B,$05
M37FD: LD A,(DE)
LD C,(HL)
EX DE,HL
LD (DE),A
LD (HL),C
INC HL
INC DE
DJNZ M37FD
EX DE,HL
RET

```

;FP_FLOAT:

```

LD B,A ;47
CALL M371D ;CD1D37
LD SP,$C00F ;310FC0
LD (BC),A ;02
AND B ;A0
; JP NZ,ME031
DFB $C2,$31,$E0 ;C231E0
INC B ;04
; JP PO ,M03C1
DFB $E2,$C1,$03 ;E2C103
; JR C,M37E7
DFB $38,$CD ;38CD
ADD A,L ;85
SCF ;37
CALL M3721 ;CD2137
RRCA ;0F
LD BC,$02C2 ;01C202
DEC (HL) ;35
XOR $E1 ;EEE1
INC BC ;03

```

```

;      JR      C,M37F2
      DFB $38,$C9          ;38C9

FP_ABS:
      LD      B,$FF        ;06FF
      JR      M3833        ;1806
;*****
; Module: CALC
; Routine: NEGATE
; negate the value at (HL)
;
; ENTRY:
;      B - if bit 7 of B is set, this forces (HL) to be positive
;      (ABS?)
;*****
FP_NEGATE:
M382D: CALL M3904          ;return if FP (HL) is zero
      RET      C           ;
      LD      B,$00        ;prep B

M3833: LD      A,(HL)       ;jump if (HL) is a FP integer format
      AND     A            ;
      JR      Z,M3842      ;

      INC     HL           ;point to the mantissa

      LD      A,B           ;get the sign mask
      AND     $80          ;/get the FP number's sign
      OR      (HL)         ;\
      RLA      ;/this will negate (HL) if B < $80,
      CCF      ;|or force (HL) positive if B > $7F
      RRA      ;\
      LD      (HL),A        ;restore the mantissa high byte
      DEC     HL           ;point back to the exponent
      RET              ;weebiedun

M3842: PUSH    DE           ;save the working registers
      PUSH    HL           ;

      CALL    INT2COMPL     ;return the integer 2's complement of
(HL)                                ; in DE

      POP     HL           ;restore HL
      LD      A,B           ;get the sign mask
      OR      C            ;form the sign byte
      ; A = B if B = 00 else
      ; A = FF if B = FF

      CPL      ;C = 0 if A = FF and B = FF else
      LD      C,A          ;C = NOT(C) if B = 0

      CALL    M314C         ;store the two's complement in CDE to (HL)
      POP     DE           ;restore DE
      RET              ;exit

```

```

        CALL M3904          ;return if FP at (HL) is zero
        RET C              ;

FP_SGN:
;save -1 if (HL) < 0, else save +1
        PUSH DE            ;save DE
        LD DE,$0001        ;we are saving +/- 1
        INC HL             ;point to the sign byte
        RL (HL)            ;CF = MSBit of sign
        DEC HL             ;point back to start of number
        SBC A,A            ;A = FF if MSBit of sign = 1
        LD C,A            ;save sign in C
        CALL M314C         ;store CDE to (HL)
        POP DE            ;restore DE
        RET               ;exit

FP_IN:
        CALL FIX_U         ;get top of FP stack to BC
        IN A,(C)          ;get the data from the port
        JR M386F

FP_PEEK:
        CALL FIX_U         ;get top of FP stack to BC
        LD A,(BC)         ;get the memory value to A

M386F:   JP M30E6          ;stack A on calc stack

FP_USR:
        CALL FIX_U         ;get top of FP stack to BC
        CALL M388E
;        LD HL,$3882
        LD HL,USRRET
        PUSH HL           ;
;        LD HL,$30E9
        LD HL,STK_BC
        PUSH HL
        PUSH BC
        RET
;*****
; Module: CALC
; Routine: USRRET
;*****
USRRET:
        POP AF
        INC A
        RET Z
        PUSH BC
        LD BC,$FF00        ;home bank, all chunks
        CALL BANK_ENABLE  ;
        POP BC
        RET

M388E:   LD HL,(SYSCON)

```

```

        INC HL
        LD A,(HL)
        CP $02
        JR NZ,M38C5
        INC HL
        INC HL
        INC HL
        LD A,B
        BIT 7,A
        JR Z,M38C5
        AND $06
        JR Z,M38BE
        SUB $04
        JP M,M38B7
        JR Z,M38B0
        LD A,(HL)
        JP M,M38C5
        JR M38CB
M38B0:  LD A,(HL)
        BIT 6,A
        JR Z,M38CB
        JR M38C5
M38B7:  LD A,(HL)
        BIT 5,A
        JR Z,M38CB
        JR M38C5
M38BE:  LD A,(HL)
        BIT 4,A
        JR Z,M38CB
        JR M38C5
M38C5:  POP HL
        LD A,$FF
        PUSH AF
        PUSH HL
        RET

M38CB:  POP HL
        PUSH AF
        PUSH HL
        PUSH BC          ;briefly save BC
        LD C,A           ;chunk map to C
        LD B,$00         ;dock bank
        CALL BANK_ENABLE ;
        POP BC           ;restore BC
        RET

FP_USRS:
        CALL PGPSTR
        DEC BC
        LD A,B
        OR C
        JR NZ,M3902
        LD A,(DE)
        CALL ALPHA?
        JR C,M38EE

```



```

        SUB    $90
        JR     C,M3902
        CP     $15
        JR     NC,M3902
        INC    A
M38EE:  DEC    A
        ADD    A,A
        ADD    A,A
        ADD    A,A
        CP     $A8
        JR     NC,M3902
        LD     BC,(UDG)
        ADD    A,C
        LD     C,A
        JR     NC,M38FF
        INC    B
M38FF:  JP     STK_BC

M3902:  RST    $08                ;error: INVALID ARGUMENT
        DFB    $09                ;
;*****
; Module: CALC
; Routine: TEST0
; test the value (FP or integer) at (HL) for zero.
; Entry:
;     HL - points to number to check
;
; Exit:
;     NZ, NC if (HL) is not zero
;     Z,  C if (HL) is zero
;*****
TEST0:
M3904:  PUSH   HL                ;/save HL, BC, and A
        PUSH   BC                ;|
        LD     B,A                ;\

        LD     A,(HL)            ;OR all of the components of the
        INC    HL                ; FP value together. this will
        OR     (HL)              ; produce zero iff the number is zero
        INC    HL                ;
        OR     (HL)              ;
        INC    HL                ;
        OR     (HL)              ;

        LD     A,B                ;/restore HL, BC, and A
        POP    BC                ;|
        POP    HL                ;\
        RET    NZ                ;return if not zero

        SCF                      ;set carry and return if zero
        RET                      ;

FP_PLUSQ:
M3914:  CALL   M3904            ;
        RET    C                ;return if (HL) is zero

```

```

        LD    A,$FF          ;A <- -1
        JR    M3922

FP_NOT:
M391C:  CALL  M3904          ;test (HL) for zero
        JR    M3926

FP_MINUSQ:
        XOR   A
M3922:  INC   HL              ;point to the next byte of the integer to be tested
        XOR   (HL)           ;set the bits
        DEC   HL              ;point back to the integer
        RLCA                  ;CF = 1, A = $FE
;*****
; Module: CALC
; Routine: STB00L
; store a boolean integer value to (HL)
; this routine will store a FP integer $0001 to (HL) if the CF
; is set upon entry, else will store $0000
;
; Entry:
; CF indicates value to store
;*****
STB00L:
M3926:  PUSH  HL              ;save HL
        LD    A,$00          ;
        LD    (HL),A         ;store 0 into the first bytes
        INC   HL              ;(exponent and integer flag)
        LD    (HL),A         ;
        INC   HL              ;
        RLA                  ;put carry flag into the lsb of the integer
        LD    (HL),A         ; this will give $0001 if the carry is set
                               ; else $0000 if the carry is clear

        RRA                  ;restore A and drop zeros into the
        INC   HL              ; remainder of the integer
        LD    (HL),A         ;
        INC   HL              ;
        LD    (HL),A         ;

        POP   HL              ;restore HL and return
        RET                   ;

;(DE) zero
; this routine will put a FP integer $0001 at (HL) if FP integer at (DE)
; is zero.
FP_OR:
        EX    DE,HL          ;test (DE) for zero
        CALL  M3904          ;
        EX    DE,HL          ;
        RET   C              ;return if (DE) is zero

```

```

        SCF                ;put 1 into (HL)
        JR    M3926        ;

; (DE) not zero
; this routine will put a FP integer $0001 at (HL) if FP integer at (DE)
; is not zero.
FP_AND:
        EX    DE,HL        ;test (DE) for zero...
        CALL  M3904        ;
        EX    DE,HL        ;
        RET    NC          ;... and return if not.

        AND    A            ;put $0001 into (HL) if (DE) was zero
        JR    M3926        ;

FP_STGAND:
        EX    DE,HL        ;test (DE) for zero and return if it is
        CALL  M3904        ; not
        EX    DE,HL        ;
        RET    NC          ;
        PUSH  DE
        DEC  DE
        XOR  A
        LD   (DE),A
        DEC  DE
        LD   (DE),A
        POP  DE
        RET

FP_LE:
FP_GE:
FP_NE:
FP_GT:
FP_LT:
FP_EQU:
FP_STLE:
FP_STGE:
FP_STNE:
FP_STGT:
FP_STLT:
FP_STEQU:
        LD    A,B
        SUB   $08
        BIT   2,A
        JR    NZ,M395E
        DEC  A
M395E:  RRCA
        JR    NC,M3969
        PUSH AF
        PUSH HL
        CALL  M37FB
        POP  DE
        EX   DE,HL
        POP  AF
M3969:  BIT   2,A
        JR    NZ,M3974

```

```

RRCA
PUSH AF
CALL M33CE
JR M39A7
M3974: RRCA
PUSH AF
CALL PGPSTR
PUSH DE
PUSH BC
CALL PGPSTR
POP HL
M397F: LD A,H
OR L
EX (SP),HL
LD A,B
JR NZ,M3990
OR C
M3986: POP BC
JR Z,M398D
POP AF
CCF
JR M39A3
M398D: POP AF
JR M39A3
M3990: OR C
JR Z,M39A0
LD A,(DE)
SUB (HL)
JR C,M39A0
JR NZ,M3986
DEC BC
INC DE
INC HL
EX (SP),HL
DEC HL
JR M397F
M39A0: POP BC
POP AF
AND A
M39A3: PUSH AF
RST $28 ;calc entry
DFB $A0 ;CONST 0 (0)
DFB $38 ;QUIT
M39A7: POP AF
PUSH AF
CALL C,M391C
POP AF
PUSH AF
CALL NC,M3914
POP AF
RRCA
CALL NC,M391C
RET

```

FP_CONCAT:

```

CALL PGPSTR
PUSH DE
PUSH BC
CALL PGPSTR
POP HL
PUSH HL
PUSH DE
PUSH BC
ADD HL,BC
LD B,H
LD C,L
RST $30
CALL M2E70
POP BC
POP HL
LD A,B
OR C
JR Z,M39D2
LDIR
M39D2: POP BC
POP HL
LD A,B
OR C
JR Z,M39DA
LDIR
M39DA: LD HL,(STKEND)
LD DE,$FFFB
PUSH HL
ADD HL,DE
POP DE
RET

FP_CHR:
CALL FP2A
JR C,M39F7
JR NZ,M39F7
PUSH AF
LD BC,$0001
RST $30
POP AF
LD (DE),A
CALL M2E70
EX DE,HL
RET

M39F7: RST $08 ;error: INTEGER OUT OF RANGE
DFB $0A ;
FP_VALS:
LD HL,(CHADD)
PUSH HL
LD A,B
ADD A,$E3
SBC A,A
PUSH AF
CALL PGPSTR

```

```

PUSH DE
INC BC
RST $30
POP HL
LD (CHADD),DE
PUSH DE
LDIR
EX DE,HL
DEC HL
LD (HL),$0D
RES 7,(IY+OFLAGS) ;FLAGS
CALL EXPRN
RST $18 ;get current character
CP $0D
JR NZ,M3A27
POP HL
POP AF
XOR (IY+OFLAGS) ;FLAGS
AND $40
M3A27: JP NZ,M1BED ;error - BAD BASIC
LD (CHADD),HL
SET 7,(IY+OFLAGS) ;FLAGS
CALL EXPRN
POP HL
LD (CHADD),HL
JR M39DA

FP_STR:
LD BC,$0001
RST $30
LD (KCUR),HL
PUSH HL
LD HL,(CURCHL)
PUSH HL
LD A,$FF ;/stream -1 (RAM write)
CALL SELECT ;\
CALL M31A1
POP HL
CALL M1248
POP DE
LD HL,(KCUR)
AND A
SBC HL,DE
LD B,H
LD C,L
CALL M2E70
EX DE,HL
RET

FP_INKEY:
CALL INS_U1 ;convert top of fp stack to value in A
CP $10 ;/invalid stream # if > 16
JP NC,M1F29 ;\
LD HL,(CURCHL) ;/get the current channel pointer
PUSH HL ;\and save it

```

```

        CALL SELECT                ;select the requested stream
        CALL M11E1
        LD  BC,$0000
        JR   NC,M3A7A
        INC  C
        RST  $30
        LD   (DE),A
M3A7A:  CALL M2E70
        POP  HL
        CALL M1248
        JP   M39DA

FP_CODE:
        CALL PGPSTR
        LD   A,B
        OR   C
        JR   Z,M3A8C
        LD   A,(DE)
M3A8C:  JP   M30E6                ;stack A on calc stack

FP_LEN:
        CALL PGPSTR
        JP   STK_BC

FP_LOOP:
        EXX
        PUSH HL
        LD   HL,BREG
        DEC  (HL)
        POP  HL
        JR   NZ,M3AA2
        INC  HL
        EXX
        RET

FP_JUMP:
M3AA1:  EXX
M3AA2:  LD   E,(HL)
        LD   A,E
        RLA
        SBC  A,A
        LD   D,A
        ADD  HL,DE
        EXX
        RET

FP_IFJUMP:
        INC  DE
        INC  DE
        LD   A,(DE)
        DEC  DE
        DEC  DE
        AND  A
        JR   NZ,M3AA1
        EXX

```

```

        INC    HL
        EXX
        RET

FP_QUIT:
        POP    AF
        EXX
        EX     (SP),HL
        EXX
        RET
;*****
; Module: FUNCTS
; Routine: INTDIV
;*****
FP_INTDIV:
        RST $28    ;calc entry
        DFB $C0          ;T -> MEM 0
        DFB $02          ;DROP
        DFB $31          ;DUP
        DFB $E0          ;MEM 0 -> T
        DFB $05          ;DIV
        DFB $27          ;INT
        DFB $E0          ;MEM 0 -> T
        DFB $01          ;SWAP
        DFB $C0          ;T -> MEM 0
        DFB $04          ;TIMES
        DFB $03          ;SUB
        DFB $E0          ;MEM 0 -> T
        DFB $38          ;QUIT
        RET
;*****
; Module: FUNCTS
; Routine: INT
;*****
FP_INT:
        RST $28    ;calc entry
        DFB $31          ;DUP
        DFB $36          ;MINUSQ
        DFB $00,$04      ;IFJUMP 3AD2
        DFB $3A          ;TRUNC
        DFB $38          ;DUP      QUIT
        DFB $C9          ;MINUSQ   RET
        DFB $31          ;DUP      DUP
        DFB $3A          ;MINUSQ   TRUNC
        DFB $C0          ;DUP      T -> MEM 0
        DFB $03          ;MINUSQ   SUB
        DFB $E0          ;DUP      MEM 0 -> T
        DFB $01          ;MINUSQ   SWAP
        DFB $30          ;NOT
        DFB $00,$03      ;IFJUMP 3ADD
        DFB $A1          ;DUP      CONST 1 (256)
        DFB $03          ;MINUSQ   SUB
        DFB $38          ;QUIT
        RET
;*****

```



```

; Module: FUNCTS
; Routine: EXP
;*****
FP_EXP:
M3ADF:  RST $28    ;calc entry
        DFB $3D                ;FLOAT
        DFB $34,$F1,$38,$AA,$3B,$29 ;LITERAL 8138AA3B29 (
1.44269504072145),3AE7
        DFB $04                ;TIMES
        DFB $31                ;DUP
        DFB $27                ;INT
        DFB $C3                ;T -> MEM 3
        DFB $03                ;SUB
        DFB $31                ;DUP
        DFB $0F                ;ADD
        DFB $A1                ;CONST 1 (256)
        DFB $03                ;SUB
        DFB $88                ;CBSV 8
        DFB $13,$36            ;6336000000 ( 1.32422428578138E-
9),3AF3
        DFB $58,$65,$66        ;6865660000 ( 5.3410985856317E-
8),3AF6
        DFB $9D,$78,$65,$40    ;6D78654000 ( 1.85069075087085E-
6),3AFA
        DFB $A2,$60,$32,$C9    ;726032C900 ( 5.3453059081221E-
5),3AFE
        DFB $E7,$21,$F7,$AF,$24 ;7721F7AF24 ( 1.23571408221324E-
3),3B03
        DFB $EB,$2F,$B0,$B0,$14 ;7B2FB0B014 ( 2.14465559984092E-
2),3B08
        DFB $EE,$7E,$BB,$94,$58 ;7E7EBB9458 ( .248762433882803),3B0D
        DFB $F1,$3A,$7E,$F8,$CF ;813A7EF8CF ( 1.45699987513945),3B12

        DFB $E3                ;MEM 3 -> T
        DFB $38                ;QUIT

        CALL FP2A
        JR     NZ,M3B20
        JR     C,M3B1E
        ADD    A,(HL)
        JR     NC,M3B27

M3B1E:  RST $08                ;error: NUMBER TOO BIG
        DFB $05                ;

M3B20:  JR     C,M3B29
        SUB    (HL)
        JR     NC,M3B29
        NEG
M3B27:  LD     (HL),A
        RET
M3B29:  RST $28    ;calc entry
        DFB $02                ;DROP
        DFB $A0                ;CONST 0 (0)
        DFB $38                ;QUIT

```

```

RET
;*****
; Module: FUNCTS
; Routine: LN
;*****
FP_LN:
    RST $28    ;calc entry
    DFB $3D          ;FLOAT
    DFB $31          ;DUP
    DFB $37          ;PLUSQ
    DFB $00,$04      ;IFJUMP  3B37
    DFB $38          ;QUIT

    RST $08          ;error: INVALID ARGUMENT
    DFB $09          ;

    DFB $A0          ;CONST 0 (0)
    DFB $02          ;DROP
    DFB $38          ;QUIT
    LD  A,(HL)
    LD  (HL),$80
    CALL M30E6        ;stack A on calc stack
    RST $28    ;calc entry
    DFB $34,$38,$00          ;LITERAL  8800000000 ( 128),3B44
    DFB $03          ;SUB
    DFB $01          ;SWAP
    DFB $31          ;DUP
    DFB $34,$F0,$4C,$CC,$CC,$CD ;LITERAL  804CCCCCD (
.8000000000046566),3B4D
    DFB $03          ;SUB
    DFB $37          ;PLUSQ
    DFB $00,$08          ;IFJUMP  3B58
    DFB $01          ;SWAP
    DFB $A1          ;CONST 1 (256)
    DFB $03          ;SUB
    DFB $01          ;SWAP
    DFB $38          ;QUIT
    INC (HL)
    RST $28    ;calc entry
    DFB $01          ;SWAP
    DFB $34,$F0,$31,$72,$17,$F8 ;LITERAL  80317217F8 (
.693147180601954),3B5F
    DFB $04          ;TIMES
    DFB $01          ;SWAP
    DFB $A2          ;CONST 1 (0.5)
    DFB $03          ;SUB
    DFB $A2          ;CONST 1 (0.5)
    DFB $03          ;SUB
    DFB $31          ;DUP
    DFB $34,$32,$20          ;LITERAL  8220000000 ( 2.5),3B69
    DFB $04          ;TIMES
    DFB $A2          ;CONST 1 (0.5)
    DFB $03          ;SUB
    DFB $8C          ;CBSV 12

```

```

10),3B6F      DFB $11,$AC                      ;61AC000000 (-3.12866177409887E-
9),3B71      DFB $14,$09                      ;6409000000 ( 1.99361238628626E-
            DFB $56,$DA,$A5                  ;66DAA50000 (-1.2726786735584E-8),3B74
            DFB $59,$30,$C5                  ;6930C50000 ( 8.23147274786606E-
8),3B77      DFB $5C,$90,$AA                  ;6C90AA0000 (-5.38915628567338E-
7),3B7A      DFB $9E,$70,$6F,$61            ;6E706F6100 ( 3.58276179213135E-
6),3B7E      DFB $A1,$CB,$DA,$96            ;71CBDA9600 (-2.43012727878522E-
5),3B82      DFB $A4,$31,$9F,$B4            ;74319FB400 ( 1.69395294506103E-
4),3B86      DFB $E7,$A0,$FE,$5C,$FC        ;77A0FE5CFC (-1.22828373969242E-
3),3B8B      DFB $EA,$1B,$43,$CA,$36        ;7A1B43CA36 ( 9.47661158716073E-
3),3B90      DFB $ED,$A7,$9C,$7E,$5E        ;7DA79C7E5E (-.081841456645634),3B95
            DFB $F0,$6E,$23,$80,$93        ;806E238093 ( .930229221237823),3B9A

            DFB $04                        ;TIMES
            DFB $0F                        ;ADD
            DFB $38                        ;QUIT
            RET
;*****
; Module: FUNCTS
; Routine: ANGLE
;*****
FP_ANGLE:
    RST $28    ;calc entry
    DFB $3D          ;FLOAT
    DFB $34,$EE,$22,$F9,$83,$6E    ;LITERAL 7E22F9836E (
.1591549430741),3BA6
    DFB $04          ;TIMES
    DFB $31          ;DUP
    DFB $A2          ;CONST 1 (0.5)
    DFB $0F          ;ADD
    DFB $27          ;INT
    DFB $03          ;SUB
    DFB $31          ;DUP
    DFB $0F          ;ADD
    DFB $31          ;DUP
    DFB $0F          ;ADD
    DFB $31          ;DUP
    DFB $2A          ;ABS
    DFB $A1          ;CONST 1 (256)
    DFB $03          ;SUB
    DFB $31          ;DUP
    DFB $37          ;PLUSQ
    DFB $C0          ;T -> MEM 0
    DFB $00,$04      ;IFJUMP 3BBC
    DFB $02          ;DROP
    DFB $38          ;QUIT

```

```

        DFB $C9                ;RET
        DFB $A1                ;CONST 1 (256)
        DFB $03                ;SUB
        DFB $01                ;SWAP
        DFB $36                ;MINUSQ
        DFB $00,$02            ;IFJUMP 3BC3
        DFB $1B                ;NEGATE
        DFB $38                ;QUIT
    RET
;*****
; Module: FUNCTS
; Routine: COS
;*****
FP_COS:
    RST $28    ;calc entry
    DFB $39                ;ANGLE
    DFB $2A                ;ABS
    DFB $A1                ;CONST 1 (256)
    DFB $03                ;SUB
    DFB $E0                ;MEM 0 -> T
    DFB $00,$06            ;IFJUMP 3BD2
    DFB $1B                ;NEGATE
    DFB $33,$03            ;JUMP 3BD2
;*****
; Module: FUNCTS
; Routine: SIN
;*****
FP_SIN:
    RST $28    ;calc entry

    DFB $39                ;ANGLE
    DFB $31                ;DUP
    DFB $31                ;DUP
    DFB $04                ;TIMES
    DFB $31                ;DUP
    DFB $0F                ;ADD
    DFB $A1                ;CONST 1 (256)
    DFB $03                ;SUB
    DFB $86                ;CBSV 6
    DFB $14,$E6            ;64E6000000 (-3.34694050252438E-
9),3BDC
    DFB $5C,$1F,$0B        ;6C1F0B0000 ( 5.92481228522956E-
7),3BDF
    DFB $A3,$8F,$38,$EE    ;738F38EE00 (-6.8293753429316E-5),3BE3
    DFB $E9,$15,$63,$BB,$23 ;791563BB23 ( 4.55900800261588E-
3),3BE8
    DFB $EE,$92,$0D,$CD,$ED ;7E920DCDED (-.142630784597714),3BED
    DFB $F1,$23,$5D,$1B,$EA ;81235D1BEA ( 1.27627896238118),3BF2

    DFB $04                ;TIMES
    DFB $38                ;QUIT
    RET
;*****
; Module: FUNCTS
; Routine: TAN

```

```

;*****
FP_TAN:
    RST $28    ;calc entry
    DFB $31          ;DUP
    DFB $1F          ;SIN
    DFB $01          ;SWAP
    DFB $20          ;COS
    DFB $05          ;DIV
    DFB $38          ;QUIT
    RET
;*****
; Module: FUNCTS
; Routine: ATN
;*****
FP_ATN:
    CALL M3656
    LD  A,(HL)
    CP  $81
    JR  C,M3C13
    RST $28    ;calc entry
    DFB $A1          ;CONST 1 (256)
    DFB $1B          ;NEGATE
    DFB $01          ;SWAP
    DFB $05          ;DIV
    DFB $31          ;DUP
    DFB $36          ;MINUSQ
    DFB $A3          ;CONST 3 (PI/2)
    DFB $01          ;SWAP
    DFB $00,$06      ;IFJUMP 3C15
    DFB $1B          ;NEGATE
    DFB $33,$03      ;JUMP 3C15
M3C13: DFB $EF          ;MEM 15 -> T
    DFB $A0          ;CONST 0 (0)
    DFB $01          ;SWAP
    DFB $31          ;DUP
    DFB $31          ;DUP
    DFB $04          ;TIMES
    DFB $31          ;DUP
    DFB $0F          ;ADD
    DFB $A1          ;CONST 1 (256)
    DFB $03          ;SUB
    DFB $8C          ;CBSV 12
    DFB $10,$B2      ;60B2000000 (-1.61890056915581E-
10),3C20
    DFB $13,$0E      ;630E000000 ( 1.03318598121405E-
9),3C22
    DFB $55,$E4,$8D  ;65E48D0000 (-6.65170318825403E-
9),3C25
    DFB $58,$39,$BC  ;6839BC0000 ( 4.32446540798992E-
8),3C28
    DFB $5B,$98,$FD  ;6B98FD0000 (-2.84962879959494E-
7),3C2B
    DFB $9E,$00,$36,$75 ;6E00367500 ( 1.91051844922185E-
6),3C2F

```

```

5),3C33      DFB $A0,$DB,$E8,$B4          ;70DBE8B400 (-1.31075976241846E-
5),3C36      DFB $63,$42,$C4            ;7342C40000 ( 9.28714871406555E-
4),3C3B      DFB $E6,$B5,$09,$36,$BE     ;76B50936BE (-6.90597501943557E-
3),3C40      DFB $E9,$36,$73,$1B,$5D     ;7936731B5D ( 5.56792102906911E-
2),3C45      DFB $EC,$D8,$DE,$63,$BE     ;7CD8DE63BE (-5.29464622668456E-
            DFB $F0,$61,$A1,$B3,$0C     ;8061A1B30C ( .881373587064445),3C4A

            DFB $04                      ;TIMES
            DFB $0F                      ;ADD
            DFB $38                      ;QUIT
            RET
;*****
; Module: FUNCTS
; Routine: ASN
;*****
FP_ASN:
    RST $28      ;calc entry
    DFB $31      ;DUP
    DFB $31      ;DUP
    DFB $04      ;TIMES
    DFB $A1      ;CONST 1 (256)
    DFB $03      ;SUB
    DFB $1B      ;NEGATE
    DFB $28      ;ROOT
    DFB $A1      ;CONST 1 (256)
    DFB $0F      ;ADD
    DFB $05      ;DIV
    DFB $24      ;ATN
    DFB $31      ;DUP
    DFB $0F      ;ADD
    DFB $38      ;QUIT
    RET
;*****
; Module: FUNCTS
; Routine: ACS
;*****
FP_ACS:
    RST $28      ;calc entry
    DFB $22      ;ASN
    DFB $A3      ;CONST 3 (PI/2)
    DFB $03      ;SUB
    DFB $1B      ;NEGATE
    DFB $38      ;QUIT
    RET
;*****
; Module: FUNCTS
; Routine: ROOT
;*****
FP_ROOT:
    RST $28      ;calc entry

```

```

        DFB $31          ;DUP
        DFB $30          ;NOT
        DFB $00,$1E      ;IFJUMP 3C87
        DFB $A2          ;CONST 1 (0.5)
        DFB $38          ;QUIT
;*****
; Module: FUNCTS
; Routine: TO_THE
;*****
FP_TO_THE:
        RST $28          ;calc entry
        DFB $01          ;SWAP
        DFB $31          ;DUP
        DFB $30          ;NOT
        DFB $00,$07      ;IFJUMP 3C78
        DFB $25          ;LN
        DFB $04          ;TIMES
        DFB $38          ;QUIT
        JP M3ADF
        DFB $02          ;DROP
        DFB $31          ;DUP
        DFB $30          ;NOT
        DFB $00,$09      ;IFJUMP 3C85
        DFB $A0          ;CONST 0 (0)
        DFB $01          ;SWAP
        DFB $37          ;PLUSQ
        DFB $00,$06      ;IFJUMP 3C87
        DFB $A1          ;CONST 1 (256)
        DFB $01          ;SWAP
        DFB $05          ;DIV
        DFB $02          ;DROP
        DFB $A1          ;CONST 1 (256)
        DFB $38          ;QUIT
        RET
;*****
; Module: TAPMSG
; Routine: SEPRMT
;*****
SEPRMT:
M3C89:  DFB $80,"Start tape, then press any key",{"."+$80}
        DFB $0D,"Program:",{" "+$80}
        DFB $0D,"Number array:",{" "+$80}
        DFB $0D,"Character array:",{" "+$80}
        DFB $0D,"Bytes:",{" "+$80}
        DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
        DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
        DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
        DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
        DFB $FF,$FF,$FF,$FF
;*****
; Module: CH_SET
; Routine: CHRSET
;*****

```

CHRSET:

DFB \$00,\$00,\$00,\$00,\$00,\$00,\$00,\$00 ;
DFB \$00,\$10,\$10,\$10,\$10,\$00,\$10,\$00 ;
DFB \$00,\$24,\$24,\$00,\$00,\$00,\$00,\$00 ;
DFB \$00,\$24,\$7E,\$24,\$24,\$7E,\$24,\$00 ;
DFB \$00,\$08,\$3E,\$28,\$3E,\$0A,\$3E,\$08 ;
DFB \$00,\$62,\$64,\$08,\$10,\$26,\$46,\$00 ;
DFB \$00,\$10,\$28,\$10,\$2A,\$44,\$3A,\$00 ;
DFB \$00,\$08,\$10,\$00,\$00,\$00,\$00,\$00 ;
DFB \$00,\$04,\$08,\$08,\$08,\$08,\$04,\$00 ;
DFB \$00,\$20,\$10,\$10,\$10,\$10,\$20,\$00 ;
DFB \$00,\$00,\$14,\$08,\$3E,\$08,\$14,\$00 ;
DFB \$00,\$00,\$08,\$08,\$3E,\$08,\$08,\$00 ;
DFB \$00,\$00,\$00,\$00,\$00,\$08,\$08,\$10 ;
DFB \$00,\$00,\$00,\$00,\$3E,\$00,\$00,\$00 ;
DFB \$00,\$00,\$00,\$00,\$00,\$18,\$18,\$00 ;
DFB \$00,\$00,\$02,\$04,\$08,\$10,\$20,\$00 ;
DFB \$00,\$3C,\$46,\$4A,\$52,\$62,\$3C,\$00 ;
DFB \$00,\$18,\$28,\$08,\$08,\$08,\$3E,\$00 ;
DFB \$00,\$3C,\$42,\$02,\$3C,\$40,\$7E,\$00 ;
DFB \$00,\$3C,\$42,\$0C,\$02,\$42,\$3C,\$00 ;
DFB \$00,\$08,\$18,\$28,\$48,\$7E,\$08,\$00 ;
DFB \$00,\$7E,\$40,\$7C,\$02,\$42,\$3C,\$00 ;
DFB \$00,\$3C,\$40,\$7C,\$42,\$42,\$3C,\$00 ;
DFB \$00,\$7E,\$02,\$04,\$08,\$10,\$10,\$00 ;
DFB \$00,\$3C,\$42,\$3C,\$42,\$42,\$3C,\$00 ;
DFB \$00,\$3C,\$42,\$42,\$3E,\$02,\$3C,\$00 ;
DFB \$00,\$00,\$00,\$10,\$00,\$00,\$10,\$00 ;
DFB \$00,\$00,\$10,\$00,\$00,\$10,\$10,\$20 ;
DFB \$00,\$00,\$04,\$08,\$10,\$08,\$04,\$00 ;
DFB \$00,\$00,\$00,\$3E,\$00,\$3E,\$00,\$00 ;
DFB \$00,\$00,\$10,\$08,\$04,\$08,\$10,\$00 ;
DFB \$00,\$3C,\$42,\$04,\$08,\$00,\$08,\$00 ;
DFB \$00,\$3C,\$4A,\$56,\$5E,\$40,\$3C,\$00 ;
DFB \$00,\$3C,\$42,\$42,\$7E,\$42,\$42,\$00 ;
DFB \$00,\$7C,\$42,\$7C,\$42,\$42,\$7C,\$00 ;
DFB \$00,\$3C,\$42,\$40,\$40,\$42,\$3C,\$00 ;
DFB \$00,\$78,\$44,\$42,\$42,\$44,\$78,\$00 ;
DFB \$00,\$7E,\$40,\$7C,\$40,\$40,\$7E,\$00 ;
DFB \$00,\$7E,\$40,\$7C,\$40,\$40,\$40,\$00 ;
DFB \$00,\$3C,\$42,\$40,\$4E,\$42,\$3C,\$00 ;
DFB \$00,\$42,\$42,\$7E,\$42,\$42,\$42,\$00 ;
DFB \$00,\$3E,\$08,\$08,\$08,\$08,\$3E,\$00 ;
DFB \$00,\$02,\$02,\$02,\$42,\$42,\$3C,\$00 ;
DFB \$00,\$44,\$48,\$70,\$48,\$44,\$42,\$00 ;
DFB \$00,\$40,\$40,\$40,\$40,\$40,\$7E,\$00 ;
DFB \$00,\$42,\$66,\$5A,\$42,\$42,\$42,\$00 ;
DFB \$00,\$42,\$62,\$52,\$4A,\$46,\$42,\$00 ;
DFB \$00,\$3C,\$42,\$42,\$42,\$42,\$3C,\$00 ;
DFB \$00,\$7C,\$42,\$42,\$7C,\$40,\$40,\$00 ;
DFB \$00,\$3C,\$42,\$42,\$52,\$4A,\$3C,\$00 ;
DFB \$00,\$7C,\$42,\$42,\$7C,\$44,\$42,\$00 ;
DFB \$00,\$3C,\$40,\$3C,\$02,\$42,\$3C,\$00 ;
DFB \$00,\$FE,\$10,\$10,\$10,\$10,\$10,\$00 ;
DFB \$00,\$42,\$42,\$42,\$42,\$42,\$3C,\$00 ;

DFB \$00,\$42,\$42,\$42,\$42,\$24,\$18,\$00 ;
 DFB \$00,\$42,\$42,\$42,\$42,\$5A,\$24,\$00 ;
 DFB \$00,\$42,\$24,\$18,\$18,\$24,\$42,\$00 ;
 DFB \$00,\$82,\$44,\$28,\$10,\$10,\$10,\$00 ;
 DFB \$00,\$7E,\$04,\$08,\$10,\$20,\$7E,\$00 ;
 DFB \$00,\$0E,\$08,\$08,\$08,\$08,\$0E,\$00 ;
 DFB \$00,\$00,\$40,\$20,\$10,\$08,\$04,\$00 ;
 DFB \$00,\$70,\$10,\$10,\$10,\$10,\$70,\$00 ;
 DFB \$00,\$10,\$38,\$54,\$10,\$10,\$10,\$00 ;
 DFB \$00,\$00,\$00,\$00,\$00,\$00,\$00,\$FF ;
 DFB \$00,\$1C,\$22,\$78,\$20,\$20,\$7E,\$00 ;
 DFB \$00,\$00,\$38,\$04,\$3C,\$44,\$3C,\$00 ;
 DFB \$00,\$20,\$20,\$3C,\$22,\$22,\$3C,\$00 ;
 DFB \$00,\$00,\$1C,\$20,\$20,\$20,\$1C,\$00 ;
 DFB \$00,\$04,\$04,\$3C,\$44,\$44,\$3C,\$00 ;
 DFB \$00,\$00,\$38,\$44,\$78,\$40,\$3C,\$00 ;
 DFB \$00,\$0C,\$10,\$18,\$10,\$10,\$10,\$00 ;
 DFB \$00,\$00,\$3C,\$44,\$44,\$3C,\$04,\$38 ;
 DFB \$00,\$40,\$40,\$78,\$44,\$44,\$44,\$00 ;
 DFB \$00,\$10,\$00,\$30,\$10,\$10,\$38,\$00 ;
 DFB \$00,\$04,\$00,\$04,\$04,\$04,\$24,\$18 ;
 DFB \$00,\$20,\$28,\$30,\$30,\$28,\$24,\$00 ;
 DFB \$00,\$10,\$10,\$10,\$10,\$10,\$0C,\$00 ;
 DFB \$00,\$00,\$6C,\$92,\$92,\$92,\$92,\$00 ;
 DFB \$00,\$00,\$78,\$44,\$44,\$44,\$44,\$00 ;
 DFB \$00,\$00,\$38,\$44,\$44,\$44,\$38,\$00 ;
 DFB \$00,\$00,\$78,\$44,\$44,\$78,\$40,\$40 ;
 DFB \$00,\$00,\$3C,\$44,\$44,\$3C,\$04,\$06 ;
 DFB \$00,\$00,\$1C,\$20,\$20,\$20,\$20,\$00 ;
 DFB \$00,\$00,\$38,\$40,\$38,\$04,\$78,\$00 ;
 DFB \$00,\$10,\$38,\$10,\$10,\$10,\$0C,\$00 ;
 DFB \$00,\$00,\$44,\$44,\$44,\$44,\$38,\$00 ;
 DFB \$00,\$00,\$44,\$44,\$28,\$28,\$10,\$00 ;
 DFB \$00,\$00,\$92,\$92,\$92,\$92,\$6C,\$00 ;
 DFB \$00,\$00,\$44,\$28,\$10,\$28,\$44,\$00 ;
 DFB \$00,\$00,\$44,\$44,\$44,\$3C,\$04,\$38 ;
 DFB \$00,\$00,\$7C,\$08,\$10,\$20,\$7C,\$00 ;
 DFB \$00,\$0E,\$08,\$30,\$08,\$08,\$0E,\$00 ;
 DFB \$00,\$08,\$08,\$08,\$08,\$08,\$08,\$00 ;
 DFB \$00,\$70,\$10,\$0C,\$10,\$10,\$70,\$00 ;
 DFB \$00,\$14,\$28,\$00,\$00,\$00,\$00,\$00 ;
 DFB \$3C,\$42,\$99,\$A1,\$A1,\$99,\$42,\$3C ;

8 EXROM Source Code

```
INCL          "C:\2068\ROM\2068.INC"

; Home ROM addresses
LED18:        EQU    $0E2F

;XFER_BYTES:   EQU    $6722

CALLx:        MACRO ADR
               CALL   $5200+ADR
               ENDM

CALLy:        MACRO ADR
               CALL   ADR
               ENDM

JPx:          MACRO ADR
               JP     $5200+ADR
               ENDM

JPy:          MACRO ADR
               JP     $5200+ADR
               ENDM

MFD32:        EQU    $FD32
M6000:        EQU    $6000

MFD90:        EQU    $FD90
M255D:        EQU    $255D

DATTYPE:      EQU    $00
FILENAME:     EQU    $01
BLKLEN:       EQU    $0B
ARG1:         EQU    $0D
ARG2:         EQU    $0F

;*****
; Module:
; Routine:
; general routines used when the EXROM is switched in
;*****

XRST0:        DI                ;disable interrupts
               JR      M0049      ;jump to bank transfer code
               DFB     $FF,$FF,$FF,$FF,$FF
```

```

;local RST 8 error handler.

XRST8:      LD    HL,(CHADD)      ;HL gets the address of the character
                                ; in dispute
                                LD    (XPTR),HL      ;put it into the "syntax error"
character
                                ; pointer
                                POP    HL            ;get the caller's address
                                LD     L,(HL)         ;/put the error code into
LD    (IY+OERRNR),L             ;\ ERRNR
                                LD     SP,(ERRSP)     ;get the error stack pointer

;jump to the home bank to handle the error

                                LD     HL,RESET       ;/address for goto bank
                                PUSH    HL            ;\
                                LD     H,$FF           ;/HOME bank ...
                                LD     L,$00          ;|... all chunks
                                PUSH    HL            ;\

;fall through to GOTO_BANK

XRST20:      PUSH    AF           ;save AF
                                LD     A,(VIDMOD)      ;/
                                AND     A            ;|check the video mode
                                NOP                 ;\
                                JR      Z,M002C       ;jump if normal video
XRST28:      POP     AF           ;restore AF
                                CALLx   GOTO_BANK+$97C0 ;jump to RAM INT service when
                                                ; in extended video mode
M002C:      POP     AF           ;restore AF
                                CALLx   GOTO_BANK     ;jump to RAM service when
                                                ; in normal video mode

XRST30:      DFB     $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF

;*****
; Interrupt routine.  called when the keyboard interrupt is
; received from the SCLD
;*****
XRST38:      PUSH    AF           ;save AF
                                DI                     ;disable interrupts
                                LD     A,(VIDMOD)      ;/
                                AND     A            ;|check the video mode
                                NOP                 ;\
                                JR      Z,M0045       ;jump if normal video
                                POP     AF           ;restore AF
                                JP      INT+$E9C0     ;jump to RAM INT service when
                                                ; in extended video mode
M0045:      POP     AF           ;restore AF
                                JP      INT+$5200     ;jump to RAM INT service when
                                                ; in normal video mode

```

```

;*****
; code executed on power-up if the SCLD has reset to in such a
; state so as to point to EXROM
;*****
M0049:      LD    A,$01                ;set HSR to point lower 8K to
DOCK/EXROM

            OUT   ($F4),A              ;
            JR     M005A

;*****
; this section of code is transferred to HOME RAM by the code
; fragment at M005A during reset, then it is jumped to by the fragment
;
;execution of the following code will cause execution to transfer to the HOME
bank
; at location $0054.
            XOR    A                    ;/set HSR and EXROM/DOCK select to
point to HOME bank
            OUT   ($F4),A              ;|HSR now selects all chunks to
HOME/EXROM
            OUT   ($FF),A              ;\bank select now transfers us to HOME

            LD     DE,$FFFF            ;
            JP     M0D31

;*****

M005A:      LD     HL,$004F            ;/
            LD     DE,$6000            ;|transfer the code fragment
            LD     BC,$000B            ;| from above to HOME ROM
            LDIR                                ;\
            JP     M6000                ;jump to it
;*****
; Module: TAPE
; Routine: W_TAPE
;
;
;*****
W_TAPE:
M0068:      LD     HL,$00E5            ;
            PUSH  HL                    ;

;set the number of header half cycles based on A.  if A>$7F, write a
; data block header else write a file header
            LD     HL,$1F80            ;number of half-cycles for the
; header
            BIT    7,A                  ;/skip if header
            JR     Z,M0076              ;\
            LD     HL,$0C98            ;number of half-cycles for the
; data header

M0076:      EX     AF,AF'              ;save the data byte
            INC    DE                    ;/preadjust the counters for the
following
            DEC    IX                    ;\ processing

```

```

                                DI                ;don't interrupt the the tape output
                                LD  A,$02          ;/start sound with '0' and set border
red                                LD  B,A          ;\ (alternates with cyan)

;write the tape header
M007E:    DJNZ  M007E    ;delay loop for tone output
                                OUT  ($FE),A      ;activate tape output bit
                                XOR  $0F          ;flip the border color and tape output
signal
                                LD  B,$A4          ;half-cycle time for 806.5Hz
                                DEC  L              ;/jump if header not done
                                JR   NZ,M007E      ;\
                                DEC  B              ;adjust B for processing time
                                DEC  H              ;/jump if header not done
                                JP   P ,M007E      ;\

;hokay, the header has now been written, so move on to writing the data

;write the single 2400Hz transition
                                LD  B,$2F          ;first half-cycle of the 2400Hz
transition
M0090:    DJNZ  M0090    ;(adjusted for processing time)
                                OUT  ($FE),A      ;
                                LD  A,$0D          ;second half-cycle of the transition
                                LD  B,$37
M0098:    DJNZ  M0098
                                OUT  ($FE),A

                                LD  BC,$3B0E      ;B=half-cycle freq for 2400Hz, Tape
output high, border = YELLOW
                                EX  AF,AF'        ;get the type byte
                                LD  L,A            ;save in L
                                JP   M00AD        ;off to the races

M00A4:    LD  A,D            ;/jump if the byte counter is '0';
                                OR   E            ;|last byte to be written
                                JR   Z,M00B4      ;\ (send the checksum)
                                LD  L,(IX+$00)    ;get the next byte

M00AB:    LD  A,H            ;get checksum
                                XOR  L            ;XOR next byte to be sent

M00AD:    LD  H,A            ;save the checksum
                                LD  A,$01        ;Tape output low, border = BLUE
                                SCF              ;sentinal bit, byte to be sent will be zero
only
                                                ; when all 8 bits have been shifted out

                                JP   M00CB        ;start sending the byte

```

```

M00B4:      LD    L,H          ;get the checksum
            JR     M00AB       ;send it

M00B7:      LD    A,C          ;Tape output high, border = YELLOW
            BIT    7,B         ;this will indicate that the 2nd half-cycle
has
                                   ; been sent

;output the data bit in the CF
M00BA:      DJNZ   M00BA       ;wait a half-cycle
            JR     NC,M00C2     ;jump if data bit is '0'

M00C0:      LD    B,$42        ;/wait a bit longer for a '1'
            DJNZ   M00C0       ;\
M00C2:      OUT    ($FE),A      ;flip the bit

            LD    B,$3E        ;half-cycle for '0'
            JR     NZ,M00B7     ;jump if next half-cycle is still to be sent

            DEC    B            ;B = $3D (make up for processing
time)
            XOR    A            ;zero A (and reset carry flag)
            INC    A            ;A=1

M00CB:      RL     L            ;get the next bit of the byte to send
            JP     NZ,M00BA     ;jump if bits are still left to be sent

            DEC    DE           ;decrement the byte count
            INC    IX           ;point to the next byte to send
            LD     B,$31        ;half-cycle value for a '0'
            LD     A,$7F        ;precharge the A register
            IN     A,($FE)      ;read the keyboard
            RRA                ;/return if key press detected
            RET    NC           ;\

            LD     A,D          ;/continue if more bytes are waiting
            INC    A            ;|to be written
            JP     NZ,M00A4     ;|(carry will be set on exit,
indicating all is OK)

            LD     B,$3B        ;/wait for one last half-cycle
M00E2:      DJNZ   M00E2       ;\
            RET                ;our job here is done
;*****
; Module: TAPE
; Routine: W_BORD
;*****
W_BORD:
            PUSH   AF           ;save AF
            LD     A,(BORDCR)   ;get attribute
            AND    $38          ;/
            RRCA                ;|form port compatible border
            RRCA                ;|color value
            RRCA                ;\
            OUT    ($FE),A      ;set the border color

```

```

        LD    A,$7F          ;/
        IN    A,($FE)        ;|check keyboard
        RRA                     ;\
        EI                      ;enable interrupts
        JR    C,M00FA        ;jump if no key pressed
        RST   $08            ;/error BREAK
        DFB   $0C            ;\
M00FA:    POP   AF            ;restore AF
        RET                    ;done
;*****
; Module: TAPE
; Routine: R_TAPE
; ENTRY: IX points to the destination, DE is the block length to load
;*****
R_TAPE:
M00FC:    INC    D                ;
        EX     AF,AF'            ;
        DEC    D                ;
        DI                     ;don't interrupt us
        LD     A,$0F            ;/set tape out=high,border to WHITE
        OUT    ($FE),A          ;\
        LD     HL,$00E5         ;
        PUSH   HL               ;
        IN     A,($FE)          ;get tape input
        RRA                     ;tape bit is now in the $20 position
        AND    $20              ;mask for the tape bit
        OR     $02              ;set bit 1
        LD     C,A              ;save the seed value for the call to R_EDGE
        CP     A                ;ensure we don't return before we
call R_EDGE
M0111:    RET    NZ              ;return if tape bit is not high

M0112:    CALL   R_EDGE          ;check for transition on the tape
input
        JR     NC,M0111         ; a key was pressed, so abort

;there was a transition detected, so let's try to read the tape.
; wait for 1044 bit times: this is a significant fraction of the time needed
; for either of the headers
        LD     HL,$0415         ;for 1044 bits
M011A:    DJNZ   M011A          ;wait for the length of time returned
by R_EDGE
                                ; at M0112
        DEC    HL               ;/
        LD     A,H              ;|jump if HL is not zero
        OR     L                ;|
        JR     NZ,M011A        ;\
        CALL   M0189           ;get a bit of data
        JR     NC,M0111        ;error detected, so exit

;check for header
M0126:    LD     B,$9C          ;
        CALL   M0189           ;get a bit of data
        JR     NC,M0111        ;error detected, so exit

```

```

        LD    A,$C6        ;/
        CP    B            ;|jump if 806.5Hz detected?
        JR    NC,M0112     ;\

        INC    H            ;
        JR    NZ,M0126     ;jump back if not enough cycles
detected

M0135:    LD    B,$C9        ;
        CALL  R_EDGE        ;check again for the edge of a bit
        JR    NC,M0111     ;error detected, so exit

        LD    A,B          ;
        CP    $D4          ;
        JR    NC,M0135     ;

        CALL  R_EDGE        ;check for the edge of a bit
        RET   NC           ;error detected, so exit

        LD    A,C          ;
        XOR   $03          ;
        LD    C,A          ;
        LD    H,$00        ;
        LD    B,$B0        ;
        JR    M016E        ;
M014F:    EX    AF,AF'      ;
        JR    NZ,M0159     ;
        JR    NC,M0163     ;
        LD    (IX+$00),L    ;save received byte at (IX)
        JR    M0168        ;
M0159:    RL    C          ;
        XOR   L            ;
        RET   NZ          ;
        LD    A,C          ;
        RRA                ;
        LD    C,A          ;
        INC   DE           ;
        JR    M016A        ;

M0163:    LD    A,(IX+$00)   ;get the last byte received
        XOR   L            ;check tha check sum
        RET   NZ           ;return if the checksum is in error

M0168:    INC   IX          ;bump the pointer
M016A:    DEC   DE          ;decrement the block length
        EX    AF,AF'      ;
        LD    B,$B2        ;
M016E:    LD    L,$01        ;
M0170:    CALL  RD_BIT      ;read a bit
        RET   NC           ;error detected, so return

        LD    A,$CB        ;
        CP    B            ;
        RL    L            ;
        LD    B,$B0        ;

```



```

        JP    NC,M0170          ;
        LD    A,H              ;
        XOR   L                ;
        LD    H,A              ;
        LD    A,D              ;
        OR    E                ;
        JR    NZ,M014F         ;
        LD    A,H              ;
        CP    $01              ;
        RET                     ;
;*****
; Module: TAPE
; Routine: RD_BIT
;*****
RD_BIT:
M0189:      CALL R_EDGE        ;find the tape tone
            RET    NC          ;return if error detected, else
                                ; move on to detect the next edge
;*****
; Module: TAPE
; Routine: R_EDGE
; find the edge of a tape tone.
; ENTRY: B contains a value through which to wait
;*****
R_EDGE:
M018D:      LD    A,$16        ;/
M018F:      DEC    A           ;|wait a 70uS
            4
            JR    NZ,M018F     ;\
            AND    A           ;reset CF
M0193:      INC    B           ;bump the timer?
            4
            RET    Z           ;return if we have waited a long
time?       10
            LD    A,$7F        ;/
            IN     A,($FE)     ;|read the keyboard and the tape
input bit
            RRA                ;|(puts tape bit into $20 position and loads
                                ;\LSB of key scan into the carry flag)
            RET    NC          ;return if key is pressed
            XOR    C           ;/this checks if there has been a
transition
                                ;| of the tape bit
            AND    $20         ;\check the tape in bit
            JR     Z,M0193     ;jump if this tape sample is the same
as the
                                ; last sample - wait for the transition

            LD    A,C          ;get previous tape input bit value
            CPL                ;complement it
            LD    C,A          ;save it again
            AND    $07         ;mask for the BORDER color
            OR     $08         ;set the tape out bit (echo the sound on the
                                ; tape)

```

```

                                OUT ($FE),A                ;output the new border color and tape
bit
                                SCF                        ;indicate all is OK
                                RET                        ;weebiedun
;*****
; Module: TAPE
; Routine: SLVM
; "Save, Load, Verify, Merge"
; this routine does the donkey work of parsing the command line for
; the tape functions.
; ENTRY: (TADDR) is used to differentiate between
; "LOAD", "SAVE", "VERIFY", or "MERGE"
;*****
SLVM:      LD    A,(TADDR)          ;low byte of the syntax table pointer
           LD    BC,PASAVE+1        ;address of PASAVE in the syntax table
           SUB   C                  ;generate a value based on the
instruction
                                ; kinda cheesy, but seems to work.
           LD    (TADDR),A          ;(TADDR)= 0 for SAVE
                                ;          1 for LOAD
                                ;          2 for VERIFY
                                ;          3 for MERGE

;get the file name. the string parameters will be on the top of the fp stack
           CALLBANKS TEM10,HOMEROM,$00      ;evaluate the next expression
and return here if it is a string

           BIT    INTPT,(IY+$01)          ;/jump if syntax checking
           JR     Z,M0238                  ;\

;prepare to allocate a 17 byte header buffer if the value at TADDR is zero
(SAVE)
           LD     BC,$0011                ;
           LD     A,(TADDR)                ;/jump if SAVE (only one buffer
needed)
           AND    A                        ;|
           JR     Z,M01DD                  ;\

;allocate two header buffers worth of memory. this is needed for
; LOAD, VERIFY, and MERGE. one buffer for the stuff coming from tape, the
other
; for the arguments coming from the command line. DE points to the start of
the
; newly allocated memory
           LD     C,$22                    ;
M01DD:     CALLBANKS ALLOCBC,HOMEROM,$00      ;allocate the header buffer

;we will return here if the buffer could be allocated
           PUSH   DE                        ;/copy the location of the header
buffer
           POP    IX                        ;\ to IX
           LD     B,$0B                    ;/
           LD     A," "                    ;|fill up the program name with
M01FB:     LD     (DE),A                    ;|spaces
           INC    DE                        ;|

```

```

        DJNZ    M01FB          ;\
        LD      (IX+FILENAME),$FF ;indicate no name present

; check the name provided to the command
;get the string descriptor from the fp stack
        CALLBANKS PGPSTR,HOMEROM,$00      ;pop BC, DE, and A from fp stack

        LD      HL,-10          ;/test the length of the program name
        DEC     BC              ;|(a length of zero will cause an
overflow
        ADD     HL,BC           ;\ hence an error)
        INC     BC              ;restore BC
        JR      NC,M0231        ;jump if the name was less than 11
characters

        LD      A,(TADDR)       ;/jump if this command does not
REQUIRE a name
        AND     A               ;| LOAD, MERGE, VERIFY
        JR      NZ,M022A        ;\

        RST     $08             ;/error F - invalid file name
        DFB     $0E             ;\

M022A:   LD      A,B            ;/jump if the length of the name is zero
        OR      C               ;|(no name to move, so don't)
        JR      Z,M0238         ;\
        LD      BC,$000A        ;length of a program name (transfer
only the first 10
                                ; characters)
M0231:   PUSH    IX              ;save the header header buffer
pointer
        POP     HL              ;get the header buffer pointer to HL
        INC     HL              ;
        EX      DE,HL           ;HL now points to the name, DE points to the
header buffer

        LDIR                    ;move the name to the header buffer
;
;now to check the variants of save/load e.g.
; SAVE f LINE m, SAVE f DATA A(), SAVE f DATA A$(), SAVE f CODE m,n, SAVE f
SCREEN$
;
M0238:   CALLBANKS GETCURCH,HOMEROM,$00      ;call GETCURCH, get
                                ; current character
                                ; skipping spaces and
                                ; control characters

        CP      $E4             ;/jump if not "DATA" token
        JP      NZ,M02F2        ;\

;
;next, find the DATA item
;
        LD      A,(TADDR)       ;/determine if we are MERGE-
ing
        CP      $03             ;\

```

```

        JP    Z ,BADBAS                ;declare a syntax error if so -
                                         ; can't MERGE DATA

;get the next character and then attempt to find it in the variables
; area will gobble the "(" of a numeric array or "$(" of a string array name
        CALLBANKD M0020,HOMEROM,$00,FIND_N,HOMEROM,$00

        SET   7,C                      ;force an array name
        JR    NC,M0295                  ;jump if we found the variable name

        LD    HL,$0000                  ;trash the pointer to the
                                         ; non-existent variable
        LD    A,(TADDR)                  ;/jump if we were LOADING
        DEC   A                          ;|(variable name not needed; the file
        JR    Z,M02A9                    ;\ header will supply it)

        RST   $08                       ;/error VARIABLE NOT FOUND
        DFB   $01                       ;\

SLVBADBAS:
M0295:        JP    NZ,BADBAS            ;variable found was not an array
variable or a string

        BIT   7,(IY+$01)                ;/jump if syntax checking
        JR    Z,M02B6                    ;\

        INC   HL                        ;/
        LD    A,(HL)                     ;|
        LD    (IX+BLKLEN),A              ;|put the length of the variable
        INC   HL                         ;| data structure in the header
buffer
        LD    A,(HL)                     ;|
        LD    (IX+BLKLEN+1),A            ;\
        INC   HL                         ;/save data type marker in buffer
M02A9:        LD    (IX+ARG1+1),C         ;\
        LD    A,$01                      ;flag byte
        BIT   6,C                         ;/jump if dealing with numbers
        JR    Z,M02B3                    ;\
        INC   A                          ;/put flag (1 for strings, 2 for
numbers)
M02B3:        LD    (IX+DATTYPE),A        ;\ in the buffer
M02B6:        EX    DE,HL                 ;

        CALLBANKS M0020,HOMEROM,$00      ;get the next character

        CP    ")"                        ;/jump if not close paren
        JR    NZ,SLVBADBAS              ;\ [must be there for SAVE f DATA
xx()])

        CALLBANKS M0020,HOMEROM,$00      ;get the next character

        BIT   7,(IY+$01)                ;/return if syntax checking - we are
done
        RET   Z                          ;\

```

```

EX    DE,HL        ;
JP    M04C9        ;

M02F2:    CP    $AA        ;/jump if not "SCREEN$" token
JR    NZ,M032E        ;\

LD    A,(TADDR)        ;/syntax error if trying to MERGE
CP    $03        ;| can't MERGE SCREEN$ data
JP    Z ,BADBAS        ;\

CALLBANKS M0020,HOMEROM,$00        ;get the next character

BIT    7,(IY+$01)        ;/return if syntax checing
RET    Z        ;\

LD    (IX+BLKLEN),00        ;/load screen length into buffer
LD    (IX+BLKLEN+1),$1B        ;\
LD    HL,$4000        ;/screen address
LD    (IX+ARG1),L ;|
LD    (IX+ARG1+1),H        ;\
JP    M0440        ;

M032E:    CP    $AF        ;/jump if not "CODE" token
JP    NZ,M0447        ;\

LD    A,(TADDR)        ;/
CP    $03        ;|syntax error if trying to MERGE
JP    Z ,BADBAS        ;\

;get the next character and then check to see if we are at the end
; of a statement (":" or newline)
CALLBANKD M0020,HOMEROM,$00,TERM?,HOMEROM,$00

JR    NZ,M0387        ;jump if not the end of a statement

LD    A,(TADDR)        ;/
AND    A        ;|syntax error if trying to SAVE
JP    Z ,BADBAS        ;\

;stack a dummy argument
CALLBANKS STKFP0,HOMEROM,$00        ;stack a zero on the fp stack
JR    M03BC        ;proceed

;evaluate the next expression. it is numeric we will return here
; to then get the next character
M0387:    CALLBANKD TEM6,HOMEROM,$00,GETCURCH,HOMEROM,$00

CP    ","        ;/jump if comma
JR    Z,M03D5        ;\

LD    A,(TADDR)        ;/syntax error if trying to SAVE
AND    A        ;| must have address and length
JP    Z ,BADBAS        ;\

;stack a second dummy argument

```

```

M03BC:          CALLBANKS STKFP0,HOMEROM,$00 ;stack 0 to the fp stack
                JR      M03FF                ;(03FF)

;get the next character then evaluate the next expression.  if numeric we
; will return here
M03D5:          CALLBANKD M0020,HOMEROM,$00,TEM6,HOMEROM,$00

M03FF:          BIT    7,(IY+$01)            ;/return if syntax checking - we are
done                                                    done
                RET    Z                      ;\

;put the top of the FP stack into BC
                CALLBANKS FIX_U,HOMEROM,$00
                LD      (IX+BLKLEN),C          ;/put the length of the code segment
                LD      (IX+BLKLEN+1),B        ;\ into the buffer

;put the top of the FP stack into BC
                CALLBANKS FIX_U,HOMEROM,$00
                LD      (IX+ARG1),C ;/put the address of the code segment
                LD      (IX+ARG1+1),B          ;\ into the buffer

                LD      H,B                    ;/put the code address into HL
                LD      L,C                    ;\

M0440:          LD      (IX+DATTYPE),03        ;put the file type into the buffer -
BINARY
                JP      M04C9                  ;jump to determine if we are SAVEing

M0447:          CP      $CA                    ;/jump if "LINE" token
                JR      Z,M0456                ;\

                BIT    7,(IY+$01)            ;/return if syntax checking
                RET    Z                      ;\

                LD      (IX+ARG1+1),$80        ;/no line number specified
                JR      M04A9                  ;\

M0456:          LD      A,(TADDR)              ;/
                AND     A                      ;|syntax error if NOT trying to SAVE
                JP      NZ,BADBAS              ;\

;get the next character and then evaluate the next expression.
; return here if its numeric
                CALLBANKD M0020,HOMEROM,$00,TEM6,HOMEROM,$00

                BIT    7,(IY+$01)            ;/return if syntax checking
                RET    Z                      ;\

;get the top of the FP stack into BC
                CALLBANKS FIX_U,HOMEROM,$00

                LD      (IX+ARG1),C            ;/put the line number into the buffer
                LD      (IX+ARG1+1),B          ;\

M04A9:          LD      (IX+$00),00            ;put file type into the buffer

```

```

        LD    HL,(ELINE)           ;/
        LD    DE,(PROG)           ;|compute the length of the program
        SCF                        ;|and variables to the $80 byte
        SBC   HL,DE                ;\just after the variables
        LD    (IX+BLKLEN),L        ;/block length to the
        LD    (IX+BLKLEN+1),H      ;\buffer
        LD    HL,(VARS)           ;/compute the variables area length
        SBC   HL,DE                ;\
        LD    (IX+ARG2),L          ;/length of the variables
        LD    (IX+ARG2+1),H        ;\to the buffer
        EX    DE,HL               ;DE=length, HL=(VARS)

M04C9:   LD    A,(TADDR)           ;/
        AND   A                   ;|jump if we are SAVEing
        JP    Z,SAVE              ;\

        PUSH  HL                  ;save the block length
        LD    BC,$0011            ;/point to the buffer used to store
        ADD   IX,BC               ;\ the header coming in from the tape
M04D6:   PUSH  IX                  ;save the header
        LD    DE,$0011            ;length of the header
        XOR   A                   ;reset the carry and zero the
accumulator
        SCF                      ;set the carry flag ("LOAD" the tape)

;read the program header
        CALL  R_TAPE              ;read the tape
        POP   IX                  ;restore the header pointer
        JR    NC,M04D6            ;loop around for unsuccessful load
                                   ; (break or error)

        LD    A,$FE               ;select channel -2
        CALLBANKS SELECT,HOMEROM,$00

        LD    (IY+$52),03         ;load the scroll count with 3
        LD    C,$80               ;
        LD    A,(IX+DATTYPE)      ;/compare the file type from the
        CP    (IX+{- $11})        ;\tape with the file type from the
command line
        JR    NZ,M050D            ;not the same, so check if it is a TS
file
        LD    C,$F6               ;-10
M050D:   CP    $04                ;
        JR    NC,M04D6            ;we found a non TS file type, so back
around
;
;we found some type of file so tell the user so
;
        LD    DE,$3CA8            ; address of "Program: " message
        PUSH  BC                  ;briefly save C

;print a message to the current channel. A contains the number of the message
to
; be printed
        CALLBANKS PUTMES,HOMEROM,$00

```

```

        POP BC                ;restore BC
        PUSH IX               ;save buffer pointer
        POP DE                ;get it to DE
        LD HL,-16             ;/subtract 16 from the buffer pointer
        ADD HL,DE              ;\ DE points to the second buffer name
        LD B,$0A              ;name length counter
        LD A,(HL)              ;get the first letter of the file name
        INC A                  ;/jump if a name was given (<>$FF)
        JR NZ,M053D           ;\
        LD A,C                 ;load a with the length of the name
        ADD A,B                ;add to the max length of a name
        LD C,A                 ;save the length of the name
;
;loop to write the file name
;
M053D:      INC DE              ;/get a character from the buffer
pointer      LD A,(DE)          ;\ (we preincrement to get past the
data type on the first iteration)
        CP (HL)                ;compare it to the received name
        INC HL                  ;move along
        JR NZ,M0544            ;jump if it does not match
        INC C                   ;if it does match, increment the match
counter
;write the character in A
M0544:      CALLBANKS WRCH,HOMEROM,$00
        DJNZ M053D             ;back around for another name
        BIT 7,C                 ;/if the names matched, C will have
rolled over
        JP NZ,M04D6            ;\
;print a newline
        LD A,$0D                ;newline
        CALLBANKS WRCH,HOMEROM,$00
        POP HL                  ;
        LD A,(IX+DATTYPE)        ;get the data type
        CP $03                  ;/jump if it is "CODE"
        JR Z,M058F              ;\
        LD A,(TADDR)             ;/
        DEC A                    ;|jump to LOAD
        JP Z ,M05CC              ;\
        CP $02                  ;/jump if character array
        JP Z ,M06E5              ;\
M058F:      PUSH HL              ;
        LD L,(IX+{- $06})         ;/get second buffer block length
        LD H,(IX+{- $05})         ;\
        LD E,(IX+BLKLEN)          ;/get this block length

```



```

        LD    D,(IX+BLKLEN+1)    ;\
        LD    A,H                ;/
        OR    L                    ;|jump if HL = 0
        JR    Z,M05AD            ;\
        SBC   HL,DE                ;
        JR    C,M05CA            ;
        JR    Z,M05AD            ;

        LD    A,(IX+DATTYPE)    ;/
        CP    $03                ;|jump if this is "CODE"
        JR    NZ,M05CA            ;\

M05AD:    POP    HL                ;
        LD    A,H                ;/
        OR    L                    ;|jump if zero
        JR    NZ,M05B8            ;\

        LD    L,(IX+ARG1)
        LD    H,(IX+ARG1+1)
M05B8:    PUSH HL
        POP    IX

        LD    A,(TADDR)          ;/jump if VERIFY
        CP    $02                ;\
        SCF                    ;force load
        JR    NZ,M05C4            ;

        AND    A                    ;force VERIFY
M05C4:    LD    A,$FF                ;specify a data block
R_TAPE1:  CALL R_TAPE                ;/return if no tape loading error
        RET    C                    ;\

M05CA:    RST    $08                ;error TAPE LOADING ERROR
        DFB    $1A                ;
;*****
; Module: TAPE
; Routine: LOAD
; this routine does the nitty-gritty of actually loading and
; merging program text and merging data from tape - ugggg!!
;*****
LOAD:
M05CC:    LD    E,(IX+BLKLEN)        ;/get the data block length
        LD    D,(IX+BLKLEN+1)    ;\
        PUSH HL                    ;save the length of the target data
structure
        LD    A,H                ;/
        OR    L                    ;|jump if it is not zero
        JR    NZ,M05DD            ;\

        INC    DE                    ;/adjust to reserve space for a name
and
        INC    DE                    ;| length bytes
        INC    DE                    ;\
        EX    DE,HL                ;HL = required block length, DE = 0
        JR    M05E9                ;

```

```

;handle an existing array with name the same name as the proposed array
M05DD:      LD    L,(IX+{- $06})      ;/get the other block length
            LD    H,(IX+{- $05})      ;\
            EX    DE,HL              ;/
            SCF                      ;|jump if the existing data block
            SBC   HL,DE              ;| is at least long enough to hold
            JR    C,M0606             ;\ the new data

M05E9:      LD    DE,$0005            ;/add 5 bytes to the block length
            ADD   HL,DE              ;|BC = number of additional bytes
needed
            LD    B,H                ;| for this data block
            LD    C,L                ;\

            CALLBANKS CHK_SZ,HOMEROM,$00      ;check for BC bytes available
;
;if we got here then there is enough space for the data coming in
;
M0606:      POP   HL                  ;restore the existing data block
length
            LD    A,(IX+DATTYPE)      ;/
            AND   A                  ;|jump if this is a program
            JR    Z,M0673             ;\

            LD    A,H                ;/jump if the existing block length is zero
            OR    L                  ;| to create the variable
            JR    Z,M0638             ;\

;
;destroy the present array with the name of the proposed data array
;
            DEC   HL                  ;point to the array size
            LD    B,(HL)              ;/
            DEC   HL                  ;|get the array size into BC
            LD    C,(HL)              ;|
            DEC   HL                  ;\
            INC   BC                  ;/add the length of the array name
            INC   BC                  ;| and array length bytes
            INC   BC                  ;\
            LD    (XPTR),IX           ;save IX

            CALLBANKS DELREC,HOMEROM,$00      ;delete the existing array

            LD    IX,(XPTR)           ;restore IX

;
;
;
M0638:      LD    HL,(ELINE)          ;/point to the $80 byte in the
variables area
            DEC   HL                  ;\
            LD    C,(IX+BLKLEN)       ;/get the required data block length
            LD    B,(IX+BLKLEN+1)     ;\
            PUSH  BC                  ;save the block length
            INC   BC                  ;/

```

```

name and      INC BC                      ;|need three more bytes for the array

data          INC BC                      ;|\length
              LD A,(IX+{- $03})          ;get the array ID for the incoming

array         PUSH AF                    ;save it

              CALLBANKS INSERT,HOMEROM,$00 ;make room at (HL) for the

              INC HL                      ;/
              POP AF                      ;|store the array name
              LD (HL),A                  ;\
              POP DE                      ;/
              INC HL                      ;|
              LD (HL),E                  ;|store the array length
              INC HL                      ;|
              LD (HL),D                  ;\
              INC HL                      ;|point to the "number of dimensions"
              PUSH HL                    ;/IX now points to the destination

address for   POP IX                      ;\ the array data
              SCF                        ;/"load..."
              LD A,$FF                  ;|"...a data block"
              JP R_TAPE1                ;\read the data block and exit via

SLVM
;
;program loading
;
M0673:        EX DE,HL                  ;DE = end address
              LD HL,(ELINE)              ;/point to the $80 byte at the end of

the variables  DEC HL                    ;\ area
              LD (XPTR),IX              ;save the buffer header pointer
              LD C,(IX+BLKLEN)          ;/
              LD B,(IX+BLKLEN+1)        ;|get the data block length and briefly

save it       PUSH BC                    ;\

              CALLBANKS DEL_DE,HOMEROM,$00 ;delete the bytes between

HL and DE     POP BC                    ;pop the block length
              PUSH HL                    ;push the data buffer address
              PUSH BC                    ;push the block length

;
; reserve space for the incoming lines
;
              CALLBANKS INSERT,HOMEROM,$00 ;make room at (HL) for BC

bytes         LD IX,(XPTR)              ;get the buffer header pointer back
              INC HL                    ;get the original address back
              LD C,(IX+ARG2)            ;/get the variables address from the

file

```

```

LD B,(IX+ARG2+1) ;\
ADD HL,BC ;/point to the new variables area
LD (VARS),HL ;\ and save it in the system vars
area
LD H,(IX+ARG1+1) ;get the putative line number
LD A,H ;/
AND $C0 ;|jump if not a valid line number
JR NZ,M06D5 ;\
LD L,(IX+ARG1) ;get the lower byte of the line number
LD (NEWPPC),HL ;store the line number into the next
line number
LD (IY+ONSPPC),00 ;subline 0
M06D5: POP DE ;restore the block length
POP IX ;
SCF ;"load..."
LD A,$FF ;"...a data block"
LD HL,(PROG) ;get the address of the program text
DEC HL ;point to the $80 byte
LD (DATADD),HL ;data terminator character (no data)
JP R_TAPE1 ;read the tape

M06E5: LD C,(IX+BLKLEN) ;/get the required data block length
LD B,(IX+BLKLEN+1) ;\
PUSH BC ;save the block length
INC BC ;room for the $80 byte

CALLBANKS ALLOCBC,HOMEROM,$00 ;expand the workspace BC bytes

LD (HL),$80 ;$80 byte for the end of the edit
buffer
EX DE,HL ;HL = the start of the newly freed space
POP DE ;pop the block length
PUSH HL ;save the pointer to the end of the
reserved area
PUSH HL ;...and again
POP IX ;point to the end of the reserved
area
SCF ;/"load..."
LD A,$FF ;|"...a data block"
CALL R_TAPE1 ;\read the program lines in from tape

POP HL ;point to the newly loaded data
LD DE,(PROG) ;get the start of the program file
M0717: LD A,(HL) ;/get the first byte of the newly
loaded BASIC program
AND $C0 ;|jump if not a valid line number
JR NZ,M0749 ;\

M071C: LD A,(DE) ;get the first byte of the old BASIC
program
INC DE ;bump the old pointer
CP (HL) ;compare to the new BASIC
INC HL ;bump the new pointer
JR NZ,M0724 ;old and new line numbers not the same
so jump

```

```

LD    A,(DE)                ;compare the last half of the line
numbers
CP    (HL)                  ;\
M0724: DEC DE                ;/bump both BASIC pointers back
DEC HL                      ;\
JR    NC,M0744              ;jump if the new line number is less
than or equal to the
                                ; old line number to insert the new line into
the program
PUSH HL                    ;save the new pointer
EX    DE,HL                ;HL=old pointer, DE=the new pointer

CALLBANKS RECLen,HOMEROM,$00    ;find the next variable or BASIC
line

POP    HL                  ;restore the new pointer
JR    M071C                ;back around to test the new line

M0744: CALL M0799            ;
JR    M0717                ;

M0749: LD    A,(HL)          ;get the character from the new code
LD    C,A                  ;/
CP    $80                  ;|return if we have reached the end of
the code
RET    Z                    ;\
;
;was not BASIC code, so try to find the entity in the variables area
;
HL    DE    BC    STK
PUSH HL                    ;save the pointer to the new code
new    old    new
LD    HL,(VARS)            ;get the pointer to the variables area
vars    old    new
M0752: LD    A,(HL)          ;/
CP    $80                  ;|jump if there are no variables
JR    Z,M0790              ;\

CP    C                    ;/jump if the variable name in the new
code starts with
JR    Z,M0776              ;\ the same letter as the one just
found
M075A: PUSH BC              ;save C
vars    old    BC,new
CALLBANKS RECLen,HOMEROM,$00    ;find the next variable or BASIC
line
;
vars    newrec    new
POP    BC                  ;restore C

EX    DE,HL                ;/put pointers back and try the next
variable name    vars    newrec
JR    M0752                ;\

```

```

M0776:      AND  $E0          ;mask out the variable discriminator
bits
            CP   $A0          ;/jump if not a multi letter variable
            JR   NZ,M078E      ;\
;
;loop through the characters of a multi-letter variable name to determine
whether the new
; name matches the one in the variables area
;
            POP  DE            ;get the pointer to the new code
            vars new
            PUSH DE            ;save it again
            vars new          new
            PUSH HL            ;save the pointer to the old code
            vars new          vars,new
M077F:      INC  HL            ;/move to the next character
            INC  DE            ;\
            LD   A,(DE)        ;/
            CP   (HL)          ;|jump if the variable names do not match
            JR   NZ,M078B      ;\
            RLA                ;/loop back around if not the last
character
            JR   NC,M077F      ;\
            POP  HL            ;restore the pointer to the old code
            vars new          new
            JR   M078E          ;
M078B:      POP  HL            ;/multi-letter name did not match, so
            vars new          new
            JR   M075A          ;\ back around to try again
M078E:      LD   A,$FF        ;will force falling through at M0799
M0790:      POP  DE            ;restore the pointer to the new code
            vars new          --
            EX   DE,HL         ;swap the pointers
new vars
            INC  A              ;
            SCF                  ;
            CALL M0799          ;
            JR   M0749          ;
M0799:      JR   NZ,M07CF      ;jump if we are just inserting a
record
            EX   AF,AF'        ;
            LD   (XPTR),HL      ;save
            EX   DE,HL          ;
vars new
;find the next line or variable, then delete it
CALLBANKD RECLen,HOMEROM,$00,DELREC,HOMEROM,$00
            EX   DE,HL          ;
            LD   HL,(XPTR)      ;

```

```

                EX    AF,AF'                ;

;insert a new record
M07CF:          EX    AF,AF'                ;
                PUSH DE                    ;save the pointer to the new code

                CALLBANKS RECLen,HOMEROM,$00    ;find the next variable or
program line

                LD     (XPTR),HL              ;save pointer to old code
                LD     HL,(PROG)              ;get the pointer to the BASIC program
                EX     (SP),HL                ;
                PUSH BC                       ;
                EX     AF,AF'                ;
                JR     C,M080E                ;
                DEC    HL                    ;

                CALLBANKS INSERT,HOMEROM,$00

                INC    HL                    ;
                JR     M0825                ;

M080E:          CALLBANKS INSERT,HOMEROM,$00

M0825:          INC    HL                    ;
                POP    BC                    ;
                POP    DE                    ;
                LD     (PROG),DE              ;
                LD     DE,(XPTR)              ;
                PUSH BC                       ;
                PUSH DE                       ;
                EX     DE,HL                  ;
                LDIR                                ;
                POP    HL                    ;
                POP    BC                    ;
                PUSH DE                       ;

                CALLBANKS DELREC,HOMEROM,$00

                POP    DE                    ;
                RET                             ;
;*****
; Module: TAPE
; Routine: SAVE
;*****
;select the keyboard and lower screen as the current channel
SAVE:
M0851:          PUSH HL                      ;save the data buffer address
                LD     A,$FD                  ;channel -3, keyboard and lower screen

                CALLBANKS SELECT,HOMEROM,$00    ;select channel -3

                XOR    A                      ;message number 0
                LD     DE,$3C89                ;

```

```

CALLBANKS PUTMES,HOMEROM,$00      ;put message 0 to current
channel

SET CLHS,(IY+OTVFLAG) ;force clear lower half screen if
keypressed

CALL M08AA      ;wait for a key to be pressed

PUSH IX          ;save the header buffer address

LD DE,$0011      ;tape header length
XOR A            ;tell W_TAPE we want to write a
header

CALL W_TAPE      ;
POP IX           ;restore the header buffer address
LD B,$32         ;/
M089A: HALT       ;|wait for approx 1000mS
DJNZ M089A       ;\
LD E,(IX+BLKLEN) ;get the length of the data block from
the

LD D,(IX+BLKLEN+1) ; header
LD A,$FF          ;tell W_TAPE we are writing data
POP IX           ;pop the data buffer address
JP W_TAPE         ;write the data and return to
                  ; caller via W_TAPE's return
;*****
; Module: TAPE
; Routine: AKEY
;*****
AKEY:
M08AA: PUSH AF          ;/
      PUSH BC          ;|briefly save registers
      PUSH DE          ;\
      LD BC,$9C40      ;/
M08B0: DEC BC          ;|wait for approx 238mS
      LD A,C           ;|
      OR B             ;|
      JR NZ,M08B0       ;\

;wait for a key press
M08B5: XOR A           ;zero the accum
      IN A,($FE)       ;poll the keyboard
      AND $1F          ;use only the keyboard bits
      CP $1F           ;/jump if no key is pressed
      JR Z,M08B5       ;\

CALLBANKS CLLHS,HOMEROM,$00      ;clear the lower half of the
screen

POP DE           ;/
POP BC           ;|restore registers
POP AF          ;\
RET              ;back to the "save" routine

BADBAS:

```



```

M08D9:      EXX                ;alt regs
            LD  HL,$1BED        ;address for BAD BASIC COMMAND
                                ; error routine
            PUSH HL             ;save jump address
            LD  L,$00           ;all chunks
            LD  H,$FF           ;home bank
            PUSH HL             ;save bank/chunk
            EXX                ;reg registers
            CALL GOTO_B         ;local local GOTO_BANK
;*****
; Module: INIT
; Routine: EXTINIT
;*****
EXTINIT:
            LD  HL,$5EEA        ;/initialize the SYSCON
            LD  (SYSCON),HL     ;\system variable
            CALL BLDSCCT        ;build the system config table
            LD  HL,(SYSCON)     ;point to SYSCON table
            LD  DE,$0008        ;/point to LROS ID
            ADD HL,DE           ;\
            LD  A,(HL)          ;/
            CP  $01             ;|jump if not LROS
            JR  NZ,M090F        ;\
            PUSH HL             ;save LROS pointer
            CALL NORMSVAR       ;initialize various system variables
            POP  HL             ;/
            INC  HL             ;|
            LD  E,(HL)          ;|push LROS entry point
            INC  HL             ;|
            LD  D,(HL)          ;|
            PUSH DE             ;\
            LD  B,$00           ;specify dock bank
            INC  HL             ;/
            LD  C,(HL)          ;|get HSR byte
            PUSH BC             ;\
            EI                  ;enable interrupts
            CALLx GOTO_BANK     ;call GOTO_BANK - goto the AROS

M090F:      LD  HL,(SYSCON)     ;get SYSCON
            INC  HL             ;/get possible LROS ID
            LD  A,(HL)          ;\
            CP  $02             ;/jump if AROS machine code
            JR  Z,M091E         ;\ cartridge
            CALL NORMSVAR       ;initialize various system variables
            JP   M099A          ;handle

M091E:      DEC  HL             ;/get the code type designator
            LD  A,(HL)          ;\
            CP  $01             ;/jump if BASIC code
            JR  Z,M0956         ;\
            CP  $02             ;/jump if machine code
            JR  NZ,LROS_ERR     ;\
            LD  DE,$0006        ;/
            ADD  HL,DE          ;|
            LD  C,(HL)          ;|BC contains the number of

```

```

INC HL ;| bytes to reserve for machine code
LD B,(HL) ;\
LD HL,$6840 ;start of LROS code buffer?
ADD HL,BC ;/
EX DE,HL ;|move BC bytes at (HL) up to (DE)
LD HL,$6840 ;| where (DE) = $6840+BC
LDIR ;\

CALL INITSYSVAR ;
LD HL,(SYSCON) ;/
LD DE,$0005 ;|get the AROS memory specification
ADD HL,DE ;|
LD A,(HL) ;\
CP $00 ;/jump if not autostart
JR Z,M099A ;\
DEC HL ;/C = chunk specification
LD C,(HL) ;\
DEC HL ;/
LD D,(HL) ;|DE = address of first BASIC line
DEC HL ;| or first machine code instruction
LD E,(HL) ;\
PUSH DE ;save the start address
LD B,$00 ;/save the chunk specification
PUSH BC ;\
EI ;enable interrupts
CALLx GOTO_BANK ;GOTO BANK (jump to the machine code)

M0956: CALL NORMSVAR ;
LD A,$80 ;/set flag to show AROS is present
LD (ARSFLG),A ;\
LD HL,AROS ;/jump to the AROS handler
PUSH HL ;\
LD B,$FF ;home bank
LD C,$00 ;all of memory
PUSH BC ;
CALLx GOTO_BANK ;GOTO BANK

LROS_ERR: RST $08 ;/error "MISSING LROS"
DFB $1B ;\

NORMSVAR: LD HL,$6840 ;point past the RAM resident services
LD DE,$0015 ;/reserve space for the CHANS table
ADD HL,DE ;\

INITSYSVAR: LD (DATADD),HL ;initialize the DATA pointer
INC HL ;/point to the start of BASIC
LD (PROG),HL ;\
LD (VARS),HL ;initialize the variables pointer
LD (HL),$80 ;/drop the sentinel at the end of the
; \ variables are
INC HL ;/initialize the input line buffer
LD (ELINE),HL ;\
LD (HL),$0D ;stuff a carriage return
INC HL ;/sentinal byte
LD (HL),$80 ;\

```

```

INC HL ;/
LD (WORKSP),HL ;|initialize system variables
LD (STKBOT),HL ;|
LD (STKEND),HL ;\
XOR A ;
LD (ARSFLG),A ;show no AROS present
LD (VIDMOD),A ;normal video mode
RET ;

M099A: LD D,$FF ;home bank
LD E,$80 ;HS select, top chunk in EXROM/DOCK
LD HL,LED18 ;jump address in HOME ROM
PUSH HL ;stack the jump address
PUSH DE ;stack bank/HS

LD HL,(SYSCON) ;/point to the first expansion bank
LD DE,$000C ;| entry
ADD HL,DE ;\
LD B,$00 ;prep BC for the next ops
M09AC: LD A,(HL) ;get the entry for an expansion bank
CP $80 ;/jump if at the end of the SYSCON
area

JR Z,M09E3 ;\
CP $00 ;/no entry for this bank, so move on
JR Z,M09DD ;\to check the next one
INC HL ;/pick up bank number
LD B,(HL) ;\
LD DE,$0014 ;/
ADD HL,DE ;|
LD A,(HL) ;|jump if active? or code?
RRCA ;|
JR C,M09C4 ;\
INC HL ;/point to the next exp bank entry in
INC HL ;|the SYSCON area
INC HL ;\
JR M09AC ;back around to check for more
M09C4: INC HL ;/get HS for this exp bank
LD A,(HL) ;\

POP DE ;remove the bank/HS from the stack
CP E ;/jump if this exp bank needs more
JR C,M09CF ;\ HS than is currently allocated
PUSH DE ;save bank/HS
INC HL ;/point to the next exp bank
INC HL ;|entry in the SYSCON
JR M09AC ;\
M09CF: POP DE ;trash the call address
LD DE,$0005 ;/point to CAL nn or JP nn
SBC HL,DE ;| in SYSCON table and
PUSH HL ;\ stack it
LD C,A ;update HS
PUSH BC ;save bank/HS
INC DE ;/
INC DE ;|point to next entry in SYSCON
ADD HL,DE ;\

```

```

M09DD:      JR    M09AC                ;back around for more
entry      LD    DE,$0018            ;/point to the next expansion bank

          ADD    HL,DE                ;\in the SYSCON area
          JR    M09AC                ;back around to check for more
M09E3:      POP    BC                ;get the bank/HS
          LD    A,B                    ;/
          CP    $FF                ;|jump if going to HOME bank
          JR    Z,M09ED                ;\
          LD    C,$58                ;allow chunks 3, 4, and 6 to be in
expansion banks
          JR    M09EF                ;
M09ED:      LD    C,$00                ;allocate all chunks to HOME
M09EF:      PUSH BC                ;save bank/HS
          EI                    ;enable interrupts
          CALLx GOTO_BANK            ;goto bank
;*****
; Module: INIT
; Routine: BLD SCT
; build the system configuration table
;*****
BLD SCT:
          LD    HL,(SYSCON)            ;point to SYSCON buffer
          XOR    A                    ;0 A
          LD    (MAXBNK),A            ;zero out MAXBANK
          LD    ($6315),A            ;zero out dispatcher's copy of MAXBANK
          LD    DE,$0008                ;/point to LROS buffer
          ADD    HL,DE                ;\
;
;transfer bytes from LROS to the SYSCON table
;
          LD    E,$FF                ;/dest bank = $FF (home)
          LD    D,$00                ;|src bank = $00 (dock)
          PUSH DE                    ;\
          LD    DE,$0001                ;/set source address
          PUSH DE                    ;\
          PUSH HL                    ;set destination address
          LD    DE,$0004                ;/set length
          PUSH DE                    ;\
          LD    DE,$0001                ;/set direction (LDIR)
          PUSH DE                    ;\
          CALLx XFER_BYTES            ;transfer the bytes

          LD    A,(HL)                ;/
          CP    $01                ;|jump if LROS signature present
          JR    Z,M0A3E                ;\
          LD    (HL),$00                ;zero LROS signature to prevent
          ; system from incorrectly seeing
          ; an LROS as present
;
;transfer bytes from AROS to the SYSCON
;
          LD    E,$FF                ;/dest bank = $FF (home)
          LD    D,$00                ;|src bank = $00 (dock)
          PUSH DE                    ;\

```

```

LD DE,$8000 ;/set source address
PUSH DE ;\
LD HL,(SYSCON) ;/get SYSCON buffer address
PUSH HL ;\and use as destination address
LD DE,$0008 ;/set length
PUSH DE ;\
LD DE,$0001 ;/set direction
PUSH DE ;\
CALLx XFER_BYTES ;transfer the bytes

INC HL ;/get AROS signature byte
LD A,(HL) ;\
CP $02 ;/jump if valid AROS ID byte
JR Z,M0A3E ;\
LD (HL),$00 ;zero out AROS ID byte, as we do not
have ; a valid AROS cartridge

;we will also get to this point if there is a valid LROS via $0A1A
M0A3E: LD HL,(SYSCON) ;get SYSCON buffer address
LD DE,$000D ;/point to AROS buffer?
ADD HL,DE ;\
LD D,$C0 ;bank status register address
LD E,$00 ;bank status register value
CALLx WRITE_BS_REG ;write to bank status register
M0A4C: CALL M0BD1 ;find valid banks
JP NC,M0AD4 ;
LD B,A ;
SET 7,B ;
LD (HL),B ;
RES 7,B ;
INC HL ;

LD C,$FE ;/dest bank = EXROM
PUSH BC ;\
LD DE,$08E7 ;/set source address
PUSH DE ;\
LD DE,$0000 ;/get SYSCON buffer address
PUSH DE ;\and use as destination address
LD DE,$0001 ;/set length
PUSH DE ;\
PUSH DE ;set direction
CALLx XFER_BYTES ;transfer the bytes

LD E,$FF ;dest bank = $FF (home)
LD D,A ;src bank provided by caller
PUSH DE ;set bank
LD DE,$0000 ;/set source adr
PUSH DE ;\
PUSH HL ;set dest address
LD DE,$0016 ;/set length
PUSH DE ;\
LD DE,$0001 ;/set direction
PUSH DE ;\
CALLx XFER_BYTES ;transfer the bytes

```

```

LD    D,(HL)                ;/
LD    A,($08E7)            ;|jump if we are not in
CP    D                    ;|the EXTROM
JP    NZ,M0AC2             ;\

LD    C,$FE                ;dest bank = $FE (external)
PUSH BC                    ;set banks
LD    DE,$0A4C             ;/set source adr
PUSH DE                    ;\
LD    DE,$0000             ;/set dest adr
PUSH DE                    ;\
LD    DE,$0001             ;length/direction
PUSH DE                    ;set length
PUSH DE                    ;set direction
CALLx XFER_BYTES           ;get bytes

LD    E,$FF                ;dest bank = home
LD    D,A                  ;src bank = user's
PUSH DE                    ;set banks
LD    DE,$0000             ;/set source adr
PUSH DE                    ;\
PUSH HL                    ;set user's destination
LD    DE,$0016             ;/set block length
PUSH DE                    ;\
LD    DE,$0001             ;/set direction
PUSH DE                    ;\
CALLx XFER_BYTES           ;transfer the bytes

LD    D,(HL)                ;/
LD    A,($08E7)            ;|get first byte of user's buffer
CP    D                    ;|and jump if user is not EXROM
JP    NZ,M0AC2             ;\
DEC    HL                  ;
DEC    HL                  ;
CALL M0ADB                 ;
LD    DE,$0015             ;
ADD    HL,DE               ;
JR     M0ACA               ;
M0AC2: LD    A,D            ;
AND    $DF                 ;
LD    (HL),A               ;
DEC    HL                  ;
CALL M0C1F                 ;
M0ACA: LD    D,$C0          ;
LD    E,$01                ;
CALLx WRITE_BS_REG         ;
JP    M0A4C                ;

M0AD4: DEC    HL            ;
LD    (HL),$80             ;
CALL M0CFB                 ;
RET                        ;

M0ADB: LD    (HL),$02       ;

```

```

PUSH BC                ;push user's bank info
LD DE,$0038            ;/set source
PUSH DE                ;\
PUSH DE                ;set destination
LD DE,$0010            ;/set block length
PUSH DE                ;\
LD DE,$0001            ;/set direction
PUSH DE                ;\
CALLx XFER_BYTES       ;transfer bytes

INC HL                ;
INC HL                ;
LD A,(HL)              ;
SET 0,A               ;
LD (HL),A              ;
LD DE,$0000            ;
LD A,$01               ;

M0AF9: EX AF,AF'       ;
PUSH HL                ;
EX DE,HL               ;
LD DE,$2000            ;
ADD HL,DE              ;
EX DE,HL               ;
POP HL                 ;

LD B,$FE               ;src bank = EXROM
LD A,(MAXBNK)          ;get MAXBANK
LD C,A                 ;dest bank = MAXBANK
PUSH BC                ;set banks
LD BC,$08E7            ;/set source
PUSH BC                ;\
PUSH DE                ;set destination
LD BC,$0001            ;/
PUSH BC                ;|set byte count and direction
PUSH BC                ;\
CALLx XFER_BYTES       ;transfer bytes

LD A,(MAXBNK)          ;get MAXBANK
LD B,A                 ;src = MAXBANK
LD C,$00               ;dest = Dock
PUSH BC                ;set banks
PUSH DE                ;set user's source
INC HL                 ;bump user's destination
PUSH HL                ;set user's destination
LD BC,$0001            ;/
PUSH BC                ;|set block length and direction
PUSH BC                ;\
CALLx XFER_BYTES       ;transfer bytes

LD B,(HL)              ;/
DEC HL                 ;|jump if destination was
LD A,($08E7)           ;|not EXROM
CP B                   ;|
JR NZ,M0B95            ;\

```

```

LD    B,$FE                ;src bank = EXROM
LD    A,(MAXBNK)           ;get MAXBANK
LD    C,A                  ;dest bank = MAXBANK
PUSH  BC                   ;set banks
LD    BC,$0A4C             ;/set source
PUSH  BC                   ;\
PUSH  DE                   ;set user's destination
LD    BC,$0001             ;/
PUSH  BC                   ;|set direction and byte count
PUSH  BC                   ;\
CALLx XFER_BYTES           ;transfer bytes

LD    A,(MAXBNK)           ;get MAXBANK
LD    B,A                  ;src bank = MAXBANK
LD    C,$00                ;dest bank = Dock
PUSH  BC                   ;set banks
PUSH  DE                   ;set user's source
INC   HL                   ;bump user's destination
PUSH  HL                   ;set user's destination
LD    BC,$0001             ;/
PUSH  BC                   ;|set direction and byte count
PUSH  BC                   ;\
CALLx XFER_BYTES           ;transfer bytes

LD    B,(HL)               ;get the value at (HL)
DEC   HL                   ;back a byte
LD    A,($0A4C)            ;A=$CD???
CP    B                    ;/jump if B is not $CD
JR    NZ,M0B95             ;\

;
;this routine sets the bit whose number is in A'
;
EX    AF,AF'               ;
LD    B,(HL)               ;get the byte at (HL)
CP    $01                  ;/
JR    NZ,M0B68             ;|if 1, set bit 1 in B
SET   1,B                  ;| then move on
JR    M0B92                ;\
M0B68: CP    $02            ;/
JR    NZ,M0B70             ;|if 2, set bit 2 in B
SET   2,B                  ;| then move on
JR    M0B92                ;\
M0B70: CP    $03            ;/
JR    NZ,M0B78             ;|if 3, set bit 3 in B
SET   3,B                  ;| then move on
JR    M0B92                ;\
M0B78: CP    $04            ;/
JR    NZ,M0B80             ;|if 4, set bit 4 in B
SET   4,B                  ;| then move on
JR    M0B92                ;\
M0B80: CP    $05            ;/
JR    NZ,M0B88             ;|if 5, set bit 5 in B
SET   5,B                  ;| then move on

```



```

M0B88:      JR      M0B92          ;\
            CP      $06           ;/
            JR      NZ,M0B90       ;|if 6, set bit 6 in B
            SET     6,B           ;| then move on
            JR      M0B92          ;\
M0B90:      SET     7,B           ;else set bit 7
M0B92:      LD      (HL),B        ;save byte back
            JR      M0BCA          ;

;
;this routine resets the bit whose number is in A'
;
M0B95:      EX      AF,AF'        ;
            LD      B,(HL)        ;
            CP      $01           ;/
            JR      NZ,M0B9F       ;|if 1, reset bit 1 in B
            RES     1,B           ;| then move on
            JR      M0BC9          ;\
M0B9F:      CP      $02           ;/
            JR      NZ,M0BA7       ;|if 2, reset bit 2 in B
            RES     2,B           ;| then move on
            JR      M0BC9          ;\
M0BA7:      CP      $03           ;/
            JR      NZ,M0BAF       ;|if 3, reset bit 3 in B
            RES     3,B           ;| then move on
            JR      M0BC9          ;\
M0BAF:      CP      $04           ;/
            JR      NZ,M0BB7       ;|if 4, reset bit 4 in B
            RES     4,B           ;| then move on
            JR      M0BC9          ;\
M0BB7:      CP      $05           ;/
            JR      NZ,M0BBF       ;|if 5, reset bit 5 in B
            RES     5,B           ;| then move on
            JR      M0BC9          ;\
M0BBF:      CP      $06           ;/
            JR      NZ,M0BC7       ;|if 6, reset bit 6 in B
            RES     6,B           ;| then move on
            JR      M0BC9          ;\
M0BC7:      RES     7,B           ;else reset bit 7
M0BC9:      LD      (HL),B        ;save byte back

M0BCA:      INC     A             ;
            CP      $08           ;
            JP      NZ,M0AF9       ;
            RET                   ;

;
;find all used banks in the machine
;
M0BD1:      LD      A,(MAXBNK)     ;get MAXBANK
            INC     A             ;bump it
            LD      (MAXBNK),A     ;save new MAXBANK
            LD      ($6315),A     ;save dispatcher's
                                   ; local copy of MAXBANK

```

```

        LD    D,$A0          ;/
        LD    E,A            ;|write to bank status register
        CALLx WRITE_BS_REG   ;\
;
;check expansion banks?
;
        LD    D,$80          ;/
        LD    E,A            ;|write to bank status register
        CALLx WRITE_BS_REG   ;\
        LD    D,$40          ;/
        LD    E,$00          ;|write to bank status register
        CALLx WRITE_BS_REG   ;\
        PUSH AF              ;save AF briefly
        LD    A,($A000)       ;/save value at $A0000
        EX    AF,AF'         ;\
        LD    A,$04          ;/store 4 into chunk 5 BSR
        LD    ($A000),A       ;\
        LD    D,$A0          ;/
        LD    E,$C0          ;|read bank status register
        CALLx READ_BS_REG     ;\
        BIT   2,E            ;/jump if 4 was successfully written
        JR    NZ,M0C0A        ;(0C0A);\
        EX    AF,AF'         ;/restore the value at $A000
        LD    ($A000),A       ;\
        POP   AF              ;restore AF
        SCF                  ;set error flag
        RET                  ;done

M0C0A:    EX    AF,AF'         ;/restore the value at $A000
        LD    ($A000),A       ;\
        POP   AF              ;restore A
        DEC   A               ;
        LD    (MAXBNK),A      ;save new MAXBANK
        LD    ($6315),A       ;save dispatcher's
                                ; local copy of MAXBANK
        LD    D,$C0          ;/
        LD    E,$04          ;|write to bank status register
        CALLx WRITE_BS_REG   ;\
        AND   A               ;
        RET                  ;

M0C1F:    DEC   HL            ;
        LD    (HL),$01        ;
        LD    DE,$0015        ;
        ADD   HL,DE           ;
        LD    A,(HL)          ;
        RRA                  ;
        JR    C,M0C2F         ;(0C2F) ;
        LD    DE,$0004        ;
        ADD   HL,DE           ;
        RET                  ;

M0C2F:    LD    C,$08         ;
        LD    A,(MAXBNK)      ;
        DEC   HL              ;

```

```

        DEC HL          ;
        DEC HL          ;
        LD D,(HL)       ;
        DEC HL          ;
        LD E,(HL)       ;
        LD H,D          ;
        LD L,E          ;
        LD B,A          ;
        PUSH HL         ;
        PUSH BC         ;
        LD BC,$0000     ;
        PUSH BC         ;
        PUSH BC         ;
        CALLx CALL_BANK ;
        LD DE,$0008     ;
        ADD HL,DE       ;
        RET             ;
;*****
; Module: INIT
; Routine: RESSCT
;*****

        XOR A
        LD (MAXBNK),A
        LD ($6315),A
        LD D,$C0
        LD E,$00
        CALLx WRITE_BS_REG
        LD HL,(SYSCON)
        LD DE,$000C
M0C60:  ADD HL,DE
        CALL M0BD1
        JP NC,M0CF5
        LD A,(HL)
        PUSH HL
        CP $80
        JR NZ,M0C72      ;(0C72)
        LD DE,$0018
        ADD HL,DE
        LD (HL),A
M0C72:  CALL M0BD1
        LD HL,$5FE9
        LD E,$FF
        LD D,A
        PUSH DE
        LD DE,$0000
        PUSH DE
        PUSH HL
        LD DE,$0016
        PUSH DE
        LD DE,$0001
        PUSH DE
        CALLx XFER_BYTES
        EX AF,AF'
        LD A,(HL)

```

```

CPL
DEC HL
LD (HL),A
EX AF,AF'
LD D,$FF
LD E,A
PUSH DE
PUSH HL
LD DE,$0002
PUSH DE
LD DE,$0001
PUSH DE
PUSH DE
CALLx XFER_BYTES
LD E,$FF
LD D,A
PUSH DE
LD DE,$0002
PUSH DE
INC HL
PUSH HL
LD DE,$0001
PUSH DE
PUSH DE
CALLx XFER_BYTES
LD A,(HL)
DEC HL
LD B,(HL)
CP B
JR NZ,M0CCA ;(0CCA)
POP HL
LD A,(HL)
CP $02
JR NZ,M0CC5 ;(0CC5)
INC HL
INC HL
JR M0CE8 ;(0CE8)
M0CC5: CALL M0ADB
JR M0CE8 ;(0CE8)
M0CCA: LD C,(HL)
POP HL
INC HL
INC HL
LD A,(HL)
CP C
JR Z,M0CE8 ;(0CE8)
PUSH HL
EX DE,HL
LD HL,$5FE9
LD BC,$0016
LDIR
POP HL
DEC HL
LD A,(MAXBNK)
SET 7,A

```

```

LD    (HL),A
CALL M0C1F
INC   HL
M0CE8: LD    D,$C0
LD    E,$01
CALLx WRITE_BS_REG
LD    DE,$0016
JP    M0C60
M0CF5: LD    (HL),$80
CALL M0CFB
RET

M0CFB: XOR    A                ;/load 0 into MAXBANK
LD    (MAXBNK),A            ;\
M0CFF: LD    HL,(SYSCON)        ;get SYSCON buffer address
LD    DE,$000C            ;
ADD   HL,DE                ;
M0D06: LD    A,(HL)            ;
CP    $80                  ;
JR    Z,M0D84              ;(0D84)
INC   HL
LD    A,(HL)
BIT   7,A
JR    NZ,M0D17              ;(0D17)
LD    DE,$0017
ADD   HL,DE
JR    M0D06                ;(0D06)
M0D17: LD    ($5FE9),HL
DEC   HL
LD    A,(HL)
CP    $02
JR    NZ,M0D28              ;(0D28)
LD    DE,$0017
ADD   HL,DE
LD    A,$FF
JR    M0D2D                ;(0D2D)
M0D28: LD    DE,$0017
ADD   HL,DE
LD    A,(HL)
M0D2D: LD    ($5FEB),A
M0D30: INC   HL
M0D31: LD    A,(HL)
CP    $80
JR    Z,M0D64              ;(0D64)
INC   HL
LD    A,(HL)
BIT   7,A
JR    NZ,M0D42              ;(0D42)
LD    DE,$0017
ADD   HL,DE
JR    M0D30                ;(0D30)
M0D42: DEC   HL
LD    A,(HL)
CP    $02
JR    NZ,M0D4E              ;(0D4E)

```

```

LD    DE,$0017
ADD   HL,DE
JR     M0D30          ;(0D30)
M0D4E: EX   DE,HL
LD     BC,$0017
ADD   HL,BC
LD     A,($5FEB)
LD     B,A
LD     A,(HL)
CP     B
JR     NC,M0D30      ;(0D30)
LD     ($5FEB),A
LD     ($5FE9),DE
JR     M0D30          ;(0D30)
M0D64: LD     A,(MAXBNK)
INC    A
LD     (MAXBNK),A
LD     HL,($5FE9)
LD     (HL),A
LD     E,$FF
LD     D,A
PUSH   DE
LD     E,(HL)
LD     D,$00
PUSH   DE
LD     E,$00
PUSH   DE
LD     E,$01
PUSH   DE
PUSH   DE
CALLx  XFER_BYTES
JP     M0CFF
M0D84: XOR    A
LD     (MAXBNK),A
LD     ($6315),A
LD     D,$C0
LD     E,$00
CALLx  WRITE_BS_REG
LD     HL,(SYSCON)
LD     DE,$000D
ADD    HL,DE
LD     D,$A0
M0D9B: CALL  M0BD1
RET    NC
LD     E,(HL)
CALLx  WRITE_BS_REG
LD     D,$C0
LD     E,$01
CALLx  WRITE_BS_REG
LD     DE,$0018
ADD    HL,DE
JR     M0D9B          ;(0D9B)
;*****
; Module: CHG_VID
; Routine: OPDFIL

```

```

; open the second display file.  move the UDG then move the stack
; and RAM resident code to high memory.  update the video mode and
; the video port.
;*****
OPDFIL:
M0DB0:      PUSH BC          ;/
            PUSH DE          ;|save caller's
            PUSH HL          ;|registers
            PUSH AF          ;\
            LD  HL,(PRAMT)    ;get physical ramtop
            LD  DE,(UDG)      ;get the pointer to the UDG
            AND  A            ;/number of bytes occupied by
            SBC  HL,DE        ;\the UDG
            LD  B,H          ;/put into byte counter
            LD  C,L          ;\
            INC  BC           ;adjust for zero base
            LD  HL,(UDG)      ;point to the present UDG
            PUSH HL           ;brefly save
            LD  DE,$0840      ;number of bytes needed by
                                ; RAM resident routines and
                                ; stack
            AND  A            ;/compute the new address
            SBC  HL,DE        ;| for the UDG
            EX  DE,HL         ;\
            POP  HL           ;restore the present UDG address
            LD  (UDG),DE      ;point to the new UDG location
            LDIR              ;move the UDG

            LD  HL,$0000      ;/HL will contain the new
            ADD  HL,SP         ;|SP used when the stack and
            LD  BC,$97C0      ;|RAM resident code are moved
            ADD  HL,BC         ;\
            DI                ;interrupts now will be really,
                                ; really bad!
            LD  SP,HL         ;update the stack pointer
            LD  DE,$F7C0      ;new location for stack and RAM
routines
            LD  HL,$6000      ;old location
            LD  BC,$0840      ;length of stuff to move
            LDIR              ;move it

            LD  HL,$1D00      ;point to the fix table
            LD  BC,$97C0      ;offset to be added to addresses
M0DEE:      LD  E,(HL)        ;/
            INC  HL           ;|get fix address
            LD  D,(HL)        ;\
            INC  HL           ;point to next entry
            LD  A,E           ;/jump if we have reached the end
            OR   D            ;|of the table
            JR   Z,M0E05      ;\
            EX  DE,HL         ;/point to moved code that
            ADD  HL,BC         ;\needs fixing
            PUSH DE           ;save the table pointer
            LD  E,(HL)        ;/
            INC  HL           ;|get the code needing fixing

```

```

LD    D,(HL)                ;\
EX    DE,HL                 ;/
ADD   HL,BC                 ;|
EX    DE,HL                 ;|fix the address and
LD    (HL),D                ;|put it back
DEC   HL                    ;|
LD    (HL),E                ;\
POP   HL                    ;restore the table pointer
JR     M0DEE                ;back around for more entries
M0E05: POP AF                ;/get caller's desired video mode
LD    (VIDMOD),A            ;\and store it
PUSH  AF                    ;save it again
EI                                ;interrupts are OK again

```

;clear the newly opened second display file

```

LD    HL,$6000              ;point to the second display file
M0E0E: XOR A                 ;zero the accum
LD    (HL),A                ;zero a byte
INC   HL                    ;point to the next
LD    A,H                   ;/zero until we reach $7AFF
CP    $7B                   ;\
JR     NZ,M0E0E              ;not done yet

```

```

POP   AF                    ;get the requested video mode
PUSH  AF                    ;save it again
AND   $7F                   ;only use the lower seven bits
                        ; (don't munge the EXROM select)
LD    B,A                   ;save it in B
IN    A,($FF)               ;get the present value of the
                        ; video select port
AND   $80                   ;save the EXROM select
OR    B                      ;OR in the request
OUT   ($FF),A               ;update the port
POP   HL                    ;/
POP   DE                    ;|restore caller's
POP   BC                    ;|registers
POP   AF                    ;\
RET                                ;

```

;*****

; Module: CHG_VID

; Routine: CLDFIL

; close the second display file

; moves the stack and RAM resident code from high memory back to

; low memory. updates SP and relocates the UDG.

;*****

CLDFIL:

```

M0E27: PUSH AF                ;/
PUSH  BC                    ;|save caller's registers
PUSH  DE                    ;|
PUSH  HL                    ;\
IN    A,($FF)               ;/get display mode port value
AND   $80                   ;|and set for black and white,
OUT   ($FF),A               ;\normal video,EXROM selected
LD    HL,$0000              ;/
ADD   HL,SP                 ;|set HL to the value the

```



```

LD    DE,$97C0          ;|SP will need once the stack
AND   A                 ;|has been moved to low memory
SBC   HL,DE             ;\
DI                      ;we can't be interrupted or we
                        ; will be in a real mess!
LD    SP,HL             ;update the stack pointer
LD    HL,$FFFF          ;/move the RAM resident code
LD    DE,$683F          ;|and the stack back to low
LD    BC,$0840          ;|memory (starting at $6000)
LDDR                      ;\
LD    HL,$1D00          ;the location of the fix table
LD    BC,$97C0          ;offset value to convert code
addresses
                        ; from high memory addresses to
                        ; low memory addresses
M0E4E:  LD    E,(HL)      ;/
INC     HL              ;|get a fix address
LD     D,(HL)          ;\
INC     HL              ;point to the next one
LD     A,E             ;/
OR      D              ;|jump if the end of the table
JR      Z,M0E66         ;\
PUSH    HL              ;save the table pointer
EX      DE,HL          ;/
LD     E,(HL)          ;|get the JP/CALL address
INC     HL              ;|in the RAM resident code
LD     D,(HL)          ;\
EX      DE,HL          ;/convert the address from
AND     A              ;|high memory to low memory
SBC     HL,BC          ;\
EX      DE,HL          ;/
LD     (HL),D          ;|modify the code
DEC     HL              ;|
LD     (HL),E          ;\
POP     HL              ;restore the table pointer
JR      M0E4E          ;back around for more entries
M0E66:  XOR     A         ;/force normal video mode
LD     (VIDMOD),A      ;\
EI                      ;we can now allow interrupts again
LD     HL,$F7BF        ;address of UDG end when code is in
                        ; high memory
LD     DE,$FFFF        ;new UDG location
PUSH    HL              ;save the old UDG address
LD     BC,(UDG)         ;get the current UDG pointer
AND     A              ;/number of bytes to move
SBC     HL,BC          ;\
LD     B,H             ;/transfer to byte counter
LD     C,L             ;\
INC     BC              ;bump by one
POP     HL              ;restore old address
LDDR                      ;move the UDG
LD     DE,$0840        ;/
LD     HL,(UDG)         ;|update the UDG pointer
ADD     HL,DE           ;|in the system variables
LD     (UDG),HL        ;\

```

```

        POP HL                ;/
        POP DE                ;|restore the caller's
        POP BC                ;|registers
        POP AF                ;\
        RET                    ;
;*****
; Module: CHG_VID
; Routine: CHG_V
; change the 2068 video mode
; Entry: requested video mode in A
;*****
CHG_V:
        PUSH BC               ;/
        PUSH DE               ;|save caller's registers
        PUSH HL               ;|
        PUSH AF               ;\
        LD B,A                ;save requester's mode

        LD A,(VIDMOD)         ;/jump if present mode is NOT
        AND A                 ;|zero
        JR NZ,M0EED           ;\

        OR B                  ;/jump if mode '0' is requested
        JP Z ,M0F3D           ;|and current mode is '0'. exit video
mode change
                                ;\with out doing anything

;intitiate a change from mode '0' to some other mode

        LD HL,$12C0           ;number of bytes needed for RAM
services
        LD B,H                ;/save for REMGSZ later on, if needed
        LD C,L                ;\
        LD DE,$0840           ;/$2B00 total bytes needed
        ADD HL,DE              ;\
        LD DE,(STKEND)        ;/add to free RAM space stored in
        ADD HL,DE              ;\STKEND
;possible error in memory check. the technical manual mentions that
; overflow is not checked
        LD DE,(RAMTOP)        ;/
        AND A                 ;|if the carry is set, RAMTOP is set
        SBC HL,DE             ;\too low to allow moving the RAM
services
        JP NC,M0F3A           ;error exit

;adjust system variable address pointers
        LD HL,$683F           ;fence value: all system variables
                                ; that point above this will be
                                ; changed.
                                ; (BC was set up earlier to $12C0)
        LD DE,REMG SZ         ;/address for REMGSZ
        PUSH DE               ;\
        LD DE,$FF00           ;/specify home bank, all chunks
        PUSH DE               ;\ in home
        LD DE,$0000           ;/

```

```

        PUSH DE                ;|no parameters in or out
        PUSH DE                ;\
        CALLx CALL_BANK        ;CALL_BANK (REMGSZ)

        LD    HL,(STKEND)      ;/
        EX    DE,HL            ;|move the fp stack
        LDDR                     ;\
        POP    AF              ;/restore caller's video mode
        PUSH AF                ;\
        CALL OPDFIL            ;open the second display file
        LD    BC,$97C0         ;/
        LD    HL,(ERRSP)       ;|point the ERRSP to the newly moved
        ADD    HL,BC            ;|stack
        LD    (ERRSP),HL       ;\
        LD    HL,(LISTSP)      ;/
        ADD    HL,BC            ;|update list stack pointer
        LD    (LISTSP),HL      ;\
        LD    HL,(MSTBOT)      ;/
        ADD    HL,BC            ;|update machine stack pointer
        LD    (MSTBOT),HL      ;\
        JR     M0F3D            ;jump to exit

M0EED:   LD    A,B              ;/jump if requested mode is
        AND    A                ;|'0' (move the stack and
        JR     Z,M0F01          ;\RAM resident code back down)

;the requested mode was NOT zero and the present mode is not zero, so
; simply update the necessary items, then exit
        AND    $7F              ;don't allow changes to the EXROM
                                ; select
        LD    B,A                ;briefly save
        IN    A,($FF)            ;get the present video control port
                                ; value
        AND    $80              ;save the EXROM select
        OR     B                 ;/OR in the requested mode
        OUT    ($FF),A           ;\ and output to the port
        LD    A,B                ;/save the requested mode to
        LD    (VIDMOD),A         ;\VIDMOD
        JR     M0F3D            ;we are outta here

;the present mode is not zero and the requested mode is zero, so
; prepare to move everything back around as needed
M0F01:   CALL CLDFIL            ;close the display file
        LD    BC,$97C0         ;/
        LD    HL,(ERRSP)       ;|update ERRSP to new
        AND    A                ;|stack location
        SBC    HL,BC            ;|
        LD    (ERRSP),HL       ;\
        LD    HL,(LISTSP)      ;/
        AND    A                ;|do the same for the
        SBC    HL,BC            ;|LISTSP
        LD    (LISTSP),HL      ;\
        LD    HL,(MSTBOT)      ;/
        AND    A                ;|yet again for MSTBOT
        SBC    HL,BC            ;|

```

```

LD    (MSTBOT),HL        ;\

LD    BC,$12C0            ;number of bytes to delete
LD    HL,$6840            ;start address
LD    DE,DELREC           ;/delete the bytes
PUSH DE                   ;\
LD    DE,$FF00            ;/HOME bank, all
PUSH DE                   ;\ chunks
LD    DE,$0000            ;/
PUSH DE                   ;|no params in or out
PUSH DE                   ;\
CALLx CALL_BANK           ;
JR    M0F3D               ;we are done

M0F3A:    SCF              ;/set carry flag (indicates error)
JR        M0F3E            ;\and exit
M0F3D:    AND    A          ;reset carry flag (all's OK)
M0F3E:    POP    AF         ;/
          POP    HL         ;|restore caller's registers
          POP    DE         ;|
          POP    BC         ;\
          RET              ;

```

```

;*****
; Module: PASSING
; Routine: PASSIN
;*****

```

PASSIN:

```
LD    BC,(CHADD)
```

```

;I don't have a clue what this is trying to do. $255D in the home
; bank is the last byte of a CALL $65D0 ($65) and then presses on
; to call END? at $1B44 that lands us on a RST $38 that calls
; the RAM interrupt service

```

```
CALL M255D
```

```

LD    HL,(CHADD)
AND    A
SBC    HL,BC
DEC    HL
LD    A,L
LD    HL,(STKEND)
LD    (HL),A
INC    HL
POP    BC
LD    (HL),B
INC    HL
LD    (HL),C
INC    HL
LD    (STKEND),HL
LD    HL,(CHADD)
DEC    HL
BIT    0,A
JR    Z,M0F73            ;

```

```

M0F67:      DEC  A
            LD   B,(HL)
            DEC  HL
            DEC  A
            JP   M ,M0F7D
            LD   C,(HL)
            DEC  HL
            PUSH BC
            JR   M0F67          ;(0F67)
M0F73:      LD   B,$20
            AND  A
            RET  Z
            LD   C,(HL)
            DEC  HL
            DEC  A
            PUSH BC
            JR   M0F67          ;(0F67)
M0F7D:      LD   HL,(STKEND)
            DEC  HL
            LD   A,(HL)
            DEC  HL
            LD   (STKEND),HL
            LD   H,(HL)
            LD   L,A
            PUSH HL
            RET

;*****
; Module: BS
; Routine: GOTO_B
;*****
GOTO_B:
M0F8A:      PUSH AF              ;/determine where the function
            LD   A,(VIDMOD)      ;| dispatcher is located
            AND  A              ;\
            JR   Z,M0F95        ;jump to normal fn dispatch
            POP  AF              ;restore AF
            JP   MFD32          ;jump to high fn dispatch
M0F95:      POP  AF              ;restore AF
            JPx  GOTO_BANK      ;

;*****
; Module: BS
; Routine: CALL_B
;*****
CALL_B:
M0F99:      PUSH AF              ;save AF
            LD   A,(VIDMOD)      ;/check VIDMOD
            AND  A              ;\
            JR   Z,M0FA4        ;jump if non-expanded video mode
            POP  AF              ;restor AF
            JP   MFD90          ;jump to expanded video mode function

dispatcher
M0FA4:      POP  AF              ;restore AF
            JPx  CALL_BANK      ;jump to normal function dispatcher

DFB $00,$00,$00,$00,$00,$00,$00,$00

```

```

DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00

STKSZ:      EQU    $200
SADDPT:     EQU    $F5
SDATPT:     EQU    $F6
HS:         EQU    $40
HS_LSN:     EQU    HS
BNA:        EQU    $80
HS_MSN:     EQU    BNA
ABN:        EQU    $A0
HSP:        EQU    ABN
STA_L:      EQU    ABN
CMD:        EQU    $C0
STA_O:      EQU    CMD
LOWNYB:     EQU    $C000
FREE_BYTES: EQU    32
PRMOUT:     EQU    8
HOR_SEL:    EQU    10
BANK:       EQU    11
UPD_K:      EQU    $02E1

DKHSPT:     EQU    $F4
HREXPT:     EQU    $FF

PRM_OUT:    EQU    8

; DISPATCH (SVC_CODE: PASSED ON STACK)
;
; SVC_CODE IS A 16 BIT QUANTITY. BIT 15 IS USED AS A JUMP FLAG: IF
; SET, THE DISPATCHER WILL TO A GOTO_BANK TO THE SPECIFIED ROUTINE,
; OTHERWISE IT WILL DO A CALL_BANK.
;
;
;
JMPTBL:     EQU    $1FFF
LAST_EXT_SVC: EQU    13
LAST_RAM_SVC: EQU    24
;
;
;
ORG $6200
ORG $1000
DISPATCH:  EQU    $6200
LD          IX,0          ;IX = SP
ADD         IX,SP         ;
PUSH        BC            ;RESERVE A WORD ON THE STACK
PUSH        AF            ;SAVE REGS
PUSH        BC

```

```

PUSH DE
PUSH HL
LD E,(IX+2)
LD D,(IX+3) ;DE = SVC_CODE
XOR A
SLA E
RL D ;DE = 2*DE
RLA
LD HL, LAST_EXT_SVC
SLA L
RL H ;HL = 2*HL
AND A
SBC HL, DE ;COMPARE HL AND DE
JR NC, D_EXT ;IF DE <= HL
LD HL, LAST_RAM_SVC
SLA L
RL H
AND A
SBC HL, DE
JR C, D_HOME
LD B, 255 ;HERE FOR RAM-BASED SERVICES
CALLX GET_STATUS ;GET STATUS OF HOME BANK
LD B, 255 ;BC=HOME BANK/HORIZ SELECT
JR D_SAVE
D_EXT: LD B, 254 ;HER FOR EXT ROM BASE SERVICES
LD C, $FE
JR D_SAVE
D_HOME: LD B, 255 ;SET BANK_ENABLE PARAMS FOR HOME
LD C, 0
D_SAVE: PUSH AF
PUSH BC ;SAVE JUMP FLAG AND BANK_ENABLE PARAMS
LD HL, JMPTBL ;CALC ADR OF TABLE ENTRY
SCF
SBC HL, DE
LD B, 254
CALLX GET_WORD ;READ TABLE ENTRY
EX DE, HL
POP BC
POP AF ;RESTORE JUMP FLAG ETC
AND A
JR Z, D_CALL
LD (IX+{-2}), C ;PUT BANK# AND HORZ SEL ON STACK
LD (IX+{-1}), B
LD L, (IX+0) ;SAVE RET ADDR
LD H, (IX+1)
LD (IX+3), H ;PUT RET ADDR BACK ON STACK
LD (IX+2), L
LD (IX+1), D ;SET UP STACK FOR GOTO_BANK
LD (IX+0), E ;PUT ADDR ON STACK
POP HL ;RESTORE REGS
POP DE
POP BC
POP AF
CALLX GOTO_BANK ;HERE IF JUMP FLAG NOT SET
D_CALL: LD L, (IX+0) ;SET UP STACK FOR CALL_BANK

```

```

LD      H,(IX+1)      ;PUT RET ADDR IN PROPER LOC
PUSH    HL
LD      L,(IX+4)
LD      H,(IX+5)
LD      (IX+{-2}),L
LD      (IX+{-1}),H
LD      L,(IX+6)      ;PUT PARM_OUT IN PROPER LOC
LD      H,(IX+7)
LD      (IX+0),L
LD      (IX+1),H
LD      (IX+2),C      ;PUT BANK#, HS ON STACK
LD      (IX+3),B
LD      (IX+4),E      ;PUT ADDR ON STACK
LD      (IX+5),D
POP     HL
LD      (IX+6),L
LD      (IX+7),H
POP     HL      ;RESTORE REGS
POP     DE
POP     BC
POP     AF
CALLx   CALL_BANK    ;HERE IF JUMP FLAG NOT SET
RET

```

;RST 56: HERE TO SERVICE INTERRUPT BY READING KEYBOARD

```

INT:    PUSH    AF
        PUSH    HL
        PUSH    IX
        LD      HL,0
        ADD     HL,SP
        PUSH    DE
        LD      A,(BS_MAX_BANK)
        LD      E,A
        LD      D,0
        INC     DE
        INC     DE
        AND     A
        SBC     HL,DE
        EX      DE,HL
        LD      IX,0
        ADD     IX,DE
        POP     DE
        LD      SP,IX
        CALLx   SAVE_STATUS
        PUSH    BC
        LD      B,$FF
        CALLx   GET_STATUS
        LD      B,$FF
        LD      A,C
        AND     $F8
        LD      C,A
        CALL    BANK_ENABLE ;AAA
        POP     BC

```



```

                LD    HL,(FRAMES)  ;INCREMENT FRAMES COUNTER
                INC   HL
                LD    (FRAMES),HL
                LD    A,H
                OR    L
                JR    NZ,LIT3
                INC   (IY+{FRAMES2-Y})      ;
LIT3:           PUSH  BC
                PUSH  DE
                CALL  UPD_K
                POP   DE
                POP   BC
PHLAF:          ;JUMP HERE TO POP HL, POP AF, ENABLE INTERRUPTS AND RETURN
                LD    IX,0
                ADD   IX,SP
                CALLx RESTORE_STATUS
                INC   IX
                LD    SP,IX
                POP   IX
                POP   HL
                POP   AF
                EI
                RET

```

```

;HERE TO SERVICE THE NON-MASKABLE INTERRUPT
;IF (NMIADD)=0 THEN RETURNS STRAIGHT AWAY;
;ELSE, JUMPS TO (NMIADD) WHIT HL (ON TOP) , AF & RETN ADDR ON
;THE STACK

```

```

NMI:           PUSH  AF
                PUSH  HL
                LD    HL,(USRNMI)
                LD    A,H
                OR    L
                JR    NZ,LNI3
                JP    (HL)
LNI3:          POP   HL
                POP   AF
                RETN

```

```

;
;
BS_MAX:        EQU   $5200+$
BS_MAX_BANK:   EQU   BS_MAX
                DFS   1      ;LOCAL COPY OF MAX_BANK

```

```

;
; GET_WORD (ADDR: HL, BANK: B, WORD: HL)
;
GET_WORD:      PUSH  AF      ;SAVE REGS
                PUSH  BC
                PUSH  DE
                CALLx GET_NUMBER ;GET BANK# OF OWNER OF ADDR
                PUSH  AF
                LD    D,B
                LD    B,A

```

```

CALLx GET_STATUS ;GET STATUS OF OWNER
PUSH BC
CALLx GET_CHUNK ;SET HS FOR GETTING AT ADDR
CPL ;PUT IN ACTIVE LOW FORMAT
LD B,D
LD C,A
CALL BANK_ENABLE ;ENABLE ADDR
LD E,(HL) ;WRITE THE WORD
INC HL
LD D,(HL)
DEC HL
EX DE,HL
POP BC
POP AF
LD B,A
CALL BANK_ENABLE ;REENABLE OWNER OF ADDR
POP DE
POP BC
POP AF
RET

```

```

;
; PUT_WORD (WORD: DE, ADDR: HL, BANK: B)
;

```

```

PUT_WORD:  PUSH AF
           PUSH BC
           CALLx GET_NUMBER
           PUSH AF
           LD D,B
           LD B,A
           CALLx GET_STATUS
           PUSH BC
           CALLx GET_CHUNK
           CPL
           LD B,D
           LD C,A
           CALL BANK_ENABLE ;BBB
           LD (HL),E
           INC HL
           LD (HL),D
           DEC HL
           POP BC
           POP AF
           CALL BANK_ENABLE ;CCC
           POP BC
           POP AF
           RET

```

```

;
; WRITE_BS_REG (REG_ADDR: D, REG_DATA: E)
;
WRITE_BS_REG: PUSH AF ;SAVE REGS
              PUSH BC
              PUSH HL
              LD H,D

```

```

LD      L,0          ;HL = MEMORY MAPPED ADR
LD      A,(LOWNYB)   ;SAVE ($E000)
PUSH    AF
LD      A,(HL)       ;SAVE (HL)
PUSH    AF
LD      A,7
OUT     (SADDPT),A   ;SAVE VALUES OF SOUND REGS $07 AND $0E
IN      A,(SDATPT)
LD      B,A
LD      A,$0E
OUT     (SADDPT),A
IN      A,(SDATPT)
LD      C,A
LD      A,7          ;SET IOA CHANNEL TO OUTPUT
OUT     (SADDPT),A
LD      A,$40
OUT     (SDATPT),A
LD      A,$0E
OUT     (SADDPT),A
XOR     A
OUT     (SDATPT),A
LD      A,2
LD      (LOWNYB),A   ;RESET NYBBLE STEERING LOGIC
LD      A,E
LD      (HL),A       ;WRITE LSN OF DATA
SRA     A
SRA     A
SRA     A
SRA     A
LD      (HL),A       ;WRITE MSN OF DATA
LD      A,7          ;RESTORE SOUND REGS
OUT     (SADDPT),A
LD      A,B
OUT     (SDATPT),A
LD      A,$0E
OUT     (SADDPT),A
LD      A,C
OUT     (SDATPT),A
POP     AF
LD      (HL),A       ;RESTORE (HL)
POP     AF
LD      (LOWNYB),A   ;RESTORE ($E000)
POP     HL           ;RESTORE REGS
POP     BC
POP     AF
RET

;
;  READ_BS_REG (LSN_ADDR: D, MSN_ADDR: E, BYTE_DATA: C)
;
READ_BS_REG: PUSH    AF          ;SAVE REGS
              PUSH    BC
              PUSH    HL
              LD      H,D
              LD      L,0        ;HL=MEMORY MAPPED ADDR

```

```

LD      A,(LOWNYB)    ;SAVE ($E000)
PUSH    AF
LD      A,(HL)        ;SAVE (HL)
PUSH    AF
LD      A,7
OUT     (SADDPT),A    ;SAVE VALUES OF SOUND REGS $07 AND $0E
IN      A,(SDATPT)
LD      B,A
LD      A,$0E
OUT     (SADDPT),A
IN      A,(SDATPT)
LD      C,A
PUSH    BC
LD      A,7           ;SET IOA CHANNEL TO OUTPUT
OUT     (SADDPT),A
LD      A,$40
OUT     (SDATPT),A
LD      A,$0E
OUT     (SADDPT),A
XOR     A
OUT     (SDATPT),A
LD      A,2
LD      (LOWNYB),A    ;RESET NYBBLE STEERING LOGIC
LD      A,(HL)        ;READ LSN OF DATA
AND     $0F
LD      C,A
LD      H,E
LD      A,(HL)        ;READ MSN OF DATA
SLA     A
SLA     A
SLA     A
SLA     A
OR      C
LD      E,A           ;RETURN BYTE DATA IN E
POP     BC            ;POP SAVED SOUND REG DATA
LD      A,7           ;RESTORE SOUND REGS
OUT     (SADDPT),A
LD      A,B
OUT     (SDATPT),A
LD      A,$0E
OUT     (SADDPT),A
LD      A,C
OUT     (SDATPT),A
POP     AF
LD      (HL),A        ;RESTORE (HL)
POP     AF
LD      (LOWNYB),A    ;RESTORE ($E000)
POP     HL            ;RESTORE REGS
POP     BC
POP     AF
RET

```

```

;
; GET_BANK_STATUS (BANK: B, STATUS: B, HORIZONTAL_SELECT: C)
;

```

```

GET_STATUS:  PUSH  AF          ;SAVE REGS
              PUSH  DE
              LD    A,B
              CP    $FE
              JR    Z,GS_EXT    ;IF BANK=254
              CP    $FF
              JR    Z,GS_HOME    ;IF BANK=255
              AND   A
              JR    Z,GS_DOCK    ;IF BANK=0
              LD    D,BNA        ;HERE IF EXP BANK
              LD    E,B
              CALLx WRITE_BS_REG
              LD    D,HS_LSN
              LD    E,HS_MSN
              CALLx READ_BS_REG  ;READ HS
              LD    A,E
              CPL
              LD    C,A
              LD    D,STA_L
              LD    E,STA_O
              CALLx READ_BS_REG  ;READ STATUS NYBBLES
              LD    B,E
              JR    GS_EXIT
GS_HOME:     LD    BC,0          ;RETURN 0 FOR HOME BANK STATUS
              JR    GS_EXIT
GS_DOCK:     IN    A,(DKHSPT)    ;RETURN DOCK BANK STATUS)
              CPL
              LD    B,A
              LD    C,0
              JR    GS_EXIT
GS_EXT:      IN    A,(HREXPT)
              AND   $80          ;CLEAR ALL BITS BUT BIT 7
              CPL
              RLCA                ;PUT ACTIVE BIT IN BIT ZERO
              LD    B,A
              IN    A,(DKHSPT)
              CPL
              AND   1
              OR    B
              LD    B,A
              LD    C,0
GS_EXIT:     POP    DE          ;RESTORE REGISTERS
              POP    AF
              RET

;
; GET_CHUNK (ADDR: HL, MASK: A)
;
GET_CHUNK:   PUSH  BC          ;SAVE B
              LD    A,H          ;CHUNK NUMBER=HIGH 3 BITS OF
              LD    B,5          ; H SO SHIFT RIGHT 3 BITS
GC_SHIFT:    SRL    A
              DJNZ  GC_SHIFT
              INC    A          ;CREATE MASK BY ROLLING A 1
              LD    B,A          ; LEFT CHUNK NUMBER+1 TIMES

```

```

        XOR    A                ; THE FLAG COMES FROM THE CARRY
        SCF
GC_ROLL:  RLA
        DJNZ   GC_ROLL
        POP    BC                ;RESTORE B
        RET

;
;GET_BANK_NUMBER (ADDR: HL, BANK_NUMBER: A)
;
GET_NUMBER:  PUSH    BC                ;SAVE REGS
             PUSH    DE
             CALLx   GET_CHUNK
             LD      C,A
             LD      A,(BS_MAX_BANK)    ;GET LARGEST BANK NUMBER
             AND     A
             JR      Z,GN_RD_DOCK ;IF NO EXP BANKS
             LD      B,A
GN_CHECK:   LD      E,B                ;SEARCH ALL EXP BANKS
             CALLx   GET_STATUS
             AND     C
             JR      Z,GN_EXP            ;FOUND THE CHUNK, SO EXIT THE LOOP
             DJNZ   GN_CHECK
GN_RD_DOCK: IN      A,(DKHSPT)    ;NOT IN EXP BANKS SO CHECK DOCK
             CPL
             AND     C
             JR      Z,GN_DOCK
             DEC     C                ;IF CHUNK>1, THEN CAN'T BE IN EXT
             JR      NZ,GN_HOME
             IN      A,(HREXPT)    ;CHECK IF IN EXT BANK
             AND     $80
             LD      D,A
             IN      A,(DKHSPT)
             AND     1
             RRCA
             AND     D
             JR      Z,GN_HOME        ;NOT IN EXT BANK
             LD      A,$FE          ;IN EXT BANK SO RETURN 254
             JR      GN_EXIT
GN_HOME:   LD      A,$FF            ;IN HOME BANK, SO RETURN 255
             JR      GN_EXIT
GN_DOCK:   XOR     A                ;FOUND IN DOCK SO RETURN 0
             JR      GN_EXIT
GN_EXP:    LD      A,B                ;RETURN EXP BANK NUMBER
GN_EXIT:   POP     DE                ;RESTORE REGS
             POP     BC
             RET

;
;BANK_ENBALE (BANK: B, HORIZONTAL_SELECT: C)
;
X_BANK_ENABLE:  PUSH    AF                ;/SAVE REGS

```

```

        PUSH    BC            ;|
        PUSH    DE            ;|
        PUSH    HL            ;\
        LD      H,B           ;bank to H
        LD      A,(BS_MAX_BANK) ;GET LARGEST BANK NUMBER
        AND     A             ;
        JR      Z,BE_SKIP     ;IF NO EXP BANKS
        LD      D,BNA         ;
        LD      E,0           ;
        CALLx   WRITE_BS_REG   ;
        LD      D,HSP         ;
        PUSH    AF            ;
        LD      A,C           ;
        CPL                     ;
        LD      E,A           ;
        POP     AF            ;
        CALLx   WRITE_BS_REG   ;TURN OFF APPROPRIATE BITS OF
                                ; ALL EXP BANKS
BE_SKIP:  LD      A,B
        AND     A
        JR      NZ,BE_NTDock
        LD      A,C
        CP      $FF
        JR      Z,BE_EXT_OK
        IN      A,(HREXPT)    ;HERE FOR DOCK
        RES     7,A
        OUT     (HREXPT),A
BE_EXT_OK: LD      A,C
        CPL
        OUT     (DKHSPT),A    ;ENABLE DOCK
        JR      BE_EXIT
BE_NTDock: LD      A,B        ;CHECK IF EXT
        CP      $FE
        JR      NZ,BE_NTEXT
        IN      A,(HREXPT)    ;HERE FOR EXT
        RLA
        RR      C
        CCF
        RRA
        OUT     (HREXPT),A
        BIT     7,A
        JR      NZ,BE_SET
        IN      A,(DKHSPT)
        RES     0,A
        OUT     (DKHSPT),A
        JR      BE_EXIT
BE_SET:   IN      A,(DKHSPT)
        SET     0,A
        OUT     (DKHSPT),A
        JR      BE_EXIT
BE_NTEXT: IN      A,(DKHSPT)  ;DISABLE DOCK
        CPL
        LD      E,A
        LD      A,C
        CPL

```

```

        OR      E
        CPL
        OUT     (DKHSPT),A
        BIT     0,C
        JR      NZ,BE_CHK_HOME
        IN      A,(HREXPT)    ;DISABLE EXT
        RES     7,A
        OUT     (HREXPT),A
        IN      A,(DKHSPT)
        RES     0,A
        OUT     (DKHSPT),A
BE_CHK_HOME: LD      A,B          ;CHECK IF HOME
        CP      $FF
        JR      Z,BE_EXIT      ;IS HOME, SO DONE
        LD      D,BNA          ;WRITE NEW EXP BANK STATUS
        LD      E,B
        CALLx   WRITE_BS_REG
        LD      D,HS
        LD      A,C
        CPL
        LD      E,A
        CALLx   WRITE_BS_REG
BE_EXIT:  POP     HL            ;RESTORE REGS
        POP     DE
        POP     BC
        POP     AF
        RET

;
; SAVE_BANK_STATUSES (STATUS_ADR: IX)
; PUSHES THE STATUS OF ALL BANKS ONTO THE STACK (IX)
;
SAVE_STATUS: PUSH   AF          ;SAVE REGS
        PUSH   BC
        PUSH   DE
        IN     A,(HREXPT)      ;SAVE EXT BANK STATUS
        NOP                    ; LEAVE BITS 0-6 ALONE.  NOPS PUT IN
        NOP                    ; TO KEEP ADDRS THE SAME
        LD     (IX+0),A
        INC    IX
        IN     A,(DKHSPT)      ;GET DOCK BANK STATUS
        LD     (IX+0),A
        INC    IX
        LD     A,(BS_MAX_BANK) ;GET NUMBER OF BANKS
        AND    A
        JR     Z,SS_EXIT
        LD     B,A             ;SET UP BANK COUNTER
SS_LOOP:  LD     E,B            ;BANK NUMBER INTO E
        CALLx  GET_STATUS      ;GET STATUS OF BANK IN b
        LD     (IX+0),C
        INC    IX
        LD     B,E
        DJNZ   SS_LOOP         ;AROUND FOR ALL
SS_EXIT:  DEC    IX
        POP    DE              ;RESTORE REGS

```



```

        POP    BC
        POP    AF
        RET

;
; RESTORE_BANK_STATUSES (STATUS_ADR: IX)
; RESTORES THE STATUS OF ALL BANKS FROM THE STACK (IX)
;
RESTORE_STATUS:    PUSH    AF            ;SAVE REGS
                   PUSH    BC
                   PUSH    DE
                   LD      A,(IX+0)      ;GET EXT ROM STATUS
                   OUT     (HREXPT),A
                   INC     IX
                   LD      A,(IX+0)      ;GET DOCK BANK STATUS
                   OUT     (DKHSPT),A
                   INC     IX
                   LD      A,(BS_MAX_BANK) ;GET NUMBER OF BANKS
                   AND     A
                   JR      Z,RS_EXIT
                   LD      B,A          ;SET UP BANK COUNTER
RS_LOOP:          LD      C,(IX+0)
                   CALL    BANK_ENABLE  ;WRITE BANK STATUS OF BANK IN B
                   INC     IX
                   DJNZ    RS_LOOP
RS_EXIT:          DEC     IX
                   POP     DE
                   POP     BC
                   POP     AF
                   RET

;
; GOTO_BANK (BANK, HORIZONTAL_SELECT, ADDR: PASSED ON STACK)
; SETS UP THE DESTINATION BANK AND JUMPS WITHOUT RETURN ADDRESS
; IN BANK
;
GOTO_BANK:        LD      IX,0          ;SET IX TO SP
                   ADD     IX,SP
                   LD      (IX+0),C      ;SAVE BC AND TRASH CALLER'S RETURN ADDRESS
                   LD      (IX+1),B
                   LD      C,(IX+2)      ;SET PARAMS FOR BANK_ENABLE
                   LD      B,(IX+3)
                   CALL    BANK_ENABLE  ;DDD
                   POP     BC          ;RESTORE BC
                   POP     IX          ;TRASH PARAMS TO GOTO_BANK
                   POP     IX          ;GET ADDR
JPIX:             JP      (IX)

;
; CALL_BANK (ADDR, BANK, HORIZONTAL_SELECT, PRM_OUT, PRM_IN)
; ALL INPUT PARAMETERS ARE PUSHED ON THE STACK

```

```

;
; CLOBBERS IX
;
; SETS UP THE BANK AND MAKES A JUMP WITH RETURN ADDRESS TO
; ADDRESS IN BANK

BS_STACK:    EQU $6200+$-$1000

                DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
                DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
                DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
                DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
                DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
                DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
                DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF
                DFB $FF,$FF,$FF,$FF,$FF,$FF,$FF,$FF

BS_SP:        EQU $5200+$
                DWL $FFFF

CALL_BANK:    EX    (SP),HL        ;GET RET ADDR
                LD    IX,(BS_SP)
                DEC   IX
                LD    (IX+0),H
                DEC   IX
                LD    (IX+0),L        ;PUSH HL ON BS_STACK
                POP    HL
                EX    (SP),HL        ;GET PARAM_IN
                DEC   IX
                LD    (IX+0),H
                DEC   IX
                LD    (IX+0),L
                LD    (BS_SP),IX
                PUSH   DE
                PUSH   BC
                PUSH   AF
                LD     HL,0
                ADD    HL,SP        ;HL = SP
                LD     D,H
                LD     E,L
                LD     A,(BS_MAX_BANK)
                LD     C,A
                LD     B,0
                INC    BC
                INC    BC        ;BC = MAX_BANK+2
                AND    A
                SBC    HL,BC
                LD     SP,HL
                LD     IX,0
                ADD    IX,DE
                EX     DE,HL        ;DE, HL NOW CONTAIN DEST, SRC
                                    ; POINTERS FOR A BLOCK MOVE
                LD     C,(IX+PRM_OUT)
                LD     B,(IX+PRM_OUT)
                LD     A,14

```

```

ADD    A,C
LD     C,A
JR     NC,CB_NC1      ;BC = PRM_OUT + 14
INC    B
CB_NC1: LDIR                ;MAKE ROOM FOR BANK STATUS
PUSH   DE
POP    IX              ;IX = DE
CALLx  SAVE_STATUS
LD     IX,0
ADD    IX,SP
LD     C,(IX+HOR_SEL)  ;GET PARAMS FOR BANK_ENABLE
LD     B,(IX+BANK)
CALL   BANK_ENABLE    ;ENABLE DESTINATION BANK
POP    AF              ;RESTORE REGS
POP    BC
POP    DE
POP    HL
POP    IX              ;TRASH PARAMS TO CALL_BANK AND GET ADDR
POP    IX
POP    IX
CALLx  JPIX
PUSH   AF
PUSH   BC
PUSH   DE
PUSH   HL
LD     IX,(BS_SP)
LD     C,(IX+0)
INC    IX
LD     B,(IX+0)
INC    IX              ;POP PRM_IN OFF BS_STACK
LD     (BS_SP),IX     ;UPDATE BS_SP
LD     IX,0
ADD    IX,SP
LD     A,8
ADD    A,C
LD     C,A
JR     NC,CB_NC2      ;BC = PRM_IN + 8
INC    B
CB_NC2: ADD    IX,BC      ;IX = SP + PRM_IN +9
PUSH   IX
POP    HL              ;HL=IX
DEC    HL              ;HL = SRC POINTER FOR BLOCK MOVE
CALLx  RESTORE_STATUS
PUSH   IX
POP    DE              ;DE= DEST POINTER FOR BLOCK MOVE
LDDR   DE,HL           ;DEALLOCATE SPACE FOR BANK STATUS
EX     DE,HL
INC    HL
LD     SP,HL           ;RESTORE SP
LD     IX,(BS_SP)
LD     C,(IX+0)
INC    IX
LD     B,(IX+0)
INC    IX              ;POP RET ADDR OF BS_STACK
LD     (BS_SP),IX     ;UPDATE BS_SP

```

```

        PUSH    BC
        POP     IX
        POP     HL
        POP     DE                ;RESTORE REGISTERS
        POP     BC
        POP     AF
        PUSH    IX
        RET

;
; HERE ARE SOME EQUATES WHICH ARE USED BY XFER_BYTES AND THE ROUTINES IT
; CALLS
;
DIRECTION: EQU    0
BUF_PTR:    EQU    0
LENGTH:     EQU    2
DEST_ADDR:  EQU    4
SRC_ADDR:   EQU    6
DEST_BANK:  EQU    8
SRC_BANK:   EQU    9

;
; MOVE_BYTES (BYTES_TO MOVE: DE, DIRECTION: A)
;
MOVE_BYTES: PUSH    HL                ;SAVE REGISTERS
            PUSH    DE
            PUSH    BC
            LD      C,B
            LD      B,(IX+SRC_BANK)
            CALL    BANK_ENABLE        ;FFF
            LD      B,D
            LD      C,E
            LD      E,(IX+BUF_PTR)
            LD      D,(IX+BUF_PTR+1)
            LD      L,(IX+SRC_ADDR)
            LD      H,(IX+SRC_ADDR+1)
            RLCA
            RRCA
            JR      C,MB_RV1          ;IF A<0
            LDIR
            ADD     HL,BC
            JR      MB_UP1
MB_RV1:    LDDR
            AND     A
            SBC     HL,BC              ;DECREMENT POINTER
MB_UP1:    LD      (IX+SRC_ADDR),L      ;STORE NEW POINTER VALUE
            LD      (IX+SRC_ADDR+1),H
            POP     BC
            POP     HL
            PUSH    HL
            PUSH    BC
            LD      B,(IX+DEST_BANK)
            CALL    BANK_ENABLE        ;SELECT DEST BANK
            LD      B,H                ;MOVE FROM STACK TO DEST

```

```

LD      C,L
LD      E,(IX+DEST_ADDR)
LD      D,(IX+DEST_ADDR+1)
LD      L,(IX+BUF_PTR)
LD      H,(IX+BUF_PTR+1)
RLCA
RRCA
JR      C,MB_RV2          ;IF A<0
LDIR
ADD     HL,BC             ;INCREMENT POINTER
JR      MB_UP2
MB_RV2:  LDDR
        AND     A
        SBC     HL,BC      ;DECREMENT POINTER
MB_UP2:  LD      (IX+DEST_ADDR),L  ;STORE NEW POINTER VALUE
        LD      (IX+DEST_ADDR+1),H
        POP     BC
        POP     DE
        POP     HL
        RET

;
; CREATE_BITMAP (ADDR: HL, BITMAP: A)
;
CREATE_BITMAP:  LD      D,H          ;SAVE START ADR
               LD      E,L
               LD      C,(IX+LENGTH)
               LD      B,(IX+LENGTH+1)
               LD      A,(IX+DIRECTION)
               RLCA                  ;CALCULATE END ADDR
               RRCA
               JR      C,CB_SUB      ;IF A<0
               ADD     HL,BC
               JR      CB_CONT
CB_SUB:  SBC     HL,BC
CB_CONT: CALLx   GET_CHUNK          ;GET END CHUNK BIT
               CPL
               LD      B,A
               EX      DE,HL
               CALLx   GET_CHUNK          ;GET START CHUNK BIT
               CPL
               LD      C,A
               XOR     B
               JR      Z,CB_EXIT
               LD      A,C            ;HERE IF START AND END CHUNKS
               AND     B              ; ARE NOT THE SAME
               LD      B,A            ;PUT START AND END BITS TOGETHER AND
               LD      C,0            ; FILL BETWEEN THEM WITH ZEROES
               SCF
CB_NB1:  LD      A,B                ;TEST NEXT BIT
               RL      C
               AND     C
               JR      NZ,CB_NB1      ;OTHERWISE, FOUND FIRST ZERO
CB_NB2:  LD      A,B                ;TEST NEXT BIT
               RL      C

```

```

        AND    C
        JR     Z,CB_EXIT          ;FOUND LAST ZERO
        XOR    B                  ;OTHERWISE, UPDATE BITMAP
        LD     B,A
        JR     CB_NB2
CB_EXIT: LD     A,B
        RET                                ;RETURN BITMAP

;
; XFER_BYTES (DIRECTION, LENGTH, DEST_ADDR, SRC_ADDR, DEST_BANK,
;           SRC_BANK,: PASSED ON STACK IN ORDER SHOWN:  STATUS_CODE: A)
;
;  ALL PARAMETERS ON STACK HAVE OFFSETS DEFINED ABOVE

XFER_BYTES:  PUSH  AF              ;SAVE REGS
             PUSH  BC
             PUSH  DE
             PUSH  HL
             LD    HL,0
             ADD   HL,SP
             LD    DE,10
             ADD   HL,DE
             EX    DE,HL          ;DE POINTS TO START OF PARAMS
             LD    A,(BS_MAX_BANK)
             LD    C,A
             LD    B,0
             LD    HL,0
             ADD   HL,SP
             AND   A
             SBC   HL,BC
             DEC   HL
             DEC   HL              ;HL=SP-MAX_BANK-2
             PUSH  HL
             POP   IX              ;IX POINTS TO LOCATION TO SAVE STATUS
             LD    SP,IX
             CALLX SAVE_STATUS     ;SAVE BANKS' STATUS
             PUSH  DE
             POP   IX              ;IX POINTS TO PARAMS
             LD    L,(IX+SRC_ADDR)
             LD    H,(IX+SRC_ADDR+1)
             CALLX CREATE_BITMAP   ;GET SRC BITMAP
             PUSH  AF              ;SAVE ON STACK TEMPORARILY
             LD    L,(IX+DEST_ADDR)
             LD    H,(IX+DEST_ADDR+1)
             CALLX CREATE_BITMAP   ;GET DESTINATION BIT MAP
             LD    C,A              ;C=DEST BITMAP
             POP   AF
             LD    B,A              ;B=SRC BITMAP
             LD    A,(IX+SRC_BANK)
             LD    D,(IX+DEST_BANK)
             CP    D                ;COMPARE SRC AND DEST BANK NUMBERS
             JR    NZ,XB_DIFF_BANKS
             LD    A,B              ;HERE IF BANK NUMBERS ARE DIFFERENT
             AND   C

```

```

                LD      B,A                ;B=UNION OR SRC AND DEST BITMAPS
                JR      XB_DO_MOVE
XB_DIFF_BANKS: LD      A,B                ;CHECK FOR OVERLAP BETWEEN SRC
AND
                OR      C                ; DEST CHUNKS
                CP      $FF
                JR      NZ,XB_OVERLAP
                LD      E,B                ;HERE IF NO OVERLAP
                LD      B,D
                CALL    BANK_ENABLE        ;SELECT DEST BANK
XB_DO_MOVE:    LD      B,(IX+SRC_BANK)
                LD      C,E
                CALL    BANK_ENABLE        ;SELECT DEST BANK
                LD      L,(IX+SRC_ADDR)
                LD      H,(IX+SRC_ADDR+1)
                LD      E,(IX+DEST_ADDR)
                LD      D,(IX+DEST_ADDR+1)
                LD      C,(IX+LENGTH)
                LD      B,(IX+LENGTH+1)
                LD      A,(IX+DIRECTION)
                RLCA
                RRCA
                JR      C,XB_REVERSE        ;IF A<0
                LDIR
                JR      XB_EXIT
XB_REVERSE:    LDDR
                JR      XB_EXIT
XB_OVERLAP:    LD      HL,MSTBOT
                PUSH    BC
                LD      B,255
                CALLx   GET_WORD
                POP     BC
                LD      DE,STKSZ
                AND     A
                SBC     HL,DE                ;HL=ADDRESS OF STACK LIMIT
                LD      DE,FREE_BYTES
                ADD     HL,DE
                EX      DE,HL                ;DE=SP_NEW
                LD      HL,0
                ADD     HL,SP                ;HL=SP_OLD
                INC     DE                ;COMPARE SP_OLD AND SP_NEW
                AND     A
                SBC     HL,DE
                JR      NC,XB_SPACE        ;IF SP_OLD-SP_NEW > 0
                LD      A,1                ;RETURN ERROR CODE
                JR      XB_EXIT
XB_SPACE:     DEC     DE
                EX      DE,HL
                LD      SP,HL                ;SET SP TO SP_NEW
                INC     DE                ;DE=BUF_SZ
                LD      A,(IX+DIRECTION)
                LD      (IX+BUF_PTR),L
                LD      (IX+BUF_PTR+1),H
                LD      L,(IX+LENGTH)
                LD      H,(IX+LENGTH+1)

```

```

XB_MOVE_LOOP: AND    A                ;HL = BYTES LEFT TO MOVE
                SBC    HL,DE          ;DE = BYTES TO MOVE THIS TIME
                JR     C,XB_LAST_MOVE ;IF LESS THAN BUF_SZ BYTES LEFT
                CALLx  MOVE_BYTES
                JR     XB_MOVE_LOOP
XB_LAST_MOVE: ADD    HL,DE
                EX     DE,HL
                CALLx  MOVE_BYTES
                EX     DE,HL
                LD     L,(IX+BUF_PTR)
                LD     H,(IX+BUF_PTR+1)
                ADD    HL,DE            ;HL = BUF_PTR+BUF_SZ
                LD     SP,HL           ;RESTORE SP
XB_EXIT:        XOR    A              ;RETURN CODE FOR SUCCESSFUL COMPLETION
                LD     IX,0
                ADD    IX,SP
                CALLx  RESTORE_STATUS ;RESTORE STATE AND RETURN ZERO
CODE
                INC    IX
                LD     SP,IX
                POP    HL              ;RESTORE REGS
                POP    DE
                POP    BC
                POP    AF
                POP    IX             ;CLEAN UP PARAMS
                EX     (SP),IX
                POP    IX
                EX     (SP),IX
                POP    IX
                EX     (SP),IX
                POP    IX
                EX     (SP),IX
                POP    IX
                EX     (SP),IX
                RET

;
; GOTO_EXT_INIT (ADDR: HL)
;
GOTO_EXT:      POP    IX              ;TRASH RET ADDR
                PUSH   AF
                IN     A,(HREXPT)
                SET    7,A
                OUT    (HREXPT),A
                LD     A,1
                OUT    (DKHSPT),A
                POP    AF
                JP     (HL)

;
                ORG $1624
;$1624 to $17FF
BLK1:          DFB $00,$00,$00,$00,$00,$00,$00,$00
                DFB $00,$00,$00,$00,$00,$00,$00,$00
                DFB $00,$00,$00,$00,$00,$00,$00,$00

```


;\$1700

TS2068 Technical Manual

```
; $1800 to $1BFF
; $1800
```

[illegible]


```

DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00

```

```

ORG    $1D00

```

```

FIXTBL:

```

```

        DWL    DISPATCH+$32
DWL     DISPATCH+$4D
DWL     DISPATCH+$72
DWL     DISPATCH+$AB

```

```

DWL $5200+INT+$0A
DWL    $5200+INT+$1F
DWL    $5200+INT+$25
DWL    $5200+INT+$2E
DWL    $5200+INT+$4D

```

```

DWL    $5200+GET_WORD+$04
DWL    $5200+GET_WORD+$0A
DWL    $5200+GET_WORD+$0E
DWL    $5200+GET_WORD+$14
DWL    $5200+GET_WORD+$1F

```

```

DWL    $5200+PUT_WORD+$03
DWL    $5200+PUT_WORD+$09
DWL    $5200+PUT_WORD+$0D
DWL    $5200+PUT_WORD+$13
DWL    $5200+PUT_WORD+$1C

```

```

DWL    $5200+GET_STATUS+$12
DWL    $5200+GET_STATUS+$19
DWL    $5200+GET_STATUS+$23

```

```

DWL    $5200+GET_NUMBER+$03
DWL    $5200+GET_NUMBER+$07
DWL    $5200+GET_NUMBER+$0F

```

```

        DWL    BANK_ENABLE+$06
        DWL    BANK_ENABLE+$13
DWL    BANK_ENABLE+$1A
        DWL    BANK_ENABLE+$75
        DWL    BANK_ENABLE+$7D

```

```

DWL    $5200+SAVE_STATUS+$14
        DWL    $5200+SAVE_STATUS+$1C

```

```

DWL    $5200+RESTORE_STATUS+$12

```

```

        DWL      $5200+RESTORE_STATUS+$1C

DWL      BS_SP

DWL      $5200+GOTO_BANK+$13

DWL      $5200+CALL_BANK+$03
        DWL      $5200+CALL_BANK+$1D
        DWL      $5200+CALL_BANK+$29
        DWL      $5200+CALL_BANK+$4E
        DWL      $5200+CALL_BANK+$5D
        DWL      $5200+CALL_BANK+$6A
        DWL      $5200+CALL_BANK+$72
        DWL      $5200+CALL_BANK+$80
        DWL      $5200+CALL_BANK+$96
        DWL      $5200+CALL_BANK+$A2
        DWL      $5200+CALL_BANK+$B0

DWL      $5200+MOVE_BYTES+$08
        DWL      $5200+MOVE_BYTES+$34

DWL      $5200+CREATE_BITMAP+$15
        DWL      $5200+CREATE_BITMAP+$1B

DWL      $5200+XFER_BYTES+$0E
        DWL      $5200+XFER_BYTES+$2D
        DWL      $5200+XFER_BYTES+$2E
        DWL      $5200+XFER_BYTES+$38
        DWL      $5200+XFER_BYTES+$54
        DWL      $5200+XFER_BYTES+$5B
        DWL      $5200+XFER_BYTES+$85
        DWL      $5200+XFER_BYTES+$BA
        DWL      $5200+XFER_BYTES+$C1
        DWL      $5200+XFER_BYTES+$D4

DWL      0                                ;TABLE TERMINATOR

;*****
; Module: DISPATCH
; Routine: UNUSED1
;*****

DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00
DFB $00,$00,$00,$00,$00,$00,$00,$00

```

```

;*****
; Module: DISPATCH
; Routine: JMPTBL
;*****
          DWL    $1795          ;PUT_LN:
          DWL    $2813          ;DRAWLN:
          DWL    $2624          ;F_PNT:
          DWL    $0939          ;SCRL:
          DWL    $08A6          ;K_CLS:
          DWL    $073F          ;PUTMES:
          DWL    $0566          ;P_NL:
          DWL    $0554          ;P_RT:
          DWL    $053A          ;P_LFT:
          DWL    $02B0          ;K_SCAN:
          DWL    $0010          ;WRCH:
          DWL    $11ED          ;SENDCH:
          DWL    $11CF          ;RDCH:
          DWL    $3C6C          ;TO_THE:
          DWL    $3C65          ;ROOT:
          DWL    $3C5E          ;ACS:
          DWL    $3C4E          ;ASN:
          DWL    $3BFD          ;ATN:
          DWL    $3BF5          ;TAN:

```

DWL \$3BD0	;SIN:
DWL \$3BC5	;COS:
DWL \$3B9E	;ANGLE:
DWL \$3B2E	;LN:
DWL \$3ADF	;EXP:
DWL \$3ACA	;INT:
DWL \$3ABB	;INTDIV:
DWL \$3656	;FLOAT:
DWL \$35D3	;TRUNC:
DWL \$356E	;DIVIDE:
DWL \$3489	;TIMES:
DWL \$3468	;MULT:
DWL \$33D3	;ADD:
DWL \$33CE	;TSSUB:
DWL \$31A1	;OUTPUT:
DWL \$3193	;FP2A:
DWL \$3160	;FP2BC:
DWL \$30F9	;ININT:
DWL \$30E9	;STK_EC:
DWL \$30E6	;STK_A:
DWL \$3059	;STKUSN:
DWL \$2FC0	;DIM:
DWL \$2FAF	;POPSTR:
DWL \$2EBD	;LET:
DWL \$2E74	;PAEDCB:
DWL \$2E70	;PSHSTR:
DWL \$2C70	;FIND_N:
DWL \$29F2	;F_INKY:
DWL \$29E5	;F_PI:
DWL \$29B6	;RND:
DWL \$28D7	;F_ATTR:
DWL \$288E	;F_SCRN:
DWL \$2854	;EXPRN:
DWL \$2810	;DRAW_L:
DWL \$26DB	;DRAW:
DWL \$2679	;CIRCLE:
DWL \$2660	;GET_XY:
DWL \$263E	;PLOTBC:
DWL \$2635	;PLOT:
DWL \$2603	;SCRMBl:
DWL \$241D	;HIFLSh:
DWL \$23DE	;COLOR:
DWL \$2380	;NOTKBQ:
DWL \$226B	;ISEQ:
DWL \$222B	;INPUT:
DWL \$217E	;P_SEQ:
DWL \$2159	;K_PRIN:
DWL \$2155	;K_LPR:
DWL \$201D	;DEF:
DWL \$2009	;BREAKQ:
DWL \$1FEB	;PAUSE:
DWL \$1FD4	;RETURN:
DWL \$1FBB	;CHK_SZ:
DWL \$1F99	;GO_SUB:
DWL \$1F39	;CLR_BC:

DWL	\$1F36	;CLEAR:	
DWL	\$1F23	;FIX_U:	
DWL	\$1F1E	;FIX_U1:	
DWL	\$1EF1	;JUMP:	
DWL	\$1EE4	;CONT:	
DWL	\$1ED4	;RAND:	
DWL	\$1ECA	;RESTBC:	
DWL	\$1E82	;DATA:	
DWL	\$1D97	;READ:	
DWL	\$1D55	;NEXT:	
DWL	\$1C59	;STOP:	
DWL	\$1C78	;FOR:	
DWL	\$1AD8	;EXCUTE:	
DWL	\$1A27	;SYNTAX:	
DWL	\$1788	;PUT_BC:	
DWL	\$1750	;DELREC:	
DWL	\$1720	;RECLen:	
DWL	\$16F0	;SUBLIN:	
DWL	\$16D6	;FIND_L:	
DWL	\$160D	;FLASHA:	
DWL	\$25D0	;MOVE:	
DWL	\$25CC	;FORMAT:	
DWL	\$25D4	;ERASE:	
DWL	\$25C8	;CAT:	
DWL	\$1465	;OPCHAN:	
DWL	\$142A	;OPEN:	
DWL	\$13BE	;CLCHAN:	
DWL	\$139F	;CLOSE:	
DWL	\$1354	;RESET:	
DWL	\$12BB	;INSERT:	
DWL	\$1230	;SELECT:	
DWL	\$11E1	;INCH:	
DWL	\$0D31	;INIT:	
DWL	\$0D1D	;K_NEW:	
DWL	\$0D0D	;DESLUG:	
DWL	\$0A4A	;PRSCAN:	
DWL	\$0A23	;DUMPPR:	
DWL	\$08EA	;CLS:	
DWL	\$08A9	;CLLHS:	
DWL	\$0888	;R_ATT:	
DWL	\$0710	;ATTBYT	
DWL	\$05B2	;SET_AT	
DWL	\$0500	;SEND_TV	
DWL	\$0A02	;K_DUMP	
DWL	\$0436	;BEEP	
DWL	\$03F3	;PARP	
DWL	\$02E1	;UD_K	start of home rom services
DWL	\$FFFF	;	RESERVED24:
DWL	\$FFFF	;	RESERVED23:
DWL	\$FFFF	;	RESERVED22:
DWL	\$FFFF	;	RESERVED21:
DWL	\$FFFF	;	RESERVED20:
DWL	\$6721	;XFER_BYTES? (\$6722)	RESERVED19:
DWL	\$65CF	;CALL_BANK? (\$65CF)	RESERVED18:
DWL	\$6571	;GOTO_BANK? (\$6571)	RESERVED17:

9 Assembler Include File

```
;clock frequency
;14.114MHz/4 = 3.528MHz
;283.446712nS

;some useful addresses
DF1:      EQU $4000
DF2:      EQU $6000      ;if present
PRBUF:    EQU $5B00
FDISPZ:   EQU $6200
FDISPNZ:  EQU $F9C0

REG_F:    EQU PRBUF
REG_A:    EQU REG_F+1
REG_C:    EQU REG_A+1
REG_B:    EQU REG_C+1
REG_E:    EQU REG_B+1
REG_D:    EQU REG_E+1
REG_L:    EQU REG_D+1
REG_H:    EQU REG_L+1
REG_IXL:  EQU REG_H+1
REG_IXH:  EQU REG_IXL+1
REG_IYL:  EQU REG_IXH+1
REG_IYH:  EQU REG_IYL+1
USR_DAT:  EQU REG_IYH+1

;HOME ROM addresses
WRCH:     EQU $0010
GETCURLCH: EQU $0018
M0020:    EQU $0020
ALLOCBC:  EQU $0030
PUTMES:   EQU $073F
CLLHS:    EQU $08A9      ;clear the lower half screen

SELECT:   EQU $1230
INSERT:   EQU $12BB
REMGSZ:   EQU $12CA

RESET:    EQU $1354      ;reset
RECLN:    EQU $1720
DEL_DE:   EQU $174D
DELREC:   EQU $1750
AROS:     EQU $18C6
PASAVE:   EQU $19E0
TEM6:     EQU $1BE5
TEM10:    EQU $1BEF
STKFP0:   EQU $1C51
```

```

FIX_U:      EQU $1F23
CHK_SZ:     EQU $1FBB

TERM?:     EQU $21E7
FIND_N:     EQU $2C70
PGPSTR:     EQU $2FAF

SEPRMT:     EQU $3C89           ;tape message table

;function dispatcher addresses
;DISPATCH: EQU $6200
;INT:       EQU $62AE
;GET_WORD:  EQU $6316
;PUT_WORD:  EQU $633B
;GET_STATUS: EQU $6405
;GET_NUMBER: EQU $645E
BANK_ENABLE: EQU $6499
;SAVE_STATUS: EQU $651E
;RESTORE_STATUS: EQU $654A
;BS_STACK:  EQU $658E
;BS_SP:     EQU $65CE
;GOTO_BANK: EQU $6572
;CALL_BANK: EQU $65D0
;MOVE_BYTES: EQU $668C
;CREATE_BITMAP: EQU $66E8
;XFER_BYTES: EQU $6722

;EXROM addresses
SLVM:      EQU $01AB

;useful equates
HOMEROM:   EQU $FF
DOCKBANK:  EQU $00
EXROM:     EQU $FE
EDITKEY:   EQU 7

;SYSTEM VARIABLES
KSTATE:    EQU $5C00           ;
KS_A1:     EQU $5C00           ;bit 7=1 means no key stored
KS_C1:     EQU $5C01           ;debounce count 0
KS_D1:     EQU $5C02           ;repeat count 0
KS_B1:     EQU $5C03           ;char 0
KS_A2:     EQU $5C04           ;bit 7=1 means no key stored
KS_C2:     EQU $5C05           ;debounce count 1
KS_D2:     EQU $5C06           ;repeat count 1
KS_B2:     EQU $5C07           ;character 1

LASTK:     EQU $5C08           ;stores newly pressed key
REPDEL:    EQU $5C09           ;repeat delay
REPPER:    EQU $5C0A           ;key repeat delay

```

```

DEFADD:      EQU $5C0B      ;address of user defined function
argument
KDATA:      EQU $5C0D      ;stores second byte of color controls
                        ; entered from keyboard
TVDATA:      EQU $5C0E      ;
STRMS:      EQU $5C10      ;addresses of channels attached to
streams
  CH_M3:     EQU STRMS      ; Chan -3:  channel for keyboard and lower
screen
  CH_M2:     EQU CH_M3+2    ; Chan -2:  main screen
  CH_M1:     EQU CH_M2+2    ; Chan -1:  RAM write
  CH_0:      EQU CH_M1+2    ; Chan 0:  output to lower screen
  CH_1:      EQU CH_0+2     ; Chan 1:  INPUT command
  CH_2:      EQU CH_1+2     ; Chan 2:  PRINT/LIST
  CH_3:      EQU CH_2+2     ; Chan 3:  LPRINT/LLIST
  CH_4:      EQU CH_3+2     ; user chan 1      (4)
  CH_5:      EQU CH_4+2     ; user chan 2      (5)
  CH_6:      EQU CH_5+2     ; user chan 3      (6)
  CH_7:      EQU CH_6+2     ; user chan 4      (7)
  CH_8:      EQU CH_7+2     ; user chan 5      (8)
  CH_9:      EQU CH_8+2     ; user chan 6      (9)
  CH_10:     EQU CH_9+2     ; user chan 7      (10)
  CH_11:     EQU CH_10+2    ; user chan 8      (11)
  CH_12:     EQU CH_11+2    ; user chan 9      (12)
  CH_13:     EQU CH_12+2    ; user chan 10     (13)
  CH_14:     EQU CH_13+2    ; user chan 11     (14)
  CH_15:     EQU CH_14+2    ; user chan 12     (15)

CHARS:      EQU $5C36      ;address of character table - 256
RASP:      EQU $5C38      ;length of warning buzz
PIP:      EQU $5C39      ;length of keyboard click
ERRNR:     EQU $5C3A      ;error number (IY points here)
Y:         EQU ERRNR      ;
FLAGS:      EQU $5C3B      ;various flags
  SPC:      EQU 0          ; suppress space before tokens
  PR:      EQU 1          ; print to printer, not TV
  LMODE1:   EQU 2          ; L mode, not K at current character
  LMODE2:   EQU 3          ; L mode, not K at cursor
  UNKFLAG:  EQU 4          ; UNKNOWN FLAG: used all over the place
  KEYHIT:   EQU 5          ; keyhit found
  NUM:      EQU 6          ; expression is numerical, not string
  INTPT:    EQU 7          ; interpret rather than check syntax

TVFLAG:     EQU $5C3C      ;flags associated with the TV
LHS:      EQU 0          ; printing to lower half of screen
EDIT:      EQU 1          ; outputting line for edit or number
for string

  ECHREQ:   EQU 3          ; echo requested if inputting from keyboard
  TVLIST:   EQU 4          ; outputting and automatic listing
  CLHS:     EQU 5          ; clear lower half screen when key
pressed

ERRSP:      EQU $5C3D      ;bottom item on machine stack

```

```

LISTSP:      EQU $5C3F      ;return address from automatic
listing
MODE:        EQU $5C41      ;0 = K or L, 1 = F, 2 = G
NEWPPC:      EQU $5C42      ;line to be jumped to
NSPPC:       EQU $5C44      ;subline to be jumped to (bit 7 off
forces jump)
PPC:         EQU $5C45      ;line # of instruction being
interpreted
SUBPPC:      EQU $5C47      ;number within line of instruction
being interpreted
BORDCR:      EQU $5C48      ;border color
EPPC:        EQU $5C49      ;line # of "current" line in listing
VARS:        EQU $5C4B      ;first record for a variable (last is
1 byte of $80)
DEST:        EQU $5C4D      ;var matched by template code 1 or 4
(text or record)
CHANS:       EQU $5C4F      ;channel data (including floppy
buffers)

; each item comprises
; the address of an output routine for WRCH
; the address of an input routine for RDCH
; a 1 byte code for the device type and

where

; appropriate, a file name, additional data

and

; a buffer

CURCHL:      EQU $5C51      ;pointer to data for current channel
PROG:        EQU $5C53      ;address of BASIC program start
NXTLIN:      EQU $5C55      ;next line of source code
DATADD:      EQU $5C57      ;terminator of last data item
ELINE:       EQU $5C59      ;line being edited
KCUR:        EQU $5C5B      ;current char in input buffer
CHADD:       EQU $5C5D      ;current char when syntax checking
etc.
XPTR:        EQU $5C5F      ;1st char not syntactically OK (0 of
all OK)

; also stores (CHADD) during read and input
WORKSP:      EQU $5C61      ;temporary workspace
STKBOT:      EQU $5C63      ;bottom of calculator stack
STKEND:      EQU $5C65      ;next free place on calculator stack
STKNXT:      EQU STKEND    ;alias for STKEND
BREG:        EQU $5C67      ;calculator's BREG
MEM:         EQU $5C68      ;pointer to calculator memory area
FLAGS2:      EQU $5C6A      ;more flags
  ALOS:       EQU 0        ; automatic listing on screen
  PRLEFT:     EQU 1        ; printer buffer not empty
  L_STR:      EQU 2        ; inside string when doing KB mode in LISTCH
  CAPS_L:     EQU 3        ; capitals shift lock on
  RETPOS:     EQU 4        ; retype possible after syntax error
  DELREP:     EQU 5        ; delete key repeat (key held down)

DFSZ:        EQU $5C6B      ;# lines in 2nd half of screen
including

; separating blank line

```

STOP:	EQU \$5C6C		;line # (in program) of top line on screen
OLDPPC:	EQU \$5C6E		;line # of E.G. interruted statement
OSPCC:	EQU \$5C70		;(old sub PPC) statement number
within line for OLDPPC			
FLAGX:	EQU \$5C71		;flags associated with assignment
FLEX:	EQU 0		; flexible length assignment required
UNFND:	EQU 1		; destination of assignment not found
INPLN:	EQU 5		; req input value rather than line of program
NO:	EQU 6		; reqd type is numeric
LINPLN:	EQU 7		; linput (input line) rather than straight input
STRLEN:	EQU \$5C72		;length of destination when string type
TADDR:	EQU \$5C74		;next byte in template
SEED:	EQU \$5C76		;last random # before scaling
FRAMES:	EQU \$5C78		;ls 2 byte of 3 byte frame counter
FRAMES2:	EQU \$5C7A		;ms byte of 3 byte frame counter
UDG:	EQU \$5C7B		;1st byte of UDG
XCOORD:	EQU \$5C7D		;x coord of last plot, etc.
YCOORD:	EQU \$5C7E		;y coord of last plot, etc.
PPOSN:	EQU \$5C7F		;column no. of printer pos.
PRCC:	EQU \$5C80		;ls byte of address of next char for printer
NOT_USED:	EQU \$5C81		;
ECHOE:	EQU \$5C82		;coords in lower half of end of keyboard input buffer
DFCC:	EQU \$5C84		;screen char under print cursor
DFCCL:	EQU \$5C86		;like DFCC for lower half screen
SPOSNCOL:	EQU \$5C88		;column for next screen output
SPOSNLIN:	EQU \$5C89		;line for next screen output
SPOSNLCOL:	EQU \$5C8A		;column for next screen output on lower half screen
SPOSNLLIN:	EQU \$5C8B		;line for next screen output on lower half screen
SCRCT:	EQU \$5C8C		;scroll count - decremented for each scroll
ATTRP:	EQU \$5C8D		;current printing attributes
FOREG:	EQU 0		;ls bit of foreground color (ink)
BLUEFG:	EQU 0		;
REDFG:	EQU 1		;
GRNFG:	EQU 2		;
BACKG:	EQU 3		;
BLUEBG:	EQU 3		;
REDBG:	EQU 4		;
GRNBG:	EQU 5		;
HILITE:	EQU 6		;
FLASH:	EQU 7		;
MASKP:	EQU \$5C8E		;current permanent printing atributes
mask;			
; 0: take attrib from ATTRP, 1: take attrib from screen			

```

ATTRT:      EQU $5C8F                ;current temp. printing attributes
(see ATTRP for bit definitions)
MASKT:      EQU $5C90                ;current temporary printing
attributes mask
PFLAG:      EQU $5C91                ;additional flags for printing:
temporary flags in even bits,
                                ; premanent flags in odd bits
                                ; new characters XORed into old
                                ; new characters inverted
XOR_CH:     EQU 0
INV_CH:     EQU 2
F_CB:       EQU 4                    ; foreground := complement of
background
B_CF:       EQU 6                    ; background := complement of
foreground

MEMBOT:     EQU $5C92                ;bottom of calculator memory (6
numbers)
MEM0:       EQU $5C92
MEM1:       EQU $5C97
MEM2:       EQU $5C9C
MEM3:       EQU $5CA1
MEM4:       EQU $5CA6
MEM5:       EQU $5CAB

USRNMI:     EQU $5CB0                ;address of NMI service routine
RAMTOP:     EQU $5CB2                ;last address of BASIC system area
PRAMT:      EQU $5CB4                ;last byte of physical RAM
ERRLN:      EQU $5CB6                ;pointer to ONERROR line number for
at GOTO
ERRC:       EQU $5CB8                ;store line number in which error
occurred
ERRS:       EQU $5CBA                ;stores statement number in which
error occurred
ERRT:       EQU $5CBB                ;store for "error type" after and
ONERROR
SYSCON:     EQU $5CBC                ;system configuration table
MAXBNK:     EQU $5CBE                ;largest bank number assigned
CRCBN:      EQU $5CBF                ;bank number of current channel
MSTBOT:     EQU $5CC0                ;address of location above machine
stack
VIDMOD:     EQU $5CC2                ;video mode
ARSBUF:     EQU $5CC4                ;pointer to AROS buffer
ARSFLG:     EQU $5CC6                ;AROS flag
                                ; bit 7 set indicates AROS present
                                ; bit 4 set indicates NXTLIN pointing to
AROS
                                ; bit 3 set indicates DATADD pointing to
AROS
ADATLN:     EQU $5CC7                ;pointer to the start of the current
data line (AROS only)
DTLNLN:     EQU $5CC8                ;length of the current data line
(AROS only)
STRMN:      EQU $5CCB                ;current stream number used for bus
expansion unit devices

```



```

;IY offsets
;OERRNR:      EQU $00          ;ERRNR
;OFLAGS:      EQU $01          ;FLAGS
;OTVFLAG:     EQU $02          ;TVFLAG
;OMODE:       EQU $07          ;MODE
;ONSPPC:      EQU $0A
;OSUBPPC:     EQU $0D
;OBORDCR:     EQU $0E
;OEPPC:       EQU $0F
;OKCUR:       EQU $21
;OXPTR:       EQU $25
;OBREG:       EQU $2D          ;BREG
;OFLAGS2:     EQU $30          ;FLAGS2
;ODFSZ:       EQU $31          ;DFSZ
;OOSPCC:      EQU $36
;OFLAGX:      EQU $37
;OSTRLEN:     EQU $38
;OFRAMES:     EQU $40          ;FRAMES
;OPPOSN:      EQU $45
;OPRCC:       EQU $46
;OSPOSNLIN:   EQU $4F
;OSPOSNLCOL:  EQU $50
;OSCRCT:      EQU $52
;OPFLAG:      EQU $57          ;PFLAG
;OERRLN:      EQU $7C

```

```

OERRNR:      EQU ERRNR-Y
OFLAGS:      EQU FLAGS-Y
OTVFLAG:     EQU TVFLAG-Y
OMODE:       EQU MODE-Y
ONSPPC:      EQU NSPPC-Y
OSUBPPC:     EQU SUBPPC-Y
OBORDCR:     EQU BORDCR-Y
OEPPC:       EQU EPPC-Y
OKCUR:       EQU KCUR-Y
OXPTR:       EQU XPTR-Y
OBREG:       EQU BREG-Y
OFLAGS2:     EQU FLAGS2-Y
ODFSZ:       EQU DFSZ-Y
OOSPCC:      EQU OSPCC-Y
OFLAGX:      EQU FLAGX-Y
OSTRLEN:     EQU STRLEN-Y
OFRAMES2:    EQU FRAMES-Y+2
OPPOSN:      EQU PPOSN-Y
OPRCC:       EQU PRCC-Y
OSPOSNLIN:   EQU SPOSNLIN-Y
OSPOSNLCOL:  EQU SPOSNLCOL-Y
OSCRCT:      EQU SCRCT-Y
OPFLAG:      EQU PFLAG-Y
OERRLN:      EQU ERRLN-Y

```

FD_RESERVED1:	EQU	1	;
FD_RESERVED2:	EQU	2	;
FD_RESERVED3:	EQU	3	;
FD_RESERVED4:	EQU	4	;
FD_RESERVED5:	EQU	5	;
FD_RESERVED6:	EQU	6	;
FD_RESERVED7:	EQU	7	;
FD_RESERVED8:	EQU	8	;
FD_RESERVED9:	EQU	9	
FD_RESERVED10:	EQU	10	;
FD_RESERVED11:	EQU	11	;
FD_RESERVED12:	EQU	12	;
FD_RESERVED13:	EQU	13	;
FD_GET_STATUS:	EQU	14	;(0EH)
FD_GET_NUMBER:	EQU	15	;(0FH)
FD_RESERVED16:	EQU	16	;
FD_RESERVED17:	EQU	17	;
FD_RESERVED18:	EQU	18	;
FD_RESERVED19:	EQU	19	;
FD_RESERVED20:	EQU	20	;
FD_RESERVED21:	EQU	21	;
FD_RESERVED22:	EQU	22	;
FD_RESERVED23:	EQU	23	;
FD_RESERVED24:	EQU	24	;
FD_UPD_K:	EQU	25	;(19H)
FD_PARP:	EQU	26	;(1AH)
FD_BEEP:	EQU	27	;(1BH)
FD_K_DUMP:	EQU	28	;(1CH)
FD_SENDTV:	EQU	29	;(1DH)
FD_SETAT:	EQU	30	;(1EH)
FD_ATTBYT:	EQU	31	;(1FH)
FD_R_ATTS:	EQU	32	;(20H)
FD_CLLHS:	EQU	33	;(21H)
FD_CLS:	EQU	34	;(22H)
FD_DUMPPIR:	EQU	35	;(23H)
FD_PRSCAN:	EQU	36	;(24H)
FD_DESLUG:	EQU	37	;(25H)
FD_K_NEW:	EQU	38	;(26H)
FD_INIT:	EQU	39	;(27H)
FD_INCH:	EQU	40	;(28H)
FD_SELECT:	EQU	41	;(29H)
FD_INSERT:	EQU	42	;(2AH)
FD_RESET:	EQU	43	;(2BH)
FD_CLOSE:	EQU	44	;(2CH)
FD_CLCHAN:	EQU	45	;(2DH)
FD_OPEN:	EQU	46	;(2EH)
FD_OPCHAN:	EQU	47	;(2FH)
FD_CAT:	EQU	48	;(30H)
FD_ERASE:	EQU	49	;(31H)

FD_FORMAT:	EQU	50	; (32H)
FD_MOVE:	EQU	51	; (33H)
FD_FLASHA:	EQU	52	; (34H)
FD_FIND_L:	EQU	53	; (35H)
FD_SUBLIN:	EQU	54	; (36H)
FD_RECLIN:	EQU	55	; (37H)
FD_DELREC:	EQU	56	; (38H)
FD_PUT_BC:	EQU	57	; (39H)
FD_SYNTAX:	EQU	58	; (3AH)
FD_EXCUTE:	EQU	59	; (3BH)
FD_FOR:	EQU	60	; (3CH)
FD_STOP:	EQU	61	; (3DH)
FD_NEXT:	EQU	62	; (3EH)
FD_READ:	EQU	63	; (3FH)
FD_DATA:	EQU	64	; (40H)
FD_RESTBC:	EQU	65	; (41H)
FD_RAND:	EQU	66	; (42H)
FD_CONT:	EQU	67	; (43H)
FD_JUMP:	EQU	68	; (44H)
FD_FIX_U1:	EQU	69	; (45H)
FD_FIX_U:	EQU	70	; (46H)
FD_CLEAR:	EQU	71	; (47H)
FD_CLR_BC:	EQU	72	; (48H)
FD_GO_SUB:	EQU	73	; (49H)
FD_CHK_SZ:	EQU	74	; (4AH)
FD_RETURN:	EQU	75	; (4BH)
FD_PAUSE:	EQU	76	; (4CH)
FD_BREAKQ:	EQU	77	; (4DH)
FD_DEF:	EQU	78	; (4EH)
FD_K_LPR:	EQU	79	; (4FH)
FD_K_PRIN:	EQU	80	; (50H)
FD_P_SEQ:	EQU	81	; (51H)
FD_INPUT:	EQU	82	; (52H)
FD_ISEQ:	EQU	83	; (53H)
FD_NOTKBQ:	EQU	84	; (54H)
FD_COLOR:	EQU	85	; (55H)
FD_HIFLSH:	EQU	86	; (56H)
FD_SCRMBL:	EQU	87	; (57H)
FD_PLOT:	EQU	88	; (58H)
FD_PLOTBC:	EQU	89	; (59H)
FD_GET_XY:	EQU	90	; (5AH)
FD_CIRCLE:	EQU	91	; (5BH)
FD_DRAW:	EQU	92	; (5CH)
FD_DRAW_L:	EQU	93	; (5DH)
FD_EXPRN:	EQU	94	; (5EH)
FD_F_SCRN:	EQU	95	; (5FH)
FD_F_ATTR:	EQU	96	; (60H)
FD_RND:	EQU	97	; (61H)
FD_F_PI:	EQU	98	; (62H)
FD_F_INKY:	EQU	99	; (63H)
FD_FIND_N:	EQU	100	; (64H)
FD_PSHSTR:	EQU	101	; (65H)
FD_PAEDCB:	EQU	102	; (66H)
FD_LET:	EQU	103	; (67H)
FD_POPSTR:	EQU	104	; (68H)

FD_DIM:	EQU	105	; (69H)
FD_STKUSN:	EQU	106	; (6AH)
FD_STK_A:	EQU	107	; (6BH)
FD_ININT:	EQU	109	; (6DH)
FD_FP2BC:	EQU	110	; (6EH)
FD_FP2A:	EQU	111	; (6FH)
FD_OUTPUT:	EQU	112	; (70H)
FD_TSSUB:	EQU	113	; (71H)
FD_ADD:	EQU	114	; (72H)
FD_MULT:	EQU	115	; (73H)
FD_TIMES:	EQU	116	; (74H)
FD_DIVIDE:	EQU	117	; (75H)
FD_TRUNC:	EQU	118	; (76H)
FD_INTDIV:	EQU	120	; (78H)
FD_INT:	EQU	121	; (79H)
FD_EXP:	EQU	122	; (7AH)
FD_LN:	EQU	123	; (7BH)
FD_ANGLE:	EQU	124	; (7CH)
FD_COS:	EQU	125	; (7DH)
FD_SIN:	EQU	126	; (7EH)
FD_TAN:	EQU	127	; (7FH)
FD_ATN:	EQU	128	; (80H)
FD_ASN:	EQU	129	; (81H)
FD_ACS:	EQU	130	; (82H)
FD_ROOT:	EQU	131	; (83H)
FD_TO_THE:	EQU	132	; (84H)
FD_RDCH:	EQU	133	; (85H)
FD_SENDCH:	EQU	134	; (86H)
FD_WRCH:	EQU	135	; (87H)
FD_K_SCAN:	EQU	136	; (88H)
FD_P_LFT:	EQU	137	; (89H)
FD_P_RT:	EQU	138	; (8AH)
FD_P_NL:	EQU	139	; (8BH)
FD_PUTMES:	EQU	140	; (8CH)
FD_K_CLS:	EQU	141	; (8DH)
FD_SCRL:	EQU	142	; (8EH)
FD_F_PNT:	EQU	143	; (8FH)
FD_DRAWLN:	EQU	144	; (90H)
FD_PUT_LN:	EQU	145	; (91H)

SCRNCHAN: EQU 2 ;main screen channel number

F_PUT_LN:	EQU	\$1795
F_DRAWLN:	EQU	\$2813
F_F_PNT:	EQU	\$2624
F_SCRL:	EQU	\$0939
F_K_CLS:	EQU	\$08A6
F_PUTMES:	EQU	\$073F
F_P_NL:	EQU	\$0566
F_P_RT:	EQU	\$0554
F_P_LFT:	EQU	\$053A
F_K_SCAN:	EQU	\$02B0
F_WRCH:	EQU	\$0010

F_SENDCH:	EQU	\$11ED
F_RDCH:	EQU	\$11CF
F_TO_THE:	EQU	\$3C6C
F_ROOT:	EQU	\$3C65
F_ACS:	EQU	\$3C5E
F_ASN:	EQU	\$3C4E
F_ATN:	EQU	\$3BFD
F_TAN:	EQU	\$3BF5
F_SIN:	EQU	\$3BD0
F_COS:	EQU	\$3BC5
F_ANGLE:	EQU	\$3B9E
F_LN:	EQU	\$3B2E
F_EXP:	EQU	\$3ADF
F_INT:	EQU	\$3ACA
F_INTDIV:	EQU	\$3ABB
F_FLOAT:	EQU	\$3656
F_TRUNC:	EQU	\$35D3
F_DIVIDE:	EQU	\$356E
F_TIMES:	EQU	\$3489
F_MULT:	EQU	\$3468
F_ADD:	EQU	\$33D3
F_TSSUB:	EQU	\$33CE
F_OUTPUT:	EQU	\$31A1
F_FP2A:	EQU	\$3193
F_FP2BC:	EQU	\$3160
F_ININT:	EQU	\$30F9
F_STK_BC:	EQU	\$30E9
F_STK_A:	EQU	\$30E6
F_STKUSN:	EQU	\$3059
F_DIM:	EQU	\$2FC0
F_POPSTR:	EQU	\$2FAF
F_LET:	EQU	\$2EBD
F_PAEDCB:	EQU	\$2E74
F_PSHSTR:	EQU	\$2E70
F_FIND_N:	EQU	\$2C70
F_F_INKY:	EQU	\$29F2
F_F_PI:	EQU	\$29E5
F_RND:	EQU	\$29B6
F_F_ATTR:	EQU	\$28D7
F_F_SCRN:	EQU	\$288E
F_EXPRN:	EQU	\$2854
F_DRAW_L:	EQU	\$2810
F_DRAW:	EQU	\$26DB
F_CIRCLE:	EQU	\$2679
F_GET_XY:	EQU	\$2660
F_PLOTBC:	EQU	\$263E
F_PLOT:	EQU	\$2635
F_SCRMBL:	EQU	\$2603
F_HIFLSH:	EQU	\$241D
F_COLOR:	EQU	\$23DE
F_NOTKBQ:	EQU	\$2380
F_ISEQ:	EQU	\$226B
F_INPUT:	EQU	\$222B
F_P_SEQ:	EQU	\$217E
F_K_PRIN:	EQU	\$2159

F_K_LPR:	EQU	\$2155
F_DEF:	EQU	\$201D
F_BREAKQ:	EQU	\$2009
F_PAUSE:	EQU	\$1FEB
F_RETURN:	EQU	\$1FD4
F_CHK_SZ:	EQU	\$1FBB
F_GO_SUB:	EQU	\$1F99
F_CLR_BC:	EQU	\$1F39
F_CLEAR:	EQU	\$1F36
F_FIX_U:	EQU	\$1F23
F_FIX_U1:	EQU	\$1F1E
F_JUMP:	EQU	\$1EF1
F_CONT:	EQU	\$1EE4
F_RAND:	EQU	\$1ED4
F_RESTBC:	EQU	\$1ECA
F_DATA:	EQU	\$1E82
F_READ:	EQU	\$1D97
F_NEXT:	EQU	\$1D55
F_STOP:	EQU	\$1C59
F_FOR:	EQU	\$1C78
F_EXCUTE:	EQU	\$1AD8
F_SYNTAX:	EQU	\$1A27
F_PUT_BC:	EQU	\$1788
F_DELREC:	EQU	\$1750
F_RECLN:	EQU	\$1720
F_SUBLIN:	EQU	\$16F0
F_FIND_L:	EQU	\$16D6
F_FLASHA:	EQU	\$160D
F_MOVE:	EQU	\$25D0
F_FORMAT:	EQU	\$25CC
F_ERASE:	EQU	\$25D4
F_CAT:	EQU	\$25C8
F_OPCHAN:	EQU	\$1465
F_OPEN:	EQU	\$142A
F_CLCHAN:	EQU	\$13BE
F_CLOSE:	EQU	\$139F
F_RESET:	EQU	\$1354
F_INSERT:	EQU	\$12BB
F_SELECT:	EQU	\$1230
F_INCH:	EQU	\$11E1
F_INIT:	EQU	\$0D31
F_K_NEW:	EQU	\$0D1D
F_DESLUG:	EQU	\$0D0D
F_PRSCAN:	EQU	\$0A4A
F_DUMPFR:	EQU	\$0A23
F_CLS:	EQU	\$08EA
F_CLLHS:	EQU	\$08A9
F_R_ATT:	EQU	\$0888
F_ATTBYT:	EQU	\$0710
F_SET_AT:	EQU	\$05B2
F_SEND_TV:	EQU	\$0500
F_K_DUMP:	EQU	\$0A02
F_BEEP:	EQU	\$0436
F_PARP:	EQU	\$03F3
F_UD_K:	EQU	\$02E1

```

;RAM SERVICES
;F_XFER_BYTES? EQU $6721
;F_CALL_BANK? EQU $65CF
;F_GOTO_BANK? EQU $6571
;F_BANK_ENABLE EQU $6499
;F_GET_NUMBER EQU $645E
;F_GET_STATUS EQU $6405
;EXROM SERVICES
;F_W_BORD EQU $00E5
;F_RESERVED8: EQU $0EA3
;F_SAVE EQU $0851
;F_RESERVED6: EQU $06E5
;F_LOAD EQU $05CC
;F_SLVM EQU $01AB
;F_R_EDGE EQU $018D
;F_RD_BIT EQU $0189
;F_RD_TAPE EQU $00FC

```

;REPORT CODES

```

RPT_OK: EQU 0
RPT_NEXTWOFOR: EQU 1
RPT_VARNOTFOUND: EQU 2
RPT_BADSUBSCR: EQU 3
RPT_OUTOFMEM: EQU 4
RPT_OUTOFSCRN: EQU 5
RPT_NUMTOOBIG: EQU 6
RPT_RTNWOGOSUB: EQU 7
RPT_ENDOFFILE: EQU 8
RPT_STOP: EQU 9
RPT_INVLDARG: EQU 10
RPT_INTTOOBIG: EQU 11
RPT_BADBASIC: EQU 12
RPT_BREAK: EQU 13
RPT_OUTOFDATA: EQU 14
RPT_BADSAVENAME: EQU 15
RPT_NOROOM4LINE: EQU 16
RPT_STOPININPUT: EQU 17
RPT_FORWONEXT: EQU 18
RPT_INVLDIODEV: EQU 19
RPT_INVLDCOLOR: EQU 20
RPT_BREAK1: EQU 21
RPT_RAMTOPBAD: EQU 22
RPT_STMTLOST: EQU 23
RPT_STMTNOGOOD: EQU 24
RPT_INLDSTRM: EQU 25
RPT_FNWODEFFN: EQU 26
RPT_TAPELOADERR: EQU 27

```

;

2068 KEYBOARD CODES

```

;LSN ->0      1      2      3      4      5      6      7      8
9      A      B      C      D      E      F      PRINT_COMMA  EDIT
;0  UNUSED  UNUSED  UNUSED  UNUSED  UNUSED  UNUSED  PRINT_COMMA  EDIT
CSR_LT CSR_RT  CSR_DN  CSR_UP  DELETE  CR      SLUG  UNUSED
;1  INK      PAPER  FLASH  BRIGHT  INVERT  OVER      AT      TAB
UNUSED  UNUSED  UNUSED  UNUSED  UNUSED  UNUSED  UNUSED  UNUSED
;2  SPACE    !      "      #      $      %      &      '      (
)      *      +      ,      -      .      /
;3  0      1      2      3      4 5      6      7      8      9
:      ;      <      =      >      ?
;4  @      A      B      C      D      E      F      G      H      I      J
K      L      M      N      O
;5  P      Q      R      S      T      U      V      W      X      Y      Z
[      \      ]      ^      _
;6  POUND    a      b      c      d      e      f      g      h      i      j
k      l      m      n      o
;7  p      q      r      s      t      u      v      w      x      y      z
{      STICK      }      FREE  COPYRIGHT
;
ONERR      SOUND      RESET
;8  BLKGR0    BLKGR1  BLKGR2  BLKGR3  BLKGR4  BLKGR5      BLKGR6  BLKGR7
BLKGR8 BLKGR9  BLKGR10  BLKGR11  BLKGR12  BLKGR13  BLKGR14  BLKGR15
;9  UDG0      UDG1      UDG2      UDG3      UDG4      UDG5      UDG6      UDG7      UDG8
UDG9      UDG10     UDG11     UDG12     UDG13     UDG14     UDG15
;A  UDG16     UDG17     UDG18     UDG19     UDG20     RND      INKEY$      PI      FN
POINT SCREEN$ ATTRIB  AT      TAB      VAL$      CODE
;B  VAL      LEN      SIN      COS      TAN      ASN      ACS      ATN      LN
EXP      INT      SQR      SGN      ABS      PEEK      IN
;C  USR      STR$     CHR$     NOT      BIN      OR      AND      <=      =>
<>      LINE      THEN      TO      STEP      DEF      CAT
;D  FORMAT    MOVE    DELETE    OPEN    CLOSE    MERGE      VERIFY      BEEP      ARC
FOREGND BKGND  FLASH  BRIGHT  INVERT  OVER      OUT
;E  LPRINT    LLIST    STOP      READ      DATA  RESTORE      NEW      BORDER  CONT
DIM      REM      FOR      GOTO      GOSUB    INPUT    LOAD
;F  LIST      LET      PAUSE     NEXT      POKE     PRINT      PLOT      RUN      SAVE
RANDOM  IF      CLS      DRAW      CLEAR      RETURN  COPY

```

```

;MACROS
CALLBANKS:  MACRO  CALLADR,DESTBANK,HORIZSEL
              PUSH IX
              EXX
              LD  HL,CALLADR      ;
              PUSH HL              ;
              LD  L,HORIZSEL      ;horizontal select
              LD  H,DESTBANK      ;home bank
              PUSH HL              ;
              LD  HL,$0000        ;
              PUSH HL              ;
              PUSH HL              ;
              EXX                  ;
              CALL M0F99          ;CALL BANK
              POP  IX
            ENDM

```



```

CALLBANKD:  MACRO CALLADR1,DESTBANK1,HORIZSEL1,CALLADR2,DESTBANK2,HORIZSEL2
              PUSH IX
              EXX
              LD  HL,CALLADR1          ;
              PUSH HL                  ;
              LD  L,HORIZSEL1          ;
              LD  H,DESTBANK1          ;
              PUSH HL                  ;
              LD  HL,$0000             ;
              PUSH HL                  ;
              PUSH HL                  ;
              EXX                      ;
              CALL M0F99               ;CALL BANK

              EXX
              LD  HL,CALLADR2          ;
              PUSH HL                  ;
              LD  L,HORIZSEL2          ;
              LD  H,DESTBANK2          ;
              PUSH HL                  ;
              LD  HL,$0000             ;
              PUSH HL                  ;
              PUSH HL                  ;
              EXX                      ;
              CALL M0F99               ;CALL BANK
              POP  IX
              ENDM

```